

Универсальное пособие • полный исходный код

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 14.09.2026, 11:27:28
Файлов исходного кода: 99
Суммарный объем кода: 1.41 MB
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: seg-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref-sum]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planar-coloring]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler-cycle]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракса (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

.....

- 24. src/data/vizStepBus.ts

- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_5.ts

- 27. src/data/tickets/ticket14_20.ts

- 28. src/data/tickets/ticket21_24.ts

- 29. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 30. src/components/ArticulationPointsViz.tsx

- 31. src/components/BellmanFordViz.tsx

- 32. src/components/BfsViz.tsx

- 33. src/components/ChapterImage.tsx

- 34. src/components/ChapterNav.tsx

- 35. src/components/ChapterView.tsx

- 36. src/components/ColoringViz.tsx

- 37. src/components/ComplexityClassesViz.tsx

- 38. src/components/ComponentsViz.tsx

- 39. src/components/DfsBridgesSimulator.tsx

- 40. src/components/DijkstraViz.tsx

- 41. src/components/DPVisualizer.tsx

- 42. src/components/EulerSimulator.tsx

- 43. src/components/ExpressQuiz.tsx

- 44. src/components/FloydViz.tsx

- 45. src/components/GraphTraversalViz.tsx

- 46. src/components/HeapViz.tsx

- 47. src/components/hints/demoKit.tsx

- 48. src/components/hints/demosExtra.tsx

- 49. src/components/hints/demosGraph.tsx

- 50. src/components/hints/demosStruct.tsx

- 51. src/components/hints/MiniDemo.tsx

- 52. src/components/hints/SimulatorModal.tsx

- 53. src/components/hints/TermPopover.tsx

- 54. src/components/JohnsonViz.tsx

- 55. src/components/KosarajuViz.tsx

- 56. src/components/KruskalSimulator.tsx

- 57. src/components/MnemonicCards.tsx

```
.....
88. scripts/export/common.mjs
89. scripts/export/guide-pdf-entry.tsx
90. scripts/export/render-pages.mjs
91. scripts/selftest/_cats.tmp.ts
92. scripts/selftest/_gloss.tmp.ts
93. scripts/selftest/_harness-entry.tmp.ts
94. scripts/selftest/_regcheck.tmp.ts
95. scripts/selftest/_runner.tmp.ts
96. scripts/selftest/bundle.ts
97. scripts/selftest/lazy.ts

# CI / автоматизация
98. .github/workflows/deploy-pages.yml
99. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

```
.....
ФАЙЛ 1/99: add.md
```

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

```
.....
```

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```
.....
 cache: npm

 - name: Install dependencies
 run: npm ci --no-audit --no-fund

 - name: Build for the repository subpath
 env:
 VITE_BASE: /algv0/
 run: npm run build

 - name: Make GitHub Pages fallback
 run: cp dist/index.html dist/404.html

 - name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
.....

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
```



```
.....
  with:
    fetch-depth: 0
    persist-credentials: true

- name: Setup Node.js
  uses: actions/setup-node@v7
  with:
    node-version: "24"
    cache: npm

- name: Install ImageMagick + DejaVu fonts
  run: |
    sudo apt-get update -qq
    sudo apt-get install -y --no-install-recommends
    # На Ubuntu политика ImageMagick иногда запрещает экспорт
    # разрешаем то, что нужно пайплайну (text -> pdf)
    for policy in /etc/ImageMagick-6/policy.xml /etc/ImageMagick-6/policy.xml; do
      if [ -f "$policy" ]; then
        sudo sed -i 's/rights="none" pattern="\(\P\)/rights="write" pattern="\(\P\)/' "$policy"
      fi
    done
    convert -version

- name: Install dependencies
  run: npm ci --no-audit --no-fund

- name: Step 1 – код → txt
  run: npm run export:txt

- name: Step 2 – txt → png
  env:
    EXPORT_DENSITY: ${github.event.inputs.density:-300}
  run: npm run export:png

- name: Step 3 – png → pdf
  run: npm run export:pdf

- name: Type-check приложения
```

Универсальное пособие • полный исходный код

```
.....
    if git diff --cached --quiet; then
        echo "Экспорт не изменился — коммит не нужен"
        exit 0
    fi
    git commit -m "chore(export): пересборка full"
    git push origin "HEAD:${GITHUB_REF_NAME}"
....
```

Patch относительно `main`

```
````diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:
```

steps:

```
 - name: Checkout
 uses: actions/checkout@v4
+ uses: actions/checkout@v6
 with:
 fetch-depth: 0
 persist-credentials: true

 - name: Setup Node.js
 uses: actions/setup-node@v4
+ uses: actions/setup-node@v7
 with:
 node-version: "22"
+ node-version: "24"
 cache: npm

 - name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
 } >> "$GITHUB_STEP_SUMMARY"

 - name: Upload artifacts (PDF + TXT)
```

## Универсальное пособие • полный исходный код

```
.....
```diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
   <head>
     <meta charset="UTF-8" />
     <meta name="viewport" content="width=device-width,
+    <link rel="icon" type="image/svg+xml" href="%BASE_
     <title> Дискретка для тупых – Интерактивный гид по
   </head>
   <body class="bg-slate-950 text-slate-100 min-h-screen
.....
```

`public/favicon.svg`

Полное содержимое после изменений

```
```xml
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```
```

Patch относительно `main`

```
```diff
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbbbba0
--- /dev/null
+++ b/public/favicon.svg
@@ -0,0 +1,5 @@
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>
```





```

 goto(id);
 }}
 />
 </main>
) : (
 <PythonCompiler />
)}

<SimulatorModal vizId={modalVizId} onClose={() =>

 <footer className="p-8 text-center text-slate-600"
 © 2026 Universal Educational Guide. Интерактивный
 </footer>
</div>
);
}
....

```

### Patch относительно `main`

```

````diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
-  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
  const [activeChapterId, setActiveChapterId] = useState<string>("1");
  const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {

```

```

        setBusy(true);
        try {
            await downloadBookHtml(chapters);
        } finally {
            setBusy(false);
        }
    };

    return (
        <header className="bg-slate-900 border-b border-slate-800">
            <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-3 lg:gap-0">
                <div className="flex items-center gap-3 w-full">
                    <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 w-6 h-6">
                        <Sparkles className="w-6 h-6" />
                    </div>
                    <div className="min-w-0 flex-1">
                        <h1 className="text-xl font-extrabold text-white">
                            Универсальное пособие
                        </h1>
                        <p className="text-xs text-indigo-400 font-medium">
                            Подготовка к экзаменам: Алгоритмы, Структуры
                        </p>
                    </div>
                </div>
            </div>

            <div className="flex items-center w-full lg:w-auto">
                <button
                    id="tab-guide-btn"
                    onClick={() => setActiveTab('guide')}
                    className={`flex-1 lg:flex-none flex items-center justify-center text-white duration-200 whitespace-nowrap ${
                        activeTab === 'guide'
                            ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
                            : 'text-slate-400 hover:text-white'
                    }`}
                >

```

Универсальное пособие • полный исходный код

```
.....
    </a>
    <a
      id="download-txt-btn"
      href={` ${import.meta.env.BASE_URL}export/full_
download="full_code_all_pages.txt"
      target="_blank"
      rel="noreferrer"
      className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
      title="Скачать полный TXT файл со всем кодом
    >
      <FileText className="w-4 h-4" /> TXT
    </a>
    <button
      id="download-html-btn"
      onClick={saveBook}
      disabled={busy}
      className="flex items-center justify-center
er: bg-amber-600/20 border border-amber-500/
      title="Скачать всё пособие одним автономным
    >
      {busy ? <Loader2 className="w-4 h-4 animate
    </button>
  </div>
</div>
</header>
);
};
...

```

Patch относительно `main`

```
```diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@

```

```
.....

 <a
 id="download-txt-btn"
 href="/export/full_code_all_pages.txt"
+ href={` ${import.meta.env.BASE_URL}export/full_code_all_pages.txt`
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 >>>

`src/components/PythonCompiler.tsx`

Полное содержимое после изменений

```tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};
```

```

.....
    const runtime = await getPythonRuntime();
    setRuntimeReady(true);
    setRuntimeMessage("Python готов – код выполняется");

    runtime.setStdout({ batched: (message) => stdout.write(message) });
    runtime.setStderr({ batched: (message) => stderr.write(message) });
    runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
    await runtime.runPythonAsync(code);

    const text = [...stdout, ...stderr].join("");
    setOutput(text || "Готово: программа ничего не вывела");
  } catch (error) {
    const captured = [...stdout, ...stderr].join("");
    setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}Ошибка: ${error.message}`);
    setRuntimeMessage("Не удалось выполнить код – произошла ошибка");
  } finally {
    setRunning(false);
  }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
  setCode(STARTER_CODE);
  setStdin("");
  setOutput("Нажмите «Запустить», чтобы увидеть результат");
};

return (
  <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
    <div className="max-w-6xl mx-auto">
      <div className="flex flex-col sm:flex-row sm:items-center">
        <div>
          <div className="inline-flex items-center gap-2">
            <Terminal className="w-4 h-4" /> Боковая панель
          </div>
          <h2 className="text-2xl sm:text-3xl font-extrabold">Запустить
          <p className="text-slate-400 mt-2 max-w-2xl">
            Пишите небольшой код и запускайте его прямо в браузере,
            который переводит CPython в WebAssembly.

```

```

        </button>
      </div>
    </div>

    <textarea
      value={code}
      onChange={({event}) => setCode(event.target.value)}
      onKeyDown={({event}) => {
        if ((event.ctrlKey || event.metaKey) && event.preventDefault());
        void runCode();
      }}
      spellCheck={false}
      aria-label="Код Python"
      className="block w-full min-h-[360px] resize-y bg-transparent focus:ring-2 focus:ring-inset focus:ring-slate-700"
      placeholder="Напишите Python-код..."
    />

    <div className="border-t border-slate-800 border-b border-slate-800 border-l border-slate-800 border-r border-slate-800"
      <label className="block px-4 pt-3 text-slate-700"
        Ввод для input() • по одной строке на клавишу
      </label>
      <textarea
        id="python-stdin"
        value={stdin}
        onChange={({event}) => setStdin(event.target.value)}
        spellCheck={false}
        rows={2}
        placeholder={'Например: \nАлиса'}
        className="block w-full resize-y bg-transparent focus:ring-2 focus:ring-inset focus:ring-slate-700"
      />
    </div>
  </section>

  <section className="rounded-2xl border border-slate-800"

```

Универсальное пособие • полный исходный код

```
````diff
diff --git a/src/components/PythonCompiler.tsx b/src/com
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python
+name = "алгоритмы"
+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>;
+ setStdout: (options: { batched: (message: string) => void }) => void;
+ setStderr: (options: { batched: (message: string) => void }) => void;
+ setStdin: (options: { stdin: () => string | number | null }) => void;
+};
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = null;
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL).then(
 (module) => {
 const { loadPyodide } = module as PyodideModule;
 return loadPyodide({ indexURL: PYODIDE_INDEX_URL });
 }
);
 }
+}
```



```

+ await runtime.runPythonAsync(code);
+
+ const text = [...stdout, ...stderr].join("");
+ setOutput(text || "Готово: программа ничего не вывела");
+ } catch (error) {
+ const captured = [...stdout, ...stderr].join("");
+ setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
+ setRuntimeMessage("Не удалось выполнить код – произошла ошибка: " + error);
+ } finally {
+ setRunning(false);
+ }
+ }, [code, stdin, running, runtimeReady]));
+
+ const reset = () => {
+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть результат");
+ };
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:items-center">
+ <div>
+ <div className="inline-flex items-center gap-2">
+ <Terminal className="w-4 h-4" /> Боковая панель
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-extrabold">
+ <p className="text-slate-400 mt-2 max-w-2xl">
+ Пишите небольшой код и запускайте его прямо в браузере,
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.5"

```

```

+ onKeyDown={(event) => {
+ if ((event.ctrlKey || event.metaKey) &
+ event.preventDefault());
+ void runCode();
+ }
+ }}
+ spellCheck={false}
+ aria-label="Код Python"
+ className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+ placeholder="Напишите Python-код..."
+ />
+
+ <div className="border-t border-slate-800
+ <label className="block px-4 pt-3 text-{
+ Ввод для input() • по одной строке на
+ </label>
+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.ta
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-tr
ceholder:text-slate-700"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border borde
+ <div className="flex items-center justify-f
+ <div className="flex items-center gap-2
+ <Terminal className="w-4 h-4 text-emera
+ </div>
+ <span className={`inline-flex items-centr
+ {runtimeReady && <CheckCircle2 classNa
+ {runtimeReady ? "готов" : "не загружен

```

Универсальное пособие • полный исходный код

```
.....
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локально в /
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});

```

### Patch относительно `main`

```
```diff
diff --git a/vite.config.ts b/vite.config.ts
```

Универсальное пособие • полный исходный код

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2

Gemini App Canvas Instructions

Ты – AI-ассистент, работающий в Gemini App Canvas (или а
При запросе пользователя создать или сгенерировать новую
строго придерживаться готовых шаблонов разметки,

Правила для малых страниц (Билетов / Глав)

1. ****Единый стиль:**** Всегда используй Tailwind CSS и HTML
2. ****Структура JSON/TS:**** Каждая новая малая страница д
 {``, `content: ...`) для вставки в `src/data/content.
 `content: ...`)
3. ****Строгая структура контента:****
 - Заголовок с цветным бейджем (например, `bg-indigo-100`)
 - Определение/правило в центре (`border-blue-500` или `border-blue-500`)
 - Обязательная сетка аналогий (2 колонки: неформальный и формальный)
 - "Душные" детали (код, формулы, сложные доказательства)
4. ****Не ломай верстку:**** Не придумывай свои CSS-классы в файлах `src/data/content.ts`. Не используй чистый markdown

ФАЙЛ 4/99: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 6

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg" />
    <title> Универсальное пособие по алгоритмам – билет №1
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen">
    <div id="root"></div>
```


Универсальное пособие • полный исходный код

.....
* `EXPORT\_GRAYSCALE=1` — оттенки серого вместо 1-битного

Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается **на ка
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док
и коммитит обновлённые `public/export/full_code_all_page
(с меткой `[skip ci]`, чтобы не зациклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full_co

Аудит контента

```bash

node scripts/audit-coverage.mjs # какие темы без

node scripts/audit-coverage.mjs --md # markdown-табли

```

Актуальный отчёт: [`TOPICS\_AUDIT.md`](./TOPICS_AUDIT.md)

Как устроен интерфейс

* **Темы 1–24** — канонический список билетов; одна стр
(`src/data/content.ts`, таблица `CANON` задаёт точные

* **Python-компилятор** — боковая панель (кнопка «Python
отладчик. Список «переменных темы» для каждой страниц
`src/data/vizSync.ts`; значения из выполняемого Python
объекты демонстрации (матрицы, графы, деревья) напрям
эталонный код, инициализация подставляет заранее зада

* **Содержание** — на десктопе боковая панель с поиском
на телефоне — липкая строка «Содержание · N из M» и в

* **Переходы между темами** — кнопки «Назад / Вперёд» в
«Предыдущая / Следующая тема» под текстом; на клавиатуре

* **Всплывающие подсказки по терминам** — текст главы а
известные термины (BFS, куча, бор, суффиксная ссылка,
подчёркиваются, а по наведению/тапу показывается карт
мини-анимацией и кнопкой «Показать симулятор» (симуля
Словарь — `src/data/glossary.ts`, мини-анимации — `src

Универсальное пособие • полный исходный код

Картинки и визуализации (Схемы)

Вместо обычных `` в приложении используются векторы

* Они помещены внутрь оберток с классом `

* Все отрисовки находятся внутри векторного тега ``

* Подпись к картинке, если она есть, идет сразу после SVG

Списки

* Стандартные списки оформлены через `

Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `

` и `` заменяются на `##` и `###`

2. Элементы ``.analogy-badge` получают выделение маркера`

3. Теги `` заменяются строкой-заглушкой `{ Иллюстрация)`.

4. Элементы списка `- ` получают маркер `.`.

5. Результат извлекается в виде текста и чистится от лишнего

ФАЙЛ 8/99: TOPICS_AUDIT.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 106 | РАЗМЕР: 1.5 КБ

Аудит покрытия тем: где нет «объяснения на пальцах» и

> Сгенерировано скриптом `node scripts/audit-coverage.m`

> Критерии:

> * **Аналогия** — в HTML главы есть блок «Аналогия ...»

> * **2D** — к главе привязан интерактивный визуализатор

>

> Всего глав в пособии: **25**.

...

Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава | id | Что делать |

Универсальное пособие • полный исходный код

```
.....
| Эйлер: Путь ≠ Цикл | `euler-path-vs-cycle` | Ключевая
```

```
...
```

```
## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации (
```

```
> Замкнуто: «5. Двумерная матрица префиксных сумм» полн
```

```
| Глава | id | Чего не хватает |
```

```
| --- | --- | --- |
```

```
| 3. Декартово дерево (Trear) | `trear` | Нет ни визуализации
```

```
|
```

```
| 8. Графы. Компоненты связности | `graph-components` |
```

```
| 24. Классы сложности, сведение задач | `complexity-class` |
```

```
...
```

```
## Сводная таблица по всем главам
```

```
| # | Глава | Аналогий | 2D-виз. | SVG | Картинка |
```

```
| --- | --- | --- | --- | --- | --- |
```

```
| 2 | 2. Двумерная разреженная матрица (Sparse Table) |
```

```
| 3 | 3. Декартово дерево (Trear) | 2 | - | - | - |
```

```
| 4 | 4. Splay-дерево | 2 | | - | - |
```

```
| 5 | 5. Двумерная матрица префиксных сумм | 1 | | | - |
```

```
| - | Планарность: K5, K3,3 и формула Эйлера | - | | | - |
```

```
| 6 | 6. Графы. Представление, DFS, BFS | 4 | | - | - |
```

```
| 8 | 8. Графы. Компоненты связности | 1 | - | - | - |
```

```
| 9 | 9. Графы. Топологическая сортировка | 1 | | - | - |
```

```
| 10 | 10. Графы. Компоненты сильной связности (SCC) |
```

```
| 12 | 12. Графы. Точки сочленения | 1 | | - | - |
```

```
| 14 | 14. Графы. Кратчайший путь. Дейкстра | 3 | | - | - |
```

```
| 15 | 15. Графы. Кратчайший путь. Форд-Беллман | 1 |
```

```
| 16 | 16. Графы. Кратчайший путь. Флойд | - | | - | - |
```

```
| 17 | 17. Графы. Алгоритм Джонсона | - | | - | - |
```

```
| 18 | 18. Графы. Остовное дерево. Краскал | 1 | | - | - |
```

```
| 19 | 19. Графы. Остовное дерево. Прима | 1 | | - | - |
```

```
| 20 | 20. Графы. Остовное дерево. Борувка | 1 | | - | - |
```



```
.....  
    "resolveJsonModule": true,  
    "isolatedModules": true,  
    "noEmit": true,  
    "jsx": "react-jsx",  
  
    /* Path mapping */  
    "baseUrl": ".",  
    "paths": {  
      "@/*": ["src/*"]  
    },  
  
    /* Linting */  
    "strict": true,  
    "noUnusedLocals": true,  
    "noUnusedParameters": true,  
    "noFallthroughCasesInSwitch": true  
  },  
  "include": ["src", "vite.config.ts"]  
}
```

.....
ФАЙЛ 10/99: vite.config.ts

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 36 | РАЗМЕР: 1

```
.....  
  
import path from "path";  
import { fileURLToPath } from "url";  
import tailwindcss from "@tailwindcss/vite";  
import react from "@vitejs/plugin-react";  
import { defineConfig } from "vite";  
import { viteSingleFile } from "vite-plugin-singlefile"  
  
const __filename = fileURLToPath(import.meta.url);  
const __dirname = path.dirname(__filename);  
  
// https://vite.dev/config/  
export default defineConfig({
```

```

.....
import { Sidebar } from "../components/Sidebar";
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/Simu
import { PythonCompiler } from "../components/PythonComp

export default function App() {
  const [activeChapterId, setActiveChapterId] = useState<string>()
  const [modalVizId, setModalVizId] = useState<string>()
  /** Боковая (на мобильном – нижняя) панель компилятора */
  const [compilerOpen, setCompilerOpen] = useState(false)
  /** Ширина панели компилятора (desktop), растягивается */
  const [compilerWidth, setCompilerWidth] = useState<number>()
  const cached = typeof window !== "undefined" ? Number(window.localStorage.getItem("compilerWidth")) : null
  return Number.isFinite(cached) && cached >= 380 ? cached : null
});
useEffect(() => {
  window.localStorage.setItem("compilerWidth", String(compilerWidth));
}, [compilerWidth]);
/** Высота терминала: null оставляет адаптивную высоту */
const [compilerHeight, setCompilerHeight] = useState<number>()
const cached = typeof window !== "undefined" ? Number(window.localStorage.getItem("compilerHeight")) : null
return Number.isFinite(cached) && cached >= 260 ? cached : null
});
useEffect(() => {
  if (compilerHeight !== null) window.localStorage.setItem("compilerHeight", String(compilerHeight));
}, [compilerHeight]);
/** Содержание свернуто (по умолчанию сворачиваем сам компилятор) */
const [sidebarCollapsed, setSidebarCollapsed] = useState<boolean>()
const cached = typeof window !== "undefined" ? window.localStorage.getItem("sidebarCollapsed") : null
if (cached !== null) return cached === "1";
return typeof window !== "undefined" && window.innerWidth < 380
});
const toggleSidebar = useCallback(() => {
  setSidebarCollapsed((c) => {
    window.localStorage.setItem("sidebarCollapsed", c ? "1" : null);
    return !c;
  });
}, []);

```



```

.....
        <ChapterView
            key={activeChapter.id}
            chapter={activeChapter}
            prev={prev}
            next={next}
            index={index}
            total={chapters.length}
            onGo={goTo}
            onOpenSimulator={setModalVizId}
            onOpenCompiler={openCompilerWithViz}
        />
    </main>
</div>
</>

<footer className="p-8 text-center text-slate-600">
    © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() =>
    {
        compilerOpen && (
            <PythonCompiler
                chapterId={activeChapter.id}
                chapterTitle={activeChapter.title}
                width={compilerWidth}
                height={compilerHeight}
                onWidthChange={setCompilerWidth}
                onHeightChange={setCompilerHeight}
                onOpenGuide={() =>
                    document.getElementById("chapter-viz")?.scrollTo(0,0)
                }
                onClose={() => setCompilerOpen(false)}
            />
        )
    }
/>
</div>
);

```

```
.....
* повесить всплывающую карточку (как превью ссылок в ВБ)
*/
export function annotateTerms(root: HTMLElement): void {
  if (!cachedRegex) cachedRegex = buildRegex();
  const re = cachedRegex;

  const perTerm = new Map<string, number>();
  const perSurface = new Map<string, number>();
  const walker = document.createTreeWalker(root, NodeFilter.
    acceptNode(node) {
      const parent = (node as Text).parentElement;
      if (!parent) return NodeFilter.FILTER_REJECT;
      if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
      if (!node.nodeValue || node.nodeValue.trim().length === 0)
        return NodeFilter.FILTER_REJECT;
      return NodeFilter.FILTER_ACCEPT;
    },
  ));

  const targets: Text[] = [];
  let current = walker.nextNode();
  while (current) {
    targets.push(current as Text);
    current = walker.nextNode();
  }

  for (const textNode of targets) {
    const text = textNode.nodeValue ?? "";
    re.lastIndex = 0;
    if (!re.test(text)) continue;
    re.lastIndex = 0;

    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;

    while ((m = re.exec(text)) !== null) {
      const surface = m[1].toLowerCase();
      const id = labelToId.get(surface);
    }
  }
}
```

```
.....
    annotateTerms(e1);
  } catch {
    /* если браузер не умеет lookbehind – просто оста
  }
};

// при смене главы
// eslint-disable-next-line react-hooks/exhaustive-dep
useEffect(run, deps);

// страховка: контент перерисовали, а подсказок в нём
useEffect(() => {
  const e1 = ref.current;
  if (!e1) return;
  if (!e1.querySelector(".term-hint")) run();
});
}
```

.....
ФАЙЛ 13/99: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 K

```
.....
@import "tailwindcss";

@layer utilities {
  .neon-glow-blue {
    box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
  }
  .neon-glow-emerald {
    box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
  }
  .neon-glow-rose {
    box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
  }
  .neon-glow-yellow {
    box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
  }
}
```

```
.....
    opacity: 0;
    transform: translateY(4px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.term-popover {
  animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в В
.term-hint {
  cursor: help;
  border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
  color: #c7d2fe;
  border-radius: 3px;
  padding: 0 1px;
  transition:
    background-color 0.15s ease,
    color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
  background: rgba(99, 102, 241, 0.22);
  color: #fff;
  outline: none;
}
@media (hover: none) {
  .term-hint {
    border-bottom-style: solid;
    background: rgba(99, 102, 241, 0.12);
  }
}

/* Контент главы приходит готовым HTML — чиним его под
```

```
.....
    font-size: 1.25rem !important;
    line-height: 1.3 !important;
}
.chapter-body :where(h3) {
    font-size: 1.05rem !important;
    line-height: 1.35 !important;
}
.chapter-body :where(.p-6, .p-8) {
    padding: 0.9rem !important;
}
.chapter-body :where(.text-3xl) {
    font-size: 1.4rem !important;
}
.chapter-body :where(.text-2xl) {
    font-size: 1.2rem !important;
}
.chapter-body :where(.text-xl) {
    font-size: 1.05rem !important;
}
}
```

```
/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
    width: 8px;
    height: 8px;
}
::-webkit-scrollbar-track {
    background: #0f172a;
}
::-webkit-scrollbar-thumb {
    background: #334155;
    border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
    background: #475569;
}
```

```
/* Индикатор длительности кадра в пошаговых визуализаци
```



```
.....  
  category?: string;  
}
```

```
.....  
ФАЙЛ 16/99: src/vite-env.d.ts
```

```
КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В  
.....
```

```
/// <reference types="vite/client" />
```

```
declare module '*.jpg' {  
  const src: string;  
  export default src;  
}  
declare module '*.jpg?url' {  
  const src: string;  
  export default src;  
}  
declare module '*.jpeg' {  
  const src: string;  
  export default src;  
}  
declare module '*.png' {  
  const src: string;  
  export default src;  
}
```

```
.....  
РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ  
.....
```

```
.....  
ФАЙЛ 17/99: src/data/additionalTickets.ts
```

```
КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 866 | РАЗМ  
.....
```



```

.....
    <p class="text-slate-400 text-sm mb-1">
летки дважды. Сумма ≠ идемпотентна.</p>
    <p class="text-slate-400 text-sm mb-1">
ации.</p>
    <p class="text-slate-400 text-sm">• Они
го минимума. Выбирай не «какая круче», а «к
</div>
</div>
</div>
</section>`,
}, {
  id: "treap",
  title: "3. Декартово дерево (Treap)",
  type: "html",
  description: "Дерево поиска по ключу и Куча по приор
category: "Продавленные структуры",
  content: `
<section id="treap" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-1">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
          двоичная куча. Пара – точка на декартовой
      </div>

    <p class="text-slate-300 text-sm mb-4">Постр
      платит  $\approx n \cdot \log n$  сравнений, counting по ма
      точка вставляется спуском от корня:  $x \leq \text{уз}$ 
      единственно. Весь процесс – в симуляторе под
      «соединить сам»</b> – кликаешь две точки,
      ь родитель», «получился бы цикл», «равные »,
      «Миф  $2k/2k+1$ »</b> и <b>«Поиск ≠ сортировка»

```

```
.....
    <p class="text-slate-400 text-sm">• «Уб
аются merge за  $O(h)$ .</p>
</div>

<p class="text-slate-300 text-sm mt-4"><b c
"bg-slate-950 px-2 py-0.5 rounded text-xs f
оение – <code class="bg-slate-950 px-2 py-0
code> (сортировка + n спусков). Худший случа
text-xs font-mono text-rose-300 border bord
ed text-xs font-mono text-rose-300 border b

<p class="text-slate-500 text-xs mt-3 leadi
и останется набор букв, как treap без y оста
но кричат по «!»: корень самый громкий, род
; равные x – идут влево по уставу.</p>

<details class="bg-slate-900/40 rounded-lg
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код: split (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounded
pair<Node*, Node*> split(Node* t, int x) {
    if (!t) return {nullptr, nullptr};
    if (t->val <= x) {
        auto [L, R] = split(t->right, x);
        t->right = L;
        return {t, R};
    } else {
        auto [L, R] = split(t->left, x);
        t->left = R;
        return {L, t};
    }
}</pre>
    </div>
</details>
```

```

.....
  title: "4. Splay-дерево",
  type: "html",
  description: "Самонастраивающееся дерево: поднимаем
  category: "Продвинутые структуры",
  content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        ротов поднимает x в корень. Единственный ин
      </div>
    </div>

    <div class="grid grid-cols-1 md:grid-cols-3">
      <div class="bg-slate-900 rounded-lg p-4">
        <p class="text-xs uppercase tracking-wider">
        <p class="text-white font-bold">мож
        <p class="text-slate-400 text-xs mt-2">
      </div>
      <div class="bg-slate-900 rounded-lg p-4">
        <p class="text-xs uppercase tracking-wider">
        <p class="text-emerald-300 font-bold">
        <p class="text-slate-400 text-xs mt-2">
      </div>
      <div class="bg-slate-900 rounded-lg p-4">
        <p class="text-xs uppercase tracking-wider">
        <p class="text-white font-bold">0(n
        <p class="text-slate-400 text-xs mt-2">
      </div>
    </div>

    <p class="text-slate-300 text-sm mb-4">От к

```

```

.....
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Смотрим
<div class="overflow-x-auto -mx-2 px-2">
  <table class="w-full text-xs sm:text-sm">
    <thead>
      <tr class="text-left text-slate-300">
        <th class="py-2 pr-3 font-bold">Имя
        <th class="py-2 pr-3 font-bold">Возраст
        <th class="py-2 pr-3 font-bold">Пол
        <th class="py-2 font-bold">Детали
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-slate-500">
        <td class="py-3 pr-3 font-bold">Дед
        <td class="py-3 pr-3">деда
        <td class="py-3 pr-3">сам
        <td class="py-3">х – корень
      </tr>
      <tr class="border-b border-slate-500">
        <td class="py-3 pr-3 font-bold">Бабушка
        <td class="py-3 pr-3">х и п
        <td class="py-3 pr-3"><span class="font-italic">
        <td class="py-3">ветка укоренения
      </tr>
      <tr>
        <td class="py-3 pr-3 font-bold">Потомки
        <td class="py-3 pr-3">х и п
        <td class="py-3 pr-3"><span class="font-italic">
        <td class="py-3">п и г стали
      </tr>
    </tbody>
  </table>
</div>
<p class="text-slate-300 text-sm mt-3">И по
у. Zig случается только последним.</p>

<div class="bg-rose-950/30 rounded-xl borde

```

Универсальное пособие • полный исходный код

```
.....
    во меняется даже при чтении (плохо для многих)

    <details class="bg-slate-900/40 rounded-lg"
      <summary class="p-4 font-bold text-slate-300"
        -b border-slate-700/50">
        Код: rotate + splay + find (Скрыто)
      </summary>
      <div class="p-5 text-sm text-slate-300"
        <pre class="bg-slate-950 p-4 rounded"
# узел: { key, left, right, parent }

def rotate_up(x):                                # x встаёт на место
    p = x.parent
    g = p.parent
    if p.left is x:                               # правый поворот
        p.left = x.right
        if x.right: x.right.parent = p
        x.right = p
    else:                                          # левый (зеркально)
        p.right = x.left
        if x.left: x.left.parent = p
        x.left = p
    p.parent = x
    x.parent = g
    if g:
        if g.left is p: g.left = x
        else: g.right = x

def splay(x):                                    # поднять x в корень
    while x.parent:
        p, g = x.parent, x.parent.parent
        if g is None:                            # ZIG
            rotate_up(x)
        elif (g.left is p) == (p.left is x):    # ZIG-ZIG
            rotate_up(p)
            rotate_up(x)
        else:                                     # ZIG-ZAG: НИЖНЯЯ
            rotate_up(x)
```

```

.....
    <div class="grid grid-cols-1 xl:grid-cols-2"
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4
rder border-emerald-500 shadow-md">
          ~ Аналогия 1: Вырезание прямоу
        </div>
        <p class="text-slate-300 text-sm mb
з него маленький целевой прямоугольник в цел
ый верхний угол отрезается дважды! Поэтому
        </div>
      </div>
    </div>
  </div>
</section>',
},
{
  id: "graph-planar-colors",
  title: "7. Графы. Планарные. Покраска",
  type: "html",
  description: "Формула Эйлера и теорема о 4 красках.",
  category: "Графы. Теория",
  content: `
<section id="graph-planar-colors" class="mb-12 scroll-m
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-blue-400
      <div class="bg-slate-900 p-4 rounded-lg mb-
        <p class="text-lg text-blue-300 italic
        <p class="text-xl font-mono text-white">
ый граф можно раскрасить 4 цветами.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg
          <div class="absolute -top-3 left-4

```



```
.....
        <p class="text-slate-300 text-sm mb-4">
            еская сортировка – это расписание, которое
        </p>
    </div>
    <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 right-4 bottom-4
            border-rose-500 shadow-md">
            Ограничения: Циклы
        </div>
        <p class="text-slate-300 text-sm mb-4">
            получить опыт, нужна работа". Алгоритм выявляет
        </p>
    </div>
</div>
</section>`,
},
{
    id: "scc-kosaraju",
    title: "10. Графы. Компоненты сильной связности (SCC)",
    type: "html",
    description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
    category: "Графы. Продвинутое",
    content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            Теория
        </span>
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
            10. Графы. Компоненты сильной связности (SCC)
        </h2>
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
            <h3 class="text-xl font-bold text-emerald-400">
                Введение
            </h3>
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-emerald-300 italic">
                    В графе с сильной связностью (SCC) вершины можно
                </p>
                <p class="text-xl font-mono text-white">
                    пронумеровать так, чтобы каждая вершина имела номер
                    непертированному в порядке top-sort.</p>
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-4 rounded-lg">
                    <p class="text-sm text-slate-300">
                        Если вершина  $v$  является источником в графе,
                    </p>
                </div>
                <div class="bg-slate-800 p-4 rounded-lg">
                    <p class="text-sm text-slate-300">
                        Если вершина  $v$  является стоком в графе,
                    </p>
                </div>
            </div>
        </div>
    </div>
</section>`
}
```

```
.....
        Аналогия 1: Единственный мост
    </div>
    <p class="text-slate-300 text-sm mb-6">
        этот мост, экономикой обоих кусков придет
    </div>
</div>
</div>
</div>
</section>',
},
{
    id: "graph-articulation",
    title: "12. Графы. Точки сочленения",
    type: "html",
    description: "Вершины, удаление которых убивает связность",
    category: "Графы. Продвинутые",
    content: `
<section id="graph-articulation" class="mb-12 scroll-mt-12">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
                Точки сочленения
            </h2>
        </span>
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
            <h3 class="text-xl font-bold text-rose-400">
                Аналогия 1: Единственный мост
            </h3>
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-rose-300 italic">
                    Рассмотрим граф, состоящий из двух
                    <p class="text-xl font-mono text-white">
                        вершин и одного ребра.
                    </p>
                </p>
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                        Аналогия 1: Центральный роутер
                    </div>
                    <p class="text-slate-300 text-sm mb-6">
                        правильный роутер - и два сегмента офиса б
```

```

</section>',
  },
  {
    id: "pathfinding-accel",
    title: "17. Графы. Ускорения поиска",
    type: "html",
    description: "",
    category: "Графы. Пути",
    content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">17. Графы. Ускорения поиска
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">Ускорения поиска
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Представьте себе, что вы копаете туннель под
        <p class="text-xl font-mono text-white">длина.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия: Копаем туннель под
          </div>
          <p class="text-slate-300 text-sm mb-4">
            ней земли. Но если начать копать одновременно
            в 2 РАЗА меньше работы (геометрически: площадь
          </div>
        </div>
      </div>
    </div>
  </div>
</section>',
  },
  {

```

```

description: "",
category: "Графы. Остовные",
content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          которое торчит из посещенных в непосещенных
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            * Аналогия: Заражение вирусом
          </div>
          <p class="text-slate-300 text-sm mb-4">
            ли вирус). Мы стартуем из одного города, и
            Сеть растет только из одного связного куска
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
},
{
  id: "mst-boruvka",
  title: "20. Графы. Остовное дерево. Борувка",
  type: "html",
  description: "",
  category: "Графы. Остовные",
  content: `

```

```

.....
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
</div>
<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
        <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-blue-300 italic">
                    <p class="text-xl font-mono text-white">
                        i}, который одновременно является её суффиксом.
                </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">
                <!-- Аналогия 1 -->
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                        Аналогия 1: Бумеранг (Префикс)
                    </div>
                    <p class="text-slate-300 text-sm mb-4">
                        идишь "АБРА". Если ты ошибешься при поиске
                        концовка "АБРА" совпадает с началом "АБРА"
                        моя перепроверок!</p>
                    </div>
                </div>
            <details class="bg-slate-900/40 rounded-lg">
                <summary class="p-4 font-bold text-slate-300">
                    -b border-slate-700/50">
                        Код (Скрыто)
                    </summary>
                    <div class="p-5 text-sm text-slate-300">
                        <pre class="bg-slate-950 p-4 rounded">
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
                    </div>
                </div>
            </div>
        </div>
    </div>

```

```

        -b border-slate-700/50">
            Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
            <pre class="bg-slate-950 p-4 rounded">
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "complexity-classes",
        title: "24. Классы сложности, сведение задач",
        type: "html",
        description: "",
        category: "Теория сложности",
        content: `
<section id="complexity-classes" class="mb-12 scroll-mt">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-purple-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-purple-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-purple-300 italic">
                <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">

```



```
.....
    <p class="text-blue-400">Теорема Эйлера
    <p>
        Связный граф содержит эйлеров цикл
        <br>Эйлеров путь (не цикл)  $\Leftrightarrow$  ровно
    </p>
    <p class="text-rose-400">Почему Гамильтон
    <p>
        Задача коммивояжёра (TSP) – это взв
        TSP – NP-трудная: полиномиальный ал
        В общем, не парься. Просто запомни
    </p>
    </div>
    </details>
</div>
</section>',
},
{
    id: "bridges-code",
    title: "Мосты в графах: код + визуализация",
    type: "html",
    content: `<section id="bridges-code" class="mb-20 s
    <div class="flex items-center mb-6">
        <span class="bg-emerald-600 text-white px-4 py-
        <h2 class="text-3xl font-bold text-white">Мосты
    </div>

    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl bord
            <h3 class="text-xl font-bold text-emerald-4

        <div class="bg-slate-900 p-4 rounded-lg mb-1
            <p class="text-lg text-emerald-300 italic
            <div class="text-left font-mono text-sm
                <p><strong class="text-white">tin{v
                <p><strong class="text-white">low{v
            <div class="p-3 bg-slate-950 rounde
                <span class="text-rose-400 font
                <p class="text-xl font-bold tex
```



```
.....

<details class="bg-slate-900/40 rounded-lg border
  <summary class="p-4 font-bold text-slate-400
    order-slate-700/50">
    Полный код на C++ (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300 space
    <p>Классическая реализация – <code class="
    оходов.</p>

    <div class="bg-slate-950 p-4 rounded-lg
      <pre class="text-xs font-mono text-

#include <vector>;
#include <algorithm>;

using namespace std;

int n;                // число вершин
vector<vector<int>>> adj; // список смежност
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;           // устанавливаем

    for (int to : adj[v]) {
        if (to == p) continue;          // не идём назад

        if (visited[to]) {
            // обратное ребро: обновляем low
            low[v] = min(low[v], tin[to]);
        } else {
            // ребро дерева DFS
            dfs(to, v);                   // рекурсивный вызов
            low[v] = min(low[v], low[to]);
        }
    }
}
```

.....

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-blue-300 italic">
  <p class="text-3xl font-mono text-white">
  <div class="text-sm text-slate-400 space-x-2">
    <p><span class="text-blue-400 font-italic">
    <p><span class="text-emerald-400 font-italic">
    <p><span class="text-rose-400 font-italic">
  </div>
</div>
```

```
</div>
```

```
<p class="text-slate-300 text-sm">
  <strong>Следствие:</strong> Для планарного графа:
  <code>ode>.
  Если ребер больше  $n-1$  – граф гарантированно не планарен.
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
    <div class="absolute -top-3 left-4 bg-slate-800 border border-slate-700
order-green-500 shadow-md">
```

K5: Полный граф на 5 вершинах

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

```
  У K5: <code class="text-white">V=5, E=10 &gt; 9</code> – БАХ, непланарен!
  Все 5 вершин соединены со всеми. На K5 не применим критерий Эйлера.
  Теорема Куратовского: K5 – эталон непланарности.
  Если внутри графа спрятан K5 – забудь про планарность!
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
  <div class="absolute -top-3 left-4 bg-rose-500 border border-rose-400
der-rose-500 shadow-md">
```

K3,3: 3 дома и 3 колодца

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4
rder border-fuchsia-500 shadow-md">
```

Аналогия: карта на стене

```
</div>
```

```
<p class="text-slate-300 text-sm mb"
  Красим карту регионов: соседние
  Регион = вершина, общая граница
  Для плоской карты (планарный граф)
  теорема о четырёх красках (Аппеля)
```

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4
er border-amber-500 shadow-md">
```

Что нужно знать на экзамен

```
</div>
```

```
<ul class="text-slate-300 text-sm sp"
  <li><strong class="text-white">
вободный у соседей. Дал  $\leq \Delta + 1$  цветов ( $\Delta$  — макс.
  <li><strong class="text-white">
тит 2 цветов, хоть он и непланарен.</li>
  <li><strong class="text-white">
 $= n$ ,  $\chi(\text{цикла чётной длины}) = 2$ ,  $\chi(\text{цикла нечётной длины}) = 3$ ,
  <li><strong class="text-white">
й и не нечётный цикл,  $\chi(G) \leq \Delta$ .</li>
```

```
</ul>
```

```
</div>
```

```
</div>
```

```
<p class="text-slate-400 text-xs mt-4">
```

Почему раскраска «сложная»: проверить раскраску — NP-полная задача.

Даже «раскрасима ли карта в 3 цвета?» — NP-полная задача.

```
</p>
```

```
.....
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border-rose-500 shadow-md">
    Аналогия: Главный перекресток
  </div>
  <p class="text-slate-300 text-sm mb-4">
    Представь город, где два района соединены
    её перекроют (удалят вершину) – районы будут
    g>хрупкость</strong> системы.
  </p>
</div>

<div class="bg-slate-900 p-6 rounded-lg border-emerald-500 shadow-md">
  Телеком: Центральный свитч
</div>
<p class="text-slate-300 text-sm mb-4">
  У тебя несколько компьютеров в одной
  абелем (мостом). Сами свитчи – это точки со
</p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border-slate-700/50">
  <summary class="p-4 font-bold text-slate-400">
    Душный мод: Код и корень DFS (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-2">
    <p class="text-blue-400 font-bold">Особ
    <p>
      Для стартовой вершины обхода правил
      то выше неё прыгать некуда). Поэтому для не
      у него <strong>больше 1 независимого ребенка
    </p>
    <div class="bg-slate-950 p-4 rounded-lg
```

```
.....
        о и есть критическая SCC! Если удалить эту
        . Одно слабое звено изолирует целые касты S
    </ul>
</div>
</div>
</section>`,
    },
};
```

.....
ФАЙЛ 19/99: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 129 | РАЗМ

```
.....
import { Chapter } from "../types";
import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets1to5 } from "./tickets/ticket1_5";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { chapters as extraChapters } from "./content_te
```

```
/**
```

```
 * Канонический порядок тем (билеты 1–24) и «правильные
 * одна страница может раскрывать одну тему или несколь
 * поэтому названия выверяются по единому списку ниже.
 */
```

```
const CANON: Record<string, { num: number; title: string }> = {
    "segment-trees": { num: 1, title: "1. Дерево отрезков" },
    "sparse-table": { num: 2, title: "2. Двумерная разреж" },
    "treap": { num: 3, title: "3. Декартово дерево (Treap)" },
    "splay-tree": { num: 4, title: "4. Splay-дерево" },
    "prefix-sums-2d": { num: 5, title: "5. Двумерная матри" },
    "graph-dfs-bfs": { num: 6, title: "6. Графы. Представ" },
    "planarity-euler-formula": { num: 7, title: "7. Графы" },
    "graph-components": { num: 8, title: "8. Графы. Компон" },
};
```

```

// Дубликаты: берём последнюю (самую свежую) версию строки
const dedup = new Map<string, Chapter>();
merged.forEach((c) => {
  if (c.id === "alg-map") return; // страница-заглушка
  if (stale.has(c.id)) return;
  dedup.set(c.id, c);
});

// Нормализуем названия по каноническому списку тем, а не по
// по учебным блокам: блоки идут строго в порядке чтения
// номерам билетов. Близкие темы (1–5, 14–17, 18–20, 21–24)
const BLOCKS: [number, number, string][] = [
  [1, 5, "Билеты 1–5 · Структуры и матрицы"],
  [6, 13, "Билеты 6–13 · Графы: обходы и связность"],
  [14, 17, "Билеты 14–17 · Кратчайшие пути"],
  [18, 20, "Билеты 18–20 · Остовные деревья"],
  [21, 23, "Билеты 21–23 · Строки"],
  [24, 24, "Билет 24 · Теория сложности"],
];
const blockOf = (num: number): string => BLOCKS.find(([, end]) => num <= end);

const withCanon = Array.from(dedup.values()).map((c) => {
  const canon = CANON[c.id];
  if (canon) return { ...c, title: canon.title, category: canon.category };
  return { ...c, category: "Введение" }; // страницы вне канона
});

// Сначала вводная страница, затем билеты 1–24 по порядку
const numbered = withCanon.filter((c) => CANON[c.id]);
numbered.sort((a, b) => CANON[a.id].num - CANON[b.id].num);
const extras = withCanon
  .filter((c) => !CANON[c.id])
  .sort((a, b) => a.title.localeCompare(b.title, "ru"));

export const chapters: Chapter[] = [...extras, ...numbered];

/**

```

Универсальное пособие • полный исходный код

```
.....  
* под sys.settrace и на каждой строке записывает фрейм  
* как панель Variables в обычном отладчике.  
*/
```

```
export type DebugValue = null | boolean | number | string
```

```
export interface DebugStep {  
  /** Номер строки в коде пользователя (1-based). */  
  line: number;  
  /** Имя функции-фрейма {"<module>" – верхний уровень) */  
  func: string;  
  /** Локальные переменные: имя → герг (усечённый), для  
  locals: Record<string, string>;  
  /**  
  * JSON-снимок доступных имён (globals + locals). В о  
  * можно напрямую отдать визуализации: i/] остаются ч  
  */  
  values?: Record<string, DebugValue>;  
}
```

```
export interface DebugResult {  
  steps: DebugStep[];  
  /** Локали в момент возврата из модуля – «результат» */  
  finalLocals?: Record<string, string>;  
  /** Финальный машинно-читаемый снимок для визуализации  
  finalValues?: Record<string, DebugValue>;  
  /** Текст последней необработанной ошибки (или пустая  
  error: string;  
  /** true, если уперлись в лимит шагов. */  
  truncated: boolean;  
}
```

```
/** Лимит шагов трассы – защита от бесконечных циклов и  
export const DEBUG_STEP_CAP = 1200;
```

```
/**  
* Строит самодостаточный python-скрипт: выполняет user  
* и возвращает JSON с шагами (строка + func + locals)
```

```

.....
    # предел 80 незаметно обрезал уровни 4x4 и 8x8
    dict_limit = 192 if depth == 0 else limit
    return {str(k): __snapshot(x, depth + 1) for k,
# deque, heap-подобные и другие итерируемые учебные
if type(v).__name__ == "deque":
    return {__snapshot(x, depth + 1) for x in list(v)}
return __safe(v)

def __visible(items):
    return {
        k: v for k, v in items
        if not k.startswith("__") and type(v).__name__
    }

def __capture(frame):
    local_values = __visible(frame.f_locals.items())
    # Внутри функции алгоритма важные структуры (memo,
    # глобальные. Локальные значения имеют приоритет на
    scope_values = __visible(frame.f_globals.items())
    scope_values.update(local_values)
    loc = {k: __safe(v) for k, v in local_values.items()}
    values = {k: __snapshot(v) for k, v in scope_values.items()}
    return loc, values

def __tracer(frame, event, arg):
    global __was_truncated
    if event == "line" and frame.f_code.co_filename ==
        and not (frame.f_code.co_name.startswith("<
    loc, values = __capture(frame)
    # PEP 709: с 3.12 компрехеншны встроены во фрейм
    # на своей строке каждую итерацию. Склеиваем по
    if __steps and __steps[-1]["line"] == frame.f_lineno:
        __steps[-1]["locals"] = loc
        __steps[-1]["values"] = values
    else:
        __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
    if len(__steps) >= __CAP:
        # Нельзя просто отключить trace: бесконечный

```



```

.....
export function changedVars(
  prev: Record<string, string> | null | undefined,
  cur: Record<string, string>
): Record<string, boolean> {
  const out: Record<string, boolean> = {};
  for (const k of Object.keys(cur)) {
    out[k] = !prev || !(k in prev) || prev[k] !== cur[k];
  }
  return out;
}

/** Базовое имя из карточки переменной: "tin[v]" → "tin"
export const baseVarName = (name: string): string => name.slice(0, -1);

/**
 * Порядок показа в панели переменных: сначала переменные
 * страницы (те, что подсвечивает визуализация), остальные
 */
export function orderVars(
  cur: Record<string, string>,
  syncNames: string[]
): [string, string][][] {
  const inSync = new Set(syncNames);
  const syncEntries = Object.entries(cur).filter(([k]) => inSync.has(k));
  const rest = Object.entries(cur)
    .filter(([k]) => !inSync.has(k))
    .sort(([a], [b]) => a.localeCompare(b));
  return [...syncEntries, ...rest];
}

```

.....
ФАЙЛ 21/99: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 636 | РАЗМЕР: 1.5 КБ

.....
import type { DemoKind } from "../components/hints/MiniDemo";

```

.....
    id: "dfs",
    labels: ["DFS", "обход в глубину", "поиск в глубину"],
    title: "DFS – обход в глубину",
    short:
        "Прёшь вперёд до упора, упёрся – откатываешься на
    formal:
        "Стек (или рекурсия). Из DFS растут: времена вход
    complexity: "O(V + E)",
    demo: "dfs",
    simulator: "stack-dfs",
},
{
    id: "binsect",
    labels: ["binsect", "бинсект", "бинарный поиск", "д
    title: "Бинарный поиск (и его самозванец binsect)",
    short:
        "Бинарный поиск НАХОДИТ готовый элемент в уже отс
        о не сортирует. «binsect» из конспекта – шутлив
        quicksort.",
    formal:
        "Поиск одного элемента в отсортированном массиве
        O(n log n). Бинарный поиск сортировку не выполн
    complexity: "поиск: O(log n); сортировкой не являет
},
{
    id: "heap",
    labels: ["куча", "кучу", "кучи", "куче", "кучей", "
    title: "Куча (priority queue)",
    short:
        "Не отсортированный список, а «мешок, из которого
    formal:
        "Полное бинарное дерево в массиве: дети узла i ле
    complexity: "вставка/извлечение O(log n), «посмотре
    demo: "heap",
    simulator: "heap-beam-search",
},
{
    id: "stack",

```

```

.....
    },
    {
      id: "bfs-on-trie",
      labels: [
        "обход дерева",
        "обход бора",
        "обход в ширину по бору",
        "какой-то обход",
        "обходом дерева",
        "конечный автомат",
        "конечного автомата",
        "автомат",
        "автомата",
      ],
      title: "Обход бора в ширину (тот самый «какой-то обход»)",
      short:
        "В Ахо–Корасике бор обходят именно BFS: суффиксную версию, то есть строго по уровням.",
      formal:
        "Кладём в очередь детей корня (их ссылка = корень) и  

        кладём с в очередь.",
      complexity: "O(суммарной длины словаря × алфавит)",
      demo: "trie-bfs",
      simulator: "aho-corasick",
    },
    {
      id: "suffix-link",
      labels: ["суффиксная ссылка", "суффиксные ссылки"],
      title: "Суффиксная ссылка",
      short:
        "«Куда откатиться, если следующая буква не подошла к  

        строки.",
      formal: "link(v) = go(link(parent(v)), ch). Считает",
      demo: "suffix-link",
      simulator: "aho-corasick",
    },
    {
      id: "toposort",

```

```

.....
    },
    title: "Разреженная таблица (метод двоичных подъёмов)",
    short:
        "Заранее считаем ответ для всех отрезков длины 1,
         М.",
    formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
    complexity: "препроцессинг O(n log n), запрос O(1)",
    demo: "sparse-table",
    simulator: "sparse-table",
},
{
    id: "segment-tree",
    labels: ["дерево отрезков", "деревья отрезков", "деревья"],
    title: "Дерево отрезков",
    short:
        "Матрёшка из отрезков: корень отвечает за весь массив,
         левый и правый куски.",
    formal: "Строится за O(n), запрос и обновление — O(log n)",
    demo: "segment-tree",
    simulator: "segment-trees",
},
{
    id: "bridge",
    labels: ["мост", "моста", "мосты", "мостов", "мостовые"],
    title: "Мост",
    short:
        "Ребро, после удаления которого граф разваливается на две
         компоненты.",
    formal: "Ребро (u,v) — мост, если low[v] > tin[u], где tin[u] — время
         первого посещения u, а low[v] — минимальное tin среди всех
         ребро в родителя — нет).",
    complexity: "O(V + E) одним DFS",
    demo: "bridge",
    simulator: "bridges-code",
},
{
    id: "articulation",
    labels: ["точка сочленения", "точки сочленения", "точки"],
    title: "Точка сочленения",
    short:

```

```

.....
    title: "Класс NP и NP-полнота",
    short:
        "NP – «решение проверить легко, найти трудно» (судить о полноте в NP.",
    formal: "Задача NP-полна, если она в NP и к ней полнота сводится",
    demo: "np",
},
{
    id: "potential",
    labels: ["потенциал", "потенциалы", "потенциалов", "потенциальность"],
    title: "Потенциалы (перевзвешивание Джонсона)",
    short:
        "Трюк «поднять все дороги на одинаковую высоту»: для ребра (u,v) положить вес w'(u,v) = w(u,v) + h(u) - h(v), где h – потенциал. Тогда кратчайший путь сохраняется.",
    formal: "w'(u,v) = w(u,v) + h(u) - h(v), где h – потенциал",
    demo: "relax",
    simulator: "johnson-algo",
},
{
    id: "scc",
    labels: ["компонента сильной связности", "компоненты сильной связности"],
    title: "Компоненты сильной связности",
    short:
        "Группы вершин, где из каждой можно дойти до каждой другой",
    formal: "Косарайю: DFS с сохранением порядка выхода из вершин",
    complexity: "O(V + E)",
    demo: "scc",
    simulator: "scc-kosaraju",
},
{
    id: "prefix-function",
    labels: [
        "префикс-функция", "префикс-функции", "префикс-функция",
        "префикс", "префикса", "суффиксом", "суффикс", "суффиксы",
        "подстроки", "подстроку", "подстрока",
    ],
    title: "Префикс-функция π (КМП)",
    short:

```

Универсальное пособие • полный исходный код

```
.....
    id: "shortest-path",
    labels: ["кратчайший путь", "кратчайшие пути", "кра
    title: "Кратчайший путь",
    short:
        "Самый дешёвый маршрут из A в B. «Дешёвый» – по с
    formal:
        "Дейкстра – неотрицательные веса,  $O(E \log V)$ . Фор
        ы,  $O(V^3)$ .",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "negative-cycle",
    labels: ["отрицательный цикл", "отрицательного цикла
    title: "Отрицательный цикл",
    short:
        "Круг, пройдя по которому вы становитесь «богаче»
    formal:
        "Форд–Беллман: если на  $V$ -й итерации ещё удастся с
    demo: "relax",
    simulator: "bellman-ford",
},
{
    id: "greedy",
    labels: [
        "жадный алгоритм", "жадный", "жадно", "жадная",
        "самое дешевое ребро", "самое дешёвое ребро", "са
    ],
    title: "Жадный алгоритм",
    short:
        "На каждом шаге хватаем то, что лучше прямо сейчас
    formal:
        "Корректность обычно доказывается «леммой о разре
    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "euler",
```

```

.....
    short:
        "Матрица смежности — таблица «есть ли ребро», быс
        й, экономно для разреженных графов.",
    formal: "Матрица: проверка  $O(1)$ , память  $O(V^2)$ . Спис
},
{
    id: "dijkstra",
    labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс
    title: "Алгоритм Дейкстры",
    short:
        "Жадный «навигатор»: всегда раскрываем ближайший
        льные.",
    formal:
        "Куча хранит пары (расстояние, вершина). Достали
    complexity: " $O(E \log V)$  с бинарной кучей",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "bellman-ford",
    labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман
    title: "Алгоритм Форда-Беллмана",
    short:
        "Тупой, но честный:  $V-1$  раз проходим по ВСЕМ рёбра
        ",
    formal:
        "После  $k$ -й итерации гарантированно верны все крат
        ось — есть отрицательный цикл.",
    complexity: " $O(V \cdot E)$ ",
    demo: "relax",
    simulator: "bellman-ford",
},
{
    id: "floyd",
    labels: [
        "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше
        "промежуточная вершина", "промежуточную вершину",
    ],

```

```

.....
    labels: ["Краскал", "Краскала", "Краскалу", "Kruska
    title: "Алгоритм Краскала",
    short:
        "Сортируем все рёбра по весу и берём подряд те, ч
    formal:
        "Сортировка  $O(E \log E)$  + E операций union/find. C
    complexity: "O(E log E)",
    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "components",
    labels: [
        "компонента связности", "компоненты связности", "
        "компоненте связности", "связности", "связный", "
    ],
    title: "Компоненты связности",
    short:
        "Граф-архипелаг: острова, между которыми нет мост
        бой.",
    formal:
        "Считаются одним проходом: пока есть непосещённая
    complexity: "O(V + E)",
    demo: "components",
    simulator: "graph-dfs-bfs",
},
{
    id: "graph",
    labels: [
        "граф", "графа", "графе", "графы", "графов", "гра
        "вершины", "вершина", "вершин", "ребра с весом",
    ],
    title: "Граф",
    short:
        "Точки (вершины) и связи между ними (рёбра). Горо
    formal:
        " $G = (V, E)$ . Ребро бывает ориентированным (улица
    demo: "graph",

```



```

},
{
  id: "inclusion-exclusion",
  labels: [
    "включений-исключений", "включения-исключения", "вырезание прямоугольников",
  ],
  title: "Включения-исключения на префиксных суммах",
  short:
    "Чтобы получить сумму прямоугольника, берём больший отрезали дважды.",
  formal:
    "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
  complexity: "запрос O(1) после O(nm) предподсчёта",
  demo: "prefix-sum",
  simulator: "prefix-sums-2d",
},
{
  id: "reduction",
  labels: ["сведение", "сведения", "сведени", "сведё"],
  title: "Сведение задач",
  short:
    "«Я не умею решать A, зато умею B». Если A можно свести к B, то B можно решить.",
  formal:
    "A ≤p B: есть полиномиальная функция, переводящая A в B, решаемую за полиномиальное время.",
  demo: "np",
},
{
  id: "range",
  labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
  title: "Запрос на отрезке",
  short:
    "Спрашиваем не про один элемент, а про кусок массива. Ответ можно получить за O(1) предподсчёта.",
  formal:
    "Суммы – префиксные суммы O(1). Минимум без изменений.",
  demo: "prefix-sum",
}

```

```

    },
    {
      id: "zig-zig",
      labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
      title: "Zig-Zig – x и родитель с одной стороны",
      short:
        "Дерево вытянулось в «бамбук»: g → p → x идут в од...
        м (p, x). Именно это вдвое сокращает глубину вс...
      formal:
        "Если крутить наивно (два раза поднимать x), бамб...
      demo: "zig-zig",
      simulator: "splay-rotations",
    },
    {
      id: "zig-zag",
      labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
      title: "Zig-Zag – x и родитель с разных сторон",
      short:
        "Узлы идут «змейкой»: p – левый сын g, а x – прав...
        p и g становятся двумя детьми x.",
      formal: "Эквивалентно двойному повороту AVL-дерева",
      demo: "zig-zag",
      simulator: "splay-rotations",
    },
  ],
};

```

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => [e.id, e]));

/** Все метки, отсортированные от длинных к коротким (чтобы в UI было удобно). */
export const GLOSSARY_LABELS: { label: string; id: string }[] =
  GLOSSARY.map((e) => e.labels.map((label) => ({ label, id: e.id })))
    .sort((a, b) => b.label.length - a.label.length);

```

```
.....  
<div class="space-y-8">
```

```
  <div class="bg-slate-700/50 p-6 rounded-xl border
```

```
    <h3 class="text-xl font-bold text-emerald-400">
```

```
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
      <p class="text-lg text-emerald-300 italic">
```

```
      <p class="text-xl font-mono text-white">
```

```
    </div>
```

```
  <div class="grid grid-cols-1 xl:grid-cols-2">
```

```
    <!-- Аналогия 1 -->
```

```
    <div class="bg-slate-800 p-6 rounded-lg">
```

```
      <div class="absolute -top-3 left-4 right-4  
border border-emerald-500 shadow-md">
```

```
        Аналогия 1: Позитивное планирование
```

```
      </div>
```

```
      <p class="text-slate-300 text-sm mb-4">
```

```
        (нельзя поехать и заработать). Он всегда выигрывает
```

```
      <div id="slot-chapter-image-dijkstra" class="h-40 w-40">
```

```
    </div>
```

```
    <!-- Аналогия 2 -->
```

```
    <div class="bg-slate-900 p-6 rounded-lg">
```

```
      <div class="absolute -top-3 left-4 right-4  
border border-rose-500 shadow-md">
```

```
        Ограничения
```

```
      </div>
```

```
      <p class="text-slate-300 text-sm mb-4">
```

```
        (невозможное путешествие), Дейкстра сломается, так как не
```

```
      </div>
```

```
    </div>
```

```
<details class="bg-slate-900/40 rounded-lg border">
```

```
  <summary class="p-4 font-bold text-slate-300">
```

```
    Строгое доказательство (Скрыто)
```

```
  </summary>
```

```
  <div class="p-5 text-sm text-slate-300">
```

```
.....
        еряет ВСЕ возможные дороги V-1 раз подряд.
        <div id="slot-chapter-image-bellman"
        </div>

        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg"
        <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
            Отрицательный цикл
        </div>
        <p class="text-slate-300 text-sm mb-4"
        г станет ЕЩЕ короче – значит мы попали во в
        </div>
    </div>

    <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</section>`
},
{
    id: "floyd",
    title: "Билет 16. Флойд-Уоршелл",
    type: "html",
    category: "Графы",
    content: `<section id="floyd-content" class="mb-12">
<div class="flex items-center mb-6">
    <span class="bg-blue-600 text-white px-4 py-1 rounded"
    <h2 class="text-3xl font-bold text-white">Флойд
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-indigo-400">

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
```

```
.....  
<h2 class="text-2xl sm:text-3xl font-bold text-white">  
</div>
```

```
<div class="space-y-8">  
  <div class="bg-slate-700/50 p-6 rounded-xl border-l-1">  
    <h3 class="text-xl font-bold text-blue-400 mb-4">  
  
    <div class="bg-slate-900 p-4 rounded-lg mb-6 text-white">  
      <p class="text-sm text-blue-300 italic mb-2">0c  
      <p class="text-lg font-mono text-white">Бор (Тр  
    </div>  
    <p class="text-slate-300 text-sm">Позволяет найти  
      -mono text-amber-300 bg-black/30 px-1 rounded">  
      y.</p>  
  </div>
```

```
<!-- Аналогии -->
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">  
  <div class="bg-slate-800 p-6 rounded-lg border border-bo  
    <div class="absolute -top-3 left-4 bg-slate-700  
      emerald-500 shadow-md">  
      Аналогия 1: Паутина (Бор)  
    </div>  
    <p class="text-slate-300 text-sm mb-4">Представ  
      мы движемся по веткам. Если нужного продолж  
      ("нити страховки") в самый длинный из извест  
  </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-2  
  <div class="absolute -top-3 left-4 bg-rose-900  
    -500 shadow-md">  
    Аналогия 2: Матёшка (Терминальные)  
  </div>  
  <p class="text-slate-300 text-sm mb-4">Представ  
    нашли и "he", потому что оно спрятано внутри  
    минальные ссылки</b> (term_link) – прямые мос  
  <p>  
</div>
```

```

.....
        if (nodes[nodes[child].link].is_terminal) {
            nodes[child].term_link = nodes[child].link;
        } else {
            nodes[child].term_link = nodes[nodes[child].link];
        }

        q.push(child);
    }
}

</div>
</div>
</details>
</div>
</section>`
},
{
    id: "alg-map",
    title: "Алгоритмы на карте",
    type: "html",
    category: "Графы",
    content: `<section id="alg-map-content" class="mb-1">
<div class="flex items-center mb-6">
    <span class="bg-purple-600 text-white px-4 py-1">
    <h2 class="text-3xl font-bold text-white">Алгоритмы на карте
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-purple-400">
        <p class="text-slate-300 text-sm mb-4">Когда мы имеем дело с графами, которые могут содержать вершины, которые могут быть большими дробными числами (долгота и широта), мы можем использовать массивы чисел, выходящих за пределы от 0 до N, и уже по ним строим графы.</p>
    </div>
    </div>
</div>
</section>`
}
];

```

```

.....
{
    question: "Куда ведёт эйлеров путь, если нечётных
options: [
    "Всё ещё можно пройти по всем рёбрам ровно раз",
    "Такого графа не существует",
    "Эйлерова пути нет – нужно удалять лишние рёбра",
],
correctIndex: 2,
explanation: "Если нечётных вершин больше 2, эйлеров
    ровно 2 нечётные или 0."
}
],
"bridges-code": [
{
    question: "После DFS у нас low[u] = 4, а tin[v] =
options: [
    "Да, так как low[u] (4) > tin[v] (2)",
    "Нет, low[u] должно быть меньше или равно",
    "Только если граф связный и неориентированный"
],
correctIndex: 0,
explanation: "Условие моста: low[u] > tin[v]. В на
},
{
    question: "Для чего нужен массив low[v]?",
options: [
    "Для того чтобы было больше переменных в коде",
    "Чтобы знать минимальный tin предка, до которого",
    "Чтобы хранить глубину рекурсии"
],
correctIndex: 1,
explanation: "low[v] хранит минимальное время входа
    поиска мостов и точек сочленения."
},
{
    question: "Какова временная сложность алгоритма по
options: [
    "O(V^2) – ищем мосты в квадрате",

```

```
.....
    }
  }
};
```

ФАЙЛ 24/99: src/data/vizStepBus.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 158 | РАЗМ

```
.....
import { createContext, useContext, useEffect, useState } from "react";
import type { DebugValue } from "../debugRunner";
```

```
/**
```

```
 * Прямая связь «компилятор « открытая визуализация».
```

```
 *
```

```
 * Компилятор передаёт не «магический номер строки эталонного
```

```
 * выполненного Python-кода. Визуализация читает захардкоженные
```

```
 * (i, j, k, v, dist, P, st...) и подсвечивает соответствующие
```

```
 * любой пользовательский код работает, если в нём исполнялись
```

```
 */
```

```
export const VizChapterContext = createContext("");
```

```
const CHANNEL = "algo:viz-state";
```

```
export interface VizStateEventDetail {
```

```
  chapterId: string;
```

```
  /** Строка и функция реально выполненного пользователя */
```

```
  line?: number;
```

```
  func?: string;
```

```
  /** Машинно-читаемые globals + locals: числа остаются
```

```
  variables?: Record<string, DebugValue>;
```

```
  /** Имена, изменённые выделенной строкой. */
```

```
  changed?: string[];
```

```
}
```

```
/** Последний снимок хранится, чтобы поздно смонтировать
```



```

.....
    }, [chapterId]));

    return runtime;
}

export type VizStepSelector = (variables: Record<string>

/**
 * Подписка старых покадровых демонстраций. Если переда
 * номер кадра выбирается по именам/значениям переменных
 */
export function useVizStepSync(
    currentStep: number,
    goToStep: (n: number) => void,
    maxStep: number,
    selectByVariables?: VizStepSelector
) {
    const chapterId = useContext(VizChapterContext);

    useEffect(() => {
        if (!chapterId) return;

        const apply = (detail?: VizStateEventDetail) => {
            if (!detail || detail.chapterId !== chapterId) re
            // Без явного селектора компонент либо рисует use
            // либо остаётся ручным. Номер Python-строки нико
            const requested =
                selectByVariables && detail.variables ? selectB
            // null означает: компонент рисует runtime-перемен
            // не увидел знакомых захардкоженных имён.
            if (requested === null || !Number.isFinite(request
            const target = Math.max(0, Math.min(Math.max(0, m
            if (target !== currentStep) goToStep(target);
        };

        apply(latestByChapter.get(chapterId));
        const handler = (e: Event) => apply((e as CustomEvent
        window.addEventListener(CHANNEL, handler);
    }
}

```

Универсальное пособие • полный исходный код

```
.....
* Подписка: панель компилятора следует за активной вкладкой
* Сразу отдаём последнее значение: компилятор часто отменяет подписку
* как пользователь переключился с 1D на 2D.
*/
export function onVizDemo(chapterId: string, cb: (demoId: string) => void) {
  if (typeof window !== "undefined") return () => {};
  const latest = latestDemoByChapter.get(chapterId);
  if (latest !== undefined) cb(latest);
  const handler = (e: Event) => {
    const detail = {e as CustomEvent<VizDemoEventDetail>};
    if (detail && detail.chapterId === chapterId) cb(detail.demoId);
  };
  window.addEventListener(DEMO_CHANNEL, handler);
  return () => window.removeEventListener(DEMO_CHANNEL, handler);
}
```

.....
ФАЙЛ 25/99: src/data/vizSync.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 883 | РАЗМЕР: 10.5 КБ

```
.....
/**
 * Захардкоженная «карта синхронизации» компилятора с визуализацией
 *
 * Для каждой страницы (главы) здесь вручную перечислены переменные,
 * которые понимает её визуализация: i, j, k, v, dist, ...
 * Компилятор выполняет любой Python-код и передаёт результат
 * имён напрямую; привязки к номерам или тексту строк эмулируются
 */
```

```
export interface SyncVariable {
  /** Имя переменной, как в коде визуализации: i, j, k, ... */
  name: string;
  /** Роль в алгоритме (показывается во всплывающей подсказке) */
  role: string;
  /** Характерный диапазон/значение на демонстрации. */
  range?: string;
}
```

Универсальное пособие • полный исходный код

```

.....
print("корень:", tree[l])`,
  variables: {
    { name: "n", role: "длина исходного массива; рекурсия"},
    { name: "v", role: "текущая вершина дерева – подсвеченная"},
    { name: "l, r", role: "границы отрезка, за которым"},
    { name: "m", role: "середина отрезка  $m = (l + r) // 2$ "},
    { name: "ul, ur", role: "границы массового обновления"},
    { name: "add", role: "добавка  $A[ul..ur] += add$  – массив"},
    { name: "lazy", role: "массив обещаний:  $lazy[v] \neq 0$ "},
  },
},
treap: {
  vizTitle: "Treap: плоскость → дерево → Split/Merge/Join",
  stepNote: "Шаг = вставка очередной пары (x; y) или удаление",
  code: `pairs = [(45, 8), (3, 6), (6, 5), (2, 4), (9, 7)]
pairs.sort(key=lambda p: -p[1])          # сортируем по y – по приоритету
x, y = 6, 5                              # подсвеченная пара
i = 0                                     # индекс шага: сортировка

def insert(root, x, y):                  # вставка спуском
  ...                                   # ключ <= узла – вставляем
def split(t, x):                         # разрез по ключу:
  ...                                   # узел целиком уходит влево
  variables: {
    { name: "x", role: "ключ – координата точки по горизонтальной о-набору: 1..45" },
    { name: "y", role: "приоритет – координата по вертикали: 1..45" },
    { name: "i", role: "индекс шага: сортировка (0..3)" },
  },
},
"splay-tree": {
  vizTitle: "Splay-дерево: большое дерево",
  stepNote: "Шаг = один кадр splay(key): прицел ребра",
  code: `keys = [50, 40, 35, 32, 30, 28, 12, 60, 55, 45, 38, 25, 20, 15, 10, 5]
key = 32                                # какой ключ поднимаем в корень
i = 0                                   # кадр анимации: 0 ... (число кадров)
case = "zig-zig"                        # вкладка «Случай поворота»

```

Универсальное пособие • полный исходный код

```
.....
    { name: "a", role: "исходный массив демо (вкладка
    { name: "n, LOG", role: "длина массива и число ур
    { name: "i", role: "начало блока: st[i][j] отвеча
    { name: "j", role: "уровень таблицы: блок длины 2
    { name: "st", role: "таблица уровней – подсвечива
    ],
},
"prefix-sums-2d": {
    vizTitle: "Префиксные суммы (1D и 2D)",
    stepNote: "Шаг = заполнение очередного  $P[i] = P[i-1]$ 
    code: `a = [3, 1, 4, 1, 5, 9, 2, 6]
i = 0

P = [0]
for i in range(1, len(a) + 1):
    P.append(P[i - 1] + a[i - 1])
print("P =", P)`,
    variables: [
        { name: "a", role: "исходный массив демо (вкладка
        { name: "i", role: "индекс элемента, который сейч
        { name: "P", role: "префиксные суммы:  $P[i] = a[0]$ 
    ],
},
"dynamic-programming": {
    vizTitle: "Динамическое программирование (мемоизаци
    stepNote: "Шаг = событие внутри fibМемо(k): вызов,
    code: `n = 5
memo = {}

def fib(k):
    if k in memo:
        return memo[k]
    if k <= 2:
        return 1
    print(f"считаем fib({k-1}) + fib({k-2})")
    r1 = fib(k - 1)
    r2 = fib(k - 2)
    memo[k] = r1 + r2
```

Универсальное пособие • полный исходный код

```

.....
    { name: "top", role: "что сняли с вершины — текущ
    },
},
"queue-bfs": {
    vizTitle: "Очередь и обход в ширину",
    stepNote: "Шаг = dequeue вершины из головы очереди
    code: `from collections import deque

q = deque(["A"])          # очередь BFS (FIFO)
q.append("B")             # enqueue соседа в хвост
v = q.popleft()          # шаг: обрабатываем голову
print("v =", v, "очередь:", list(q))`,
    variables: [
        { name: "q", role: "очередь (массив): гости стран
        { name: "v", role: "вершина, извлечённая из головы
    ],
},
"graph-dfs-bfs": {
    vizTitle: "DFS и BFS на графе",
    stepNote: "Шаг = переход из текущей вершины v к неп
    code: `adj = {"A": ["B", "C"], "B": ["A", "D", "E"]}
visited, order = set(), [] # порядок обхода
stack = ["A"]             # DFS: стек (рекурсия)
q = []                   # BFS: очередь (deque)
v = "A"                  # шаг: достали вершину из

# --- тело алгоритма ---
while stack:              # ---- DFS ----
    v = stack.pop()
    if v in visited:
        continue
    visited.add(v)
    order.append(v)
    stack += [u for u in adj[v] if u not in visited]
print("DFS:", order)

visited, order, q = set(), [], ["A"] # ---- BFS ----
while q:

```

```

.....
        comp[to] = comp[v]
        q.append(to)
print(comp, "компонент:", count)`,
    variables: {
        { name: "v", role: "текущая вершина обхода — подсвечивается"},
        { name: "comp", role: "словарь меток: номер компонента"},
        { name: "q", role: "текущая очередь BFS", range: 1000},
        { name: "count", role: "число запусков обхода = число компонент"},
    },
    "top-sort": {
        vizTitle: "Топологическая сортировка",
        stepNote: "Шаг = выход рекурсии из вершины v: она добавлена в order",
        code: `adj = {0: [1, 2], 1: [3], 2: [3], 3: [4], 4: [5]}`,
        used, order = set(), []

def dfs(v):
    used.add(v)
    for to in adj[v]:          # шаг: ребро v → to
        if to not in used:
            dfs(to)
    order.append(v)           # выход из v!

# --- тело алгоритма ---
for v in adj:
    if v not in used:
        dfs(v)
print(order[::-1])          # разворот — ответ`,
    variables: {
        { name: "v", role: "текущая вершина DFS — подсвечивается"},
        { name: "to", role: "вершина, куда ведёт ребро из v"},
        { name: "order", role: "порядок выхода из рекурсии"},
        { name: "used", role: "множество посещённых вершин"},
    },
},
    "scc-kosaraju": {
        vizTitle: "Компоненты сильной связности (Косарайю)",
        stepNote: "Шаг = вершина из order (в обратном порядке) добавлена в comp",
        code: `adj = {0: [1, 2], 1: [3], 2: [3], 3: [4], 4: [5]}`,
        used, order, comp = set(), [], {}

def dfs(v):
    used.add(v)
    for to in adj[v]:
        if to not in used:
            dfs(to)
    order.append(v)

def dfs2(v, comp):
    used.add(v)
    for to in adj_rev[v]:
        if to not in used:
            dfs2(to, comp)
    comp[v] = len(comp)

for v in adj:
    if v not in used:
        dfs(v)
for v in order[::-1]:
    if v not in used:
        dfs2(v, comp)

print(comp, "компонент:", count)`,
    variables: {
        { name: "v", role: "текущая вершина обхода — подсвечивается"},
        { name: "comp", role: "словарь меток: номер компонента"},
        { name: "q", role: "текущая очередь BFS", range: 1000},
        { name: "count", role: "число запусков обхода = число компонент"},
    },
    "top-sort": {
        vizTitle: "Топологическая сортировка",
        stepNote: "Шаг = выход рекурсии из вершины v: она добавлена в order",
        code: `adj = {0: [1, 2], 1: [3], 2: [3], 3: [4], 4: [5]}`,
        used, order = set(), []

def dfs(v):
    used.add(v)
    for to in adj[v]:          # шаг: ребро v → to
        if to not in used:
            dfs(to)
    order.append(v)           # выход из v!

# --- тело алгоритма ---
for v in adj:
    if v not in used:
        dfs(v)
print(order[::-1])          # разворот — ответ`,
    variables: {
        { name: "v", role: "текущая вершина DFS — подсвечивается"},
        { name: "to", role: "вершина, куда ведёт ребро из v"},
        { name: "order", role: "порядок выхода из рекурсии"},
        { name: "used", role: "множество посещённых вершин"},
    },
},
    "scc-kosaraju": {
        vizTitle: "Компоненты сильной связности (Косарайю)",
        stepNote: "Шаг = вершина из order (в обратном порядке) добавлена в comp",
        code: `adj = {0: [1, 2], 1: [3], 2: [3], 3: [4], 4: [5]}`,
        used, order, comp = set(), [], {}

def dfs(v):
    used.add(v)
    for to in adj[v]:
        if to not in used:
            dfs(to)
    order.append(v)

def dfs2(v, comp):
    used.add(v)
    for to in adj_rev[v]:
        if to not in used:
            dfs2(to, comp)
    comp[v] = len(comp)

for v in adj:
    if v not in used:
        dfs(v)
for v in order[::-1]:
    if v not in used:
        dfs2(v, comp)

print(comp, "компонент:", count)`,
    variables: {
        { name: "v", role: "текущая вершина обхода — подсвечивается"},
        { name: "comp", role: "словарь меток: номер компонента"},
        { name: "q", role: "текущая очередь BFS", range: 1000},
        { name: "count", role: "число запусков обхода = число компонент"},
    },
}

```

Универсальное пособие • полный исходный код

```

.....
    { name: "v", role: "текущая вершина DFS (проход 1",
    { name: "order", role: "порядок выхода прохода 1",
    { name: "adjT", role: "транспонированный граф: все",
    { name: "comp", role: "номер SCC каждой вершины",
    },
},
"graph-articulation": {
    vizTitle: "Точки сочленения",
    stepNote: "Шаг = обновление low[v] по ребру; вершина",
    code: `# демо-граф страницы: треугольник 0-1-2, мост
# DFS из 0: tin/low после полного обхода; точки сочленения
tin = {0: 1, 1: 2, 2: 3, 3: 4, 4: 5, 5: 6, 6: 7}
low = {0: 1, 1: 1, 2: 1, 3: 4, 4: 5, 5: 5, 6: 5}

v, to = 3, 4                # шаг: возврат из 4 в 3
is_cut = low[to] >= tin[v]   # 5 >= 4 → 3 – точка
v2, to2 = 1, 2
is_cut2 = low[to2] >= tin[v2] # 1 >= 2 → нет {1 с
print(f"v={v}: точка сочленения? {is_cut}; v={v2}: {is_
    variables: {
        { name: "tin", role: "время захода DFS в вершину"
        { name: "low", role: "минимальный tin, достижимый
        { name: "v, to", role: "ребро дерева DFS: проверя
        { name: "is_cut", role: "low[to] ≥ tin[v] и v – н
    },
},
"bridges-code": {
    vizTitle: "Мосты: DFS + tin/low",
    stepNote: "Шаг = строка псевдокода слева; ребро (v,
    code: `# демо-граф страницы: A-B, B-C, A-C, C-D, D-
# DFS из A: tin/low после полного обхода
tin = {"A": 1, "B": 2, "C": 3, "D": 4, "E": 5, "F": 6,
low = {"A": 1, "B": 1, "C": 1, "D": 4, "E": 4, "F": 4,

v, to = "B", "G"            # шаг: возврат из G в B (G
is_bridge = low[to] > tin[v] # 7 > 2 → мост!
v2, to2 = "C", "D"
is_bridge2 = low[to2] > tin[v2] # 4 > 3 → C-D тоже

```

Универсальное пособие * полный исходный код

```
.....
# смежности демо-графа из 7 вершин A..G
adj = [[1, 4, 5], [0, 2, 5], [1, 3, 5, 6], [2, 4, 5, 6],
       [3, 0, 5], [0, 1, 2, 3, 4, 6], [2, 3, 5]]
color = [-1] * 7          # color[v] – цвет вершины (0..6)
k, v = 0, 0               # сколько цветов использовано
for v in range(7):        # жадная раскраска
    used = {color[u] for u in adj[v] if color[u] >= 0}
    c = 0
    while c in used: c += 1
    color[v] = c
    k = max(k, c + 1)
    variables: [
        { name: "V", role: "число вершин графа", range: "1..7" },
        { name: "E", role: "число рёбер", range: "K5: 10" },
        { name: "F", role: "число граней в плоской укладке" },
        { name: "v", role: "вершина, которую красят (метка)" },
        { name: "color", role: "массив/словарь цветов вершин" },
        { name: "k", role: "сколько цветов уже использовано" },
    ],
},
dijkstra: {
    vizTitle: "Дейкстра",
    stepNote: "Шаг = старт, извлечение ближайшей вершины",
    code: `import heapq

g = {"A": [("B", 4), ("C", 2)], "B": [("G", 4), ("D", 5)],
     "C": [("D", 5), ("E", 1)], "E": [("F", 1)], "F": []}
dist = {"A": 0}
prev = {}
done = set()
pq = [(0, "A")]
while pq:
    du, u = heapq.heappop(pq)
    if u in done:
        continue
    done.add(u)
    print(f"обрабатываем {u} (dist={du})")
    for v, w in g[u]:
```


Универсальное пособие • полный исходный код

```

.....
    { name: "dist", role: "расстояния от S; улучшение
    },
},
"floyd": {
    vizTitle: "Флойд на матрице аэропортов",
    stepNote: "Шаг = проверка пары (i, j) при посреднике
    code: `INF = 999` # «бесконечность»

    # поэтому отдельные проверки
d = [[0, 4, INF, 2, INF, INF, INF], [INF, 0, 3, INF, INF,
    INF, INF, INF, 0, INF, 1], [INF, INF, INF, INF, INF, 0
n = 7
k, i, j = 0, 0, 0 # шаг: посредник k, летим

# --- тело алгоритма ---
for k in range(n): # посредник k
    for i in range(n):
        for j in range(n):
            if d[i][k] + d[k][j] < d[i][j]: # через k
                d[i][j] = d[i][k] + d[k][j]
print("итог: все пары посчитаны"),
    variables: [
        { name: "d", role: "матрица смежности → кратчайшие"},
        { name: "n", role: "число вершин (аэропортов)", g
        { name: "k", role: "посредник: разрешённые пути и"},
        { name: "i, j", role: "клетка d[i][j], которую пр"},
        { name: "d[i][j]", role: "текущая длина кратчайше"},
    ],
},
"johnson-algo": {
    vizTitle: "Алгоритм Джонсона (перевзвешивание)",
    stepNote: "Шаг = Беллман-Форд из фиктивной вершины
    code: `# демо-граф страницы: S→{A,B,C} (0), A→B=-2,
# потенциалы – реальные расстояния от фиктивной S (Белл
h = {"A": 0, "B": -2, "C": -1}

u, v, w = "A", "B", -2 # ребро с отрицательным ве
w_new = w + h[u] - h[v] # w' = w + h[u] - h[v] = -
print(f"w'({u}→{v}) = {w_new}")

```

Универсальное пособие • полный исходный код

```

.....
    { name: "i", role: "индекс текущего ребра в отсор
    { name: "parent", role: "DSU: представитель компо
    { name: "mst", role: "рёбра остова; в ответе равн
    ],
},
"mst-prima": {
    vizTitle: "Прим",
    stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
    code: `import heapq

# смежность демо-графа страницы: 9 вершин A..I
adj = {"A": [(2, "B"), (5, "D")], "B": [(2, "A"), (4, "C"),
      "C": [(4, "B"), (9, "F")], "D": [(5, "A"), (3, "E"),
      "F": [(7, "E"), (9, "C"), (13, "I")], "G": [(8, "H"),
      "I": [(12, "H"), (13, "F")]}
heap = [(0, "A", "-")]          # (вес, вершина, откуда)
taken, mst = set(), []
w, v, parent = 2, "B", "A"    # шаг: ребро из кучи
i = 0

# --- тело алгоритма ---
while heap and len(taken) < len(adj):
    w, v, parent = heapq.heappop(heap)
    if v in taken:
        continue
    taken.add(v)
    if parent != "-":
        mst.append((w, parent, v))
    for w2, u in adj[v]:
        if u not in taken:
            heapq.heappush(heap, (w2, u, v))
print("остов:", mst, "вес:", sum(w for w, _, _ in mst))
variables: [
    { name: "adj", role: "список смежности демо-графа",
    { name: "w, v, parent", role: "ребро из кучи: вес",
    { name: "heap", role: "приоритетная очередь канди
    { name: "taken", role: "вершины уже в дереве — «з
    { name: "mst", role: "рёбра остова, вес 51", gang

```

Универсальное пособие * полный исходный код

```

.....
    { name: "comp", role: "DSU: номер компоненты (плем
    { name: "cheapest", role: "самое дешёвое исходяще
    { name: "u, v, w", role: "ребро-кандидат: концы и
    { name: "phase", role: "номер фазы параллельных с
    },
},
"string-kmp": {
    vizTitle: "Префикс-функция (КМП)",
    stepNote: "Шаг = обработка символа s[i]: откаты по
    code: `pattern = "aba"
text = "abacaba"
s = pattern + "#" + text      # демо-страница ищет pattern
i, j = 1, 0
pi = [None] * len(s)         # пустые клетки: значения п
pi[0] = 0

# --- тело алгоритма ---
for i in range(1, len(s)):   # шаг: считаем pi[i]
    j = pi[i - 1]             # длина совпадения
    while j > 0 and s[i] != s[j]:
        j = pi[j - 1]         # откат назад
    if s[i] == s[j]:
        j += 1
    pi[i] = j
print(pi)`
variables: [
    { name: "pattern", role: "образец: питон подставл
    { name: "text", role: "текст, в котором ищем", га
    { name: "s", role: "склейка pattern + \"#\n\" + tex
    { name: "i", role: "индекс текущего символа – счи
    { name: "j", role: "длина текущего совпавшего пре
    { name: "pi", role: "префикс-функция:  $\pi[i]$  – наиб
    ],
},
"string-z-func": {
    vizTitle: "Z-функция",
    stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l
    code: `pattern = "aba"

```

Универсальное пособие • полный исходный код

```

.....
    { name: "i", role: "текущая позиция в тексте – логический индекс",
      { name: "v", role: "номер текущей вершины автомата",
        { name: "c / state", role: "символ и состояние – символ и индекс",
          },
        },
    },
    "complexity-classes": {
      vizTitle: "Классы сложности: матрёшка  $P \subseteq NP \subseteq PSPACE$ ",
      stepNote: "Шаг = подсветка класса/задачи по имени и номеру",
      code: `cls = "NP"                                # класс: "P", "NP", "PSPACE"
task = "3-SAT"                                         # задача: "3-SAT", "Vertex Cover"

# сведение A ≤p B: стрелка между задачами
reductions = [{"3-SAT", "Vertex Cover"}]
for a, b in reductions:
    print(f"{a} ≤p {b}")`
    variables: [
      { name: "cls", role: "какой класс подсветить: P / NP / PSPACE",
        { name: "task", role: "какую задачу подсветить: 3-SAT / Vertex Cover",
          { name: "reductions", role: "список пар задач – пометка",
            },
          },
        ],
    },
    intro: {
      // мнемодкарточки без шагов алгоритма
      variables: [],
    },
    "alg-map": {
      variables: [],
    },
  },
  /* — Дополнительные демонстрации внутри страниц (включая диаграммы)

  "sparse-table#2d-build": {
    vizTitle: "Разреженная таблица 2D: построение",
    stepNote: "Шаг = новый уровень k: блоки  $2^k \times 2^k$  дополнены нулями",
    code: `A = [[45, 12, 88, 34, 11, 76, 23, 90], [67, 88, 29, 60], [25, 41, 16, 92, 9, 17, 56, 31], [51, 1, 42, 85, 14]]
r, c, k = 0, 0, 0

```

Универсальное пособие • полный исходный код

```
.....
    vizTitle: "Префиксные суммы 2D (прямоугольники)",
    stepNote: "Демонстрация кликабельна вручну: наведи
    code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):
    S.append([0] + [None] * m)
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1]

r1, c1, r2, c2 = 1, 1, 2, 2 # клетки 1-индексные, как
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`,
    variables: [
        { name: "i, j", role: "текущая ячейка таблицы S {
        { name: "S[i][j]", role: "сумма прямоугольника {0
        },
        { name: "r1, c1, r2, c2", role: "углы запроса в 1
        ний", range: "1 ... n, 1 ... m" },
    ],
},
};
```

```
/** Безопасный доступ: страница без записи получает пус
export const getPageSync = (chapterId?: string | null,
    (demoId && chapterId && PAGE_SYNC[`${chapterId}#${demoId}`]
    (chapterId && PAGE_SYNC[chapterId]) ||
    { variables: [] });
```

```
/**
```

* Стартовое содержимое редактора для страницы.

*

* Формат: короткий комментарий-шапка + исполняемый реф

* Визуализация читает из его трассы знакомые имена (i,

* но точно так же примет пользовательский код с теми же

Универсальное пособие • полный исходный код

```
.....
? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#
: `# «${chapterTitle}»\n# демо на странице нет – св

if (!sync.code) return head;
const init = extractInit(sync.code);
return `${head}\n${init}\n`;
}
```

РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

ФАЙЛ 26/99: src/data/tickets/ticket1_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 335 | РАЗМ

```
.....

import { Chapter } from '../../../types';

export const tickets1to5: Chapter[] = [
  {
    id: "segment-trees",
    title: "1. Дерево отрезков с операциями на отрезках",
    type: "html",
    description: "Дерево отрезков – структура данных дл",
    category: "Оптимизации и Продвинутое",
    content: `
<section id="segment-trees" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>

  <div class="space-y-8">
    <div id="intro" class="bg-slate-700/50 p-6 round
      <h3 class="text-xl font-bold text-blue-400`
  }
]
```

```

        lazy[2 * node] += lazy[node];
        tree[2 * node] += lazy[node];
        lazy[2 * node + 1] += lazy[node];
        tree[2 * node + 1] += lazy[node];
        lazy[node] = 0;
    }
}
</pre>
</div>
</details>
</div>
</div>
</section>
`
},
{
    id: "sparse-table",
    title: "2. Двумерная разреженная матрица (Sparse Table)",
    type: "html",
    description: "Разреженная таблица для идемпотентных операций",
    category: "Продвинутые структуры",
    content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">Сложность
    <h2 class="text-2xl sm:text-3xl font-bold text-white">2. Двумерная разреженная матрица (Sparse Table)
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">Задача
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Дана матрица  $A$  размера  $N \times M$ . Для каждого запроса  $(L, R, k)$  найти значение
        <p class="text-xl font-mono text-white">

$$A_{L, R, k} = \bigoplus_{i=L}^R A_{i, \lfloor \frac{R-i}{2^k} + 1 \rfloor}$$

        </p>
        <p>где  $\oplus$  — идемпотентная операция (например,  $\min$  или  $\max$ ).
        </p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            <div class="flex justify-between text-sm text-emerald-400">
              <div>Входные данные:  $N, M, Q$ 
            </div>
            <div>Выходные данные:  $Q$  значений
          </div>
        </div>
      </div>
    </div>
  </div>
`

```

```
.....
        Префиксная матрица сумм (биле
    </div>
    <ul class="text-slate-300 text-sm s
        <li>• Размер: <code class="bg-s
-800">N × M</code> – одна копия матрицы, и
        <li>• Запрос суммы на прямоугол
        <li>• Умеет <b>только сумму</b>
        <li>• Менять элементы нельзя –
    </ul>
</div>
<div class="bg-slate-900 p-6 rounded-lg
    <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
        2D Sparse table минимумов (эт
    </div>
    <ul class="text-slate-300 text-sm s
        <li>• Размер: <code class="bg-s
0">N × M × log N × log M</code> – жирнее в
        <li>• Запрос минимума на прямоу
        <li>• Умеет <b>min, max, gcd</b>
        <li>• Менять элементы тоже нель
    </ul>
</div>
</div>
<div class="grid grid-cols-1 xl:grid-cols-2
    <div class="bg-slate-800 p-6 rounded-lg
        <div class="absolute -top-3 left-4
er border-amber-500 shadow-md">
        Аналогия 3: Слоёный тортик
    </div>
    <p class="text-slate-300 text-sm">П
й по каждой оси – log (для 1000×1000 это 10
ена O(1) для минимума: запекли все возможные
    </div>
    <div class="bg-slate-900 p-6 rounded-lg
        <div class="absolute -top-3 left-4
border-sky-500 shadow-md">
        Аналогия 4: Аккумулятор
```



```

</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4
rder border-emerald-500 shadow-md">
      Аналогия 1: Удар катаной (Sp
    </div>
    <p class="text-slate-300 text-sm mb
по значению X. Все, кто меньше X улетают в.
    </div>
    <div class="bg-slate-900 p-6 rounded-lg"
      <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
        Аналогия 2: Слияние капель (M
      </div>
      <p class="text-slate-300 text-sm mb
вого). Мы просто смотрим на корни: у кого Y
как потомок.</p>
    </div>
  </div>
<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
    Код (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300"
    <pre class="bg-slate-950 p-4 rounded
pair&lt;Node*, Node*&gt; split(Node* t, int x) {
  if (!t) return {nullptr, nullptr};
  if (t->val &lt;= x) {
    auto [L, R] = split(t->right, x);
    t->right = L;
    return {t, R};
  } else {
    auto [L, R] = split(t->left, x);
    t->left = R;
    return {L, t};
  }
}

```

```
.....
    <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
        Аналогия 2: Балансировка чере
    </div>
    <p class="text-slate-300 text-sm mb">
и становиться кривым как бамбук. Но как тол
</p>
    </div>
</div>
<details class="bg-slate-900/40 rounded-lg">
    <summary class="p-4 font-bold text-slate-
    -b border-slate-700/50">
        Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300">
        <pre class="bg-slate-950 p-4 rounded">
// Zig, Zag
function rotate(x) {
    let p = x.parent;
    if (p.left === x) { // Right rotate
        p.left = x.right;
        x.right = p;
    } else { // Left rotate
        p.right = x.left;
        x.left = p;
    }
}</pre>
    </div>
</details>

<div class="bg-indigo-950/60 p-6 rounded-xl">
    <h4 class="text-lg font-bold text-indigo">

    <div class="space-y-4 text-sm text-slate">
        <div>
            <strong class="text-white block">
корень?</strong>
            <p>Оно поднимет узел, на котором
```

```

.....
<div class="bg-slate-700/50 p-6 rounded-xl border
  <h3 class="text-xl font-bold text-emerald-4
  <div class="bg-slate-900 p-4 rounded-lg mb-1
    <p class="text-lg text-emerald-300 italic
    <p class="text-xl font-mono text-white">
      , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
  </div>
  <div class="grid grid-cols-1 xl:grid-cols-2
    <div class="bg-slate-800 p-6 rounded-lg
      <div class="absolute -top-3 left-4
  rder border-emerald-500 shadow-md">
    ~ Аналогия 1: Вырезание прямоу
  </div>
    <p class="text-slate-300 text-sm mb
  з него маленький целевой прямоугольник в цел
  ый верхний угол отрезается дважды! Поэтому
  </div>
    <div class="bg-slate-900 p-6 rounded-lg
      <div class="absolute -top-3 left-4
  border-sky-500 shadow-md">
    Аналогия 2: Аккумулятор
  </div>
    <p class="text-slate-300 text-sm mb
  аккумулирует</b>: каждая клетка P[i][j] уже
  </b> по включениям-исключениям. Фокус держи
  я печь слёный тортик из ~100 копий матрицы.
  </div>
</div>
<div class="bg-slate-900/40 rounded-lg border
  <p class="text-slate-400 text-sm"> <b
  <b class="text-emerald-300">суммы</b>; раз
  d</b>. Сумму через sparse table считать нел
  тной операции).</p>
  </div>
</div>
</div>
</section>
}

```

```

        </div>
        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #f8d7da; border-radius: 10px; padding: 10px; margin: 10px 0;">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md">
                Ограничения
            </div>
            <p class="text-slate-300 text-sm mb-0">
                дный и не умеет пересматривать уже "зафиксирова
            </p>
        </div>
        <div id="slot-dijkstra-viz" class="mt-8"></div>
    </div>
</div>
</section>`
    },
    {
        id: "bellman-ford",
        title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
        type: "html",
        category: "Графы. Пути",
        content: `
<section id="bellman-ford-content" class="mb-12 scroll-py-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
        <h2 class="text-2xl sm:text-3xl font-bold text-white">15. Графы. Поиск кратчайшего пути. Форд-Беллман
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-rose-400/50">
            <h3 class="text-xl font-bold text-rose-400">Ограничения
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-rose-300 italic">Аналогия 1
                <p class="text-xl font-mono text-white">
                    <math>D[u, v] = \min_{(u, w) \in E} (D[u, w] + w(u, w))</math>
            </p>
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">

```

```

.....
    <div class="bg-slate-800 p-6 rounded-lg border-2
      <div class="absolute -top-3 left-4 bg-slate-800
        border-emerald-500 shadow-md">
          Суть фокуса (По шагам)
        </div>
      <p class="text-slate-300 text-sm mb-4">
        Главный герой – внешний цикл
        шаг class="bg-slate-900 text-pink-400">
        шу себе проходить через вершину k?»</i>
      </p>
      <ul class="text-sm text-slate-300 space-y-2">
        <li>Шаг k=0: Ты знаешь только
        <li>Шаг k=1: Разрешаем пересадки
        e» через путь class="text-indigo-300">
        <li>Шаг k=2: Разрешаем пересадки
        нутри этих путей уже могут быть пересадки ч
        <li>...</li>
        <li>Шаг k=N: Ты проверил все
      </li>
    </ul>
  </div>
</div>
</section>`
  },
  {
    id: "johnson-algo",
    title: "17. Графы. Алгоритм Джонсона (Фокус с пересадками)",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-2

```

```

.....
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
    <h3 class="text-xl font-bold text-emerald-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">
      <p class="text-xl font-mono text-white">
        (проверяем через DSU) - берем в ответ.</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 right-4 bottom-4
          border border-emerald-500 shadow-md">
          Аналогия: Прокладка интернет
        </div>
        <p class="text-slate-300 text-sm mb-4">
с самых дешевых дорог в стране". Он строит
единую сеть.</p>
      </div>
    </div>
  </div>
</div>
</section>`
},
{
  id: "mst-prima",
  title: "19. Графы. Остоаное дерево. Прима",
  type: "html",
  category: "Графы. Остовные",
  content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
  </div>

```

```
.....
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
ается с соседом.</p>
</div>
<div class="grid grid-cols-1 gap-6">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 b
rder border-emerald-500 shadow-md">
            Аналогия: Племена
        </div>
        <p class="text-slate-300 text-sm mb
изкому племени для объединения. К вечеру пл
ства шлют гонцов к ближайшему чужому царств
    </div>
</div>
</div>
</section>`
    }
};
```

ФАЙЛ 28/99: src/data/tickets/ticket21_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 262 | РАЗМ

```
.....

import { Chapter } from '../types';

export const tickets21to24: Chapter[] = [
  {
    id: "string-kmp",
    title: "21. Префикс-функция (п) – Взгляд назад (КМП)",
    type: "html",
    category: "Строки",
    content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
```

```

        </p>
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate-
  -b border-slate-700/50">
    Пример вычисления (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300"
    <p>Тот же пример: <b>abcabcd</b></p>
    <ul class="list-disc pl-5 space-y-2"
      <li><b>a</b>: Префиксов нет. <code>
      <li><b>ab</b>: Из начала ("a")
    >
      <li><b>abc</b>: Опять мимо. <code>
      <li><b>abca</b>: 0! Начало ("a"
    </code></li>
      <li><b>abcab</b>: Начало ("ab")
    /li>
      <li><b>abcabc</b>: Начало ("abc
    e></li>
      <li><b>abcabcd</b>: "d" всё исп
    с' – не совпало. Откат по  $\pi$ :  $j = \pi[2] = 0$ .
     $\pi[6] = 0$ </code></li>
    </ul>
  </div>
</details>
<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate-
  -b border-slate-700/50">
    Код C++ (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300"
    <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];

```


.....

Тут мы берём всю строку и «прикидываю начну читать строку отсюда, насколько длинная». Это самый быстрый способ найти `<code>Z[i]`, считаешь Z-функцию, и там, где `<code>Z[i]`

`</p>`
`</div>`

`<!-- Аналогия 2 -->`
`<div class="bg-slate-900 p-6 rounded-lg"`
`<div class="absolute -top-3 left-4 border-rose-500 shadow-md">`
Аналогия 2: LLM Самокопиаст
`</div>`
`<p class="text-slate-300 text-sm mb"`
В LLM Z-функция может применять `[i]</code> (где <i>i</i> указывает назад на сама себя.
</p>
</div>
</div>`

`<details class="bg-slate-900/40 rounded-lg"`
`<summary class="p-4 font-bold text-slate-300 border-slate-700/50">`

Пример вычисления (Скрыто)
`</summary>`
`<div class="p-5 text-sm text-slate-300"`
`<p>Тот же пример: abcbcd</p>`
`<ul class="list-disc pl-5 space-y-2"`
`<li><code>Z[0]</code> – не счита`
`<li><code>Z[1]</code> (bcbcd`
`[1] = 0</code></li>`
`<li><code>Z[2]</code> (cabcd`
`</li>`
`<li><code>Z[3]</code> (abcd`
? Нет, в начале 'a', тут 'd'). Совпало 3 бу
`<li><code>Z[4], Z[5], Z[6]</code>`
`</ul>`

```
.....
    <p class="text-sm text-blue-300 italic mb-2">Осн
    <p class="text-lg font-mono text-white">Бор (Тр
  </div>
  <p class="text-slate-300 text-sm">Позволяет найти
    го один проход.</p>
</div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
      Аналогия 1: Паутина (Бор)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      ет – мы падаем по <b>суффиксной ссылке</b> ("
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border-2
      <div class="absolute -top-3 left-4 bg-rose-900
        -500 shadow-md">
        Аналогия 2: Матёшка (Терминальные)
      </div>
      <p class="text-slate-300 text-sm mb-4">Представ
        нашли и "he", потому что оно спрятано внутри
      </div>
    </div>

  <details class="bg-slate-900/40 rounded-lg border b
    <summary class="p-4 font-bold text-slate-400 ou
      r-slate-700/50">
      Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300 space-y-
      <pre class="bg-slate-950 p-4 rounded overfl
int get_link(int v) {
  if (t[v].link == -1) {
    if (v == 0 || t[v].p == 0) t[v].link = 0;
```

```

.....
        <div class="bg-slate-800 p-6 rounded-lg
            <div class="absolute -top-3 left-4
rder border-emerald-500 shadow-md">
                Аналогия 1: Судoku (P vs NP)
            </div>
            <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс <b>NP</b> –
енную сетку (ответ), он БЫСТРО (за класс P)
            </div>
            <!-- Аналогия 2 -->
            <div class="bg-slate-900 p-6 rounded-lg
            <div class="absolute -top-3 left-4
rder border-purple-500 shadow-md">
                Аналогия 2: Машина-переводчик
            </div>
            <p class="text-slate-300 text-sm mb
ь отличный переводчик на английский (Сведен
аче В, а сама В лежит в NP, то В – <b>NP-по
            </div>
        </div>
    </div>
</div>
</section>
}
};

```

ФАЙЛ 29/99: src/data/tickets/ticket6_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМ

```
import { Chapter } from "../../types";
```

```
export const tickets6to13: Chapter[] = [
```

```

    {
        id: "graph-dfs-bfs",
        title: "6. Графы. Представление, DFS, BFS",
    },

```

```

.....
    { /* DFS vs BFS */ }
    <div class="bg-slate-700/50 p-6 rounded-xl border
      <h3 class="text-xl font-bold text-indigo-400"

    <div class="grid grid-cols-1 xl:grid-cols-2
      <!-- Аналогия DFS -->
      <div class="bg-slate-800 p-6 rounded-lg
        <div class="absolute -top-3 left-4 i
        rder border-indigo-500 shadow-md">
          ♂ DFS (Поиск в глубину) = Жадный
        </div>
        <p class="text-slate-300 text-sm mb
          <b>Что делает:</b> Жадный алгоритм
          ую вершину (например, минимальную по номеру
          (рекурсивно) и пробует другой путь.
        </p>
        <ul class="text-xs text-indigo-300"
          <li><b>Где используется:</b></li>
          <li>Топологическая сортировка (
          <li>Поиск мостов и точек сочленен
          <li>Сильно связанные компоненты К
          <li>Поиск циклов и просто провер
        </ul>
      </div>

      <!-- Аналогия BFS -->
      <div class="bg-slate-900 p-6 rounded-lg
        <div class="absolute -top-3 left-4 i
        border border-emerald-500 shadow-md">
          ♂ BFS (Поиск в ширину) = Волна
        </div>
        <p class="text-slate-300 text-sm mb
          <b>Что делает:</b> Концентрические
          Работает через Очередь (FIFO). Продвигается
        </p>
        <ul class="text-xs text-emerald-300"
          <li><b>Где используется:</b></li>
          <li>Кратчайший путь в Невзвешен

```



```

},
{
  id: "top-sort",
  title: "9. Графы. Топологическая сортировка",
  type: "html",
  description: "Разложение задач по порядку выполнения",
  category: "Графы",
  content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500 shadow-md">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        перед.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
            Аналогия 1: Учеба в вузе
          </div>
          <p class="text-slate-300 text-sm mb-4">
            еская сортировка – это расписание, которое
          <p>
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
            Ограничения: Циклы
          </div>
          <p class="text-slate-300 text-sm mb-4">
            получить опыт, нужна работа". Алгоритм выявля

```

```

},
{
  id: "scc-kosaraju",
  title: "10. Графы. Компоненты сильной связности (SCC)",
  type: "html",
  description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
  category: "Графы. Продвинутые",
  content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
    </div>

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">Макро-вершина — это компонент сильной связности (SCC) ориентированного графа, в котором каждая вершина достижима из каждой другой вершины в той же компоненте.
    </div>

    <p class="text-slate-300 text-sm mb-4">
      <strong>«Это просто цикл?» — Нет!</strong>
    </p>

    <ul class="list-disc list-inside text-sm text-slate-300">
      <li>Если у тебя есть цикл <code>A → B → C → A</code>, то это SCC.
      <li>Если ты добавишь вершину <strong>Г</strong> с ребрами <code>A → Г</code> и <code>Г → A</code>, то у тебя появятся четыре вершины — одна большая SCC.
      <li>Правило: <b>Если два цикла имеют хотя бы одну общую вершину, то они образуют одну SCC.</b>
    </ul>

    <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
      <div class="bg-slate-800 p-6 rounded-lg">
        <h4 class="text-sm font-medium text-slate-300">Алгоритм Косарая
        <p class="text-sm text-slate-400">Алгоритм Косарая — это алгоритм для нахождения всех компонент сильной связности (SCC) в ориентированном графе. Он основан на модифицированном алгоритме поиска в глубину (DFS).
        <ol class="list-decimal text-sm text-slate-400">
          <li>Выполнить DFS по графу, помечая вершины по времени захода в них (finishing times).
          <li>Обратить направление всех ребер в графе.
          <li>Выполнить DFS по обратному графу, начиная с вершин, помеченных в первом шаге в порядке убывания времени захода. Каждая такая DFS-компонента будет соответствовать одной SCC.
        </ol>
      </div>
      <div class="bg-slate-800 p-6 rounded-lg">
        <h4 class="text-sm font-medium text-slate-300">Пример
        <p class="text-sm text-slate-400">Рассмотрим граф с 5 вершинами (A, B, C, D, E) и следующими ребрами: A → B, B → C, C → A, C → D, D → E, E → C.
        <pre>graph TD
          A --> B
          B --> C
          C --> A
          C --> D
          D --> E
          E --> C</pre>
        <p>В этом графе есть две SCC: {A, B, C} и {D, E}.
      </div>
    </div>
  </div>
`

```

```

.....
        !</li>
      </ol>
    </div>
  </details>
</div>
</div>
</section>',
},
{
  id: "graph-bridges",
  title: "11. Графы. Мосты",
  type: "html",
  description: "Рёбра, удаление которых расхреначивает",
  category: "Графы. Продвинутое",
  content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-rose-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">
        <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
            Аналогия 1: Единственный мост
          </div>
          <p class="text-slate-300 text-sm mb-4">
            ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
          </div>
        </div>
      </div>
    </div>
  </div>
`

```



```

    </p>
  </div>

  <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
    <div class="bg-slate-800 p-6 rounded-lg border border-rose-500 shadow-md">
      Аналогия: Главный перекресток
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь город, где два района соединены
      её перекроют (удалят вершину) – районы будут
      <strong>хрупкость</strong> системы.
    </p>
  </div>

  <div class="bg-slate-900 p-6 rounded-lg border border-emerald-500 shadow-md">
    Телеком: Центральный свитч
  </div>
  <p class="text-slate-300 text-sm mb-4">
    У тебя несколько компьютеров в одной
    абелем (мостом). Сами свитчи – это точки со
  </p>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
  <summary class="p-4 font-bold text-slate-400">
    Душный мод: Код и корень DFS (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-2">
    <p class="text-blue-400 font-bold">Особ
  </p>
    Для стартовой вершины обхода правил
    то выше неё прыгать некуда). Поэтому для не
```

```

.....
        <li><strong>Как точка сочленения рушит !
        Косарайю, свернув каждую SCC в одну супер-в
        (сделаем неориентированным), то любая <strong>
        о и есть критическая SCC! Если удалить эту
        . Одно слабое звено изолирует целые касты S
    </ul>
</div>
</div>
</section>',
},
{
  id: "graph-euler",
  title: "13. Графы. Эйлеров цикл",
  type: "html",
  description: "Пройти по всем ребрам ровно 1 раз.",
  category: "Графы",
  content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-indigo-400
      <div class="bg-slate-900 p-4 rounded-lg mb-1
        <p class="text-lg text-indigo-300 itali
        <p class="text-xl font-mono text-white"
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg
          <div class="absolute -top-3 left-4
            rder border-emerald-500 shadow-md">
              Аналогия 1: Рисование не отрыв
            </div>
          <p class="text-slate-300 text-sm mb
            е сходится нечетное число линий, значит, из
            а везде должно быть четное число (зашел-выш

```

Универсальное пособие • полный исходный код

```
.....  
    { id: 3, x: 400, y: 150, label: "3" },  
    { id: 4, x: 550, y: 150, label: "4" },  
    { id: 5, x: 700, y: 50, label: "5" },  
    { id: 6, x: 700, y: 250, label: "6" },  
  ];  
  
const edges: Edge[] = [  
  { u: 0, v: 1 },  
  { u: 1, v: 2 },  
  { u: 2, v: 0 },  
  { u: 2, v: 3 },  
  { u: 3, v: 4 }, // 3 and 4 are articulation points (3  
  { u: 4, v: 5 },  
  { u: 5, v: 6 },  
  { u: 6, v: 4 },  
];  
  
// Adjacency list  
const adj: number[][] = Array.from(  
  { length: Object.keys(nodes).length },  
  () => [],  
);  
edges.forEach((e) => {  
  adj[e.u].push(e.v);  
  adj[e.v].push(e.u);  
});  
  
interface StepInfo {  
  desc: string;  
  visited: Set<number>;  
  ap: Set<number>;  
  currentNode: number | null;  
  tin: number[];  
  low: number[];  
}  
  
export const ArticulationPointsViz: React.FC = () => {  
  const [steps, setSteps] = useState<StepInfo[]>([]);
```

.....

```

    if (visited.has(to)) {
        currentLow[v] = Math.min(currentLow[v], currentLow[to]);
        recordStep(
            `Обратное ребро ${v}-${to}. Обновляем low[${v}],
            v,
        );
    } else {
        children++;
        dfs(to, v);
        currentLow[v] = Math.min(currentLow[v], currentLow[to]);

        recordStep(
            `Возврат в ${v} из ${to}. Обновляем low[${v}],
            v,
        );

        if (currentLow[to] >= currentTin[v] && p !== ap.add(v);
            recordStep(
                `Найдена точка сочленения: вершина ${v}],
                v,
            );
        }
    }
}

if (p === -1 && children > 1) {
    ap.add(v);
    recordStep(
        `Корень дерева DFS ${v} имеет >1 детей. Это
        v,
    );
} else if (p === -1) {
    recordStep(
        `Корень дерева DFS ${v} имеет <=1 ребенка, он
        v,
    );
}

```

```

.....
};

return (
  <div className="bg-slate-900 rounded-xl border bord
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Info className="w-5 h-5 text-rose-400" />
        Моделирование поиска точек сочленения
      </h3>

    <div className="flex items-center gap-2 bg-slate
      <button
        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-
        text-white"
      >
        {isPlaying ? (
          <Pause className="w-4 h-4 ml-1" />
        ) : (
          <Play className="w-4 h-4 ml-1" />
        )}
        {isPlaying ? "Пауза " : "Старт "}
      </button>

      <button
        onClick={reset}
        className="flex items-center justify-center
        ors"
      >
        <RotateCcw className="w-4 h-4" />
      </button>
    </div>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-3 g
    <div className="lg:col-span-2 relative h-96 lg:h
      <svg className="w-full h-full" viewBox="0 0 8
        { /* Edges */ }

```

```

.....
      ? "#f43f5e"
      : isVisited
      ? "#34d399"
      : "#334d55";
const r = isAp ? 24 : 20;

return (
  <g key={node.id}>
    <circle
      cx={node.x}
      cy={node.y}
      r={r}
      fill={fill}
      stroke={stroke}
      strokeWidth="3"
      className="transition-all duration-1
    />
    <text
      x={node.x}
      y={node.y}
      dy=".3em"
      textAnchor="middle"
      fill="white"
      fontSize="14px"
      fontWeight="bold"
      className="select-none pointer-events
    >
      {node.label}
    </text>

    {isVisited && (
      <>
        <text
          x={node.x}
          y={node.y + r + 15}
          textAnchor="middle"
          fill="#94a3b8"
          fontSize="10px"

```

```

<div className="mb-4">
  <h5 className="text-xs text-slate-500 font-medium">
    <div className="flex items-center gap-2">
      <div className="w-3 h-3 rounded-full bg-rose-500 border-1px border-slate-500">
        Точка сочленения
      </div>
      <div className="flex items-center gap-2">
        <div className="w-3 h-3 rounded-full bg-rose-500 border-1px border-slate-500">
          Посещена
        </div>
        <div className="flex items-center gap-2">
          <div className="w-3 h-3 rounded-full bg-rose-500 border-1px border-slate-500">
            Текущая
          </div>
          <div className="flex items-center gap-2">
            <div className="w-8 h-1 bg-rose-500 border-1px border-slate-500">
              Мост
            </div>
          </div>
        </div>
      </div>
    </h5>
  </div>

```

```

<div>
  <h5 className="text-xs text-slate-500 font-medium">
    ЛОГ ДЕЙСТВИЙ:
  </h5>
  <div className="space-y-2">
    {steps
      .slice(0, currentStepIndex + 1)
      .reverse()
      .map((step, idx) => (
        <div
          key={idx}
          className={`text-xs p-2 rounded $
xt-slate-500`} `}
        >
          {step.desc}
        </div>
      ))}
  </div>

```

```

.....

export default function BellmanFordViz() {
  const [hasCycle, setHasCycle] = useState(false);
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const runtime = useVizRuntime();
  const [isPlaying, setIsPlaying] = useState(false);

  const nodes: Node[] = [
    { id: 'S', x: 60, y: 150, name: 'База (S)' },
    { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
    { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
    { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
    { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
    { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
    { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
  ];

  const getEdges = (cycle: boolean): Edge[] => {
    const defaultEdges = [
      { u: 'S', v: 'A', w: 4 },
      { u: 'S', v: 'B', w: 5 },
      { u: 'B', v: 'C', w: -2 },
      { u: 'A', v: 'C', w: 1 },
      { u: 'C', v: 'D', w: 3 },
      { u: 'C', v: 'E', w: 4 },
      { u: 'D', v: 'F', w: 2 },
      { u: 'E', v: 'F', w: 1 },
    ];
    if (cycle) {
      return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
    } else {
      return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
    }
  };

  const edges = getEdges(hasCycle);

```



```

.....
    const vDist = dist[edge.v];

    let updated = false;
    if (uDist !== 999 && uDist + edge.w < vDist) {
        dist = { ...dist, [edge.v]: uDist + edge.w };
        prev = { ...prev, [edge.v]: edge.u };
        updated = true;
        anyUpdate = true;
    }

    simulationSteps.push({
        iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
        distances: { ...dist }, previous: { ...prev },
        log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}.
            ${updated ? 'Релаксация успешна! dist[${edge.v}] = ' + (uDist + edge.w) : ''}`,
        activeCodeLine: updated ? 6 : 5,
    });
}

if (!anyUpdate && iter < vCount - 1) {
    simulationSteps.push({
        iteration: iter, checkingEdge: null, distances: { ...dist },
        log: `Ранний выход после итерации ${iter}.`,
        activeCodeLine: 10,
    });
    break;
}
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
    let cycleFound = false;
    for (const edge of edges) {
        if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
            cycleFound = true;
            simulationSteps.push({
                iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
                distances: { ...dist }, previous: { ...prev },
            });
        }
    }
    if (cycleFound) {
        simulationSteps[simulationSteps.length - 1].log +=
            `Обнаружен цикл! dist[${edge.v}] = ${dist[edge.v]} < ${dist[edge.u] + edge.w}`;
    }
}

```

```

.....
const liveW = vizNumber(vars?.w);
const liveDist = vizRecord(vars?.dist);
const compilerLinked = liveI !== null || liveU !== null;
const currentStep: Step = compilerLinked
  ? {
      ...baseStep,
      iteration: liveI ?? 0,
      checkingEdge: liveU && liveV ? { u: liveU, v: liveV },
      distances: Object.fromEntries(nodes.map((node) => [node.id, liveDist[node.id]])),
      log: `Компилятор: i=${liveI ?? "-"}, ребро ${liveU && liveV ? liveU + "-" + liveV : ""}`,
    }
  : baseStep;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border-1px border-slate-700">
    <div className="flex flex-wrap items-center justify-center gap-2">
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-blue-600 text-white rounded-lg">
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white">
            <p className="text-sm text-blue-300">Полный
          </p>
        </div>
      </div>
    </div>

    <div className="flex items-center gap-3">
      <div className="bg-slate-900 p-1 rounded-lg border-1px border-slate-800">
        <button onClick={() => setHasCycle(false)} className="p-1 bg-blue-600 text-white" : 'text-slate-400'>
        <button onClick={() => setHasCycle(true)} className="p-1 bg-blue-600 text-white" : 'text-slate-400'>
      </div>
      <div>
        <button onClick={() => { setIsPlaying(false); }} className="p-1 bg-slate-600 text-white px-3 py-2 rounded-lg">
        <button onClick={() => setIsPlaying(!isPlaying)} className="p-1 bg-slate-600 text-white px-3 py-2 rounded-lg">
        <button onClick={() => { setIsPlaying(false); }} className="p-1 bg-slate-600 text-white px-3 py-2 rounded-lg">
      </div>
    </div>
  </div>
);

```

```

.....
    && edge.v === 'D' ));

    let strokeColor = '#475569';
    let marker = 'url(#arrow-bf-normal)';
    if (isChecking) { strokeColor = '#f59e0b'
    else if (currentStep.hasNegativeCycle &&

    return (
      <g key={idx}>
        <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={
asNegativeCycle && isCyclePart) ? 4 : 2} marker={isChecking ? '4,4' : 'none'} />
        <circle cx={({uNode.x + vNode.x) / 2} cy={({uNode.y + vNode.y) / 2} stroke={
Checking ? '#f59e0b' : isNeg ? '#f43f5e' : '#475569'} strokeDash={isChecking ?
8fa9fc'} fontSize="11" fontWeight="bold" text={isChecking ? 'C' : isNeg ? 'N' : ''} />
      </g>
    );
  });
}

(nodes.map((node) => {
  const dist = currentStep.distances[node.id];
  const isCheckingNode = currentStep.checkingNode === node.id);

  return (
    <g key={node.id} className="transition-leave">
      <circle cx={node.x} cy={node.y} r={isCheckingNode ? '#7dd3fc' : '#64748b'} strokeWid
      <text x={node.x} y={node.y + 5} fill={isCheckingNode ? '#f59e0b' : '#475569'}>{dist}</text>
      <text x={node.x} y={node.y - 28} fill={isCheckingNode ? '#f59e0b' : '#475569'} fontW
ight="bold" textAnchor="middle" className="label">{node.name}</text>
      <text x={node.x} y={node.y + 35} fill={isCheckingNode ? '#f59e0b' : '#475569'}>{
split(' ')[0]}</text>
    </g>
  );
}
)
</svg>

```

$\frac{1}{n} \sum_{i=1}^n x_i = \bar{x}$

```

// Init
steps.push({
  activeLine: 1,
  queue: [],
  visited: [],
  currentNode: null,
  logs: "Инициализация: начинаем с корня (0)."

```

```

.....
    currentNode: null,
    logs: "Очередь пуста. Алгоритм завершен!"
  });

  return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
  const [stepIdx, setStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const step = STEPS[stepIdx];

  const handleNext = useCallback(() => {
    setStepIdx(prev => Math.min(prev + 1, STEPS.length
  }, []));

  // @ts-expect-error зарезервировано под кнопку «назад»
  const handlePrev = useCallback(() => {
    setStepIdx(prev => Math.max(prev - 1, 0));
  }, []);

  useEffect(() => {
    if (!isPlaying) return;
    if (stepIdx >= STEPS.length - 1) {
      setIsPlaying(false);
      return;
    }
    const timer = setTimeout(handleNext, 1200);
    return () => clearTimeout(timer);
  }, [isPlaying, stepIdx, handleNext]);

  return (
    <div className="bg-slate-900 border-2 border-slate-
      <div className="flex flex-col md:flex-row items-c
        <div>

```

```

.....
{EDGES.map((e, idx) => {
  const n1 = NODES.find(n => n.id === e.from)
  const n2 = NODES.find(n => n.id === e.to)
  const isVisited = step.visited.includes(n1)
  return (
    <line
      key={idx}
      x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
      className={`transition-all duration-500`}
    >
  )
})
  </>
);
}}}

{/* Вершины */}
{NODES.map(n => {
  const isCurrent = step.currentNode === n.id
  const isInQueue = step.queue.includes(n.id)
  const isVisited = step.visited.includes(n.id)

  let fill = '#1e293b'; // slate-800
  let stroke = '#475569'; // slate-600
  let scale = 1;
  void scale;

  if (isCurrent) {
    fill = '#3b82f6'; // blue-500
    stroke = '#60a5fa'; // blue-400
    scale = 1.3;
  } else if (isInQueue) {
    fill = '#059669'; // emerald-600
    stroke = '#34d399'; // emerald-400
    scale = 1.1;
  } else if (isVisited) {
    fill = '#0f172a'; // slate-900
    stroke = '#3b82f6'; // blue-500
  }
})
}

```

```

.....
      { line: 1, code: 'queue = Queue()' },
      { line: 2, code: 'queue.push(start); vi' },
      { line: 3, code: ' ' },
      { line: 4, code: 'while not queue.isEmp' },
      { line: 5, code: '  node = queue.pop()' },
      { line: 6, code: '  for neighbor in gra' },
      { line: 7, code: '    if neighbor not i' },
      { line: 8, code: '      visited.add(ne' },
      { line: 9, code: '      queue.push(nei' },
    ].map((cl) => {
      const isActive = step.activeLine === cl
      return (
        <div key={cl.line} className={`px-2 p
d border-l-2 border-blue-400 -ml-0.5` : 'te
        <span className="opacity-40 w-6 inl
        <span className="whitespace-pre">{c
      </div>
    );
  })}
</div>
</div>

<div className="bg-slate-800/80 border border
  <div className="text-sm font-bold text-eme
    <span>Состояние Очереди (Queue):</span>
    <span className="text-xs text-slate-500
  </div>
  <div className="flex items-center gap-2 mi
    {step.queue.length === 0 ? (
      <span className="text-slate-500 text-x
    ) : (
      step.queue.map((qId, i) => (
        <span key={`_${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
          {qId}
        </span>
      ))
    )}
  )}

```



```
.....
    src: floydImg,
    alt: 'Все ко Всем Флойд',
    caption: ' Все связаны со всеми!',
    borderColor: 'border-emerald-500/30',
    captionColor: 'text-emerald-400',
  },
};

export default function ChapterImage({ vizType }: { vizType: string }) {
  const info = imageMap[vizType];
  if (!info) return null;

  return (
    <div>
      <div className={`rounded-2xl overflow-hidden border-2 border-emerald-500`} >
        <img src={info.src} alt={info.alt} className="w-full h-full" />
      </div>
      <p className={`text-center text-xs mt-2 font-bold`} >{info.caption}</p>
    </div>
  );
}
```

ФАЙЛ 34/99: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....

import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";

interface Props {
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
}
```

```

.....
return (
  <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" >
    {prev ? (
      <button
        onClick={() => onGo(prev.id)}
        className="group text-left p-4 rounded-xl bg-white shadow-sm transition-all"
      >
        <div className="flex items-center gap-1.5 text-indigo-400 mb-1">
          <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая тема
        </div>
        <div className="text-sm font-bold text-slate-700">
          {prev.title}
        </div>
      </button>
    ) : (
      <div className="hidden sm:block" />
    )}

    {next && (
      <button
        onClick={() => onGo(next.id)}
        className="group text-right p-4 rounded-xl bg-white shadow-sm transition-all sm:col-start-2"
      >
        <div className="flex items-center justify-end gap-1.5 text-indigo-400 mb-1">
          Следующая тема <ChevronRight className="w-3.5 h-3.5" />
        </div>
        <div className="text-sm font-bold text-slate-700">
          {next.title}
        </div>
      </button>
    )}
  </nav>
);
};

```

```
.....  
import { downloadChapterHtml } from "../utils/exportHtml";
```

```
interface Props {  
  chapter: Chapter;  
  prev?: Chapter;  
  next?: Chapter;  
  index: number;  
  total: number;  
  onGo: (id: string) => void;  
  onOpenSimulator: (vizId: string) => void;  
  /** Перейти во вкладку компилятора – там уже подставлено название */  
  onOpenCompiler?: () => void;  
}
```

```
const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;
```

```
export const ChapterView: React.FC<Props> = ({ chapter,  
  const contentRef = useRef<HTMLDivElement>(null);  
  const [anchor, setAnchor] = useState<HintAnchor | null>(null);  
  const hideTimer = useRef<number | null>(null);  
  const [saving, setSaving] = useState<"idle" | "work">("idle");
```

```
  useTermHints(contentRef, [chapter.id]);
```

```
  const cancelHide = useCallback(() => {  
    if (hideTimer.current) {  
      window.clearTimeout(hideTimer.current);  
      hideTimer.current = null;  
    }  
  }, []);
```

```
  const scheduleHide = useCallback(() => {  
    cancelHide();  
    hideTimer.current = window.setTimeout(() => setAnchor(anchor), 1000);  
  }, [cancelHide]);
```

```
  useEffect(() => {  
    setAnchor(null);  
  }, []);
```

```

.....
    if (!el) return;
    e.preventDefault();
    if (anchor && anchor.termId === el.dataset.termId)
    else open(el);
};

const handleFocus = (e: React.FocusEvent) => {
    const el = (e.target as HTMLElement).closest<HTMLElement>
    if (el) open(el);
};

const viz = getViz(chapter.id);

return (
    <>
    <article className="chapter-body">
        /* панель загрузки: сохранить именно эту страницу
        <div className="flex flex-wrap items-center gap-2">
            <button
                onClick={saveHtml}
                disabled={saving === "work"}
                title="Сохранить эту тему одним автономным файлом"
                className="flex items-center gap-1.5 text-x-small font-medium border-1px
                300 hover:text-white hover:border-emerald-500"
            >
                {saving === "work" ? (
                    <Loader2 className="w-3.5 h-3.5 animate-spin" />
                ) : saving === "done" ? (
                    <Check className="w-3.5 h-3.5 text-emerald-500" />
                ) : (
                    <Download className="w-3.5 h-3.5" />
                )}
            {saving === "done" ? "Файл сохранён" : "Скачать"}
            </button>
            <button
                onClick={() => window.print()}
                title="Распечатать или сохранить в PDF средствами браузера"
                className="flex items-center gap-1.5 text-x-small font-medium border-1px

```

```

.....
        >
        <Maximize2 className="w-3.5 h-3.5" />
    </button>
    {onOpenCompiler && {
        <button
            onClick={onOpenCompiler}
            title="Открыть Python: код выполняе
рации"
            className="flex items-center gap-1.
t-emerald-400 hover:text-white hover:border
        >
            <Terminal className="w-3.5 h-3.5" />
        </button>
    }}
</div>
</div>
{viz.hint && <p className="text-xs text-sla
<VizChapterContext.Provider value={chapter.
    <viz.Component />
</VizChapterContext.Provider>
</section>
)}

<RelatedTickets chapterId={chapter.id} onGo={on

<ChapterNav prev={prev} next={next} index={inde
</article>

<TermPopover
    anchor={anchor}
    onClose={() => setAnchor(null)}
    onOpenSimulator={(id) => {
        setAnchor(null);
        onOpenSimulator(id);
    }}
    onCardEnter={cancelHide}
    onCardLeave={scheduleHide}
/>

```

```

.....
const EDGES: [number, number][] = [
  [0, 1], [1, 2], [2, 3], [3, 4], [4, 0],
  [0, 5], [1, 5], [2, 5], [3, 5], [4, 5],
  [2, 6], [3, 6], [5, 6],
];

const ADJ: number[][] = (() => {
  const adj: number[][] = NODES.map(() => []);
  EDGES.forEach(([u, v]) => {
    adj[u].push(v);
    adj[v].push(u);
  });
  return adj;
})();

const COLORS = [
  { name: "красный", hex: "#ef4444" },
  { name: "синий", hex: "#3b82f6" },
  { name: "зелёный", hex: "#22c55e" },
  { name: "жёлтый", hex: "#eab308" },
];

const CONFLICT = "#e11d48";

/** Жадная раскраска по возрастанию номеров — честный у
function greedy(): number[] {
  const color: number[] = NODES.map(() => -1);
  for (let v = 0; v < NODES.length; v++) {
    const used = new Set(ADJ[v].map((u) => color[u]).filter(c => c !== -1));
    let c = 0;
    while (used.has(c)) c++;
    color[v] = c;
  }
  return color;
}

export function ColoringViz() {
  const [colors, setColors] = useState<number[]>(NODES.map(() => -1));

```

```

.....
    const i = NODES.findIndex((n) => n.label === liveVNum)
    if (i >= 0) return i;
  }
  if (liveVNum !== null && liveVNum >= 0 && liveVNum <= 10) {
    return null;
  }
})();

const conflicts = useMemo(() => {
  const bad = new Set<number>();
  EDGES.forEach(([u, v]) => {
    const cu = shownColors[u];
    const cv = shownColors[v];
    if (cu >= 0 && cu === cv) {
      bad.add(u);
      bad.add(v);
    }
  });
  return bad;
}, [shownColors]);

const usedColors = useMemo(
  () => new Set(shownColors.filter((c) => c >= 0)).size,
  [shownColors]
);

const paint = (id: number) => {
  const next = [...colors];
  next[id] = next[id] === brush ? -1 : brush;
  setColors(next);
};

return (
  <div className="bg-slate-800/90 p-5 rounded-2xl border">
    <div className="flex flex-wrap items-center justify-center">
      <div>
        <h3 className="text-lg font-bold text-white">
          <p className="text-xs text-slate-400">
            Смежные вершины не должны совпадать по цветам
          </p>
        </h3>
      </div>
    </div>
  </div>
);

```

```

">
    {n.label}
  </text>
  {c >= 0 && {
    <text x={n.x} y={n.y - 20} textAnchor=
vents-none">
      color[{n.id}]= {c}
    </text>
  }}
</g>
  );
}}
</svg>
{compilerLinked && {
  <div className="absolute top-2 right-2 bg-emerald-300 z-20">
    Python красит: {liveVLabel ?? ({liveVNum
  </div>
})
{conflicts.size > 0 && {
  <div className="absolute bottom-2 left-1/2
-[11px] text-rose-300 z-20">
    ▲ Конфликт: смежные вершины одного цвета
  </div>
})
</div>

<div className="space-y-3">
  <div className="bg-slate-950 rounded-xl border
  <div className="text-[10px] uppercase track
    <Palette className="w-3 h-3" /> Кисть
  </div>
  <div className="grid grid-cols-4 gap-2">
    {COLORS.map(({c, i}) => {
      <button
        key={c.name}
        onClick={() => setBrush(i)}
        title={c.name}

```


Универсальное пособие * полный исходный код

```
.....  
* Тема 24 «Классы сложности, сведение задач».  
* Диаграмма вложенности классов  $P \subseteq NP \subseteq PSPACE \subseteq EXPT$ .  
* полиномиальных сведений  $A \leq_p B$ : если B решается легко  
*/
```

```
type ClassId = "P" | "NP" | "PSPACE" | "EXPTIME";
```

```
interface Problem {  
    id: string;  
    name: string;  
    cls: ClassId;  
    x: number;  
    y: number;  
    npComplete?: boolean;  
}
```

```
const PROBLEMS: Problem[] = [  
    { id: "sort", name: "сортировка\( $n \log n$ \)", cls: "P", x: 150, y: 100 },  
    { id: "mst", name: "остов (Краскал)", cls: "P", x: 150, y: 150 },  
    { id: "2sat", name: "2-SAT", cls: "P", x: 115, y: 185 },  
    { id: "shortpath", name: "Дейкстра / Флойд", cls: "P", x: 150, y: 220 },  
    { id: "sat", name: "3-SAT", cls: "NP", x: 330, y: 120 },  
    { id: "vc", name: "Vertex Cover", cls: "NP", x: 385, y: 150 },  
    { id: "ham", name: "Гамильтонов цикл", cls: "NP", x: 385, y: 185 },  
    { id: "knap", name: "Рюкзак (точный)", cls: "NP", x: 385, y: 220 },  
    { id: "circuit", name: "QBF / игра на формуле", cls: "EXPTIME", x: 385, y: 255 },  
    { id: "chess", name: "обобщённые шахматы", cls: "EXPTIME", x: 385, y: 290 },  
];
```

```
const REDUCTIONS: [string, string, string][] = [  
    ["sat", "vc", "3-SAT  $\leq_p$  VC"],  
    ["sat", "ham", "3-SAT  $\leq_p$  Гамильтон"],  
    ["vc", "knap", "VC  $\leq_p$  Рюкзак"],  
];
```

```
const CLASS_META: Record<ClassId, { title: string; color: string; note: string }> = {  
    P: { title: "P", color: "#34d399", note: "решаются за полиномиальное время" },  
    NP: { title: "NP", color: "#fbbf24", note: "решение не гарантируется за полиномиальное время" },  
    PSPACE: { title: "PSPACE", color: "#fbbf24", note: "решение не гарантируется за полиномиальное пространство" },  
    EXPTIME: { title: "EXPTIME", color: "#fbbf24", note: "решение не гарантируется за экспоненциальное время" },  
};
```

```

.....

const activeCls: ClassId | null =
  compilerLinked && liveCls
    ? (Object.keys(CLASS_META).find((k) => k.toLowerCase === selected)
      : selected;

const problemHit = (p: Problem) => {
  if (compilerLinked && liveTask) return p.name.toLowerCase === selectedProblem;
  return selectedProblem === p.id;
};

const infoText = (() => {
  if (compilerLinked && liveTask) {
    const p = PROBLEMS.find((q) => q.name.toLowerCase === selectedProblem);
    if (p) return `${p.name.replace("\n", " ")} ∈ ${p.cls}`;
  }
  if (compilerLinked && liveCls) {
    const cls = (activeCls ?? "NP") as ClassId;
    return CLASS_INFO[cls];
  }
  if (selectedProblem) {
    const p = PROBLEMS.find((q) => q.id === selectedProblem);
    return `${p.name.replace("\n", " ")} ∈ ${p.cls}${p.cls === selectedProblem ? "" : ".cls"}`;
  }
  if (activeCls) return CLASS_INFO[activeCls as ClassId];
  return "Кликни на класс или задачу. «A ≤p B» = «A не ≤p B»";
})();

const reset = () => {
  setSelected(null);
  setSelectedProblem(null);
};

return (
  <div className="bg-slate-800/90 p-5 rounded-2xl border" >
    <div className="flex flex-wrap items-center justify-center" >
      <div>

```

```

.....
    const hot = compilerLinked && liveRedPair
      ? (liveRedPair[0] && a.name.toLowerCase
        liveRedPair[1] && b.name.toLowerCase
        : selectedProblem === from || selectedP
    const mx = (a.x + b.x) / 2;
    const my = (a.y + b.y) / 2 - 4;
    return (
      <g key={`${from}-${to}`} opacity={hot ?
        <line x1={a.x} y1={a.y} x2={b.x} y2={b
ay="5 4" markerEnd="url(#arrowhead)" />
        <text x={mx} y={my} textAnchor="middle
text>
      </g>
    );
  }}}
<defs>
  <marker id="arrowhead" markerWidth="7" ma
    <polygon points="0 0, 7 2.5, 0 5" fill=
  </marker>
</defs>

{PROBLEMS.map((p) => {
  const dim = activeCls !== null && activeC
  const hit = problemHit(p);
  return (
    <g key={p.id} onClick={() => setSelected
      <rect
        x={p.x - 46} y={p.y - 13} width={92
        fill={hit ? "#c2410c" : p.npCompleto
        stroke={hit ? "#fb923c" : CLASS_MET
        strokeWidth={hit ? 2.5 : 1.2}
      />
      {p.name.split("\n").map((line, li, ar
        <text key={li} x={p.x} y={p.y + 4 +
ontWeight="600">
          {line}
        </text>
      )}}
    )}}

```


Универсальное пособие • полный исходный код

```
.....  
const push = (f: Omit<StepFrame, "comp" | "count"> &  
  frames.push({ comp: [...comp], count, ...f }));  
  
push({ queue: [], current: null, scanning: null, log:  
  ной вершины." }));  
  
for (let s = 0; s < NODES.length; s++) {  
  if (comp[s] !== null) continue;  
  count += 1;  
  const queue = [s];  
  comp[s] = count - 1;  
  push({ queue: [...queue], current: s, scanning: null,  
    арует с неё.` }));  
  while (queue.length) {  
    const v = queue.shift()!;  
    push({ queue: [...queue], current: v, scanning: null,  
      й.` }));  
    for (const to of ADJ[v]) {  
      if (comp[to] === null) {  
        comp[to] = comp[v];  
        queue.push(to);  
        push({ queue: [...queue], current: v, scanning:  
          {comp[v]}, в очередь.` }));  
      } else {  
        push({ queue: [...queue], current: v, scanning:  
          });  
      }  
    }  
  }  
  push({ queue: [], current: null, scanning: null, log:  
});  
push({ queue: [], current: null, scanning: null, log:  
  });  
return frames;  
}  
  
const FRAMES = buildFrames();
```

```

.....
    return idx >= 0 ? idx : null;
  }
  return liveVNum !== null && liveVNum >= 0 && liveVNum <= 100;
};
const liveCurrent = liveIndexOf();

const liveCompOf = (n: V): number | null => {
  if (liveCompByLabel && n.label in liveCompByLabel)
    const c = vizNumber(liveCompByLabel[n.label]);
    if (c !== null) return c;
}
  if (liveCompByLabel && String(n.id) in liveCompByLabel)
    const c = vizNumber(liveCompByLabel[String(n.id)]);
    if (c !== null) return c;
}
  return frame.comp[n.id];
};
const liveQueueLabels = (liveQ
  ? liveQ.map((x) => (typeof x === "string" ? x : vizNumber(x)))
  : null
)?.filter(Boolean);
const displayCount = liveCount ?? frame.count;
const displayLog = compilerLinked
  ? `  Компилятор: v=${liveVLabel ?? liveVNum ?? "-"}
    ES[q].label)).join(", ")`
  : frame.log;

const shownCurrent = compilerLinked ? liveCurrent : null;
const shownScanning = compilerLinked ? null : frame.scanning;
const shownQueue: number[] = liveQueueLabels
  ? liveQueueLabels
    .map((lab) => NODES.findIndex((n) => n.label === lab))
    .filter((i) => i >= 0)
  : frame.queue;

return (
  <div className="bg-slate-800/90 p-5 rounded-2xl border"
    <div className="flex flex-wrap items-center justify-between"

```

```

<div className="grid grid-cols-1 lg:grid-cols-[1fr 1fr]">
  <div className="bg-slate-950 rounded-xl border border-white/10">
    <div className="absolute inset-0 bg-[radial-gradient(circle,white 1px,transparent 1px)]" style="background-size: 16px 16px; background-position: center; background-image: linear-gradient(to right, transparent 48%, white 48% 52%, transparent 52%), linear-gradient(to bottom, transparent 48%, white 48% 52%, transparent 52%);">
      <svg className="w-full h-[320px] relative z-10" style="background-color: #000000; color: #FFFFFF;">
        {EDGES.map(({u, v}, i) => {
          const a = NODES[u];
          const b = NODES[v];
          const sameComp = liveCompOf(a) !== null && liveCompOf(b) !== null && liveCompOf(a) === liveCompOf(b);
          return (
            <line
              key={i}
              x1={a.x} y1={a.y} x2={b.x} y2={b.y}
              stroke={sameComp ? (COMP_COLORS[liveCompOf(a)] || COMP_COLORS[liveCompOf(b)]) : "#ccc"}
              strokeWidth={sameComp ? 3.5 : 2}
              opacity={sameComp ? 0.9 : 0.6}
            />
          );
        })}
        {NODES.map((n) => {
          const c = liveCompOf(n);
          const isCurrent = shownCurrent === n.id;
          const isScanned = shownScanning === n.id;
          const inQueue = shownQueue.includes(n.id);
          const color = c === null ? "#334155" : COMP_COLORS[c] || "#ccc";
          return (
            <g key={n.id}>
              {(isCurrent || isScanned) && (
                <circle cx={n.x} cy={n.y} r={18} fill={color} stroke="white" stroke-width="2" style={isCurrent ? {opacity: 1} : {opacity: 0.5}}
              )}
              <circle cx={n.x} cy={n.y} r={13} fill={color} stroke="white" stroke-width="2" style={isScanned ? {opacity: 1} : {opacity: 0.5}}
              <text x={n.x} y={n.y + 4} textAnchor="center" font-size="10" font-weight="bold">{n.label}</text>
              <text x={n.x} y={n.y - 18} textAnchor="center" font-size="10">{c === null ? "-" : `c${c}`}</text>
            </g>
          );
        })}
      </svg>
    </div>
  </div>
</div>

```

```

.....
        LORS.length] ]}]>
            <div className="text-slate-400">{n.
            <div>{c === null ? "-" : c}</div>
            </div>
        });
    ]}]
    </div>
</div>
<div className="bg-slate-900/60 rounded-xl bo
    {displayLog}
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 39/99: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

.....
import { useState, useEffect, useRef } from "react";
import { useVizStepSync, vizNumber, vizString } from '
import { Undo, Play, Pause, SkipForward, RefreshCw } fr

```

```

interface CodeLine {
  line: string;
  highlight: string;
}

```

```

const CODE_LINES: CodeLine[] = [
  { line: "void dfs(int v, int p = -1) {", highlight: "n
  { line: "    visited[v] = true;", highlight: "normal"
  { line: "    tin[v] = low[v] = timer++;", highlight:
  { line: "", highlight: "normal" },
  { line: "    for (int to : adj[v]) {", highlight: "no

```



```

.....

const GRAPH_EDGES = [
  { u: 0, v: 1, key: "0-1" },
  { u: 1, v: 2, key: "1-2" },
  { u: 2, v: 0, key: "0-2" },
  { u: 2, v: 3, key: "2-3" },
  { u: 3, v: 4, key: "3-4" },
  { u: 4, v: 5, key: "4-5" },
  { u: 5, v: 3, key: "3-5" },
  { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
  {
    currentNode: 0, visitedIndices: [0],
    tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
    activeEdge: null, backEdge: null,
    log: "Входим в A. tin[A]=1, low[A]=1.",
    activeCodeLine: [0,1,2]
  },
  {
    currentNode: 1, visitedIndices: [0,1],
    tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
    activeEdge: [0,1], backEdge: null,
    log: "Идём (A,B). B впервые. tin[B]=2, low[B]=2.",
    activeCodeLine: [4,5,7,10,11]
  },
  {
    currentNode: 6, visitedIndices: [0,1,6],
    tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
    activeEdge: [1,6], backEdge: null,
    log: "Из B идём в G. tin[G]=3, low[G]=3.",
    activeCodeLine: [4,5,7,10,11]
  },
  {
    currentNode: 1, visitedIndices: [0,1,6],
    tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
    activeEdge: null, backEdge: null,

```

```

.....
{
  currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
  tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
  activeEdge: null, backEdge: [5,3],
  log: "Из F видим D (посещён). Обратное ребро (F,D).",
  activeCodeLine: [7,8,9]
},
{
  currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
  tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
  activeEdge: null, backEdge: null,
  log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) — не мо",
  activeCodeLine: [11,13,16,17]
},
{
  currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
  tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
  activeEdge: null, backEdge: null,
  log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) — не мо",
  activeCodeLine: [11,13,16,17]
},
{
  currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
  tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
  activeEdge: null, backEdge: null,
  log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) —",
  activeCodeLine: [13,14,15,16,17]
},
{
  currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
  tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
  activeEdge: null, backEdge: null,
  log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C)",
  activeCodeLine: [11,13,16,17]
},
{
  currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
  tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},

```

```

.....
    if (prev >= DFS_STEPS.length - 1) { setDfsAuto(
      return prev + 1;
    });
  }, 2500);
}
return () => { if (dfsTimer.current) clearInterval(
}, [dfsAuto]);

const step = DFS_STEPS[dfsIdx];

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">
    <div className="flex items-center gap-1 bg-slate-700 rounded"
      <button onClick={() => { setDfsAuto(false); s
        disabled={dfsIdx === 0}
        className="p-1.5 hover:bg-slate-700 rounded
        <Undo className="h-3.5 w-3.5" />
      </button>
      <button onClick={() => setDfsAuto(!dfsAuto)}
        className={`px-2.5 py-1 rounded text-[10px]
          dfsAuto ? "bg-rose-600 text-white" : "bg-
        }`} >
        {dfsAuto ? <<Pause className="h-3 w-3" />
      </button>
      <button onClick={() => { setDfsAuto(false); s
        disabled={dfsIdx === DFS_STEPS.length-1}
        className="p-1.5 hover:bg-slate-700 rounded
        <SkipForward className="h-3.5 w-3.5" />
      </button>
      <button onClick={() => { setDfsAuto(false); s
        className="p-1.5 hover:bg-slate-700 rounded
        <RefreshCw className="h-3.5 w-3.5" />
      </button>
    </div>
  </div>

```



```
.....
    <span className="font-mono font-bold text
      {step.bridges.length > 0 ? step.bridges
    </span>
  </div>
</div>
</div>
</div>
</div>
);
}
```

.....
ФАЙЛ 40/99: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
.....
import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber, vizRecord,

interface Node { id: string; x: number; y: number; name
interface Edge { u: string; v: string; w: number; }
interface Step {
  currentNode: string | null;
  checkingEdge: { u: string; v: string } | null;
  distances: { [key: string]: number };
  visited: { [key: string]: boolean };
  previous: { [key: string]: string | null };
  log: string;
  activeCodeLine: number;
}
```

```
export default function DijkstraViz() {
  const nodes: Node[] = [
    { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
    { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
    { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
    { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist: Record<string, number> = { A: 0, B: 999 };
  const visited: Record<string, boolean> = { A: false };
  const prev: Record<string, string | null> = { A: null };

  simulationSteps.push({
    currentNode: null, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: ' Дейкстра готов к доставке. Начинаем из Пикет-Пост-Оффиса',
    activeCodeLine: 2,
  });

  for (let iter = 0; iter < nodes.length; iter++) {
    let minD = 999;
    let u = null;
    for (const n of nodes) {
      if (!visited[n.id] && dist[n.id] < minD) {
        minD = dist[n.id];
        u = n.id;
      }
    }

    if (!u) break;

    visited[u] = true;
    simulationSteps.push({
      currentNode: u, checkingEdge: null,
      distances: { ...dist }, visited: { ...visited },
      log: ` Выбираем вершину ${u} с мин. непосещёнными соседями`,
      activeCodeLine: 5,
    });
  }
});

```

```

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => {
      setCurrentStepIndex((prev) => prev + 1);
    }, 1500);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;

if (!baseStep) return <div className="text-white">3arg</div>;
const vars = runtime?.variables;
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveDist = vizRecord(vars?.dist);
const livePrev = vizRecord(vars?.prev);
const liveDone = vizArray(vars?.done) ?? vizArray(vars?.done);
const compilerLinked = liveU !== null || liveV !== null;
const currentStep: Step = compilerLinked
  ? {
    ...baseStep,
    currentNode: liveU,
    checkingEdge: liveU && liveV ? { u: liveU, v: liveV } : null,
    distances: Object.fromEntries(nodes.map((node) => [node, 0])),
    previous: Object.fromEntries(nodes.map((node) => [node, null])),
    visited: Object.fromEntries(nodes.map((node) => [node, false])),
    log: `Компилятор: u=${liveU ?? "-"}, v=${liveV ?? "-"}, dist=${liveDist}, prev=${livePrev}, done=${liveDone}`
  }
  : baseStep;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
    {/* Шапка симулятора */}
    <div className="flex flex-wrap items-center justify-between">

```

```

    { /* Граф */ }
    <div className="w-full lg:w-2/3 bg-indigo-950/20
      justify-center min-h-[400px]">
      <div className="absolute top-3 left-3 bg-indigo-950/20
        font-bold shadow-sm">
        Шаг {currentStepIndex + 1} из {steps.length}
      </div>

      <svg className="w-full h-auto max-h-[500px]">
        <defs>
          <marker id="arrow-dh-normal" markerWidth="10"
            <path d="M0,0 L6,3 L0,6 z" fill="#475568" />
          </marker>
          <marker id="arrow-dh-active" markerWidth="10"
            <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
          </marker>
          <marker id="arrow-dh-opt" markerWidth="6"
            <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
          </marker>
        </defs>
        { /* Рёбра */ }
        {edges.map((edge, idx) => {
          const uNode = nodes.find((n) => n.id === edge.u);
          const vNode = nodes.find((n) => n.id === edge.v);
          const isChecking =
            currentStep.checkingEdge &&
            (currentStep.checkingEdge.u === edge.u &&
              currentStep.checkingEdge.v === edge.v);

          const isOptimal = currentStep.previous[edge.id] === 'opt';

          let marker = 'url(#arrow-dh-normal)';
          if (isChecking) marker = 'url(#arrow-dh-active)';
          else if (isOptimal) marker = 'url(#arrow-dh-opt)';

          return (
            <g key={idx}>
              <line
                x1={uNode.x} y1={uNode.y}
                x2={vNode.x} y2={vNode.y}
                stroke={isChecking ? '#f59e0b' : isOptimal ? '#10b981' : '#475568'}
                stroke-width={isOptimal ? 2 : 1}
              />
            </g>
          );
        })}
      </svg>
    </div>
  </div>
</div>

```



```

.....
        />
        <text x={node.x} y={node.y + 5} fill=
            {node.id}
        </text>
        <text x={node.x} y={node.y - 28} fill=
"middle" className="bg-indigo-950 px-1 py-0
            {dist === 999 ? '∞' : `d=${dist}`}
        </text>
        <text x={node.x} y={node.y + 35} fill=
            {node.name.split(' ')[0]}
        </text>
    </g>
    );
  })}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded-2xl">
  <p className="font-semibold text-amber-400">
  <p className="min-h-[40px] flex items-center">
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/50">
  <h4 className="text-lg font-bold text-emerald-400">
  <div className="space-y-2.5 flex-1 overflow-y-hidden">
    {Object.keys(adjList).map((u) => {
      const isCurrentNode = currentStep.currentNode === u;
      return (
        <div key={u} className={`p-3 rounded-lg ${
          isCurrentNode ? 'bg-emerald-900/60' : 'bg-slate-300'
        }`} >
          <div className="flex items-center gap-2">
            <span className={`px-2 py-0.5 rounded` +
te-300'}`}>
              {u}
            </span>
            <span className="text-slate-400">→</span>

```

```

.....
      {nodes.map((n) => {
        const dist = currentStep.distances[n.id]
        const prev = currentStep.previous[n.id]
        const vis = currentStep.visited[n.id]
        const isCurr = currentStep.currentNode === n.id

        return (
          <tr key={n.id} className={`border-1
            isCurr ? 'bg-indigo-950/60 font-medium' : ''}>
            <td className="py-2.5 px-3 flex items-center">
              <span className={`w-2 h-2 rounded-full ${vis ? 'bg-indigo-500' : 'bg-gray-300'} mr-2`} />
              {n.name}
            </td>
            <td className="py-2.5 px-3 font-medium">
              {dist === 999 ? '∞' : dist}
            </td>
            <td className="py-2.5 px-3 text-gray-500">
              {prev || '-'}
            </td>
          </tr>
        );
      })}
    </tbody>
  </table>
</div>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 rounded-lg p-4">
  <div>
    <h4 className="text-lg font-bold text-slate-200">Алгоритм
    <div className="bg-slate-950/80 rounded-lg p-4">
      {codeLines.map((item) => {
        const isActive = currentStep.activeCodeLine === item.line
        return (
          <div key={item.line} className={`py-1 px-2 ${isActive ? 'bg-slate-900/40' : ''}`>

```

Универсальное пособие • полный исходный код

```
.....  
  /* 2 */ "    if (n in memo) return memo[n];",  
  /* 3 */ "    if (n <= 2) return 1;",  
  /* 4 */ "    memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);",  
  /* 5 */ "    return memo[n];",  
  /* 6 */ "}"  
};
```

```
export function DPVisualizer() {  
  const [targetN, setTargetN] = useState<number>(5);  
  const [steps, setSteps] = useState<DPStep[]>([]);  
  const [currentStepIndex, setCurrentStepIndex] = useState(0);  
  const runtime = useVizRuntime();  
  const [isPlaying, setIsPlaying] = useState<boolean>(false);  
  const [speed] = useState<number>(800);  
  const [activeTab, setActiveTab] = useState<'table' | 'graph'>('table');
```

```
  useEffect(() => {  
    const generatedSteps: DPStep[] = [];  
    let stepId = 0;  
    const memoMap: { [key: number]: number } = {};  
  
    function simulateFib(n: number): number {  
      generatedSteps.push({  
        id: stepId++,  
        n,  
        action: 'call',  
        desc: `Вызов fibMemo(${n}). Проверяем наличие в кэше`,  
        codeLine: 2,  
        memoState: { ...memoMap }  
      });  
  
      if (n in memoMap) {  
        generatedSteps.push({  
          id: stepId++,  
          n,  
          action: 'memo_hit',  
          desc: `✓ Кэш-хит! fibMemo(${n}) уже посчитано`,  
          codeLine: 2,  
          memoState: { ...memoMap }  
        });  
      }  
      return memoMap[n] = simulateFib(n - 1) + simulateFib(n - 2);  
    }  
  });
```

```
  if (n in memoMap) {  
    generatedSteps.push({  
      id: stepId++,  
      n,  
      action: 'memo_hit',  
      desc: `✓ Кэш-хит! fibMemo(${n}) уже посчитано`,  
      codeLine: 2,  
      memoState: { ...memoMap }  
    });  
  }  
  return {  
    targetN,  
    setTargetN,  
    steps,  
    setSteps,  
    currentStepIndex,  
    setCurrentStepIndex,  
    runtime,  
    isPlaying,  
    setIsPlaying,  
    speed,  
    activeTab,  
    setActiveTab  
  };  
}
```

.....

```

    return memoMap[n];
  }

  const finalRes = simulateFib(targetN);

  generatedSteps.push({
    id: stepId++,
    n: targetN,
    action: 'done',
    desc: `  Расчет завершен! fibMemo(${targetN}) = $
    codeLine: 6,
    memoState: { ...memoMap }
  });

  setSteps(generatedSteps);
  setCurrentStepIndex(0);
  setIsPlaying(false);
}, [targetN]);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length -
    timer = setTimeout(() => {
      setCurrentStepIndex(prev => prev + 1);
    }, speed);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

const baseStep = steps[currentStepIndex] || null;
const vars = runtime?.variables;
const liveK = vizNumber(vars?.k) ?? vizNumber(vars?.n
const liveMemo = vizRecord(vars?.memo);
const compilerLinked = liveK !== null || !!liveMemo;
const memoState = liveMemo

```

```

>
  <option value={4}>N = 4</option>
  <option value={5}>N = 5</option>
  <option value={6}>N = 6</option>
  <option value={7}>N = 7</option>
</select>
</div>

<div className="flex items-center bg-slate-950" >
  <button
    onClick={() => { setIsPlaying(false); set
    className="p-2 text-slate-400 hover:text-
    title="Сбросить"
  >
    <RotateCcw className="w-4 h-4" />
  </button>
  <button
    onClick={() => { setIsPlaying(false); set
    disabled={currentStepIndex === 0}
    className="p-2 text-slate-400 hover:text-
    title="Вар назад"
  >
    <ChevronLeft className="w-5 h-5" />
  </button>
  <button
    onClick={() => setIsPlaying(!isPlaying)}
    className={`flex items-center gap-1.5 px-
      isPlaying
        ? 'bg-amber-500 text-slate-950 hover:
        : 'bg-emerald-600 text-white hover:bg
    }`}
  >
    {isPlaying ? <Pause className="w-4 h-4" />
    {isPlaying ? 'Пауза' : 'Пуск'}}
  </button>
  <button
    onClick={() => { setIsPlaying(false); set
    disabled={currentStepIndex === steps.leng

```

```

{currentStep && (
  <div className="mt-6 pt-4 border-t border-slate-400">
    <div className="text-sm font-medium text-slate-400">
      <span className="inline-block w-2 h-2 rounded-full bg-slate-400"></span>
      <span>{currentStep.desc}</span>
    </div>
    <div className="text-xs font-mono bg-slate-400 p-2">
      Текущий N: <strong className="text-emerald-400">{currentStep.n}</strong>
    </div>
  </div>
)}
</div>

{/* Tabs */}
<div className="flex items-center gap-2 border-b border-slate-400">
  <button
    onClick={() => setActiveTab('table')}
    className={`flex items-center gap-2 px-6 py-3 border rounded-md
      ${activeTab === 'table'
        ? 'border-emerald-500 text-emerald-400 bg-slate-400'
        : 'border-transparent text-slate-400 hover:text-slate-300'}
    `}
  >
    <Eye className="w-4 h-4" />
    Степ-бай-степ состояние
  </button>
  <button
    onClick={() => setActiveTab('code')}
    className={`flex items-center gap-2 px-6 py-3 border rounded-md
      ${activeTab === 'code'
        ? 'border-emerald-500 text-emerald-400 bg-slate-400'
        : 'border-transparent text-slate-400 hover:text-slate-300'}
    `}
  >
    <Code className="w-4 h-4" />
    Интерактивный код
  </button>
</div>

```

```

        }` }
      >
      <span className="w-8 text-xs text-slate-500">
        <span>{line}</span>
      </div>
    );
  }}}
</pre>
<div className="p-6 bg-slate-900/50 border-slate-800">
  <span className="p-1.5 bg-emerald-500/20 rounded">
    <span>{currentStep?.desc}</span>
  </div>
</div>
)}
</div>

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-800">
  <div className="flex items-center justify-between">
    <span>Прогресс выполнения</span>
    <span>Шаг {currentStepIndex + 1} из {steps.length}</span>
  </div>
  <div className="h-2 bg-slate-950 rounded-full overflow-hidden">
    <div
      className="h-full bg-gradient-to-r from-emerald-500 to-emerald-400"
      style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
    ></div>
  </div>
</div>
</div>
);
}

```

```

.....
const livePath = useMemo(() => {
  if (!compilerLinked) return null;
  const ids = livePathLabels!
    .map((lab) => C1_NODES.find((n) => n.label === lab)
    .filter((x): x is number => x !== undefined);
  return ids;
}, [compilerLinked, livePathLabels]);
const shownPath = livePath ?? c1Path;
const shownVisited = useMemo(() => {
  if (livePath) {
    const keys: string[] = [];
    for (let i = 1; i < livePath.length; i++) {
      const a = livePath[i - 1];
      const b = livePath[i];
      if (C1_EDGES.some((e) => (e.u === a && e.v === b))) {
        keys.push(`${Math.min(a, b)}-${Math.max(a, b)}`);
      }
    }
    return keys;
  }
  return c1VisitedEdges;
}, [livePath, c1VisitedEdges]);

const degrees = useMemo(
  () => C1_NODES.map((n) => ({ label: n.label, deg: C1_NODES[n.label].deg })),
  []
);

const clickC1 = (vId: number) => {
  if (c1Done && c1Path.length > 0) return;
  if (c1Path.length === 0) {
    setC1Path([vId]);
    setC1Msg("Старт! Кликай соседние вершины, чтобы продолжить");
    return;
  }
  const last = c1Path[c1Path.length - 1];
  const edge = C1_EDGES.find(e => (e.u === last && e.v !== last));
  if (!edge) {
    return;
  }
  const next = edge.v === last ? edge.u : edge.v;
  setC1Path(c1Path.concat(next));
  if (c1Path.length === C1_NODES.length) {
    setC1Done(true);
    return;
  }
};

```



```

.....
<div className="bg-slate-950 rounded-xl p-4 border
  -hidden">
  <div className="absolute inset-0 bg-[radial-gra
  <svg className="w-full h-[260px] z-10 relative"
    {C1_EDGES.map((e, i) => {
      const u = C1_NODES.find(n => n.id === e.u)!
      const v = C1_NODES.find(n => n.id === e.v)!
      const key = `${Math.min(e.u,e.v)}-${Math.ma
      const used = shownVisited.includes(key);
      return <line key={i} x1={u.x} y1={u.y} x2={
        stroke={used ? "#6366f1" : "#475569"}
        strokeWidth={used ? 4 : 2}
        className="transition-all duration-300" />
    })}
    {C1_NODES.map(n => {
      const isLast = shownPath[shownPath.length-1
      const visited = shownPath.includes(n.id);
      return <g key={n.id} className="cursor-poin
        {isLast && <circle cx={n.x} cy={n.y} r={1
        <circle cx={n.x} cy={n.y} r={11}
          fill={isLast ? "#6366f1" : visited ? "#
          stroke={isLast ? "#fff" : visited ? "#6
          strokeWidth={2.5}
          className="transition-all duration-200
        <text x={n.x} y={n.y+4} textAnchor="middl
        bel}</text>
      </g>;
    })}
  </svg>
  <div className="absolute top-2 left-2 bg-slate-
    Ребёр: {shownVisited.length}/{C1_EDGES.length}
    {compilerLinked && <span className="ml-2 text
  </div>
  <div className="absolute bottom-2 left-2 bg-sla
    0">
    Степени: {degrees.map((d) => `${d.label}:${d.
  </div>
</div>

```

```
.....  
    'Мы перекрашиваем их в синий цвет и делим граф по  
    'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе полу  
    'Мы вызываем алгоритм Дейкстры для поиска нового  
    ],  
    correctAnswer: 2,  
    explanation: 'Верно! Если вершины уже в одном клане  
        Это нарушит структуру дерева.'  
},  
{  
    id: 2,  
    question: 'Почему алгоритм Дейкстры впадает в ступор  
    options: [  
        'Потому что он написан только для положительных ч  
        'Он жадный и считает, что каждый следующий шаг де  
            закрытые вершины.',  
        'Отрицательные ребра создают гравитационный коллапс  
        'Дейкстра просто не любил отрицательные балансы н  
    ],  
    correctAnswer: 1,  
    explanation: 'Именно! Дейкстра уверен: если ты прие  
        ля чего нужен алгоритм Беллмана-Форда.'  
},  
{  
    id: 3,  
    question: 'Какова ключевая фишка алгоритма Борузки  
    options: [  
        'Он использует меньше памяти благодаря коммунисти  
        'Он умеет работать с отрицательными весами без ци  
        'Он позволяет выполнять шаги слияния колхозов (ко  
        'Он был разработан в Токио и поэтому самый быстрый  
    ],  
    correctAnswer: 2,  
    explanation: 'Абсолютно верно! В Борушке каждая ком  
        лелить вычисления на GPU или многопроцессорных кл  
},  
{  
    id: 4,  
    question: 'В чем главная суть Динамического програм
```

```

.....
    setScore(prev => prev + 1);
  }
};

const handleNext = () => {
  if (currentQIndex + 1 < quizQuestions.length) {
    setCurrentQIndex(prev => prev + 1);
    setSelectedOption(null);
    setIsAnswered(false);
  } else {
    setShowResults(true);
  }
};

const handleRestart = () => {
  setCurrentQIndex(0);
  setSelectedOption(null);
  setIsAnswered(false);
  setScore(0);
  setShowResults(false);
};

if (showResults) {
  return (
    <div className="bg-slate-900/90 border border-slate-
      my-12">
      <div className="w-20 h-20 bg-gradient-to-tr from
        -6 shadow-xl shadow-indigo-500/30">
        <Award className="w-10 h-10 text-white" />
      </div>
      <h3 className="text-3xl font-bold text-white mb
      <p className="text-slate-400 text-sm mb-6">Ты н

      <div className="bg-slate-950 border border-slate
        <p className="text-slate-400 text-sm uppercase
        <p className="text-5xl font-black text-indigo
        <p className="text-slate-300 text-sm">
          {score === quizQuestions.length

```

.....

```

<div className="space-y-4">
  {currentQ.options.map((option, idx) => {
    const isSelected = selectedOption === idx;
    const isCorrect = idx === currentQ.correctAnswer

    let optionStyle = "bg-slate-800/80 hover:bg-slate-800/90"
    if (isAnswered) {
      if (isCorrect) {
        optionStyle = "bg-emerald-950/80 border-emerald-500"
      } else if (isSelected) {
        optionStyle = "bg-rose-950/80 border-rose-500"
      } else {
        optionStyle = "bg-slate-800/40 border-slate-700"
      }
    }

    return (
      <div
        key={idx}
        onClick={() => handleSelectOption(idx)}
        className={"flex items-start gap-4 p-4"}
      >
        <div className="mt-0.5 shrink-0">
          {isAnswered && isCorrect ? (
            <CheckCircle2 className="w-6 h-6 text-emerald-500" />
          ) : isAnswered && isSelected && !isCorrect ? (
            <XCircle className="w-6 h-6 text-rose-500" />
          ) : (
            <div className={"w-6 h-6 rounded-full " +
              "border-indigo-500 bg-indigo-500 text-white"}
              {String.fromCharCode(65 + idx)}
            </div>
          )}
        </div>
        <div className="flex-1 text-sm md:text-base">
          {option}
        </div>
      </div>
    )
  })}
</div>

```

ФАЙЛ 44/99: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber } from '../utils';

interface Step {
  k: number; i: number; j: number;
  matrix: (number | null)[][]; updated: boolean;
  log: string; activeCodeLine: number;
}

export default function FloydViz() {
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const runtime = useVizRuntime();
  // k/i/j/d приходят напрямую; номер строки произвольный
  const [isPlaying, setIsPlaying] = useState(false);

  const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4', 'Е 5', 'Ж 6'];
  const nodesSvg = [
    { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
    { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
    { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
    { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
    { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
    { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
    { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
  ];

  const initialMatrix = [
    [0, 4, 999, 2, 999, 999, 999],
    [999, 0, 3, 999, 999, 3, 999],
    [999, 999, 0, 999, 2, 999, 999],
    [999, 999, 999, 0, 4, 2, 999],
    [999, 999, 999, 999, 0, 999, 1],
    [999, 999, 999, 999, 999, 0, 5],
  ];
```

```

.....
    log: `→ Внешний цикл. Разрешаем пути через пром
    activeCodeLine: 3,
  });

  for (let i = 0; i < N; i++) {
    for (let j = 0; j < N; j++) {
      if (i === j || i === k || j === k) continue;

      const oldVal = dist[i][j];
      const viaVal = dist[i][k] + dist[k][j];
      if (dist[i][k] !== 999 && dist[k][j] !== 999 &
        dist[i][j] = viaVal;
      simulationSteps.push({
        k, i, j, matrix: dist.map(r => [...r]), u
        log: ` Улучшение пути [${i}]→[${j}] чере
        ∞'}.`,
        activeCodeLine: 7,
      });
    }
  }
}

simulationSteps.push({
  k: 7, i: -1, j: -1, matrix: dist.map(r => [...r])
  log: ` Все пары отработаны. Алгоритм Флойда завер
  activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []));

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length -
    timer = setTimeout(() => setCurrentStepIndex((p)

```

```

.....
        <span className="text-2xl"> </span>
      </div>
      <div>
        <h3 className="text-2xl font-bold text-white">
        <p className="text-sm text-emerald-300">Полн
      </div>
    </div>

    <div className="flex items-center gap-2">
      <button onClick={() => { setIsPlaying(false);
        text-sm">  Сброс</button>
      <button onClick={() => setIsPlaying(!isPlaying
        -amber-600' : 'bg-emerald-600')}`}>{isPlaying
      <button onClick={() => { setIsPlaying(false);
        ; }} className="px-3 py-2 bg-emerald-600 te
    </div>
  </div>

  <div className="flex flex-col lg:flex-row gap-6 m
    { /* Граф */ }
    <div className="w-full lg:w-2/3 bg-indigo-950/2
      stify-center min-h-[400px]">
      <div className="absolute top-3 left-3 bg-indi
        ont-bold shadow-sm">
        {currentStep.k >= 0 && currentStep.k < 7 ?
      </div>

      <svg className="w-full h-auto max-h-[500px]"
        <defs>
          <marker id="arrow-fw-normal" markerWidth=
          <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
          <marker id="arrow-fw-active" markerWidth=
          <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
          <marker id="arrow-fw-via" markerWidth="6"
          th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
        </defs>
        {Object.keys(adjList).flatMap((uStr) => {
          const u = parseInt(uStr);

```

```
</svg>
```

```
<div className="w-full bg-slate-950/80 p-4 rounded-2xl">
  <p className="font-semibold text-amber-400">
    <p className="min-h-[40px] flex items-center">
  </div>
```

```
</div>
```

```
/* Список смежности */
```

```
<div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded-2xl">
  <h4 className="text-lg font-bold text-emerald-400">
    <div className="space-y-2 text-sm font-mono text-white">
      {Object.keys(adjList).map((uStr) => {
        const u = parseInt(uStr);
        const isI = currentStep.i === u;
        const isK = currentStep.k === u;
        return (
          <div key={u} className={`flex flex-wrap justify-between items-center p-2 ${
            isI ? 'bg-amber-900/40 border-amber-400' :
            isK ? 'bg-indigo-900/40 border-indigo-400' : 'bg-slate-950/80'
          }`} >
            <span className="font-bold text-white text-sm">
              {u}
            </span>
            {adjList[u].map((edge, idx) => {
              const isUpd = currentStep.i === u & currentStep.j === edge.v);
              const isVia = currentStep.k !== u & currentStep.j !== edge.v);
              return (
                <span key={idx} className={`px-2 py-1 text-sm font-mono ${
                  'down' : isVia ? 'bg-indigo-500 text-white border-indigo-400' :
                  'up' : isUpd ? 'bg-amber-500 text-white border-amber-400' :
                  'normal' : 'bg-slate-950/80 text-white border-slate-400'
                }`} >
                  {edge.v} ({edge.w})
                </span>
              )
            })}
          </div>
        )
      })}
    </div>
  </div>
</div>
```



```

        </tbody>
      </table>
    </div>
  </div>

  { /* Код алгоритма */ }
  <div className="w-full lg:w-1/2 bg-slate-900/60" >
    <div>
      <h4 className="text-lg font-bold text-slate-100">Алгоритм
      <div className="bg-slate-950/80 rounded-lg p-4" >
        {codeLines.map(item => {
          <div key={item.line} className={`py-1.5 px-4`} >
            e === item.line ? 'bg-blue-900/80 text-amber-400' : 'text-slate-400'
            <span className="text-slate-600 w-5 shrink-0">{item.line}</span>
            <span className="whitespace-pre flex-grow-1">{item.code}</span>
          </div>
        })}
      </div>
    </div>
  </div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 45/99: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from 'viz';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';

type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };

```

```

function dfs(v: string) {
    vis.add(v);
    stack.push(v);
    steps.push({ type: 'visit', node: v, desc: `Зашли в узел ${v}`,
        visitedNodes: Array.from(vis) });

    for (const u of adj[v]) {
        if (!vis.has(u)) {
            steps.push({ type: 'process', node: v, desc: `Переходим к узлу ${u}`,
                stack: [...stack], visitedNodes: Array.from(vis) });
            dfs(u);
        }
    }

    stack.pop();
    steps.push({ type: 'backtrack', node: v, desc: `Возвращаемся к узлу ${v}`,
        ueOrStack: [...stack], visitedNodes: Array.from(vis) });
}

dfs(start);
return steps;
}

```

```

function generateBFSSteps(start: string) {
    const steps: Action[] = [];
    const vis = new Set<string>();
    const q: string[] = [];

    q.push(start);
    vis.add(start);
    steps.push({ type: 'visit', node: start, desc: `Начинаем с узла ${start}`,
        ray: Array.from(vis) });

    while (q.length > 0) {
        const v = q[0];
        steps.push({ type: 'process', node: v, desc: `Делаем шаг из узла ${v}`,
            q: [...q], visitedNodes: [...Array.from(vis)] });
    }
}

```


.....

```

        if (current && step.type !== 'stroke') {
            strokeColor = "stroke-amber-500";
            textColor = "fill-amber-500";
            radius = 22;
        }

        return (
            <g key={n.id} className="graph-node">
                <circle cx={n.x} cy={n.y}
                    className={`stroke-${strokeColor} fill-${fillColor} stroke-width=2` />
                <text x={n.x} y={n.y}
                    className={`text-${textColor} font-size=14` />
                    {n.label}
                </text>

                {/* Ripple for BFS */}
                {current && mode === 'bfs' && n === current ? (
                    <circle cx={n.x} cy={n.y}
                        className="stroke-amber-500 fill-transparent stroke-width=2" />
                ) : null}
            </g>
        )
    )}
</svg>
</div>

{/* Structure View (Stack / Queue) */}
<div className="bg-slate-900 border border-emerald-500 p-10" >
    {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
    <div className={`absolute -top-3 left-3 right-3 bottom-3
        border border-emerald-500
        ${mode === 'dfs' ? 'border-emerald-500' : 'border-emerald-500'}
        id-300 border border-emerald-500`} >
        {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
    </div>

    <div className="flex flex-col gap-2" >
        {step.queueOrStack && step.queueOrStack.length > 0 ? (
            <div>
                {step.queueOrStack.map((node, index) => (
                    <div key={index} className="flex items-center gap-2" >
                        <div className="flex flex-col gap-2" >
                            <div>{node.x}</div>
                            <div>{node.y}</div>
                        </div>
                        <div>{node.label}</div>
                    </div>
                ))}
            </div>
        ) : null}
    </div>
</div>

```

```
.....
                <RefreshCw strokeWidth={3} size={20} />
            </button>
        </div>

        { /* Description Log */ }
        <div className="flex-1 bg-slate-900/50 p-4 rounded-lg shadow-sm
ow-hidden">
            <div className="text-[10px] text-slate-400">
                {steps.length}</div>
            <div className="text-sm text-slate-400">
                {step.desc}
            </div>
        </div>
    </div>
</div>
);
}
```

ФАЙЛ 46/99: src/components/HearViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....

import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
import { useVizRuntime, vizArray, vizNumber } from '../hooks';

interface Patient {
  id: string;
  name: string;
  emoji: string;
  symptom: string;
  priority: number; // 1 to 99 (Severity)
  animState: 'in' | 'idle' | 'winner' | 'out';
}

// Max-heap helpers for Patients
```

Универсальное пособие • полный исходный код

```
.....
  const posInLevel = index - (Math.pow(2, level) - 1);
  const count = Math.pow(2, level);
  const x = ((posInLevel + 0.5) / count) * 100;
  const y = 10 + level * 24;
  return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
  { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '🫀' },
  { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '🦴' },
  { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
  { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🌡️' },
  { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🔪' },
  { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
  let arr: Patient[] = [];
  const start = [
    { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '🫀' },
    { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '🦴' },
    { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🌡️' },
    { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
    { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🔪' },
    { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
  ];
  start.forEach((p, i) => {
    arr = heapInsert(arr, { id: `init${i}`, ...p, animS: 0 });
  });
  return arr;
})();

let uid = 300;

export const HeapViz: React.FC = () => {
  const [heap, setHeap] = useState<Patient[]>([]);
  const runtime = useVizRuntime();
```

```

.....
    const finalSymptom = preset.symptom;

    const newPatient: Patient = {
      id: `p${uid++}`,
      name: finalName,
      emoji: finalEmoji,
      symptom: finalSymptom,
      priority,
      animState: 'in',
    };

    const nextHeap = heapInsert(heap, newPatient).map(x => {
      // Mark specifically this new patient as 'in' in the heap
      const target = nextHeap.find(x => x.priority === pr
      const marked = nextHeap.map(x => x.id === (target?.id

    setHeap(marked);
    addLog(` Поступил пациент: ${finalName} с тяжестью ${finalSymptom}`);

    setTimeout(() => {
      setHeap(nextHeap);
      setBusy(false);
    }, 500);
  }, [busy, heap]);

  const handleRandomInsert = () => {
    const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
    const randomShift = Math.floor(Math.random() * 10);
    const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
    insertPatient(preset.name, prio);
  };

  const handleCustomInsert = (e: React.FormEvent) => {
    e.preventDefault();
    if (!customName.trim()) return;
    insertPatient(customName, customPriority);
    setCustomName('');
  };

```

```

        setBusy(false);
      }, 600);
    }, 800);

    }, 350);
  }, 650);
}, [busy, heap]);

const reset = () => {
  setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 })));
  setLog([' База данных скорой помощи перезагружена.']);
  setHealing(null);
  setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 } ))), 1000);
};

// --- Render binary tree lines ---
const edges = visualHeap.flatMap((_, idx) => {
  const res: React.ReactElement[] = [];
  const p = nodePos(idx);
  const l = 2 * idx + 1;
  const r = 2 * idx + 2;
  if (l < visualHeap.length) {
    const lp = nodePos(l);
    res.push(
      <line key={`el${idx}`}
        x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
        x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
        stroke="#475569" strokeWidth="2"
      />
    );
  }
  if (r < visualHeap.length) {
    const rp = nodePos(r);
    res.push(
      <line key={`er${idx}`}
        x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
        x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
        stroke="#475569" strokeWidth="2"
      />
    );
  }
});

```



```

.....
    <form onSubmit={handleCustomInsert} className="
      <input
        type="text"
        required
        value={customName}
        onChange={e => setCustomName(e.target.value)}
        placeholder="Имя пациента"
        className="flex-1 bg-slate-800 border border-
          er-emerald-500 transition-colors placeholder
        />
      <div className="flex flex-col items-center mi
        <span className="text-[9px] font-mono text-
        <input
          type="range"
          min="10"
          max="99"
          value={customPriority}
          onChange={e => setCustomPriority(Number(e
            className="w-20 accent-emerald-500 cursor
          />
        </div>
        <button
          type="submit"
          disabled={busy || !customName.trim() || heap
          className="bg-teal-600 hover:bg-teal-500 di
          px-4 rounded-xl shadow-lg active:scale-95 t
        >
          Везти
        </button>
      </form>

      {/* Забрать самого тяжелого */}
      <button
        onClick={extractMax}
        disabled={busy || heap.length === 0}
        className="flex-1 min-w-[170px] bg-gradient-to
          40 disabled:cursor-not-allowed text-white f
          transition-all cursor-pointer flex items-c

```

```

.....
    ${patient.animState === 'in' ? 'anim
    ${patient.animState === 'winner' ?
    ${patient.animState === 'out' ? 'an
    ${isCompilerNode ? 'ring-4 ring-amb
    ${isRoot ? 'bg-rose-950/80 border-r
`
  }
  style={{
    left: `${pos.x}%`,
    top: `${pos.y}%`,
    width: isRoot ? 60 : 52,
    height: isRoot ? 60 : 52,
  }}
  >
    <span className="text-2xl leading-non
    <span className="text-[10px] font-mon
      {patient.priority}%
    </span>

    {/* Tooltip */}
    <div className="absolute bottom-full
      <div className="bg-slate-950 border
0 shadow-xl">
      <div className="font-bold text-wh
      <div className="text-emerald-400"
      </div>
    </div>
  </div>
  );
  }}}
</div>

  <div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 roun
  <div className="absolute top-4 left-4 text-[1
    Операционная койка

```

```
.....
    <div className="bg-slate-900 border border-slate-
      <div className="text-[10px] font-mono text-slate
        {log.map((entry, idx) => (
          <div
            key={idx}
            className={`text-xs font-mono flex items-ce
          >
            {idx === 0 ? <span className="text-emerald-
              <span>{entry}</span>
            </div>
          )}}
        </div>
      </div>
    );
  };
};
```

ФАЙЛ 47/99: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....
import React, { createContext, useContext, useEffect, use
```

```
/** Общие примитивы для мини-визуализаций подсказок. */
```

```
export const W = 260;
```

```
export const H = 130;
```

```
/**
```

```
 * Общий множитель скорости. Кадры были слишком короткими
```

```
 * заметить, что именно переехало, и анимация читалась
```

```
 */
```

```
export const SPEED = 2.1;
```

```
/** Длительность переезда узлов. Должна быть заметно ме
```

```
export const MOVE_MS = 800;
```

```
export const EASE = "cubic-bezier(.34,.01,.2,1)";
```

```
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;
```

```

.....
    <div className="text-[10px] text-slate-400 text-
      {caption}
    </div>
  })
  <div className="text-[9px] text-slate-600 text-center"
    {paused ? "пауза – курсор на карточке" : "наведи"}
  </div>
</div>
);
};

```

```

export const Node: React.FC<{
  x: number;
  y: number;
  r?: number;
  fill: string;
  label?: string | number;
  stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
  <g style={{ transition: MOVE }}>
    <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke} />
    {label !== undefined && (
      <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
        {label}
      </text>
    )}
  </g>
);

```

```

export const Edge: React.FC<{
  a: [number, number];
  b: [number, number];
  color?: string;
  width?: number;
  dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
  <line
    x1={a[0]}

```

```

.....
    <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done
    <text x={130} y={112} textAnchor="middle" fontSize=
      9
    </text>
    <text x={72} y={54} textAnchor="middle" fontSize=
      3
    </text>
    <text x={188} y={54} textAnchor="middle" fontSize=
      4
    </text>

    <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
    <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
    <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
  </Svg>
  });
};

```

/** Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {
  const comps: [string, string][][] = [
    [ ["A", "B"], ["B", "C"] ],
    [ ["D", "E"] ],
    [ ["F", "G"], ["G", "H"], ["F", "H"] ],
  ];
  const pos: Record<string, [number, number]> = {
    A: [26, 40], B: [62, 92], C: [26, 92],
    D: [120, 40], E: [120, 92],
    F: [190, 34], G: [232, 78], H: [178, 96],
  };
  const colors = [C.active, C.warn, C.done];
  const step = useLoop(comps.length + 1, 900);
  const compOf = (v: string) => comps.findIndex((e) =>

  return (
    <Svg caption={step === 0 ? "Граф распался на острова"
      {comps.flatMap((edges, ci) =>

```



```

.....
const step = useLoop(frames.length, ms);
const f = frames[step];
const color = (id: string) =>
  id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "r" ? "#f08080" : "#f08080";

return (
  <Svg caption={captions[step]}>
    {f.edges.map(([a, b], i) => (
      <line
        key={i}
        x1={f.pos[a][0]}
        y1={f.pos[a][1]}
        x2={f.pos[b][0]}
        y2={f.pos[b][1]}
        stroke={C.edge}
        strokeWidth={1.8}
        style={{ transition: MOVE }}
      />
    ))}
    {Object.entries(f.pos).map(([id, [x, y]]) => (
      <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
        <circle r={13} fill={color(id)} stroke={C.idl} />
        <text textAnchor="middle" dominantBaseline="bottom">
          {id}
        </text>
      </g>
    ))}
  </Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
  <SplayFrames
    captions={["p – корень, x под ним", "Один поворот: x под p, p под x"]}
    frames={[
      { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": "x", "x": "p"}]},
      { pos: { x: [130, 32], p: [182, 92] }, edges: [{"p": "x", "x": "p"}]}
    ]}
  />
);

```

Универсальное пособие • полный исходный код

```
.....
  A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
  ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
  const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
  const step = useLoop(order.length + 1, 850);
  const visited = new Set(order.slice(0, step).flat());
  const wave = step > 0 && step <= order.length ? order
  return (
    <Svg caption={`Уровень ${Math.max(0, Math.min(step,
      {GRAPH_EDGES.map(([u, v], i) => {
        <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
      }}}
      {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k}
          fill={wave.includes(k) ? C.active : visited.h
          stroke={wave.includes(k) ? "#a5b4fc" : undefi
      }}}
    </Svg>
  );
};

export const DfsDemo: React.FC = () => {
  const path = ["A", "B", "D", "E", "F", "C"];
  const step = useLoop(path.length + 1, 750);
  const visited = path.slice(0, step);
  const head = visited[visited.length - 1];
  return (
    <Svg caption={head ? `Идём вглубь: ${visited.join("
      {GRAPH_EDGES.map(([u, v], i) => {
        const iu = visited.indexOf(u);
        const iv = visited.indexOf(v);
        const onPath = iu >= 0 && iv >= 0 && Math.abs(i
        return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
      }}}
    )}
  );
};
```



```

.....
    <text x={224} y={74} textAnchor="middle" fontSize=
  </Svg>
  );
};

export const RelaxDemo: React.FC = () => {
  const step = useLoop(3, 1100);
  const dV = [12, 12, 7][step];
  const improved = step === 2;
  return (
    <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Пытаемся  

      <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"} />
      <text x={130} y={54} textAnchor="middle" fontSize=16 />
      <Node x={60} y={70} r={20} fill={C.active} label="3" />
      <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />
      <text x={60} y={106} textAnchor="middle" fontSize=16 />
      <text x={200} y={106} textAnchor="middle" fontSize=16 />
      <text x={130} y={22} textAnchor="middle" fontSize=16 />
    </Svg>
  );
};

export const BridgeDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
  ];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""} />
    {edges.map(({[u, v], i}) => {
      const isBridge = u === "C" && v === "D";
      if (isBridge && cut) return null;
      return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge && cut ? "red" : "black"} />
    })}
  );
};

```

```

.....
const step = useLoop(chosen.length + 1, 800);
const taken = new Set(chosen.slice(0, step));
return (
  <Svg caption={`Жадно берём самые лёгкие рёбра без ц
    {edges.map((e, i) => {
      const on = taken.has(`${e.u}-${e.v}`);
      return (
        <g key={i}>
          <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
          <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y=
            textAnchor="middle" fontSize={9} fontWeigh
        </g>
      );
    })}
    {Object.entries(pos).map(([k, [x, y]]) => (<Node
  </Svg>
);
};

export const SccDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["B", "C"], ["C", "A"], ["A", "D"], ["D", "B"], ["D", "C"]
  ];
  const step = useLoop(2, 1400);
  const scc = ["A", "B", "C"];
  const inScc = (u: string, v: string) => step === 1 && scc.includes(v);
  return (
    <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
      {edges.map([u, v], i) => {
        <Edge key={i} a={pos[u]} b={pos[v]} color={inScc(u, v)}
      })}
      {Object.entries(pos).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k} fill={step === 1 && scc.includes(k)}
      })}
    </Svg>
  );
};

```

```

.....
      fill={i === items.length - 1 ? (popping ? C
    <text x={130} y={111 - i * 26} textAnchor="mi
    </g>
  )))
  <text x={130} y={16} textAnchor="middle" fontSize=
</Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
  const states = [["A", "B"], ["A", "B", "C"], ["B", "C
  const step = useLoop(states.length, 800);
  const items = states[step];
  return (
    <Svg caption="pop_front слева, push_back справа – к
      <defs>
        <marker id="mdArrow" markerWidth="6" markerHeigh
          <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
        </marker>
      </defs>
      <text x={12} y={44} fontSize={9} fill={C.dim}>вых</text>
      <text x={206} y={44} fontSize={9} fill={C.dim}>вх</text>
      {items.map((v, i) => (
        <g key={v} style={{ transition: "all .3s" }}>
          <rect x={30 + i * 54} y={56} width={46} heigh
          <text x={53 + i * 54} y={71} textAnchor="midd
            >
          </g>
        )))
      <line x1={24} y1={71} x2={10} y2={71} stroke={C.d
    </Svg>
  );
};

```

```

export const DsuDemo: React.FC = () => {
  const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2
  const step = useLoop(parents.length, 950);
  const p = parents[step];

```



```

.....
    o префиксные суммы P">
    <text x={4} y={18} fontSize={8} fill={C.dim}>a</t
    {a.map((v, i) => (
      <g key={i}>
        <rect x={20 + i * 39} y={24} width={34} height={12}
          fill={show && i >= l && i <= r ? C.active : C.dim}
        <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
      </g>
    ))}
    <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
    {pref.map((v, i) => (
      <g key={i}>
        <rect x={20 + i * 33} y={78} width={29} height={12}
          fill={show && i === l ? C.bad : show && i === r ? C.dim : C.idle}
          stroke={C.idleStroke} style={{ transition: "fill 0.2s" }}
        <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
      </g>
    ))}
  </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const len = [1, 2, 4][step];
  const count = 8 - len + 1;
  return (
    <Svg caption={`Уровень j=${step}: готовые ответы для`}>
      {Array.from({ length: 8 }).map((_, i) => (
        <g key={i}>
          <rect x={12 + i * 30} y={18} width={26} height={12}
            fill={C.dim}
          <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
        </g>
      ))}
      {Array.from({ length: count }).map((_, i) => (
        <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 12}

```

```

export const PrefixFunctionDemo: React.FC = () => {
  const s = "abacaba";
  const pi = [0, 0, 1, 0, 1, 2, 3];
  const step = useLoop(s.length + 1, 700);
  const i = Math.max(0, step - 1);
  const len = step > 0 ? pi[i] : 0;
  return (
    <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс`
      {s.split("").map((ch, idx) => (
        <g key={idx}>
          <rect x={18 + idx * 33} y={26} width={28} height={20}
            fill={step > 0 && (idx < len || (idx > i - len)) ? C.fill : C.idle}
            style={{ transition: "fill .25s" }} />
          <text x={32 + idx * 33} y={40} textAnchor="middle">{ch}</text>
          <text x={32 + idx * 33} y={70} textAnchor="middle">{idx}</text>
        </g>
      ))}
    <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
    <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
    <text x={130} y={106} textAnchor="middle" fontSize={12}>
      π[i] – длина самого длинного «префикс = суффикс»
    </text>
  </Svg>
  );
};

```

```

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);
  return (
    <Svg caption={`Заполняем таблицу подзадач: готово $
      {Array.from({ length: rows }).map((_, r) =>
        Array.from({ length: cols }).map((_, c) => {
          const idx = r * cols + c;
          const filled = idx < step;

```

Универсальное пособие • полный исходный код

```
.....  
  | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"  
  | "suffix-link" | "toposort" | "relax" | "prefix-sum"  
  | "segment-tree" | "bridge" | "articulation" | "mst"  
  | "prefix-function" | "dp" | "floyd" | "components" |  
  | "zig" | "zig-zig" | "zig-zag";  
  
const REGISTRY: Record<DemoKind, React.FC> = {  
  bfs: BfsDemo,  
  dfs: DfsDemo,  
  heap: HeapDemo,  
  stack: StackDemo,  
  queue: QueueDemo,  
  dsu: DsuDemo,  
  trie: TrieDemo,  
  "trie-bfs": TrieBfsDemo,  
  "suffix-link": SuffixLinkDemo,  
  toposort: ToposortDemo,  
  relax: RelaxDemo,  
  "prefix-sum": PrefixSumDemo,  
  "sparse-table": SparseTableDemo,  
  "segment-tree": SegmentTreeDemo,  
  bridge: BridgeDemo,  
  articulation: ArticulationDemo,  
  mst: MstDemo,  
  amortized: AmortizedDemo,  
  np: NpDemo,  
  scc: SccDemo,  
  "prefix-function": PrefixFunctionDemo,  
  dp: DpDemo,  
  floyd: FloydDemo,  
  components: ComponentsDemo,  
  graph: GraphBasicsDemo,  
  "coord-compress": CoordCompressDemo,  
  zig: ZigDemo,  
  "zig-zig": ZigZigDemo,  
  "zig-zag": ZigZagDemo,  
};
```

```

.....
    if (e.key === "Escape") onClose();
  };
  window.addEventListener("keydown", onKey);
  document.body.style.overflow = "hidden";
  return () => {
    window.removeEventListener("keydown", onKey);
    document.body.style.overflow = "";
  };
}, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
  <div className="fixed inset-0 z-[110] flex items-end"
    <div className="absolute inset-0 bg-slate-950/80"
    <div className="relative w-full sm:max-w-5xl max-h-
      ded-2xl shadow-2xl flex flex-col">
      <div className="flex items-center gap-3 px-4 py-4"
        <div className="min-w-0 flex-1">
          <h3 className="font-bold text-white text-sm">
            {viz.hint} && <p className="text-[11px] text-white">
              {viz.description}
            </p>
          </div>
          <button
            onClick={onClose}
            className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
            aria-label="Заккрыть симулятор"
          >
            <X className="w-5 h-5" />
          </button>
        </div>
      <div className="overflow-y-auto overscroll-contain">
        <Component />
      </div>
    </div>
  </div>,
  document.body

```



```

        setPos(null);
        return;
    }
    const vw = window.innerWidth;
    const vh = window.innerHeight;
    const width = Math.min(CARD_W, vw - 16);
    const height = cardRef.current?.offsetHeight ?? 300

    let left = anchor.rect.left + anchor.rect.width / 2
    left = Math.max(8, Math.min(left, vw - width - 8));

    const below = anchor.rect.top < height + GAP + 8;
    const top = below ? anchor.rect.bottom + GAP : anchor

    setPos({ top: Math.max(8, Math.min(top, vh - height
    }, {anchor});

    useEffect(() => {
        if (!anchor) return;
        const onKey = (e: KeyboardEvent) => {
            if (e.key === "Escape") onClose();
        };
        window.addEventListener("keydown", onKey);
        window.addEventListener("scroll", onClose, true);
        return () => {
            window.removeEventListener("keydown", onKey);
            window.removeEventListener("scroll", onClose, true);
        };
    }, [anchor, onClose]);

    if (!anchor) return null;
    const entry = GLOSSARY_BY_ID.get(anchor.termId);
    if (!entry) return null;
    const viz = getViz(entry.simulator);
    const simulatorId = entry.simulator;

    return createPortal(
        <div

```

```

.....
        </div>
      })

      {viz && simulatorId && {
        <button
          onClick={() => onOpenSimulator(simulatorId)}
          className="w-full flex items-center justify-center
            rounded-lg py-2 transition-colors"
        >
          <Play className="w-3.5 h-3.5" /> Показать
        </button>
      }}
    </div>
  </div>
</div>,
document.body
);
};

```

.....
 ФАЙЛ 54/99: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

.....
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizSteps';

export function JohnsonViz() {
  const [step, setStep] = useState(0);
  const MAX_STEP = 7;
  useVizStepSync(step, setStep, MAX_STEP, (vars) => {
    const edge = `${vizString(vars.u)} ?? ""}${vizString(vars.v)} ?? ""`;
    return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
  });

  const nodes = [

```

```

.....
    if (e.id === 'AB') {
      if (step === 4) return "stroke-amber-400 st
      if (step > 4) return "stroke-emerald-500";
    }
    if (e.id === 'BC') {
      if (step === 5) return "stroke-amber-400 st
      if (step > 5) return "stroke-emerald-500";
    }
    if (e.id === 'CA') {
      if (step === 6) return "stroke-amber-400 st
      if (step > 6) return "stroke-emerald-500";
    }

    if (step >= 4) return "stroke-slate-600"; // ot
    return "stroke-slate-500";
  };

const getMarkerUrl = (e: any) => {
  if (e.u === 'S') {
    if (step === 0 || step === 7) return "";
    if (step === 1) return "url(#arrow-amber)";
    if (step === 2) return "url(#arrow-pink)";
    return "url(#arrow-slate-dim)";
  }
  if (e.id === 'AB' && step === 4) return "url(#a
  if (e.id === 'AB' && step > 4) return "url(#arr
  if (e.id === 'BC' && step === 5) return "url(#a
  if (e.id === 'BC' && step > 5) return "url(#arr
  if (e.id === 'CA' && step === 6) return "url(#a
  if (e.id === 'CA' && step > 6) return "url(#arr

  if (step >= 4) return "url(#arrow-slate-dim)";
  return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
  if (e.u === 'S') return e.weight;

```

```
.....
        Если мы запустим быстрый алгоритм Д
    </div>
);
case 1: return (
    <div>
        <b>Шаг 1: Точка отсчета.</b><br/>
        Добавляем фиктивную вершину S. Соед
ми</span>.
        Это наша база для расчета потенциал
    </div>
);
case 2: return (
    <div>
        <b>Шаг 2: Ищем потенциалы  $h(v)$ .</b>
        Запускаем медленный, но мощный алгор
ных вершин:
        <span className="text-pink-300 ml-1
    </div>
);
case 3: return (
    <div>
        <b>Шаг 3: Магическая формула.</b><b>
        Всем ребрам графа назначаем новый в
        <div className="text-center font-mo
        Это повышает или понижает вес ребра
    </div>
);
case 4: return (
    <div>
        <b>Шаг 3.1: Пересчет ребра  $A \rightarrow B$ .</b>
        Старый вес: <span className="text-r
        Вычисляем:  $-2 + 0 - (-2) =$  <span cl
    </div>
);
case 5: return (
    <div>
        <b>Шаг 3.2: Пересчет ребра  $B \rightarrow C$ .</b>
        Старый вес: 1. <br/>
```

```

.....
      <marker id="arrow-slate-dim
uto-start-reverse">
        <path d="M 0 0 L 10 5 L
</marker>
      <marker id="arrow-amber" vie
start-reverse">
        <path d="M 0 0 L 10 5 L
</marker>
      <marker id="arrow-pink" vie
tart-reverse">
        <path d="M 0 0 L 10 5 L
</marker>
      <marker id="arrow-emerald"
o-start-reverse">
        <path d="M 0 0 L 10 5 L
</marker>
</defs>

{/* Edges */}
{edges.map((e, i) => {
  const u = nodes.find(n => n
  const v = nodes.find(n => n

  const isS = e.u === 'S';
  if (isS && (step === 0 || s

  const midX = (u.x + v.x) / 2;
  const midY = (u.y + v.y) / 2;

  const edgeColorClass = getE
  const textHigh = isEdgeText
  const textEmer = isEdgeText

  return {
    <g key={i}>
      <line
        x1={u.x} y1={u.y}
        className={`str

```

```

.....
                                <g transform={`
fade-in zoom-in duration-500">
                                <rect x="-1
k-500 stroke-1" />
                                <text x="0"
font-bold fill-pink-400">
                                {pot}
                                </text>
                                </g>
                                )}
                                </g>
                                )
                                }}}
                                </svg>
                                </div>

```

```

<div className="w-full md:w-[320px] flex
  { /* Log Box */ }
  <div className="bg-[#0f172a] p-5 ro
l justify-center leading-relaxed text-sm te
    {getDesc()}
  </div>

```

```

  { /* Controls */ }
  <div className="flex bg-slate-900 b
    <button onClick={() => setStep(1
      className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
      <RotateCcw strokeWidth={3}
    </button>
    <button onClick={() => setStep(1
      className="p-3 bg-slate-800
-slate-400 transition-colors" title="Вар на
      <Undo strokeWidth={3} size=
    </button>
    <button onClick={() => setStep(1
      className={`flex-1 font-bol
        ${step === MAX STEP

```

```

.....
    y: number;
    label: string;
}

interface GEdge {
    u: number;
    v: number;
}

const initialNodes: GNode[] = [
    { id: 0, x: 150, y: 100, label: "0" },
    { id: 1, x: 300, y: 100, label: "1" },
    { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
    { id: 3, x: 450, y: 160, label: "3" },
    { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
    { u: 0, v: 1 },
    { u: 1, v: 2 },
    { u: 2, v: 0 },
    { u: 2, v: 3 }, // Connection edge
    { u: 3, v: 4 },
    { u: 4, v: 3 },
];

interface AlgoStep {
    phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
    desc: string;
    visited: Set<number>;
    currentNode: number | null;
    order: number[];
    transposed: boolean;
    sccMap: Record<number, number>; // node -> sccId
    currentSccId: number;
    activeEdges: GEdge[];
}

```

```

.....
    ? initialEdges.map((e) => ({ u: e.v, v: e.u }
      : [...initialEdges],
    ));
  });
};

recordStep(
  "init",
  "Инициализация алгоритма Косарайю. Начинаем первый
  null,
  false,
  -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
  visited.add(u);
  recordStep(
    "dfs1",
    `DFS 1: зашли в вершину ${u}, помечаем посещение
    u,
    false,
    -1,
  );

  for (const to of adj[u]) {
    if (!visited.has(to)) {
      dfs1(to);
    }
  }
}

order.push(u);
recordStep(
  "dfs1",
  `DFS 1: вершина ${u} полностью обработана. Запи

```



```

        true,
        currentScc,
    );

    for (const to of adjT[u]) {
        if (!visited.has(to)) {
            dfs2(to, currentScc);
        }
    }
};

// Run DFS2 using the order (reversed from back to
for (let i = order.length - 1; i >= 0; i--) {
    const u = order[i];
    if (!visited.has(u)) {
        sccId++;
        recordStep(
            "dfs2",
            `DFS 2: берем следующую непосещенную из стека
            u,
            true,
            sccId,
        );
        dfs2(u, sccId);
    } else {
        recordStep(
            "dfs2",
            `DFS 2: вершина ${u} из стека уже покрашена в
            u,
            true,
            sccId,
        );
    }
}

recordStep(
    "finished",
    `Алгоритм успешно завершен! Найдено ${sccId} комп

```

```

.....

const getSccColor = (id: number) => {
  if (id === 1) return "fill-emerald-600 stroke-emera
  if (id === 2) return "fill-indigo-600 stroke-indigo
  return "fill-slate-800 stroke-slate-500";
};

return (
  <div className="bg-slate-900 rounded-xl border bord
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Layers className="w-5 h-5 text-emerald-400"
          Визуализатор Алгоритма Косарайю (Поиск SCC)
        </h3>

      <div className="flex items-center gap-2 bg-slate
        <button
          onClick={togglePlay}
          className="flex items-center gap-2 px-3 py-
            d-500 text-white"
        >
          {isPlaying ? {
            <Pause className="w-4 h-4 ml-1" />
          } : {
            <Play className="w-4 h-4 ml-1" />
          }}
          {isPlaying ? "Пауза " : "Старт "}
        </button>

        <button
          onClick={reset}
          className="flex items-center justify-center
            ors"
        >
          <RotateCcw className="w-4 h-4" />
        </button>
      </div>
    </div>
  </div>

```

```
.....
    <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
  </marker>
</defs>
```

```

{ /* Edges */
initialEdges.map((edge, idx) => {
  const u = initialNodes.find((n) => n.id ==
  const v = initialNodes.find((n) => n.id ==

  // If transposed, draw reversed coordinates
  const x1 = step.transposed ? v.x : u.x;
  const y1 = step.transposed ? v.y : u.y;
  const x2 = step.transposed ? u.x : v.x;
  const y2 = step.transposed ? u.y : v.y;

  const isSccEdge =
    step.sccMap[edge.u] &&
    step.sccMap[edge.v] &&
    step.sccMap[edge.u] === step.sccMap[edge.v]
  let stroke = "#475569";
  let markerId = "arrow";

  if (step.transposed) {
    stroke = isSccEdge ? "#818cf8" : "#4f46e5";
    markerId = "arrow-transposed";
  } else {
    if (
      step.currentNode === edge.u ||
      step.currentNode === edge.v
    ) {
      stroke = "#10b981";
      markerId = "arrow-active";
    }
  }
}

// Adjust slightly for bidirectional link
const isBidi =
  (edge.u === 3 && edge.v === 4) ||
```

```

.....
        cx={node.x}
        cy={node.y}
        r="20"
        className={`transition-all duration
strokeWidth="3"
/>
<text
  x={node.x}
  y={node.y}
  dy=".3em"
  textAnchor="middle"
  fill="white"
  fontSize="13px"
  fontWeight="bold"
  className="select-none"
>
  {node.label}
</text>

{scssId && (
  <text
    x={node.x}
    y={node.y - 28}
    textAnchor="middle"
    fill="#10b981"
    fontSize="10px"
    fontWeight="bold"
  >
    SCC #{scssId}
  </text>
  )}
</g>
);
}}
</svg>

{step.transposed && (
  <div className="absolute top-4 left-4 bg-in

```

```

    })
  </div>
</div>

{/* List of actions log */}
<div>
  <span className="text-[10px] text-slate-500">
    ЛОГ ОПЕРАЦИЙ:
  </span>
  <div className="space-y-1.5 max-h-[150px]">
    {steps
      .slice(0, currentStepIdx + 1)
      .reverse()
      .slice(0, 5)
      .map((s, idx) => {
        <div
          key={idx}
          className={`text-[11px] p-1.5 rounded`
        >
          {s.desc}
        </div>
      })}
    </div>
  </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">
  <span>
    Шаг {currentStepIdx + 1} из {steps.length}
  </span>
  <div className="flex gap-1.5">
    <button
      disabled={currentStepIdx === 0}
      onClick={() => setCurrentStepIdx((prev) => prev - 1)}
      className="bg-slate-900 border border-slate-800 text-white px-2 py-1 rounded"
    >
      Назад
    </button>
  </div>
</div>

```

```

.....
  weight: number;
  status: 'pending' | 'inspecting' | 'included' | 'reje
  color: string;
}

interface StepLog {
  title: string;
  description: string;
  type: 'info' | 'success' | 'warning' | 'error';
}

// 9 Vertices layout
const initialVertices: Vertex[] = [
  { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
  { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
  { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
  { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
  { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
  { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
  { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
  { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
  { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
  { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
  { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
  { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
  { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
  { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
  { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
  { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
  { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
  { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
  { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
  { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
  { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' }
];

```

```

.....
    description: '«Краскал – это раскраска?». Предста
        овы начинать!',
    type: 'info',
  },
  1));

// --- KRUSKAL STEPS ---
const kruskalSteps = [
  // Step 1: Inspect A-B (2)
  () => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
      title: 'Шаг 1: Берем самое дешевое ребро A-B (в
      description: 'Оно соединяет две бесцветные верши
      type: 'info'
    }, ...prev]));
  },
  // Step 2: Include A-B
  () => {
    setVertices(prev => prev.map(v => v.id === 0 || v
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
      title: 'Успех: Ребро A-B в осто́ве',
      description: 'Вершины A and B окрашены в красны
      type: 'success'
    }, ...prev]));
  },
  // Step 3: Inspect D-E (3)
  () => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
      title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
      description: 'Оно соединяет две другие бесцветны
      type: 'info'
    }, ...prev]));
  },
  // Step 4: Include D-E
  () => {

```

```

.....
() => {
  setVertices(prev => prev.map(v => v.color === 'blue' ? v : {id: 4, color: 'blue'}));
  setEdges(prev => prev.map(e => e.status === 'included' ? e : {id: '1-4', status: 'included'}));
  setLogs(prev => [{
    title: 'Слияние кланов: Ребро A-D добавлено',
    description: 'Мы объединили красный и синий кланы',
    type: 'success'
  }, ...prev]);
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '1-4' ? {id: '1-4', status: 'excluded'} : e));
  setLogs(prev => [{
    title: 'Шаг 5: Берем ребро B-E (вес 6)',
    description: 'НО: это ребро соединяет вершины B и E, которые уже соединены!',
    type: 'warning'
  }, ...prev]);
},
// Step 10: Reject B-E
() => {
  setEdges(prev => prev.map(e => e.id === '1-4' ? {id: '1-4', status: 'excluded'} : e));
  setLogs(prev => [{
    title: 'Отказ: Цикл обнаружен!',
    description: 'Мы выбрасываем ребро B-E. Иначе получится цикл!',
    type: 'error'
  }, ...prev]);
},
// Step 11: Inspect E-F (7)
() => {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {id: '4-5', status: 'excluded'} : e));
  setLogs(prev => [{
    title: 'Шаг 6: Берем ребро E-F (вес 7)',
    description: 'Оно соединяет фиолетовую вершину E и синюю вершину F',
    type: 'info'
  }, ...prev]);
},
// Step 12: Include E-F
() => {

```



```

.....
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
        title: 'Отказ: Цикл обнаружен!',
        description: 'Выбрасываем ребро C-F.',
        type: 'error'
    }, ...prev]));
},
// Step 17: Inspect G-H (10)
() => {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
    setLogs(prev => [{
        title: 'Шаг 9: Берем ребро G-H (вес 10)',
        description: 'Соединяет фиолетовую G и бесцветную H.',
        type: 'info'
    }, ...prev]));
},
// Step 18: Include G-H
() => {
    setVertices(prev => prev.map(v => v.id === 7 ? {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро G-H в остоле',
        description: 'Вершина H перекрашена в фиолетовую.',
        type: 'success'
    }, ...prev]));
},
// Step 19: Inspect E-H (11) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setLogs(prev => [{
        title: 'Шаг 10: Берем ребро E-H (вес 11)',
        description: 'Оба конца E и H уже фиолетовые. Соединение создает цикл.',
        type: 'warning'
    }, ...prev]));
},
// Step 20: Reject E-H
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {

```

```

.....
        ритетную очередь (Heap): D-E(3), B-E(6), E-F(
    type: 'info'
    }, ...prev]);
},
// Step 2: Pop D-E (3)
() => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
        title: 'Шаг 2: Куча выдает самое дешевое ребро',
        description: 'Плесень тянется по ребру D-E к вершине E',
        type: 'info'
    }, ...prev]);
},
// Step 3: Include D-E, add D's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 3 ? {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
        ..e, status: 'in_heap' } : e));
    setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D-E (3)']);
    setLogs(prev => [{
        title: 'Успех: Плесень захватила вершину D',
        description: 'В кучу добавлены новые пути из D: D-E, D-F',
        type: 'success'
    }, ...prev]);
},
// Step 4: Pop A-D (5)
() => {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
        description: 'Плесень расширяется в сторону вершины A',
        type: 'info'
    }, ...prev]);
},
// Step 5: Include A-D, add A's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 0 ? {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {

```

```

// Step 9: Include B-C, add C's edges
() => {
  setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
      eap' } : e));
  setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (8)']);
  setLogs(prev => [{
    title: 'Успех: Вершина C захвачена',
    description: 'Ребро C-F(9) добавлено в кучу.',
    type: 'success'
  }, ...prev]);
},
// Step 10: Pop B-E (6) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '1-4' ? {
  setLogs(prev => [{
    title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
    description: 'ВНИМАНИЕ! Плесень проверяет вершины',
    type: 'warning'
  }, ...prev]);
},
// Step 11: Reject B-E
() => {
  setEdges(prev => prev.map(e => e.id === '1-4' ? {
  setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'E-G (8)']);
  setLogs(prev => [{
    title: 'Отказ: Игнорируем внутреннее ребро',
    description: 'Ребро B-E отброшено. Плесень не р
    type: 'error'
  }, ...prev]);
},
// Step 12: Pop E-F (7)
() => {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
  setLogs(prev => [{
    title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
    description: 'Плесень из центра Токио (E) тянет
    type: 'info'
  }, ...prev]);
},

```

```

.....
    setLogs(prev => [{
      title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
      description: 'Вершины C и F уже заражены плесенью',
      type: 'warning'
    }, ...prev]));
  },
  // Step 17: Reject C-F
  () => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
      setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
    }, ...prev));
    setLogs(prev => [{
      title: 'Отказ: Игнорируем ребро C-F',
      description: 'Выбрасываем из кучи.',
      type: 'error'
    }, ...prev]));
  },
  // Step 18: Pop G-H (10)
  () => {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
      setLogs(prev => [{
        title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
        description: 'Плесень ползет к вершине H.',
        type: 'info'
      }, ...prev]));
    }, ...prev));
  },
  // Step 19: Include G-H, add H's edges
  () => {
    setVertices(prev => prev.map(v => v.id === 7 ? {
      setEdges(prev => prev.map(e => e.id === '6-7' ? {
        eap' } : e));
      setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I (13)'])
    }, ...prev));
    setLogs(prev => [{
      title: 'Успех: Вершина H захвачена',
      description: 'В кучу добавлено H-I(12).',
      type: 'success'
    }, ...prev]));
  },
  // Step 20: Pop E-H (11) - Cycle!

```

```

.....
    },
    1;

// --- BORUVKA STEPS ---
const boruvkaSteps = [
  // Step 1: Five-Year Plan 1 (Inspecting outgoing)
  () => {
    // Highlight the cheapest outgoing edges for EACH
    // Node 0(A) -> A-B(2)
    // Node 1(B) -> A-B(2)
    // Node 2(C) -> B-C(4)
    // Node 3(D) -> D-E(3)
    // Node 4(E) -> D-E(3)
    // Node 5(F) -> E-F(7)
    // Node 6(G) -> D-G(8)
    // Node 7(H) -> G-H(10)
    // Node 8(I) -> H-I(12)
    setEdges(prev => prev.map(e =>
      ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8'
    ]));
    setLogs(prev => [{
      title: 'Первая пятилетка: Каждая деревня выбирает
      description: 'Каждая из 9 деревень одновременно
        А выбирает A-B(2), D выбирает D-E(3), а G вы
        !',
      type: 'info'
    }, ...prev]);
  },
  // Step 2: Concurrently merge into Kolkhozes (Components)
  () => {
    // Concurrently build chosen roads. Merge into:
    // Component 1: {A, B, C} (colored red)
    // Component 2: {D, E, F, G, H, I} (colored gold/olive)
    setVertices(prev => prev.map(v =>
      [0, 1, 2].includes(v.id) ? { ...v, color: 'red' } : v
    ));
    setEdges(prev => prev.map(e => {
      if (['0-1', '1-2'].includes(e.id)) {

```

```

    }, ...prev]);
  },
  // Step 5: Check remaining edges & mark as rejected
  () => {
    setEdges(prev => prev.map(e => e.status === 'pend
    setLogs(prev => [{
      title: ' Успех: MST построен Борувкой за 2 шага
      description: 'Все лишние ребра (В-Е, Е-Н, С-Е,
        вес: 51. Благодаря параллельности, мы потрати
      type: 'success'
    }, ...prev]);
  }
};

const currentSteps =
  activeAlgo === 'kruskal' ? kruskalSteps :
  activeAlgo === 'prim' ? primSteps :
  boruvkaSteps;

const handleNextStep = () => {
  if (stepIndex < currentSteps.length) {
    currentSteps[stepIndex]();
    setStepIndex(stepIndex + 1);
  }
};

const handleReset = (algo: 'kruskal' | 'prim' | 'boru
  setActiveAlgo(algo);
  setVertices(initialVertices);
  setEdges(initialEdges);
  setStepIndex(0);
  setHeapQueue([]);
  if (algo === 'kruskal') {
    setLogs([
      {
        title: 'Введение в раскраску (Краскал)',
        description: 'Представь, что все 9 вершин граф
        type: 'info',

```

```

.....
    if (status === 'inspecting') return '#f59e0b';
    if (status === 'rejected') return '#ef4444';
    if (status === 'in_heap') return '#0284c7';
    if (status === 'included') {
      if (color === 'red') return '#ef4444';
      if (color === 'blue') return '#3b82f6';
      if (color === 'purple') return '#a855f7';
      if (color === 'mold') return '#10b981';
    }
    return '#475569';
  };

return (
  <div className="space-y-12">

    {/* 3-in-1 MST Simulator Box */}
    <div className="bg-slate-900/90 border border-slate-800" >
      <div className="flex flex-col lg:flex-row justify-between" >
        <div>
          <div className="flex items-center gap-2 mb-2" >
            <span className="bg-gradient-to-r from-indigo-500 to-purple-500 text-white font-bold uppercase tracking-wider" >
              Сунер-Интерактив 3 в 1
            </span>
            <h3 className="text-2xl font-bold text-white" >
              MST Simulator
            </h3>
          </div>
          <p className="text-slate-400 text-sm" >
            Наблюдай за работой Краскала (Раскраска),
          </p>
        </div>

        {/* Algorithm Tabs */}
        <div className="flex flex-wrap items-center gap-2" >
          <div className="flex items-center bg-slate-800 p-2" >
            <button
              onClick={() => handleReset('kruskal')}
              className={
                "flex items-center gap-1.5 px-3 py-2"
              }
            >
              Краскала
            </button>
          </div>
        </div>
      </div>
    </div>
  );
}

```

```

.....
      onClick={() => handleReset(activeAlgo)}
      className="flex items-center justify-center gap-2
-200 font-semibold text-xs rounded-xl border
    >
      <RotateCcw className="w-3.5 h-3.5" />
      <span>Сбросить</span>
    </button>

    <button
      onClick={handleNextStep}
      disabled={stepIndex >= currentSteps.length}
      className={
        "flex items-center justify-center gap-2
initial cursor-pointer " +
        (stepIndex >= currentSteps.length
          ? "bg-slate-800 text-slate-500 border
            : activeAlgo === 'kruskal'
          ? "bg-indigo-600 hover:bg-indigo-500
            : activeAlgo === 'prim'
          ? "bg-emerald-600 hover:bg-emerald-500
            : "bg-rose-600 hover:bg-rose-500 text
        )
      >
      <SkipForward className="w-4 h-4" />
      <span>{stepIndex === 0 ? 'Начать' : stepIndex + 1}</span>
    </button>
  </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3
  {/* Graph Canvas */}
  <div className="lg:col-span-2 bg-slate-950 rounded
    active min-h-[480px] overflow-hidden">

    {/* Legend */}
    <div className="absolute top-4 left-4 bg-slate-900
      -center gap-3 text-xs">
      <div className="flex items-center gap-1.5

```



```

.....
<div className="absolute top-4 right-4 bg-slate-300" style={{
  width: 100px, height: 100px, display: flex, align-items: center, justify-content: center;
}}>
  {stepIndex} / {currentSteps.length}
</div>

{compilerLinked && (
  <div className="absolute bottom-4 right-4 font-mono text-emerald-300" style={{
    width: 100px, height: 100px, display: flex, align-items: center, justify-content: center;
  }}>
    Python: i={liveI ?? "-"}{liveU || liveV} {liveW !== null ? ', w=${liveW}` : ""}
  </div>
)}

{/* SVG Edges */}
<svg className="absolute inset-0 w-full h-full" style={{
  opacity: 0.5,
  pointerEvents: "none",
}}>
  {liveEdges.map(edge => {
    const u = vertices[edge.u];
    const v = vertices[edge.v];
    const strokeColor = getEdgeStroke(edge);
    const isInspecting = edge.status === 'inspect';
    const isRejected = edge.status === 'reject';
    const isInHeap = edge.status === 'in_heap';

    return (
      <g key={edge.id}>
        <line
          x1={u.x + "%"}
          y1={u.y + "%"}
          x2={v.x + "%"}
          y2={v.y + "%"}
          stroke={strokeColor}
          strokeWidth={isInspecting ? 4.5 : 2}
          strokeDasharray={isRejected ? '4, 4' : ''}
          className={"transition-all duration-300"}
        />
        <text
          x={({u.x + v.x} / 2 + "%")}

```

```

.....
      shadow-xl z-20">
        <p className={"font-bold text-base " +
          `ld-300' : 'text-rose-300'}`}>
          MST построено с помощью {activeAlgo}
        </p>
        <p className="text-slate-300 text-xs mt-2">
          Общий вес минимального остова: 51. Всего
        </p>
      </div>
    )}
  </div>

  {/* Logs / Heap */}
  <div className="bg-slate-950 rounded-2xl border-b border-slate-900">
    <div className="p-4 bg-slate-900 border-b border-slate-800">
      <h4 className="font-bold text-slate-200">
        Ход выполнения
      </h4>
    </div>

    {activeAlgo === 'prim' && (
      <div className="p-3 bg-slate-900/60 border-b border-slate-800">
        <p className="text-[11px] font-bold text-slate-200">
          ✂ Приоритетная очередь (Min-Heap):
        </p>
        <div className="flex flex-wrap gap-1.5">
          {heapQueue.length === 0 ? (
            <span className="text-xs text-slate-400">
              Пусто
            </span>
          ) : (
            heapQueue.map(({item, idx}) => (
              <span key={idx} className={"px-2 py-1 shadow-md animate-pulse" : "bg-slate-800"}>
                {item}
              </span>
            ))
          )}
        </div>
      </div>
    )}
  </div>

```

```
</div>
```

```
<div className="flex bg-slate-950 p-1 rounded
  <button
    onClick={() => setActiveCheatTab('kruskal')}
    className={"px-3 py-1.5 rounded text-xs font-medium"}
    === 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-800 text-white"
  >
    Краскал (Билет 18)
  </button>
  <button
    onClick={() => setActiveCheatTab('prim')}
    className={"px-3 py-1.5 rounded text-xs font-medium"}
    === 'prim' ? "bg-emerald-600 text-white" : "bg-slate-800 text-white"
  >
    Прима (Билет 19)
  </button>
  <button
    onClick={() => setActiveCheatTab('boruvka')}
    className={"px-3 py-1.5 rounded text-xs font-medium"}
    === 'boruvka' ? "bg-rose-600 text-white" : "bg-slate-800 text-white"
  >
    Борувка (Билет 20)
  </button>
</div>
```

```
</div>
```

```
{activeCheatTab === 'kruskal' && {
  <div className="grid grid-cols-1 lg:grid-cols-2"
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-2xl">
        <p className="text-slate-400 text-xs font-medium">
          <Clock className="w-3.5 h-3.5 text-indigo-400" />
        </p>
        <p className="text-2xl font-black text-white">Краскал</p>
        <p className="text-slate-400 text-xs font-medium">Минимальное остовное дерево</p>
      </div>
      <div className="bg-slate-950 p-4 rounded-2xl">
        <p className="text-slate-400 text-xs font-medium">
          <Clock className="w-3.5 h-3.5 text-indigo-400" />
        </p>
        <p className="text-2xl font-black text-white">Прима</p>
        <p className="text-slate-400 text-xs font-medium">Минимальное остовное дерево</p>
      </div>
      <div className="bg-slate-950 p-4 rounded-2xl">
        <p className="text-slate-400 text-xs font-medium">
          <Clock className="w-3.5 h-3.5 text-indigo-400" />
        </p>
        <p className="text-2xl font-black text-white">Борувка</p>
        <p className="text-slate-400 text-xs font-medium">Минимальное остовное дерево</p>
      </div>
    </div>
  </div>
}
```

```

.....
    mst.append(({u, v, w}))`}
      </pre>
    </div>
  </div>
</div>
})

{activeCheatTab === 'prim' && (
  <div className="grid grid-cols-1 lg:grid-cols-2" >
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
          <Clock className="w-3.5 h-3.5 text-emerald-400" />
        </p>
        <p className="text-2xl font-black text-white">Примитивы</p>
        <p className="text-slate-400 text-xs mt-2">Примитивы — это базовые типы данных, которые используются для хранения информации в памяти компьютера. Они являются фундаментом для построения более сложных структур данных и программ.</p>
      </div>
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
          <Award className="w-3.5 h-3.5 text-emerald-400" />
        </p>
        <p className="text-2xl font-black text-white">Награды</p>
        <p className="text-slate-400 text-xs mt-2">Награды — это специальные значки или медали, которые используются для поощрения пользователей за определенные действия. Они могут быть получены за выполнение заданий, достижение определенных результатов или за активное участие в сообществе.</p>
      </div>
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
          <Code className="w-3.5 h-3.5 text-emerald-400" />
        </p>
        <p className="text-2xl font-black text-white">Код</p>
        <pre className="text-xs text-emerald-400 font-mono">
80 flex-1">

```



```
.....
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown",
}, []);
```

```
return (
```



```
    <div className="lg:hidden sticky top-[57px] z-40" >
      <button
        onClick={() => setOpen(true)}
        className="w-full flex items-center gap-2.5 p-2"
        aria-label="Открыть содержание"
      >
        <span className="p-1.5 rounded-lg bg-indigo-600 text-white" >
          <List className="w-4 h-4" />
        </span>
        <span className="min-w-0 flex-1" >
          <span className="block text-[10px] uppercase" >
            Содержание • {index + 1} из {total}
          </span>
          <span className="block text-[13px] font-bold" >
            {title}
          </span>
        </span>
      </button>
    </div>
```

```
{open && (
  <div className="lg:hidden fixed inset-0 z-[90]" >
    <div className="absolute inset-0 bg-slate-950" >
      <div className="absolute inset-y-0 left-0 w-[100px] h-[100px]
        te-[tocIn_.18s_ease-out]" >
        <div className="flex items-center justify-between" >
          <span className="font-bold text-white text-[12px]" >
            {title}
          <button
            onClick={() => setOpen(false)}
            className="p-2 -mr-2 text-slate-400 hover:bg-slate-800"
            aria-label="Заккрыть содержание"
          >
            <X className="w-5 h-5" />
          </button>
        </div>
      </div>
    </div>
  </div>
)
```

```

.....
    await downloadBookHtml(chapters);
  } finally {
    setBusy(false);
  }
};

return (
  <header className="bg-slate-900 border-b border-slate-800">
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
      <div className="flex items-center gap-3 w-full">
        <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 w-6 h-6">
          <Sparkles className="w-6 h-6" />
        </div>
        <div className="min-w-0 flex-1">
          <h1 className="text-xl font-extrabold text-white">
            Универсальное пособие
          </h1>
          <p className="text-xs text-indigo-400 font-medium">
            Подготовка к экзаменам: Алгоритмы, Структуры данных
          </p>
        </div>
      </div>
    </div>

    <div className="flex items-center w-full lg:w-auto">
      <button
        id="tab-guide-btn"
        onClick={() => window.scrollTo({ top: 0, behavior: 'smooth' })}
        className="hidden sm:flex flex-1 lg:flex-none text-white border border-indigo-600/80 shadow-lg shadow-indigo-500/20"
        title="Содержание пособия"
      >
        <BookOpen className="w-4 h-4" /> Учебное пособие
      </button>
      <button
        id="tab-compiler-btn"

```



```
.....
      id="download-txt-btn"
      href={`${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
      target="_blank"
      rel="noreferrer"
      className="flex items-center justify-center
      :bg-sky-600/20 border border-sky-500/30 tra
      title="Скачать полный TXT файл со всем кодом
    >
      <FileText className="w-4 h-4" /> TXT
    </a>
    <button
      id="download-html-btn"
      onClick={saveBook}
      disabled={busy}
      className="flex items-center justify-center
      er:bg-amber-600/20 border border-amber-500/
      title="Скачать всё пособие одним автономным
    >
      {busy ? <Loader2 className="w-4 h-4 animate
    </button>
  </div>
</div>
</header>
  );
};
```

.....
ФАЙЛ 60/99: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....
export function PlanarityDemo() {
  return (
    <div className="space-y-4">
      <h4 className="text-base font-bold text-white">K5
      <p className="text-xs text-slate-400">Просто смотри
```

```

    }
  }
  const lines = []; const circles = [];
  for(let i=0;i<edges.length;i++) {
    const p1=findPt(edges[i][0]),p2=findPt(
    const xing = crosses.includes(i);
    lines.push(<line key={`l${i}`} x1={p1.x
    stroke={xing ? "#f43f5e" : "#4f46e5"}
  }
  const labels = ["A","B","C","D","E"];
  for(const p of pts) {
    circles.push(<g key={`c${p.id}`}>
      <circle cx={p.x} cy={p.y} r={8} fill=
      <text x={p.x} y={p.y+3} textAnchor="m
    </g>);
  }
  return <>{lines}{circles}</>;
})()})
</svg>
<div className="absolute bottom-2 left-2 right
  z-20">
  V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 bor
  <div className="absolute top-2 left-2 bg-slate
    00/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
  <svg className="w-full h-full absolute inset-
    {{{}} => {
      const pts = [
        {id:0,x:60,y:60},{id:1,x:160,y:60},{id:
        {id:3,x:60,y:220},{id:4,x:160,y:220},{id:
      ];
      const edges = [
        [0,3],[0,4],[0,5],
        [1,3],[1,4],[1,5],

```



```

.....

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_URL)
      .then((module as PyodideModule).loadPyodide({ indexURL:
      }));
  }
  return runtimePromise;
}

function errorText(error: unknown) {
  if (error instanceof Error) return error.message;
  return String(error);
}

/* .....
/*  Шаблон синхронизации: короткая шапка + минимальный
/*  скелет, шаг-в-шаг с подсветкой визуализации (см. da
/*  .....

/* .....
/*  Шпаргалка Python: команды сортировки, циклы, структ
/*  «Вставить» кладёт сниппет в редактор под курсор.
/*  .....

```

Универсальное пособие • полный исходный код

```
.....
    label: "for i, x in enumerate(...)",
    tip: "Индекс и элемент одновременно: i — позиция",
    code: "for i, x in enumerate(['a', 'b', 'c']):",
},
],
},
{
    group: "Структуры",
    items: [
        {
            label: "Стек (список)",
            tip: "LIFO, как рекурсия DFS: append = push, pop = push",
            code: 'st = []\nst.append("A")    # push\nst.append("B")\nst.pop()    # pop',
        },
        {
            label: "Очередь (deque)",
            tip: "FIFO для BFS: popleft() за O(1). list.pop(0) за O(n)",
            code: 'from collections import deque\nq = deque()\nq.append(1)\nq.append(2)\nq.popleft()',
        },
        {
            label: "Куча (heapq)",
            tip: "Мин-куча: (вес, вершина) — как очередь приоритетов",
            code: 'import heapq\nh = []\nheapq.heappush(h, (1, 'A'))\nheapq.heappush(h, (2, 'B'))\nheapq.heappop(h)',
        },
        {
            label: "Матрица n × m",
            tip: "Таблица расстояний: dist[i][j], бесконечность — float('inf')",
            code: 'n, m = 3, 4\ndist = [[float("inf")] * m for i in range(n)]',
        },
        {
            label: "Множество visited",
            tip: "Посещённые вершины: in/add за O(1).",
            code: 'visited = set()\nvisited.add("A")\nprint("A" in visited)',
        },
    ],
},
```

```

.....
const [code, setCode] = useState(() => {
  if (!savedEditor) return initTemplate;
  if (savedEditor.code !== savedEditor.template || sa
  return savedEditor.code;
});
const [stdin, setStdin] = useState("");
const [output, setOutput] = useState("Код выполнится");
const [running, setRunning] = useState(false);
const [runtimeReady, setRuntimeReady] = useState(false);
const [runtimeMessage, setRuntimeMessage] = useState("");
/** Принудительный повтор (глаз / Ctrl+Enter), даже е
const [traceRequest, setTraceRequest] = useState(0);
const [copied, setCopied] = useState(false);
const [stripTab, setStripTab] = useState<"vars" | "hi
const [outputOpen, setOutputOpen] = useState(true);
const editorRef = useRef<HTMLTextAreaElement>(null);

const [desyncedFrom, setDesyncedFrom] = useState<stri
  savedEditor &&
  savedEditor.chapterId !== chapterId &&
  savedEditor.code !== savedEditor.template &&
  savedEditor.code.trim() !== "" &&
  savedEditor.code !== template &&
  savedEditor.code !== initTemplate
    ? chapterId
    : null
);
const prevTplRef = useRef({ base: initTemplate, full:

/** Активная трасса. input нужен, чтобы изменение std
const [debug, setDebug] = useState<{ result: DebugRes
/** Esc оставляет неизменённый текст в редакторе, пока
const [editHold, setEditHold] = useState<string | null
/** Автопроход включается пользователем; первый шаг п
const [playing, setPlaying] = useState(false);
const [playbackSpeed, setPlaybackSpeed] = useState<num
  if (typeof window !== "undefined") return 1;
  const saved = Number(window.localStorage.getItem(PL

```

```

.....
    setDesyncedFrom(null);
    setStripTab("vars");
    setPlaying(false);
    setEditHold(null);
    setDebug(null);
    setTraceRequest((n) => n + 1);
}, [initTemplate, template]);

const copyCode = useCallback(async () => {
  try {
    await navigator.clipboard.writeText(code);
    setCopied(true);
    setTimeout(() => setCopied(false), 1600);
  } catch {
    setRuntimeMessage("Не получилось скопировать — бу
  }
}, [code]);

/* — Пошаговый отладчик —————

/** Связь больше не зависит от эталонных строк: совпа
const hasViz = sync.variables.length > 0;

/**
 * Снимок ПОСЛЕ выделенной строки. Контейнеры берём и
 * (там уже виден append/st[i][j]=...), но индексы цикла
 * события: после st[i][j]= Python уже мог увеличить
 */
const snapshotAt = useCallback((result: DebugResult,
  const step = result.steps[idx];
  const next = result.steps[idx + 1];
  const afterLocals = next?.locals ?? result.finalLoca
  const afterValues = next?.values ?? result.finalVal
  const beforeValues = step?.values ?? {};
  const line = step?.line ? source.split("\n")[step.l
  // Нас интересуют только простые скалярные цели (`i
  // `i +=`). Контейнеры и так всегда берутся из afte
  // внутри print(f"i={i}") не будет ошибочно принят

```

```

// Единственная точка публикации: любое перемещение т
// визуализации согласованный снимок, без side-effect
useEffect(() => {
  if (debug) publishDebug(debug.result, debug.idx, de
}, [debug, publishDebug]);

const goDebug = useCallback(
  (nextIdx: number) => {
    setDebug((d) => {
      if (!d || d.result.steps.length === 0) return d
      const idx = Math.max(0, Math.min(d.result.steps
      return { ...d, idx };
    });
  },
  []
);

const stepBy = useCallback(
  (delta: number) => {
    setDebug((d) => {
      if (!d || d.result.steps.length === 0) return d
      const idx = Math.max(0, Math.min(d.result.steps
      return { ...d, idx };
    });
  },
  []
);

const stepTo = useCallback((idx: number) => goDebug(i

const stopDebug = useCallback(() => {
  setPlaying(false);
  setEditHold(code);
  setDebug(null);
  clearVizState(chapterId);
}, [chapterId, code]);

// Базовый интервал 650 мс; множитель скорости хранит

```



```

.....
    setDebug({ result, idx: 0, src: source, input, re
    setPlaying(autoPlay && result.steps.length > 1);
    const printed = [...stdout, ...stderr].join("");
    setOutput(printed || "Программа ничего не вывела");
    setRuntimeMessage(
      result.truncated
      ? `Выполнение остановлено после ${DEBUG_STEP_
      : "Трасса готова автоматически. ← → — шаги, пр
    );
  } catch (error) {
    const message = errorText(error);
    setDebug({ result: { steps: [], error: message, t
    setOutput(`Ошибка:\n${message}`);
    setRuntimeMessage("Не удалось выполнить код — про
  } finally {
    setRunning(false);
  }
}, {chapterId, code, stdin, traceRequest, running, ru

// Код выполняется сам: debounce даёт спокойно допечата
// изменился во время загрузки Pyodide, после заверше
useEffect(() => {
  if (running || editHold === code) return;
  if (debug?.src === code && debug.input === stdin &&
  const timer = window.setTimeout(() => void debugRun
  return () => window.clearTimeout(timer);
}, {code, stdin, traceRequest, running, runtimeReady,

// Стрелки ← → в режиме отладки листают ШАГИ (а не ст
useEffect(() => {
  if (!debug) return;
  const onKey = (e: KeyboardEvent) => {
    const target = e.target as HTMLElement | null;
    if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
    if (e.metaKey || e.ctrlKey || e.altKey) return;
    if (e.key === "ArrowRight") {
      e.stopPropagation();
      e.preventDefault();

```

```

}, [debug, curDebug, snapshotAt]);
const debugChanged = curDebug?.step ? changedVars(curDebug) : [];
const debugOrdered = curDebug?.step ? orderVars(debugChanged) : [];

/** Одна и та же кнопка запуска всегда видна: из редактора
 * проигрывает её, в отладчике переключает play/pause */
const runOrTogglePlayback = useCallback(() => {
  if (running) return;
  if (!debug || debug.result.steps.length === 0) {
    void debugRun(true);
    return;
  }
  if (playing) {
    setPlaying(false);
    return;
  }
  if (debug.idx >= debug.result.steps.length - 1) stepToNext();
  setPlaying(true);
}, [debug, debugRun, playing, running, stepTo]);

/** Вставка сниппета из шпаргалки в позицию курсора.
const insertSnippet = useCallback(({snippet: string}) => {
  const ta = editorRef.current;
  setCode((current) => {
    if (!ta) return current + (current.endsWith("\n") ? "" : "\n");
    const start = ta.selectionStart ?? current.length;
    const end = ta.selectionEnd ?? start;
    const padded = (start > 0 && !current.slice(0, start).endsWith("\n")) ? "\n" : "";
    const next = current.slice(0, start) + padded + snippet + current.slice(end, current.length);
    requestAnimationFrame(() => {
      ta.focus();
      ta.selectionStart = ta.selectionEnd = start + padded.length;
    });
    return next;
  });
}, []);

return {

```

```

.....
    <div className="absolute left-1/2 top-1/2 h-1
      digo-400 transition-colors" />
    </div>
  })

  {/* Левый край меняет ширину на десктопе. */}
  {onWidthChange && (
    <div
      role="separator"
      aria-orientation="vertical"
      aria-label="Потяните, чтобы изменить ширину п
      title="Потяните, чтобы изменить ширину панели
      className="hidden lg:block absolute top-0 bot
      onPointerDown={(e) => {
        const el = e.currentTarget;
        el.setPointerCapture(e.pointerId);
        const startX = e.clientX;
        const startW = width ?? 560;
        const onMove = (ev: PointerEvent) => {
          const next = Math.round(startW + (startX
            onWidthChange(Math.max(380, Math.min(next
        });
        const onUp = () => {
          el.removeEventListener("pointermove", onM
          el.removeEventListener("pointerup", onUp
        };
        el.addEventListener("pointermove", onMove a
        el.addEventListener("pointerup", onUp as Ev
      })
    >
    <div className="absolute left-1/2 top-1/2 -tr
      digo-500 transition-colors" />
    </div>
  })
  {/* — Заголовок панели —————
  <div className="shrink-0 flex items-center gap-2
    <span className={`w-2 h-2 rounded-full shrink-0
    <Terminal className="w-4 h-4 text-emerald-400 s

```

```

.....
      type="button"
      onClick={resync}
      className="p-1.5 rounded-lg text-slate-400"
      aria-label="Сбросить к шаблону страницы"
    >
      <RotateCcw className="w-4 h-4" />
    </button>
  </Tooltip>
  <Tooltip side="bottom" content="Скрыть панель"
    <button
      type="button"
      onClick={onClose}
      className="p-1.5 rounded-lg text-slate-400"
      aria-label="Закрыть компилятор"
    >
      <X className="w-4 h-4" />
    </button>
  </Tooltip>
  <Tooltip
    side="bottom"
    content="Вставить полный эталонный код этого"
  >
    <button
      type="button"
      onClick={() => {
        setCode(template);
        setDesyncedFrom(null);
        setPlaying(false);
        setEditHold(null);
        setDebug(null);
        setTraceRequest((n) => n + 1);
      }}
      className="p-1.5 rounded-lg text-indigo-300"
      aria-label="Вставить эталонный код"
    >
      <Eye className="w-4 h-4" />
    </button>
  </Tooltip>

```

```

stripTab === "hints" ? "bg-slate-800 text-white" : "bg-white text-slate-800"
  }` }
>
  <HelpCircle className="w-3.5 h-3.5" /> Подсказка
</button>
<div className="ml-auto pr-1">
  <Tooltip
    side="bottom"
    content={
      hasViz
        ? "Связь идёт по именам ниже: реальный узел
а не сравнивается."
        : "На выбранной странице нет интерактивных элементов"
    }
  >
    <span
      tabIndex={0}
      className={`cursor-help inline-flex items-center gap-1.5
        hasViz ? "text-emerald-400" : "text-slate-400"}
    >
      <Link2 className="w-3 h-3" />
      {hasViz ? "синхронизировано" : "нет визуализации"}
    </span>
  </Tooltip>
</div>
</div>

{stripTab === "vars" ? (
  <div className="px-3 pb-2.5 space-y-2 max-h-300">
    <div className="flex items-center gap-2 min-h-300">
      <Tooltip side="bottom" content="Открыть эту страницу в редакторе"
        >
        <button
          type="button"
          onClick={onOpenGuide}
          className="inline-flex items-center gap-1.5 text-white
700 text-slate-300 hover:text-white hover:bg-slate-700"
        >

```

```

.....
        tabIndex={0}
        className="cursor-help rounded-md
font-bold text-emerald-300 hover:bg-emerald-300
        >
        {v.name}
      </code>
    </Tooltip>
  )))
  {sync.stepNote && (
    <Tooltip side="bottom" content={`War
    <span
      tabIndex={0}
      className="cursor-help inline-flex
text-[10px] text-slate-400 hover:text-white
    >
      <ArrowRightLeft className="w-3 h-3
      <span className="truncate max-w-5
    </span>
    </Tooltip>
  )}
</div>
)}
</div>
) : (
  <div className="px-3 pb-2.5 max-h-36 overflow
    {SNIPPETS.map((gItem) => (
      <div key={gItem.group}>
        <div className="text-[9px] font-bold up
        <div className="flex flex-wrap gap-1.5">
          {gItem.items.map((s) => (
            <Tooltip key={s.label} side="bottom
            <button
              type="button"
              onClick={() => insertSnippet(s.
              className="inline-flex items-center
- [10.5px] text-indigo-300 hover:text-white
              title=""
            >

```

```

</Tooltip>
<Tooltip content="Шаг назад (←) – автопро
  <button type="button" onClick={() => {
    -md text-slate-400 hover:text-white hover:b
    <ChevronLeft className="w-4 h-4" />
  } />
</button>
</Tooltip>
<Tooltip content="Шаг вперёд (→) – автопро
  <button type="button" onClick={() => {
    sult.steps.length - 1} className="p-1 round
    tion-colors">
    <ChevronRight className="w-4 h-4" />
  } />
</button>
</Tooltip>
<Tooltip content="В конец трассы">
  <button type="button" onClick={() => {
    tep || debug.idx >= debug.result.steps.leng
    0 disabled:opacity-40 transition-colors">
    <SkipForward className="w-4 h-4" />
  } />
</button>
</Tooltip>
<span className="ml-1.5 text-[10px] text-
  {curDebug?.step ? (
    <
      var {debug.idx + 1}/{debug.result.s
      {curDebug.step.func !== "<module>"
    } />
  ) : (
    "шагов нет – исправьте ошибку и начини
  )}
</span>
{debug.result.truncated && (
  <Tooltip content={`Выполнение остановлен
    <span tabIndex={0} className="cursor-i
    rounded-full px-1.5 py-0.5">
      ЛИМИТ
    </span>
  </Tooltip>

```

```

>
    {active ? "▶" : ▶}
  </span>
  <span>{line || " "}</span>
</div>
  );
  }}}
</div>

{/* панель переменных – «Variables», как в
<div className="shrink-0 max-h-[34%] overflow-hidden" >
  <div className="flex items-center gap-1.5" >
    <Variable className="w-3 h-3 text-indigo-500" />
    Переменные на этом шаге
    <span className="normal-case tracking-normal" />
  </div>
  {debug.result.error && (
    <pre className="mt-1 whitespace-pre-wrap" />
  )}
  {debugOrdered.length === 0 && !debug.result.error && (
    <div className="mt-1 text-[11px] text-slate-500" />
  )}
  </div>
  {
    <div className="mt-1 flex flex-wrap gap-1" >
      {debugOrdered.map(([name, value]) => {
        const changed = debugChanged[name];
        const synced = syncVarSet.has(name);
        return (
          <span
            key={name}
            className={`inline-flex items-baseline gap-1.5
              ${changed ? "border-amber-500/50 bg-amber-500/10" : ""}
              ${synced ? "border-emerald-500/40 bg-emerald-500/10" : ""}
              ${!changed & !synced ? "border-slate-700 bg-slate-800" : ""}
            `}
          >
            {value}
          </span>
        );
      })}
    </div>
  }
}

```



```

.....
        id="python-stdin"
        value={stdin}
        onChange={(event) => {
            setEditHold(null);
            setStdin(event.target.value);
        }}
        spellCheck={false}
        placeholder="строки ввода через ↵"
        className="w-full bg-transparent font-mono"
    />
</div>
</div>

{/* — Вывод ————— */}
<div className="shrink-0 border-t border-slate-800" >
    <button
        type="button"
        onClick={() => setOutputOpen((o) => !o)}
        className="flex items-center justify-between"
        colors"
    >
        <span className="inline-flex items-center gap-2" >
            <Terminal className="w-3.5 h-3.5 text-emerald-500" />
        </span>
        <span className="inline-flex items-center gap-2" >
            {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />}
            <span className="text-slate-600">{outputOpen ? "Закрыть" : "Открыть"}</span>
        </span>
    </button>
    {outputOpen && (
        <pre className="flex-1 min-h-[70px] overflow-hidden leading-5 text-slate-300">
            {output}
        </pre>
    )}
</div>
</aside>

```



```

        setTimeout(() => {
          // Step 2: Guest leaves with happy face, queue sh
          setQueue(prev => prev.map((c, idx) => idx === 0 ?

          setServedHistory(prev => [`${headGuest.emoji} ${h

          setTimeout(() => {
            setQueue(prev => prev.slice(1));
            setIsServing(false);
            addLog(`  ${headGuest.name} поел и ушел сытым.
          }, 400);
        }, 900);
      }, [queue, isServing]));

const reset = () => {
  counter = 3;
  setQueue([
    { id: 'r1', name: 'Студент Вася', emoji: ' ', c
      т голода после матана!', animState: 'in' },
    { id: 'r2', name: 'Кодер Семён', emoji: ' ',
      апишу ИИ за порцию борща!', animState: 'in' },
    { id: 'r3', name: 'Кот Борис', emoji: ' ', col
      без сметаны, плиз.', animState: 'in' },
  ]);
  setServedHistory([]);
  setIsServing(false);
  setLog([' Столовая сброшена. Снова голодная толпа!
  setTimeout(() => {
    setQueue(prev => prev.map(c => ({ ...c, animState
  }, 500);
};

return {
  <div className="flex flex-col gap-6">
    /* МНЕМОНИКА / ПРАВИЛО */
    <div className="bg-indigo-950/40 border border-in
      <div className="text-3xl"> </div>

```



```

.....
        <div className={`
            w-14 h-14 rounded-2xl border-1
            relative
            ${customer.color}
            ${isHead ? 'ring-4 ring-amber-500' : ''}
        `}>
            <span className="text-3xl">{customer.name}</span>

            {/* Индексы HEAD/TAIL */}
            {(isHead || isTail) && (
                <span className={`
                    absolute -bottom-2 text-[0.8em]
                    ${isHead ? 'bg-amber-500 text-black' : ''}
                `}>
                    {isHead && isTail ? 'h+t' : isHead ? 'h' : 't'}
                </span>
            )}
        </div>
    </div>
</div>

{/* Пол кухни */}
<div className="w-full h-3 bg-gradient-to-r from-amber-500 to-amber-200">
</div>

{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
<div className="md:col-span-3 bg-slate-900 rounded-2xl p-4">
    <div className="absolute top-4 left-4 text-[0.8em] text-white">
        Сытые гости (Архив FIFO)
    </div>

    <div className="absolute top-4 right-4 flex items-center justify-between">

```



```

.....
        count++;
        newNodes[curr].children[c] = count;
        newNodes[count] = {
            id: count,
            char: c,
            parent: curr,
            children: {},
            fail: 0,
            term_link: 0,
            is_terminal: false,
            words: [],
            depth: newNodes[curr].depth + 1
        };
    }
    curr = newNodes[curr].children[c];
}
if (!newNodes[curr].words.includes(word)) {
    newNodes[curr].words.push(word);
    newNodes[curr].is_terminal = true;
}
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
    const childKeys = Object.values(newNodes[u].children);
    newNodes[u].y = 40 + depth * paddingY;

    if (childKeys.length === 0) {
        newNodes[u].x = 40 + currentX * paddingX;
        currentX++;
    } else {
        let leftX = -1;
        let rightX = -1;
        childKeys.forEach(v => {
            layoutDFS(v, depth + 1);
        });
    }
}

```

```

.....
const removeWord = (wordToRemove: string) => {
  if (words.length <= 1) {
    setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем продолжить!');
    return;
  }
  const updated = words.filter((w) => w !== wordToRemove);
  setWords(updated);
  buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {
  const rootChildren = Object.values(nodes[0].children);
  const queue = [...rootChildren];
  const updatedNodes = { ...nodes };
  rootChildren.forEach((childId) => {
    updatedNodes[childId].fail = 0;
  });

  setNodes(updatedNodes);
  setBfsQueue(queue);
  setSimulationStep(1);
  setCurrentNode(null);
  setSpeechText(
    'Начинаем обход в ширину (BFS)! Все вершины на главной вершине  

    уже суффикса просто нет. Жми «Следующий шаг»!'
  );
};

// Один шаг BFS
const stepBFS = () => {
  if (bfsQueue.length === 0) {
    setSimulationStep(2);
    setCurrentNode(null);
    setSpeechText(
      'Ура! Очередь пуста! Мы успешно построили полную таблицу переходов!  

      Теперь можно полюбоваться таблицей переходов!'
    );
  }
};

```



```

.....
};

return {
  <div className="bg-slate-800/90 rounded-2xl p-6 border"
    /* Шапка виджета */
    <div className="flex flex-wrap items-center justify-between"
      <div className="flex items-center space-x-3">
        <span className="text-3xl"> </span>
        <div>
          <h4 className="text-2xl font-bold text-white">
            Интерактивный конструктор: Говорящий Эхо-
          </h4>
          <p className="text-sm text-slate-400">
            Построение дерева и расчет суффиксных ссы
          </p>
        </div>
      </div>
    <div className="flex items-center gap-2">
      <button
        onClick={() => buildInitialTrie(words)}
        className="flex items-center gap-1 bg-slate-700 text-white
          transition-colors"
        title="Сбросить автомат"
      >
        <RotateCcw className="w-4 h-4" /> Сбросить
      </button>
    </div>
  </div>

  /* Верхняя панель: Говорящий лосось и диалоговое
  <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
    /* Аватар Лосося (Сеня) с анимацией моргания и
    <div className="flex flex-col items-center justify-center"
      <div className="w-40 h-40 relative flex items-center justify-center
        full border border-blue-500/30 shadow-inner"
        /* SVG Лосося */
        <svg
          viewBox="0 0 200 200"

```

```

        ) : (
          <path d="M 165 110 Q 155 115 168 118" s
        )}

    {/* Улыбка / жабры */}
    <path d="M 130 80 Q 125 100 130 120" stro
    <path d="M 120 85 Q 115 100 120 115" stro
  </svg>

  {/* Декоративные пузыри */}
  <div className="absolute -top-1 right-4 tex
  <div className="absolute top-8 right-1 text
</div>
<span className="mt-3 bg-rose-600/30 border b
  Эхо-Лосось Сеня
</span>
</div>

{/* Пузырь с текстом (Баббл) */}
<div className="md:col-span-2 bg-slate-800 bord
  tween min-h-[140px]">
  {/* Треугольник баббла */}
  <div className="absolute -left-3 top-1/2 -tra
    slate-800 border-b-[12px] border-b-transpar

  <div className="flex items-center space-x-2 t
    <Volume2 className={`w-4 h-4 ${isTalking ?
    <span>Прямое вещание из аквариума:</span>
  </div>

  <p className="text-slate-200 text-base leadin
    {speechText}
  </p>

  <div className="mt-4 pt-3 border-t border-sla
    <div className="text-xs text-slate-400 flex
      <Info className="w-3.5 h-3.5 text-blue-40
      Статус: {simulationStep === 0 ? 'Бор рот

```

```
.....
<div className="mb-8 bg-slate-900 border border-s
  <h5 className="text-white font-bold mb-3 flex i
    <span> </span> Дерево и Суффиксные (fail) ссы
  </h5>
```

```
{(() => {
  let maxX = 0;
  let maxY = 0;
  Object.values(nodes).forEach(n => {
    if (n.x && n.x > maxX) maxX = n.x;
    if (n.y && n.y > maxY) maxY = n.y;
  });
  const svgWidth = Math.max(maxX + 80, 400);
  const svgHeight = Math.max(maxY + 60, 200);

  return (
    <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
      <defs>
        <marker id="arrowhead" markerWidth="6"
          <polygon points="0 0, 6 3, 0 6" fill=
        </marker>
        <marker id="fail-arrow" markerWidth="6"
          <polygon points="0 0, 6 3, 0 6" fill=
        </marker>
      </defs>

      </* Draw Edges */>
      {Object.values(nodes).map(node => {
        if (node.id === 0) return null;
        const parent = nodes[node.parent];
        if (!parent.x || !parent.y || !node.x ||

        const isCurrent = currentNode === node.

        return (
          <g key={`edge-${node.id}`}>
            <line
              x1={parent.x}
```

```

.....
        fill="none"
        stroke={isCurrent ? '#f43f5e' : 'rgb(0,0,0)'}
        strokeWidth={isCurrent ? 2 : 1.5}
        strokeDasharray="4 4"
        markerEnd="url(#fail-arrow)"
      />
    );
  })}

{/* Draw Nodes */}
Object.values(nodes).map(node => {
  if (!node.x || !node.y) return null;
  const isRoot = node.id === 0;
  const isCurrent = currentNode === node.id;
  const isInQueue = bfsQueue.includes(node.id);

  let fill = '#1e293b';
  let stroke = '#475569';

  if (isCurrent) {
    fill = '#2563eb';
    stroke = '#60a5fa';
  } else if (isInQueue) {
    fill = '#b45309';
    stroke = '#fbbf24';
  } else if (node.is_terminal) {
    fill = '#059669';
    stroke = '#34d399';
  }

  return (
    <g key={`node-${node.id}`}>
      <circle
        cx={node.x}
        cy={node.y}
        r="16"
        fill={fill}
        stroke={stroke}

```

```

.....
      {words.map((w) => (
        <span
          key={w}
          className="bg-slate-800 border border-
er gap-1 group"
        >
          {w}
          <button
            onClick={() => removeWord(w)}
            className="text-slate-500 hover:tex
            title="Удалить"
          >
            ×
          </button>
        </span>
      )})
    </div>
  </div>

  <form onSubmit={handleAddWord} className="fle
    <input
      type="text"
      value={newWord}
      onChange={(e) => setNewWord(e.target.valu
      placeholder="НОВОЕ СЛОВО..."
      maxLength={8}
      className="bg-slate-800 border border-sla
      outline-none focus:border-blue-500"
    />
    <button
      type="submit"
      className="bg-slate-700 hover:bg-slate-60
      ition-colors"
    >
      <Plus className="w-3.5 h-3.5" /> Добавить
    </button>
  </form>
</div>

```

```

.....
      : 'hover:bg-slate-800/40'
    }` }
  >
  <td className="py-2.5 px-3 flex items-center" >
    <span
      className={`w-2 h-2 rounded-full
        node.id === 0
          ? 'bg-purple-500'
          : isCurrent
          ? 'bg-blue-400 animate-pulse'
          : isInQueue
          ? 'bg-amber-400'
          : 'bg-slate-600'
        }` }
    ></span>
    <span>#{node.id}</span>
    {node.id === 0 && <span className="font-size-2" >
  </td>
  <td className="py-2.5 px-3">
    <span className="bg-slate-800 border border-1 border-slate-600 p-1" >
      {node.char}
    </span>
  </td>
  <td className="py-2.5 px-3 text-slate-500">
    {node.id === 0 ? '-' : `#${node.id}`}
  </td>
  <td className="py-2.5 px-3">
    {node.id === 0 ? {
      <span className="text-slate-500" >
    } : {
      <span className="bg-indigo-950 border border-1 border-slate-600 p-1" >
        #{node.fail}
      </span>
    }}
  </td>
  <td className="py-2.5 px-3">
    {node.id === 0 || node.term_line === 0}
    <span className="text-slate-500" >

```

Универсальное пособие • полный исходный код

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
import { useVizRuntime, vizNumber } from '../data/vizStore';
```

```
// ===== TYPES =====
```

```
interface Step {
  id: number;
  desc: string;
  codeLine: number;
  treeState: Record<number, number>; // node index
  highlightNodes: number[]; // current highlights
  mergeChildren?: [number, number]; // pair being merged
  arrayHighlight?: [number, number]; // [L, R] range
  result?: number; // partial result
  lazyState?: Record<number, number>; // node index for lazy
  promisePushed?: number; // узел, у которого есть promise
}
```

```
// ===== PURE LOGIC: build tree =====
```

```
export function buildTreeFull(arr: number[]): Record<number, number> {
  const n = arr.length;
  const tree: Record<number, number> = {};
  function go(v: number, l: number, r: number) {
    if (l === r) { tree[v] = arr[l]; return; }
    const m = (l + r) >> 1;
    go(2 * v, l, m);
    go(2 * v + 1, m + 1, r);
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
  }
  go(1, 0, n - 1);
  return tree;
}
```

```
// ===== STEP GENERATORS =====
```

```

.....
    state: { ...tree }, highlightNodes: [v], mergeChildren: true
  }
  go(l, 0, n - 1);
  steps.push({ id: id++, desc: `Дерево построено! Корень: ${v}`
    Highlight: [0, n - 1] });
  return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
  const n = arr.length;
  const tree = buildTreeFull(arr);
  const steps: Step[] = [];
  let id = 0;

  function go(v: number, l: number, r: number, ql: number, qr: number) {
    if (ql > r || qr < l) {
      steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`
        codeLine: 2, treeState: tree, highlightNodes: [v]
      });
      return 0;
    }
    if (ql <= l && r <= qr) {
      steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`
        value: arr[v], codeLine: 4, treeState: tree, highlightNodes: [v]
      });
      return tree[v];
    }
    const m = (l + r) >> 1;
    steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`
      codeLine: 6, treeState: tree, highlightNodes: [v]
    });
    const left = go(2 * v, l, m, ql, qr);
    const right = go(2 * v + 1, m + 1, r, ql, qr);
    const res = left + right;
    steps.push({ id: id++, desc: `Возврат в узел ${v}: ${res}`
      mergeChildren: [2 * v, 2 * v + 1], arrayHighlight: [l, r]
    });
    return res;
  }

  const ans = go(1, 0, n - 1, ql, qr);
  steps.push({ id: id++, desc: `Запрос sum([${ql}..${qr}]: ${ans}`

```



```

.....
    r >>= 1;
    if (l < r) {
        steps.push({ id: id++, desc: `Поднимаемся: l = ${l}, r = ${r}, arrayHighlight: [qL, qR], result: res });
    }
}
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}], arrayHighlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val: number; left: VNode; right: VNode; }
```

```

function buildVisualTree(treeState: Record<number, number>) {
    if (l === r) return { v, l, r, val: treeState[v] ?? null, left: null, right: null };
    const m = (l + r) >> 1;
    return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(treeState, l, m), right: buildVisualTree(treeState, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```

const BUILD_CODE = [
    "build(v, l, r) {",
    "    if (l === r) { tree[v] = A[l]; return; }",
    "    // -----",
    "    // -----",
    "    mid = (l + r) >> 1;",
    "    build(2*v, l, mid);",
    "    build(2*v+1, mid+1, r);",
    "    tree[v] = tree[2*v] + tree[2*v+1];",
    "}"
]

```

```
};
```

```

const QUERY_TD_CODE = [
    "query(v, l, r, ql, qr) {",
    "    if (ql > r || qr < l) return 0;",
    "    // -----",
    "    if (ql <= l && r <= qr) return tree[v];",
    "}"
]

```

```

.....

export function genLazySteps(arr: number[], uL: number,
  const n = arr.length;
  const steps: Step[] = [];
  let id = 0;
  const tree: Record<number, number> = {};
  const lazy: Record<number, number> = {};
  for (const k of Object.keys(buildTreeFull(arr))) {
    tree[Number(k)] = buildTreeFull(arr)[Number(k)];
    lazy[Number(k)] = 0;
  }
  const full = buildTreeFull(arr);

  const snap = {desc: string, codeLine: number, hl: number,
    steps.push({
      id: id++,
      desc,
      codeLine,
      treeState: { ...tree },
      highlightNodes: [...hl],
      lazyState: { ...lazy },
      ...extra,
    });
  };
  snap(`Старт: дерево построено (корень = ${full[1]}).
  `);

function pushDown(v: number, l: number, r: number) {
  if (lazy[v] !== 0) {
    const m = (l + r) >> 1;
    const d = lazy[v];
    snap(`push(${v}): обещание +${d} с узла ${v} уходит
      lazy.` , 8, [v, 2 * v, 2 * v + 1], { promisePush: true });
    tree[2 * v] = (tree[2 * v] ?? 0) + d * (m - l + 1);
    lazy[2 * v] = (lazy[2 * v] ?? 0) + d;
    tree[2 * v + 1] = (tree[2 * v + 1] ?? 0) + d * (r - m);
    lazy[2 * v + 1] = (lazy[2 * v + 1] ?? 0) + d;
    lazy[v] = 0;
  }
}

```

```

.....
    res += tree[v] ?? 0;
    snap(`query(v=${v}, [l..r]): целиком внутри
      sult: res ));
    return;
  }
  pushDown(v, l, r);
  const m = (l + r) >> 1;
  snap(`query(v=${v}, [l..r]): частичное пересече
    );
  query(2 * v, l, m);
  query(2 * v + 1, m + 1, r);
}
res = 0;
snap(`Теперь запрос sum([uL..uR]) – проверим, ч
query(1, 0, n - 1);
snap(`  sum([uL..uR]) = ${res} – совпадает с че
return steps;
}

```

// ===== PLAYER COMPONENT =====

```

function Player({ steps, color }: { steps: Step[]; color: string }) {
  const [idx, setIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [speed, setSpeed] = useState(700);

  useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

  useEffect(() => {
    if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
      const t = setTimeout(() => setIdx(i => i + 1), speed);
      return () => clearTimeout(t);
    }
  }, [playing, idx, steps, speed]);

  const step = steps[idx] || steps[0];
  if (!step) return null;

  const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500',
    border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-600'
  }

```

```

.....
    <span className="text-xs font-extrabold font-mono" style={
      {promise !== 0 && {
        <span className="mt-0.5 px-1 rounded-full border border-gray-200 whitespace-nowrap">
          +{promise}
        </span>
      }}
    </div>
    {(nd.left || nd.right) && (
      <div className="flex flex-col items-center mt-1" style={
        <div className="w-px h-2 bg-slate-700" />
        <div className="flex justify-around w-full" style={
          {nd.left && renderNode(nd.left)}
          {nd.right && renderNode(nd.right)}
        </div>
      </div>
    )}
  </div>
);
}, [step, clr, liveNode]);

return (
  <div className={`rounded-2xl border overflow-hidden`} style={
    ? 'border-emerald-500/30' : 'border-amber-500/30'
  } /* Header */
  <div className={`px-4 py-2.5 flex items-center justify-between`} style={
    'bg-emerald-950/50' : 'bg-amber-950/50'
  } border-bottom border-gray-200>
    <h4 className="font-bold text-sm text-white flex-grow-1">
      <span className="text-[10px] font-mono text-slate-300">
    </div>

    /* Array strip */
    <div className="px-3 py-2 bg-slate-950 border-bottom border-gray-200">
      {arr.map((v, i) => {
        const inRange = step.arrayHighlight && i >= step.start
        return (
          <div key={i} className={`flex flex-col item-center`} style={
            inRange ? 'bg-indigo-950 border-indigo-500' : 'bg-transparent'
          }>

```

```

        </span>
      )}
    </div>

    {/* Controls */}
    <div className="px-3 py-2.5 bg-slate-950 border-t-2 border-l-2 border-r-2 border-b-2" >
      <div className="flex items-center bg-slate-900 rounded" >
        <button onClick={() => { setPlaying(false); setSpeed(1200); }}
          rounded" title="Сброс"><RotateCcw className="w-4 h-4" />
        <button onClick={() => { setPlaying(false); setSpeed(700); }}
          0 hover:text-white hover:bg-slate-800 rounded" title="Норма"><RotateCcw className="w-4 h-4" />
        <button onClick={() => setPlaying(!playing)}
          white transition-all ${playing ? 'bg-rose-500' : 'bg-slate-900'}
          {playing ? <Pause className="w-3 h-3" /> : <Play className="w-3 h-3" />}
        </button>
        <button onClick={() => { setPlaying(false); setSpeed(120); }}
          .5 text-slate-400 hover:text-white hover:bg-slate-800 rounded" title="Быстро"><RotateCcw className="w-4 h-4" />
        </button>
      </div>
      <select value={speed} onChange={e => setSpeed(Number(e.target.value))}
        text-[9px] font-bold rounded px-1.5 py-1 outline-1 outline-slate-700" >
        <option value={1200}>Медл.</option>
        <option value={700}>Норма</option>
        <option value={350}>Быстро</option>
        <option value={120}>Турбо</option>
      </select>
    </div>

    {/* Progress bar */}
    <div className="h-1 bg-slate-950"><div className="bg-rose-500" style={{ width: ((currentTime + 1) / total) * 100 : 0}% }} /></div>
  </div>
);
}

// =====
// MAIN EXPORTED COMPONENT
// =====

```

Универсальное пособие • полный исходный код

```
.....
// Питон умеет рулить добавкой: переменная add из кода
const effAdd = liveAdd ?? uAdd;
const lazySteps = useMemo(() => genLazySteps(arr, uL,

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
  <div className="bg-slate-900 rounded-2xl border border-
    /* ---- TOP BAR: array editor + query range ---- */
    <div className="mb-6 space-y-4">
      <div className="flex flex-col lg:flex-row gap-4">
        /* Array input */
        <form onSubmit={handleApply} className="flex-
          <div className="flex items-center justify-b
            <label className="text-[10px] font-bold up
              label>
                <button type="button" onClick={randomize}
              "><RefreshCw className="w-3.5 h-3.5" /></bu
            </div>
            <div className="flex gap-2">
              <input
                value={customInput}
                onChange={e => setCustomInput(e.target.v
                className="flex-1 bg-slate-900 border b
              one focus:border-indigo-500"
                placeholder="5, 8, 3, 12, 7, 2"
              />
              <button type="submit" className="bg-indig
                ition-colors">Применить</button>
            </div>
            <div className="flex flex-wrap gap-1.5 pt-1
              {arr.map((v, i) => (
                <span key={i} className={`border px-2 p
                  ext-amber-200 ring-2 ring-amber-400/30` : "
                  <span className={liveI === i ? "text-
                    </span>
                  </span>
                )}}
              <span className="text-xs text-slate-500 f
```

```

{ /* ---- TABS ---- */
<div className="flex flex-wrap items-center gap-1"
  <button onClick={() => setView('build')} className="flex items-center justify-center gap-1"
    -colors ${view === 'build' ? 'border-indigo-500 text-white' : 'border-slate-200'}`}>
    <Hammer className="w-3.5 h-3.5" /> Построить
  </button>
  <button onClick={() => setView('queries')} className="flex items-center justify-center gap-1"
    on-colors ${view === 'queries' ? 'border-emerald-500 text-white' : 'border-slate-200'}`}>
    <Search className="w-3.5 h-3.5" /> Запрос: с
  </button>
  <button onClick={() => setView('lazy')} className="flex items-center justify-center gap-1"
    colors ${view === 'lazy' ? 'border-amber-500 text-white' : 'border-slate-200'}`} title="Массовое обновление на отрезке"
    Обновление на отрезке (Lazy)
  </button>
  <button onClick={() => setView('theory')} className="flex items-center justify-center gap-1"
    n-colors ${view === 'theory' ? 'border-blue-500 text-white' : 'border-slate-200'}`}>
    <Info className="w-3.5 h-3.5" /> Почему / Сравн
  </button>
</div>

{ /* ---- TAB CONTENT ---- */
{view === 'build' && (
  <SimPanel
    key={`build-${arr.join('-')}`}
    title="Построение дерева (рекурсия)"
    icon=""
    steps={buildSteps}
    code={BUILD_CODE}
    color="indigo"
    arr={arr}
  />
)}
}

```

```

.....
    <input type="number" value={effAdd} onChange={
      border-slate-700 text-white text-xs font-mono
    } ? 'задана переменной add в Python-компиляторе' : ''
  </div>
  <p className="text-[11px] text-slate-500 margin-top-10">
    Ко всем элементам отрезка прибавляем <b>сумму</b> (в
    емся только по границе, а внутри вешаем стилизацию
    суммы).
    {liveAdd !== null && <span className="ml-2">{liveAdd}</span>
    {liveV !== null && <span className="ml-2">{liveV}</span>
  </p>
</div>
<SimPanel
  key={`lazy-${arr.join('-')}-${uL}-${uR}-${effAdd}`}
  title={`update([${uL}..${uR}], +${effAdd})`}
  icon=""
  steps={lazySteps}
  code={LAZY_CODE}
  color="amber"
  arr={arr}
  liveNode={liveV}
/>
</div>
)}

{view === 'theory' && (
  <div className="space-y-5">
    {/* Why it splits this way */}
    <div className="bg-slate-950 p-5 rounded-xl border">
      <h4 className="text-base font-bold text-white">
        Почему разбиение именно так для N={arr.length}?
      </h4>
      <p className="text-sm text-slate-300 leading-relaxed">
        Каждый узел берёт <code>className="bg-slate-900 px-1"</code>
        и получает <code>className="bg-slate-900 px-1"</code>
        -slate-900 px-1 rounded font-mono text-emerald-400
        евая часть забирает на 1 больше (округление)
      </p>
      <div className="bg-slate-900 p-4 rounded-lg">

```



```

.....
      <button
        onClick={() => setQuery("")}
        className="absolute right-2 top-1/2 -tran
        aria-label="Очистить поиск"
      >
        <X className="w-3.5 h-3.5" />
      </button>
    })
  </div>
</div>

<div className="flex-1 overflow-y-auto px-3 pb-4
  {groups.length === 0 && <p className="text-sm t

  {groups.map(([category, items]) => (
    <div key={category}>
      <div className="text-[10px] font-bold upper
      <div className="space-y-1">
        {items.map((chapter) => {
          const isSelected = chapter.id === selec
          const hasViz = Boolean(VIZ_REGISTRY[cha
          return (
            <button
              key={chapter.id}
              onClick={() => {
                setSelectedChapterId(chapter.id);
                onNavigate?.();
                window.scrollTo({ top: 0, behavio
              }}
              className={`w-full text-left px-3 p
                isSelected
                  ? "bg-indigo-600/15 border-indi
                  : "bg-slate-800/40 border-transp
              }`}
            >
              {hasViz && (
                <span
                  className="w-1.5 h-1.5 rounded-

```

Универсальное пособие • полный исходный код

```
.....
import React, { useState } from "react";
import { SplayRotationsViz } from "../SplayRotationsViz";
import SplayWalkViz from "../SplayWalkViz";
import { SplayRotationSandbox } from "../SplayRotationSandbox";

/**
 * Страница splay – три вкладки по нарастанию сложности
 * 1. «Механика» – покадровый разбор Zig / Zig-Zig / Zig-Zag
 *    (прицел → поворот, призраки старых позиций, обход)
 * 2. «Большое дерево» – те же случаи внутри настоящего дерева
 *    поворот – кадры «прицел → отстёгиваем среднее по высоте»
 * 3. «Собери сам» – песочница: подними узел кликами, построй дерево
 */
type Tab = "cases" | "big" | "play";

export const SplayPlayground: React.FC<{ defaultTab?: Tab }> = ({ defaultTab }) => {
  const [tab, setTab] = useState<Tab>(defaultTab);

  return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
      <div className="flex flex-wrap items-center justify-between">
        <div className="flex items-center gap-3">
          <div className="p-2.5 bg-indigo-600 text-white rounded">
            <span className="text-2xl"> </span>
          </div>
          <div>
            <h3 className="text-2xl font-bold text-white">
              <p className="text-sm text-indigo-300">Один
            </p>
          </div>
        </div>
      </div>

      <div className="flex flex-wrap gap-2 mb-5">
        {[
          { id: "cases" as Tab, label: "1 · Механика: Zig / Zig-Zig / Zig-Zag" },
          { id: "big" as Tab, label: "2 · Большое дерево" },
          { id: "play" as Tab, label: "3 · Собери сам" }
        ]}.map((t) => (
```

Универсальное пособие • полный исходный код

```
.....
*
* Здесь ничего не подсказывается: дано настоящее дерев
* поднять отмеченный узел в корень. Клик по узлу = оди
* Порядок поворотов выбираешь сам; правильный ли он по
* только после того, как узел окажется наверху (или по
*/

export interface TNode {
  key: number;
  left: TNode | null;
  right: TNode | null;
}

const leaf = (key: number): TNode => ({ key, left: null

const clone = (t: TNode | null): TNode | null =>
  t ? { key: t.key, left: clone(t.left), right: clone(t

/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
export const PRESETS: { name: string; hintCase: string;
  {
    name: "Линия влево",
    hintCase: "Zig-Zig",
    target: 1,
    build: () => {
      const n1 = leaf(1);
      const n2: TNode = { key: 2, left: n1, right: leaf
      const n4: TNode = { key: 4, left: n2, right: leaf
      return { key: 6, left: n4, right: leaf(7) };
    },
  },
  {
    name: "Змейка",
    hintCase: "Zig-Zag",
    target: 3,
    build: () => {
      const n3 = leaf(3);
      const n2: TNode = { key: 2, left: leaf(1), right:
```

```

.....
const path = findPath(root, key);
if (path.length < 2) return root;

const x = path[path.length - 1];
const p = path[path.length - 2];
const gg = path.length >= 3 ? path[path.length - 3] :

if (p.left === x) {
  p.left = x.right;
  x.right = p;
} else {
  p.right = x.left;
  x.left = p;
}

if (!gg) return x;
if (gg.left === p) gg.left = x;
else gg.right = x;
return root;
}

```

```

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number)
  const path = findPath(root, key);
  if (path.length < 2) return null;
  const x = path[path.length - 1];
  const p = path[path.length - 2];
  const g = path.length >= 3 ? path[path.length - 3] :

  if (!g) return { node: x.key, kind: "Zig" };

  const xLeft = p.left === x;
  const pLeft = g.left === p;
  if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
  return { node: x.key, kind: "Zig-Zag (сначала нижняя" };
}

```

```

/* ..... раскладка .....

```

```

.....
const [presetIdx, setPresetIdx] = useState(0);
const preset = PRESETS[presetIdx];

const [tree, setTree] = useState<TNode>(() => preset.build());
const [history, setHistory] = useState<TNode[]>([]);
const [moves, setMoves] = useState<{ node: number; correct: boolean }>([]);
const [showAnswer, setShowAnswer] = useState(false);

const target = preset.target;
const solved = tree.key === target;

const reset = useCallback(
  (idx = presetIdx) => {
    setPresetIdx(idx);
    setTree(PRESETS[idx].build());
    setHistory([]);
    setMoves([]);
    setShowAnswer(false);
  },
  [presetIdx]
);

const { nodes, edges, width, height } = useMemo(() => preset.build(), [preset]);
const path = useMemo(() => new Set(findPath(tree, target)), [tree, target]);
const expected = useMemo(() => (solved ? null : canonicalNext(tree, target)), [tree, target]);

const clickNode = (key: number) => {
  if (solved) return;
  const p = findPath(tree, key);
  if (p.length < 2) return; // корень крутить не с кем

  const want = canonicalNext(tree, target);
  setHistory((h) => [...h, clone(tree) as TNode]);
  setMoves((m) => [...m, { node: key, correct: want?.key === key }]);
  setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {

```

```

        {p.name}
      </button>
    )))
  </div>

  <div className="bg-slate-900 rounded-xl border bo
    <svg
      viewBox={`0 0 ${width} ${height}`}
      className="h-auto block mx-auto"
      style={{ minWidth: Math.min(width * 1.5, 420)
      role="img"
    >
      {edges.map(([a, b], i) => {
        const na = nodes.find((n) => n.key === a);
        const nb = nodes.find((n) => n.key === b);
        if (!na || !nb) return null;
        const onPath = path.has(a) && path.has(b);
        return (
          <line
            key={i}
            x1={na.x}
            y1={na.y}
            x2={nb.x}
            y2={nb.y}
            stroke={onPath ? "#6366f1" : "#475569"}
            strokeWidth={onPath ? 3 : 1.8}
            style={{ transition: "all 700ms cubic-b
          />
        );
      })}
    {nodes.map((n) => {
      const isTarget = n.key === target;
      const clickable = !solved && n.depth > 0;
      return (
        <g
          key={n.key}
          onClick={() => clickable && clickNode(n

```

```

.....
        <RotateCcw className="w-3.5 h-3.5" /> Заново
    </button>
    <button
      onClick={() => setShowAnswer((s) => !s)}
      className={`flex items-center gap-1.5 px-3 py-2
        ${showAnswer
          ? "bg-amber-600/20 border-amber-500 text-amber-500"
          : "bg-slate-800 border-slate-700 text-slate-200"}
        `}
    >
      {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : ""}
      {showAnswer ? "Скрыть разбор" : "Сдаться / показать"}
    </button>

    <span className="text-[11px] text-slate-500 ml-2" >
      поворотов: {moves.length} • минимум: {minimalMoves}
    </span>
  </div>

  {showAnswer && !solved && expected && (
    <div className="mt-3 text-[12px] bg-amber-950/40 p-3" >
      Сейчас splay крутил бы узел{" "}
      <span className="font-mono font-bold">{expected}
    </div>
  )}

  {solved && (
    <div
      className={`mt-3 rounded-lg border px-3 py-2.5
        ${wrong === 0
          ? "bg-emerald-950/40 border-emerald-700/60 text-emerald-500"
          : "bg-rose-950/30 border-rose-800/60 text-rose-500"}
        `}
    >
      <div className="flex items-center gap-2 font-[12px]" >
        {wrong === 0 ? <CheckCircle2 className="w-4 h-4" /> : <X className="w-4 h-4" />}
        {wrong === 0
          ? `Узел ${target} в корне за ${moves.length}

```


Универсальное пособие • полный исходный код

```
.....
*   * у узлов настоящие числа (20 / 40 / 60), а не абстр
*   сразу видно, что дерево остаётся деревом поиска;
*   * под деревом печатается обход слева направо: он ОД
*   это и есть доказательство, что поворот ничего не
*   * кадр «прицел» ничего не двигает, только подсвечива
*   * на кадре «результат» остаются призраки – пунктирн
*   местах и стрелки «откуда → куда»;
*   * по умолчанию анимация НЕ крутится сама: пользоват
*   идёт в своём темпе. Автопрокрутка включается кноп
*/
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, {number, number}>>;
```

```
interface Frame {
  pos: Pos;
  edges: {NodeId, NodeId}[];
  kind: "start" | "aim" | "result";
  title: string;
  text: string;
  /** Пара, которую крутим на этом шаге: кадр-«прицел».
  focus?: {NodeId, NodeId};
  /** Кто переехал на этом кадре – рисуем призрак и стр
  moved?: NodeId[];
  ms: number;
}
```

```
interface Case {
  key: "zig" | "zig-zig" | "zig-zag";
  label: string;
  when: string;
  rotations: string;
  /** Номера узлов: роль → число. */
  keys: Partial<Record<NodeId, number>>;
  /** Обход слева направо – одинаковый на всех кадрах.
  inorder: string[];
  /** Что означают поддеревья A..D. */
  ranges: string;
```

```

.....
    {
      pos: { x: [160, 50], A: [80, 130], p: [245, 130],
edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
kind: "result",
title: "Стало: 20 – корень",
text: "40 стало правым сыном двадцатки. Поддерезо
      ева от 40 – это одно и то же место.",
moved: [{"x", "p", "B"}],
ms: RESULT_MS,
    },
  ],
};

/* ..... Zig-Zig .....
const ZZ_S0 = {
  pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
  pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
  key: "zig-zig",
  label: "Zig-Zig",
  when: "x и p – дети с ОДНОЙ стороны (оба слева или оба
rotations: "2 поворота, первым – верхний",
  keys: { x: 20, p: 40, g: 60 },
  inorder: ["A", "20", "B", "40", "C", "60", "D"],
  ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
  frames: [
    {
      ...ZZ_S0,
      kind: "start",
      title: "Было: прямая линия 60 → 40 → 20",
      text: "Три узла выстроились в «бамбук»: каждый сл

```

```

.....
    },
};

/* ..... Zig-Zag .....
const ZG_S0 = {
  pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
  edges: [{"g", "p"}, {"g", "D"}, {"p", "A"}, {"p", "x"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZG_S1 = {
  pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
  edges: [{"g", "x"}, {"g", "D"}, {"x", "p"}, {"x", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZAG: Case = {
  key: "zig-zag",
  label: "Zig-Zag",
  when: "x и p – дети с РАЗНЫХ сторон (змейка)",
  rotations: "2 поворота, первым – нижний",
  keys: { x: 40, p: 20, g: 60 },
  inorder: ["A", "20", "B", "40", "C", "60", "D"],
  ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
  frames: [
    {
      ...ZG_S0,
      kind: "start",
      title: "Было: 60 → влево на 20 → вправо на 40",
      text: "Узлы идут «змейкой»: сначала шаг влево, по",
      ms: RESULT_MS,
    },
    {
      ...ZG_S0,
      kind: "aim",
      title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
      text: "Дед (60) стоит на месте и в этом повороте",
      focus: ["p", "x"],
      ms: AIM_MS,
    }
  ]
};

```

```

.....

const ROLE_RU: Partial<Record<NodeId, string>> = {
  x: "x — ищем его",
  p: "p — родитель",
  g: "g — дед",
};

const isSubtree = (id: NodeId) => id === "A" || id ===

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
  const { pos, edges, focus, moved } = frame;
  const dim = (id: NodeId) => (focus ? !focus.includes(
  const ghosts = frame.kind === "result" ? (moved ?? [])

return (
  <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
    <defs>
      <marker id="splay-arrow" viewBox="0 0 10 10" re
        <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
      </marker>
    </defs>

    /* призраки: где узел стоял до поворота */
    {ghosts.map((id) => {
      const from = prev[id]!;
      const to = pos[id]!;
      const r = isSubtree(id) ? 15 : 20;
      const dx = to[0] - from[0];
      const dy = to[1] - from[1];
      const len = Math.hypot(dx, dy) || 1;
      const pad = r + 8;
      return (
        <g key={`ghost-${id}`} opacity={0.55}>
          <circle cx={from[0]} cy={from[1]} r={r} fil
            {len > pad * 2 + 6 && (
              <line
                x1={from[0] + (dx / len) * pad}
                y1={from[1] + (dy / len) * pad}

```

```

.....
    const hot = Boolean(focus && focus.includes(id))
    return (
      <g
        key={id}
        style={{
          transform: `translate(${p[0]}px, ${p[1]}px)`,
          transition: MOVE,
          opacity: dim(id) ? 0.28 : 1,
        }}
      >
        {hot && <circle r={r + 7} fill="none" stroke="black" />
        <circle r={r} fill={c.fill} stroke={c.stroke} />
        <text
          textAnchor="middle"
          dominantBaseline="central"
          fontSize={sub ? 12 : 15}
          fontWeight="700"
          fill={sub ? "#94a3b8" : "#fff"}
        >
          {sub ? id : keys[id]}
        </text>
        {!sub && (
          <text textAnchor="middle" y={-r + 7} font-size={12} font-weight="bold">
            {id}
          </text>
        )}
      </g>
    );
  }
}
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
  const liveCase = vizString(vars?.case);
  const liveFrame = vizNumber(vars?.i);

```

```

<button
  key={c.key}
  onClick={() => setCaseIdx(i)}
  className={`px-3 sm:px-4 py-2 rounded-lg te
    i === caseIdx ? "bg-indigo-600 text-white
  `}
>
  {c.label}
</button>
)}}
</div>

<div className="mb-3 text-xs sm:text-[13px] text-
  <span className="font-bold text-indigo-300">Kor,
  <span className="text-slate-500">({active.rotat
</div>

{/* шаг + пояснение стоят НАД картинкой: сначала
<div className="mb-3 rounded-xl border border-sla
  <div className="flex items-center gap-2 mb-1 fl
    <span
      className={`text-[10px] font-bold uppercase
        current.kind === "aim"
          ? "bg-amber-500/20 text-amber-300"
          : current.kind === "start"
            ? "bg-slate-700 text-slate-300"
            : "bg-indigo-500/20 text-indigo-300"
      `}
    >
      {current.kind === "aim" ? "прицел" : current
    </span>
    <span className="text-sm font-bold text-white
      Шаг {idx + 1} из {total}. {current.title}
    </span>
  </div>
  <p className="text-[13px] text-slate-400 leadin
</div>

```

```

    onClick={() => {
      setPlaying(false);
      setFrame((f) => (f - 1 + total) % total);
    }}
    disabled={idx === 0}
    className="px-3 py-2 rounded-lg bg-slate-800 i
      d: hover:text-slate-300 text-xs font-bold tr
  >
    ← Назад
</button>
<button
  onClick={() => {
    setPlaying(false);
    setFrame((f) => (f + 1) % total);
  }}
  className="px-4 py-2 rounded-lg bg-indigo-600
    w-indigo-900/40"
  >
    {last ? "Сначала" : "Дальше →"}
</button>
<button
  onClick={() => setPlaying((p) => !p)}
  className="flex items-center gap-1.5 px-3 py-
    ext-xs font-bold transition-colors"
  >
    {playing ? <Pause className="w-3.5 h-3.5" />
      {playing ? "Стоп" : "Авто"}}
</button>
<button
  onClick={() => setSlow((s) => !s)}
  title="Автопрокрутка ещё медленнее"
  className={`flex items-center gap-1.5 px-3 py
    slow
      ? "bg-emerald-600/20 border-emerald-500 t
      : "bg-slate-800 border-slate-700 text-sla
    `}
  >
    <Gauge className="w-3.5 h-3.5" /> {slow ? "0.5

```

```

.....

/**
 * Splay на БОЛЬШОМ дереве – по урокам tgear-прохода.
 *
 * Главный принцип: НИКАКИХ «хоп». Каждый одиночный поворот
 *   aim – прицел на тройку g–p–x, назван случай;
 *   cut – отстёгиваем среднее поддеревцо β (пунктир, поворот);
 *   rot – поворот: зелёным подсвечены рёбра, которых не было;
 * Рядом с логом всегда висит врезка «Механика одного поворота»;
 * На каждом кадре – живая автопроверка in-order (сортировка);
 *
 * Чистые функции (splaySteps, rotateUp, bstInsert, cloneWithParents);
 */

export interface SN {
  key: number;
  left: SN | null;
  right: SN | null;
  parent: SN | null;
}

export function cloneWithParents(t: SN | null): SN | null {
  if (!t) return null;
  const n: SN = { key: t.key, left: null, right: null, parent: null };
  const l = t.left ? cloneWithParents(t.left) : null;
  const r = t.right ? cloneWithParents(t.right) : null;
  if (l) {
    n.left = l;
    l.parent = n;
  }
  if (r) {
    n.right = r;
    r.parent = n;
  }
  return n;
}

export function bstInsert(root: SN | null, key: number) {

```



```
.....
  if (g.left === p) g.left = child;
  else g.right = child;
}
```

```
/** Рёбра в виде "p→c" — для diff между кадрами. */
export function collectEdges(t: SN | null): string[] {
  const es: string[] = [];
  const walk = (n: SN | null) => {
    if (!n) return;
    if (n.left) es.push(`${n.key}→${n.left.key}`);
    if (n.right) es.push(`${n.key}→${n.right.key}`);
    walk(n.left);
    walk(n.right);
  };
  walk(t);
  return es.sort();
}
```

```
const diffEdges = (before: string[], after: string[]): string[] {
  const rm = before.filter((e) => !after.includes(e));
  const ad = after.filter((e) => !before.includes(e));
  const parts: string[] = [];
  for (const a of ad) {
    const [p, c] = a.split("→");
    if (rm.includes(`${c}→${p}`)) parts.push(`ребро ${p}→${c}`);
    else parts.push(`поддерево ${c} переехало к ${p}`);
  }
  return parts.join("; ");
};
```

```
export type SplayStepKind = "search" | "aim" | "cut" | "move" | "erase";
export interface SplayStep {
  kind: SplayStepKind;
  root: SN | null;
  aimEdge?: [number, number];
  /** Кадр cut: ребро, по которому отстёгивается среднее */
  cutEdge?: [number, number];
  cutLabel?: string;
}
```

```

.....
    note: isInsert
      ? `Вставили ${key} обычным BST-спуском. Теперь по
      : found
      ? `Спуск по ключу ${key}: путь ${pathKeys.join(
      : `Ключа ${key} нет. Спуск уперся в ${last.key}
    rotCount: 0,
  ));

// --- подъём ---
let rotCount = 0;
let target: SN = x as SN;
const fixRoot = () => {
  while (root && root.parent) root = root.parent;
};
/** Один поворот = три кадра: aim → cut → rot. */
const emitRotation = (child: SN, label: string, aimEdge: string) => {
  steps.push({
    kind: "aim",
    root: cloneWithParents(root)!,
    aimEdge,
    caseLabel: label,
    note: aimNote,
    rotCount,
  });
  const p = child.parent!;
  const mid = p.left === child ? child.right : child.left;
  steps.push({
    kind: "cut",
    root: cloneWithParents(root)!,
    cutEdge: [child.key, mid ? mid.key : child.key],
    cutLabel: mid ? `β = поддерево ${mid.key}` : "β пусто",
    cutDest: mid ? (p.left === child ? "справа у p" : "слева у p") : "в p",
    caseLabel: label,
    note: mid
      ? `Отстёгиваем среднее поддерево β = ${mid.key}
        и у них общий — после поворота переедет ${p.left ? p.left.key : p.right.key}
        : `Среднего поддерева нет (β пусто): ${child.key}
    rotCount,
  });

```

```

.....
    steps.push({
      kind: "aim",
      root: cloneWithParents(root)!,
      aimEdge: [target.parent.key, target.key],
      caseLabel: "Zig-Zig",
      note: `Теперь обычный зиг: ребро ${target.parent.key} над ${target.key}`,
      rotCount,
    });
    emitRotation(
      target,
      "Zig-Zig",
      [target.parent.key, target.key],
      `Доводим: ${target.key} над ${target.parent.key}`,
    );
  } else {
    const side = `${g.key} → ${pIsLeft ? "влево" : "вправо"}`;
    emitRotation(
      target,
      "Zig-Zag",
      [p.key, target.key],
      `Zig-Zag (змейка: ${side}). Порядок ОБРАТНЫЙ: п. ${p.key} над ${target.key}`,
    );
    steps.push({
      kind: "aim",
      root: cloneWithParents(root)!,
      aimEdge: [target.parent.key, target.key],
      caseLabel: "Zig-Zag",
      note: `Второе ребро: ${target.parent.key}—${target.key}`,
      rotCount,
    });
    emitRotation(
      target,
      "Zig-Zag",
      [target.parent.key, target.key],
      `Доводим: ${target.key} над ${target.parent.key}`,
    );
  }
}
}

```

Универсальное пособие • полный исходный код

```
.....
{ line: 5, text: "      elif p,g,x на одной линии:      # Z
{ line: 6, text: "          rotate(p over g)              # B
{ line: 7, text: "          rotate(x over p)" },
{ line: 8, text: "      else:                             # Z
{ line: 9, text: "          rotate(x over p)              # H
{ line: 10, text: "          rotate(x over g)" },
};
```

```
/* ..... врезка «механика одного по
```

```
const MechanismInset: React.FC<{ step: SplayStep | null
const isCut = step?.kind === "cut";
const isRot = step?.kind === "rot";
return (
  <div className="bg-slate-950/80 rounded-lg border b
    <h4 className="text-xs font-bold text-slate-300 m
    <p className="text-[10px] text-slate-500 mb-2">
      Как вправление вывиха: одна кость ходит вокруг
    </p>
    <svg viewBox="0 0 340 190" className="w-full">
      <g transform="translate(10,6)">
        <text y="10" fill="#94a3b8" fontSize="11" fon
        <line x1="55" y1="46" x2="30" y2="86" stroke=
        <line x1="55" y1="46" x2="80" y2="86" stroke=
        <line x1="30" y1="86" x2="12" y2="126" stroke=
        <line
          x1="30" y1="86" x2="48" y2="126"
          stroke={isCut ? "#f59e0b" : "#64748b"}
          strokeWidth={isCut ? 3 : 2}
          strokeDasharray={isCut ? "5,3" : "none"}
        </>
        <circle cx="55" cy="46" r="15" fill="#1e293b"
        <text x="55" y="50" textAnchor="middle" fill=
        <circle cx="30" cy="86" r="15" fill="#78350f"
        <text x="30" y="90" textAnchor="middle" fill=
        <circle cx="12" cy="126" r="13" fill="#0f172a
        <text x="12" y="130" textAnchor="middle" fill=
        <circle
```



```

        setTreeVers((v) => v + 1);
        return;
    }
    timer.current = setTimeout(() => setIdx((v) => v + 1));
    return () => {
        if (timer.current) clearTimeout(timer.current);
    };
}, [playing, idx, steps]);

const step = steps ? steps[Math.min(idx, steps.length - 1)] : null;
const viewRoot = step ? step.root : baseRef.current;
const pos = useMemo(() => layout(viewRoot), [viewRoot]);
const nodes = useMemo(() => [...pos.entries()].map(([key, node]) => node), [pos]);
const edges = useMemo(() => {
    const es: { a: number; b: number }[] = [];
    const walk = (n: SN | null) => {
        if (!n) return;
        if (n.left) es.push({ a: n.key, b: n.left.key });
        if (n.right) es.push({ a: n.key, b: n.right.key });
        walk(n.left);
        walk(n.right);
    };
    walk(viewRoot);
    return es;
}, [viewRoot, treeVers, idx]);

const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.w));
const autoH = Math.max(300, ...nodes.map((n) => n.py));

const dBefore = useMemo(() => (steps ? depthOf(steps[steps.length - 1]) : 0), [steps]);
const dNow = steps ? depthOf(viewRoot, steps[steps.length - 1]) : 0;
const hBefore = useMemo(() => (steps ? heightOf(steps[steps.length - 1]) : 0), [steps]);
const hNow = heightOf(viewRoot);

// переехавшие рёбра: разница с предыдущим кадром
const prevEdgeSet = useMemo(() => {
    if (!steps || !step || idx <= 0) return null;
    return new Set(collectEdges(steps[idx - 1].root));
}, [steps, step, idx]);

```

```

.....
    if (step.pathKeys?.includes(k)) return "#38bdf8";
    return "#64748b";
  };

  return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border-2 border-indigo-500" /* Шанка */>
      <div className="flex flex-wrap items-center justify-center gap-3">
        <div className="p-2.5 bg-indigo-600 text-white text-2xl"> </div>
        <div>
          <h3 className="text-2xl font-bold text-white">
            <p className="text-sm text-indigo-300">Кликните по кнопке
          </p>
        </div>
      </div>
      <div className="flex flex-wrap items-center gap-3">
        <input
          value={input}
          onChange={(e) => setInput(e.target.value)}
          onKeyDown={(e) => {
            if (e.key === "Enter") {
              const v = parseInt(input, 10);
              if (!Number.isNaN(v)) {
                const has = inorderKeys(baseRef.current);
                if (has) runKey(v);
                else doInsert();
                setInput("");
              }
            }
          }}
          placeholder="ключ + Enter"
          className="w-28 bg-slate-950 border border-indigo-500"
        />
        <button
          onClick={() => {

```

```

</button>
<button onClick={() => setPlaying(!playing)}
  er-600" : "bg-emerald-600"}`>
  {playing ? " Пауза" : "▶ Пуск"}
</button>
<button onClick={() => { setPlaying(false); s
  className="px-3 py-1.5 rounded-lg text-xs fo
  Шаг →
</button>
<span className="text-xs text-slate-400">
  Шаг {idx + 1} из {steps.length} • поворотов
</span>
{step?.caseLabel && (
  <span className="px-2.5 py-1 rounded-full t
  00/60">
    {step.caseLabel}
  </span>
)}
</div>
)}

{/* Дерево */}
<div className="bg-indigo-950/20 rounded-xl p-4 b
  {!steps && (
    <div className="absolute top-3 left-3 bg-indi
    ont-bold">
      Кликни по любому узлу – splay покадрово
    </div>
  )}
<svg viewBox={`0 0 ${autoW} ${autoH}`} className
  {edges.map((e) => {
    const a = pos.get(e.a);
    const b = pos.get(e.b);
    if (!a || !b) return null;
    const isAim = step?.aimEdge && step.aimEdge
    const isCutEdge =
      step?.kind === "cut" && step.cutEdge && s
      .cutEdge[1]);

```



```

.....
        className="transition-all duration-300"
      />
      <text y={4} fill="#ffffff" fontSize="13">
    >
      {n.key}
    </text>
  </g>
);
}}
</svg>

{/* Лог + механика */}
<div className="w-full flex flex-col lg:flex-row" >
  <div className="flex-1 bg-slate-950/80 p-4 rounded" >
    <p className="font-semibold text-amber-400" >
      {step?.kind !== "aim"
        ? " Прицел:"
        : step?.kind !== "cut"
        ? " Отстёгиваем среднее поддерево:"
        : step?.kind !== "rot"
        ? " Поворот:"
        : step?.kind !== "search"
        ? " Спуск:"
        : "Статус:"}
    </p>
    <p className="min-h-[40px] flex items-center" >
      {step ? step.note : "Кликни по узлу или в
      ке справа – механика одного поворота."}
    </p>
    {steps && step && step.kind !== "search" &&
      <p className={`text-xs mt-2 ${checkOk ? "text-amber-400" : "text-slate-400"}`} >
        {checkOk ? " " : " "} проверка кадра: 0
        {inorderKeys(viewRoot).length}
        {movedCount > 0 ? ` · переехало рёбер: ` : ""}
      </p>
    )}
  </div>
  <div className="w-full lg:w-[300px] shrink-0">

```

```

.....
    const zz2 = step?.caseLabel === "Zig-Zig"
    const za1 = step?.caseLabel === "Zig-Zag"
    const za2 = step?.caseLabel === "Zig-Zag"
    const active = zig || zz1 || zz2 || za1 || za2
    return (
      <div key={\line} className={`py-1 px-2 border-400` : "text-slate-400"}`}>
        <span className="text-slate-600 select-none">
          {\.text}
        </span>
      </div>
    );
  }
}
</div>
</div>
</div>
</div>
);
};

```

export default SplayWalkViz;

ФАЙЛ 70/99: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

.....
import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizSteps';

```

```

interface Plate {
  id: string;
  name: string;
  emoji: string;
  color: string;
  isDirty: boolean;
  animState: 'in' | 'idle' | 'washing' | 'clean';
}

```

```

.....
const [showSoap, setShowSoap] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>()

const addLog = (msg: string) => setLog(prev => [...prev, msg],

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length >= 7) {
    addLog('⚠ Стопка слишком высокая, тарелки могут р
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }

  const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
  counter++;

  const newPlate: Plate = {
    id: Date.now().toString(),
    ...preset,
    isDirty: true,
    animState: 'in',
  };

  setDirtyStack(prev => [...prev, newPlate]);
  addLog(` PUSH: Положили ${newPlate.emoji} сверху с

  setTimeout(() => {
    setDirtyStack(prev => prev.map(p => p.id !== newP
  }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length === 0) {

```

```

.....
    { id: 'r3', name: 'Блюдец от торта',      emoji:
      mState: 'in' },
  });
  setCleanRack([]);
  setIsWashing(false);
  setShowSoap(false);
  setLog([' Стол сброшен. Посуда снова грязная!']);
};

const topIndex = displayStack.length - 1;

return (
  <div className="flex flex-col gap-6">
    { /* МНЕМОНИКА / ПРАВИЛО */ }
    <div className="bg-blue-950/40 border border-blue-
      <div className="text-3xl"> </div>
      <div>
        <h3 className="font-black text-blue-400 text-
        <p className="text-xs text-slate-300 mt-1 lea
          Ты складываешь грязные тарелки друг на друга
          А когда приходит время мыть, ты берёшь именн
          Попытаешься вытащить нижнюю – всё рухнет!
        </p>
      </div>
    </div>

    { /* УПРАВЛЕНИЕ */ }
    <div className="flex items-center gap-3 flex-wrap
      <button
        onClick={pushPlate}
        disabled={isWashing}
        className="flex-1 min-w-[150px] bg-gradient-t
          acity-40 disabled:cursor-not-allowed text-w
          ale-95 transition-all cursor-pointer flex i
        >
        <span> Положить грязную (PUSH)</span>
      </button>

```

```

.....
    <div className="absolute inset-0 pointer-events-none">
      <div className="absolute w-6 h-6 rounded-2xl" style={{
        background: 'white',
        border: '1px solid black',
        opacity: 0.5,
      }}></div>
      <div className="absolute w-4 h-4 rounded-2xl" style={{
        background: 'white',
        border: '1px solid black',
        opacity: 0.5,
      }}></div>
      <div className="absolute w-5 h-5 rounded-2xl" style={{
        background: 'white',
        border: '1px solid black',
        opacity: 0.5,
      }}></div>
      <div className="absolute text-4xl anim-curve-in-out">
        {topIndex}
      </div>
    </div>
  ) : (
    <div className={`flex-1 flex flex-col justify-between`} style={{
      height: 100,
    }}>
      {/* Указатель TOP */}
      <div
        className="absolute right-8 flex items-center"
        style={{ bottom: `${24 + topIndex * 44}px` }}
      >
        <span className="text-xs font-mono text-slate-500">
          er gap-1">
            <ArrowUp className="w-3.5 h-3.5" />
            <span>ВЕРШИНА (TOP)</span>
          </span>
          <span className="text-blue-400 animate-pulse">
            {topIndex}
          </span>
        </div>

      {/* Тарелки (отрисовываем в обратном порядке) */}
      {displayStack.slice().reverse().map((plate, topIndex) => {
        const idx = displayStack.length - 1 - topIndex;
        const isTop = idx === topIndex;
        return (

```

```

</div>

<div className="flex-1 flex flex-col justify-between"
  {cleanRack.length === 0 ? (
    <div className="flex-1 flex flex-col item-center"
      <span className="text-5xl mb-2"> </span>
      <p className="text-xs font-bold font-mono">
      <p className="text-[10px] mt-1">Здесь 6
    </div>
  ) : (
    <div className="flex flex-col gap-2 w-full"
      {cleanRack.map(plate => (
        <div
          key={plate.id}
          className="w-full h-8 rounded-full flex items-center justify-center gap-2 opacity-1
        >
          <span className="text-sm"> {plate.name}
          <span className="text-[10px] text-end">
        </div>
      )
    )}
  </div>
)}
</div>

{/* Столешница сушилки */}
<div className="w-full h-4 bg-gradient-to-r from-slate-900 to-slate-800"
  </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4"
  <div className="text-[10px] font-mono text-slate-200"
    {log.map((entry, idx) => (
      <div
        key={idx}
        className={`text-xs font-mono flex items-center justify-between`}
      >

```

```

.....
    }

    if (s[i] === s[j]) {
        steps.push({ i, j, pi: [...pi], highlight:
            ска увеличивается.` });
        j++;
    } else {
        steps.push({ i, j, pi: [...pi], highlight:
            овпадения нет.` });
    }

    pi[i] = j;
    steps.push({ i, j, pi: [...pi], desc: `Записывае

    if (j === pattern.length) {
        steps.push({ i, j, pi: [...pi], found: true
    }
}
return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
    if (!pattern) pattern = " ";
    const s = pattern + "#" + text;
    const n = s.length;
    const z = new Array(n).fill(0);
    const steps: any[] = [];

    let l = 0, r = 0;
    steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

    for (let i = 1; i < n; i++) {
        steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

        if (i <= r) {
            steps.push({ i, l, r, z: [...z], desc: `Вни
                !` });
            z[i] = Math.min(r - i + 1, z[i - l]);

```

```

// Ensure inputs are valid dynamically
const safePattern = pattern || " ";
const safeText = text || " ";

const { s: demoString, steps, patternLength: demoPa
  if (mode === "kmp") return generateKmpSteps(safeP
  else return generateZSteps(safePattern, safeText
}, {mode, safePattern, safeText});
const runtime = useVizRuntime();
const runtimeString = vizString(runtime?.variables?
const runtimePattern = vizString(runtime?.variables
const s = runtimeString ?? demoString;
// Длина образца: приоритет – переменная pattern из
// разделителя "#" в склейке pattern#text; иначе вс
const patternLength = runtimePattern
  ? runtimePattern.length
  : runtimeString
  ? (runtimeString.includes("#") ? runtimeString
  : demoPatternLength;

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
  setStepIdx(0);
  setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
  if (autoPlay) {
    timer.current = setInterval(() => {
      setStepIdx(prev => {
        if (prev >= steps.length - 1) { set
        return prev + 1;
      });
    });
  }
});

```



```

.....
        КМП (π-ФУНКЦИЯ)
      </button>
      <button onClick={() => setMode("z")}
        className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
        Z-ФУНКЦИЯ
      </button>
    </div>

    <div className="flex items-center gap-2"
      <div className="bg-slate-800 p-1 fl
        <span className="text-[10px] te
        <input value={pattern} onChange=
xt-white text-xs font-bold font-mono px-2 p
      </div>
      <div className="bg-slate-800 p-1 fl
        <span className="text-[10px] te
        <input value={text} onChange={e
te text-xs font-bold font-mono px-2 py-1 ou
      </div>
    </div>
  </div>

  <div className="bg-slate-950 rounded-xl bor
px] flex items-center justify-start">
    <div className="flex gap-1.5 px-4 h-ful
      {s.split("").map((char, index) => {
        const isPattern = index < patte
        const isSeparator = index === p

        let borderColor = "border-slate
        let bgColor = "bg-slate-900";
        let textColor = isSeparator ? "

      // KMP Logic
      if (mode === "kmp") {
        if (step.highlight?.include
          borderColor = "border-a

```

```

.....
        <div className="text-[1
z-20">
        slate-700/50 font-mono font-bold">
            {(mode === "kmp" ?
        </div>

        {/* KMP fingers */}
        {mode === "kmp" && step
            <div className="abs

                <span className
            </div>
        }}
        {mode === "kmp" && step
            <div className="abs
                <span className
            </div>
        }}
        {mode === "kmp" && step
            <div className="abs

                <span className
            </div>
        }}

        {/* Z fingers */}
        {mode === "z" && step.i
            <div className="abs

                <span className
            </div>
        }}
        {mode === "z" && step.z
            <div className="abs
                <span className
            </div>
        }}
        {mode === "z" && step.z
            <div className="abs

```

```

</div>

    { /* Description Log */
    <div className="flex-1 bg-slate-900/50 flex-direction-hidden">
        <div className="absolute top-0 right-0">
            <span className="font-mono text-white">
                {steps[steps.length - 1].desc}
            </span>
        </div>
        <div className="text-[10px] text-slate-400">
            {steps.length}
        </div>
        <div className="text-sm text-slate-400">
            {step.desc}
        </div>
    </div>
</div>

{ /* Why this matters card */
<div className="bg-slate-800 p-4 rounded-xl">
    <span className="text-2xl mt-1"> </span>
    <p className="text-xs text-slate-400">
        Обрати внимание на хитрость с решёткой.
        Мы пишем <b>Шаблон + # + Текст</b>.
        Когда значени {mode === 'kmp' ? "проблема не
о было найдено!
        Пройди шаги вручную, посмотри, как работает алгоритм
ля пропуска работы (в Z).
    </p>
</div>

{ /* Code Sneak Peek */
<div className="bg-slate-900 border border-slate-800">
    <div className="bg-slate-800 px-4 py-2 border border-slate-400 uppercase">
        <span>Код C++ ({mode === 'kmp' ? 'КМП' : 'ЗКМ'})
    </div>
    <pre className="p-4 text-xs font-mono text-slate-400">
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {`

```

Универсальное пособие • полный исходный код

```
.....
* Всплывающая подсказка (как title="", только красивая
* Появляется по наведению курсора и по фокусу с клавиатуры
*/
export const Tooltip: React.FC<TooltipProps> = ({ content, side }) => {
  const [open, setOpen] = useState(false);

  const hiddenShift = side === "top" ? "translate-y-1" : "translate-y-0";

  return (
    <span
      className={`relative inline-flex ${className ?? ""}
        onMouseEnter={() => setOpen(true)}
        onMouseLeave={() => setOpen(false)}
        onFocus={() => setOpen(true)}
        onBlur={() => setOpen(false)}
      >
      {children}
      <span
        role="tooltip"
        className={`pointer-events-none absolute left-1/2 top-1/2
          bg-slate-900 px-3 py-2 text-left text-[11px]
          duration-150 ${
            side === "top" ? "bottom-full mb-2" : "top-full mt-2"
          } ${open ? "visible translate-y-0 opacity-100" : "invisible"}
        >
        {content}
        <span
          className={`absolute left-1/2 top-1/2 -translate-x-50% -translate-y-50%
            side === "top" ? "-bottom-[5px] border-b border-slate-900" : ""
          }`
        />
      </span>
    </span>
  );
};
.....
```

```

.....
    { u: 2, v: 3 },
    { u: 3, v: 4 },
  ];

interface TStep {
  desc: string;
  visited: Set<number>;
  fullyProcessed: Set<number>;
  currentNode: number | null;
  topoList: number[];
  dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
  const [steps, setSteps] = useState<TStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
  const v = vizNumber(vars.v);
  const orderLength = vizArray(vars.order)?.length;
  if (v === null && orderLength === undefined) return null;
  const candidates = steps
    .map((step, index) => ({ step, index }))
    .filter(({ step }) => (v === null || step.currentNode !== v));
  return candidates.at(-1)?.index ?? null;
});

const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
  const listSteps: TStep[] = [];
  const visited = new Set<number>();
  const fullyProcessed = new Set<number>();
  const topoList: number[] = [];
  const dfsStack: number[] = [];

  const recordStep = (desc: string, currentNode: number) => {
    listSteps.push({
      desc,

```

```

.....
    dfsStack.pop();
    topoList.push(u); // prepend behavior: we write in reverse
    recordStep(
      `DFS: закончили обработку '${dagNodes[u].name}'
      ,
      u,
    );
  };

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
  if (!visited.has(i)) {
    recordStep(
      `Запускаем новый обход DFS от нетронутой вершины
      ,
      null,
    );
    dfs(i);
  }
}

recordStep(
  `Топологическая сортировка завершена! Полученный список
  ,
  null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {
  desc: "Запуск...",
  visited: new Set(),
  fullyProcessed: new Set(),
  currentNode: null,
  topoList: [],
  dfsStack: [],
};

useEffect(() => {

```

```

        {isPlaying ? (
            <Pause className="w-4 h-4 ml-1" />
        ) : (
            <Play className="w-4 h-4 ml-1" />
        )}
        {isPlaying ? "Пауза " : "Старт "}
    </button>

    <button
        onClick={reset}
        className="flex items-center justify-center
        ors"
    >
        <RotateCcw className="w-4 h-4" />
    </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
    {/* Playfield SVG */}
    <div className="lg:col-span-8 bg-[#0B1120] rela
        <svg className="w-full h-full max-w-[650px]" >
            <defs>
                <marker
                    id="dag-arrow"
                    viewBox="0 0 10 10"
                    refX="24"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"
                >
                    <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
                </marker>
                <marker
                    id="dag-arrow-active"
                    viewBox="0 0 10 10"
                    refX="24"

```

.....

```

let fill = "#1e293b";
let stroke = "#334155";
let textColor = "text-slate-400";
void textColor;

if (isCurrent) {
  fill = "#312e81";
  stroke = "#818cf8";
  textColor = "text-white font-bold";
} else if (isProcessed) {
  fill = "#022c22";
  stroke = "#059669";
  textColor = "text-emerald-300";
} else if (isVisited) {
  fill = "#3730a3";
  stroke = "#6366f1";
  textColor = "text-indigo-200";
}

return (
  <g key={node.id}>
    <rect
      x={node.x - 55}
      y={node.y - 22}
      width="110"
      height="44"
      rx="8"
      fill={fill}
      stroke={stroke}
      strokeWidth="2"
      className="transition-all duration-300"
    />
    <text
      x={node.x}
      y={node.y - 4}
      textAnchor="middle"
      fill="white"
    />
  </g>
)

```



```

.....
    <span className="text-[10px] text-slate-500">
      РЕЗУЛЬТАТ (МАТРУТ ОБУЧЕНИЯ):
    </span>
    <div className="bg-slate-900/60 p-2.5 rounded-3xl
pr-1 text-slate-300">
      {step.topoList.length === 0 ? (
        <span className="text-slate-600 text-[10px]">
          ожидаем обработку узлов...
        </span>
      ) : (
        getTopoListString()
      )}
    </div>
  </div>

  {/* Color key */}
  <div className="space-y-1">
    <span className="text-[10px] text-slate-500">
      ВЕРШИНЫ В DFS:
    </span>
    <div className="flex items-center gap-2">
      <span className="w-3.5 h-3.5 rounded bg-slate-400">
        <span className="text-slate-400 text-[10px]">
          Серые (зашли)
        </span>
      </div>
      <div className="flex items-center gap-2">
        <span className="w-3.5 h-3.5 rounded bg-slate-400">
          <span className="text-slate-400 text-[10px]">
            Черные (готово)
          </span>
        </div>
      </div>
    </div>
  </div>

  <div className="mt-4 pt-3 border-t border-slate-200">
    <span>
      Шаг {currentStepIdx + 1} из {steps.length}
    </span>
  </div>

```

Универсальное пособие • полный исходный код

```
.....
*          координатная плоскость НАВЕРХУ (ассоциация)
*          чек-лист инвариантов (автопроверка, как у
* split  — разрез по x с двумя корзинами (грамотная
* merge  — склейка двух деревьев по y.
* erase   — удаление ПРОИЗВОЛЬНОЙ точки: найти по x,
* layers — миф «дети в массиве по  $2k/2k+1$ »: работает
* search — «отсортирую бинарным поиском»: поиск не с
*          сортировка платит  $\geq n \cdot \log n$  сравнений, со
*
* Канонический набор: все x и все y различны  $\Rightarrow$  дерево
* Узел идентифицируется парой id = "x:y".
*/
```

```
export interface Pair {
  key: number;
  grade: number;
}
type NodeId = string;
const pairId = (key: number, grade: number): NodeId =>

interface TNode {
  id: NodeId;
  key: number;
  grade: number;
  left: TNode | null;
  right: TNode | null;
}

export const POINTS: Pair[] = [
  { key: 45, grade: 8 },
  { key: 3, grade: 6 },
  { key: 6, grade: 5 },
  { key: 2, grade: 4 },
  { key: 9, grade: 3 },
  { key: 7, grade: 2 },
  { key: 12, grade: 0 },
  { key: 1, grade: -4 },
];
```

Универсальное пособие • полный исходный код

```
.....
export function splitTree(root: TNode | null, x: number) {
  const trace: SplitResult["trace"] = [];
  const go = (t: TNode | null): { l: TNode | null; r: TNode | null } {
    if (!t) return { l: null, r: null };
    if (t.key <= x) {
      const sub = go(t.right);
      trace.push({ atId: t.id, goLeft: true, note: `(${t.key}
        о правое поддерезво.` });
      t.right = sub.l;
      return { l: t, r: sub.r };
    }
    const sub = go(t.left);
    trace.push({ atId: t.id, goLeft: false, note: `(${t.key}
      о левое поддерезво.` });
    t.left = sub.r;
    return { l: sub.l, r: t };
  };
  const res = go(root);
  return { l: res.l, r: res.r, trace };
}
```

```
export interface MergeResult {
  root: TNode | null;
  trace: { aId: NodeId | null; bId: NodeId | null; chosenId: NodeId | null };
}
export function mergeTree(a: TNode | null, b: TNode | null) {
  const trace: MergeResult["trace"] = [];
  const go = (a: TNode | null, b: TNode | null): TNode | null {
    if (!a || !b) {
      trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, chosenId: null,
        note: `оное целиком.` });
      return a ?? b;
    }
    if (a.grade > b.grade) {
      trace.push({ aId: a.id, bId: b.id, chosenId: a.id,
        note: `ем его правое поддерезво с (${b.key}, ${b.grade})` });
      a.right = go(a.right, b);
      return a;
    }
    if (b.grade > a.grade) {
      trace.push({ aId: a.id, bId: b.id, chosenId: b.id,
        note: `ем его левое поддерезво с (${a.key}, ${a.grade})` });
      b.left = go(a, b.left);
      return b;
    }
    return null;
  };
  const res = go(a, b);
  return { root: res, trace };
}
```

Универсальное пособие • полный исходный код

```
.....
let bst = true;
let heap = true;
let prev: number | null = null;
const walk = (n: TNode | null, min: number, max: number, parentGrade: number) => {
  if (!n) return;
  if (n.key <= min || n.key > max) bst = false;
  if (parentGrade !== null && n.grade > parentGrade) heap = false;
  walk(n.left, min, n.key, n.grade);
  if (prev !== null && n.key < prev) bst = false;
  prev = n.key;
  walk(n.right, n.key, max, n.grade);
};
walk(root, -Infinity, Infinity, null);
return { bst, heap };
}

/* ----- раскладка сцены дерева ----- */

const layoutTree = (root: TNode | null) => {
  const pos = new Map<NodeId, { px: number; py: number }>();
  let slot = 0;
  const walk = (n: TNode | null, depth: number) => {
    if (!n) return;
    walk(n.left, depth + 1);
    pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 20 });
    slot += 1;
    walk(n.right, depth + 1);
  };
  walk(root, 0);
  return pos;
};

const treeEdges = (root: TNode | null): { from: NodeId; to: NodeId }[] => {
  const es: { from: NodeId; to: NodeId }[] = [];
  const walk = (n: TNode | null) => {
    if (!n) return;
    if (n.left) { es.push({ from: n.id, to: n.left.id }); }
    if (n.right) { es.push({ from: n.id, to: n.right.id }); }
    walk(n.left);
    walk(n.right);
  };
  walk(root);
  return es;
};
```

```

.....
        strokeWidth={2.5}
        strokeDasharray={edgeDash && edgeDash(e.f
        className="transition-all duration-500"
      />
    );
  }}}
  {nodes.map((n) => {
    const p = pairParts(n.id);
    const pulse = pulseId === n.id;
    return (
      <g key={`tn-${n.id}`} transform={`translate
        <circle r={pulse ? 24 : 20} fill={nodeColo
      } strokeWidth={pulse ? 4 : 2} className="tr
        <text y={-2} fill="#fff" fontSize="13" fo
        <text y={13} fill="#cbd5e1" fontSize="10"
      </g>
    );
  }}}
</svg>
{subtitle && <p className="text-xs text-slate-400
</div>
);
};

const pairParts = (id: NodeId): [number, number] => {
  const [a, b] = id.split(":").map(Number);
  return [a, b];
};

/* ----- плоскость (клетчатая бумага) -----

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H

interface PlaneProps {
  points: Pair[];

```

```

.....
    const a = byPair(e.a);
    const b = byPair(e.b);
    const isBad = badEdge && ((badEdge.a.key === e.a.key &&
        badEdge.b.key === e.b.key) || (badEdge.a.key === e.b.key &&
        badEdge.b.key === e.a.key));
    return <line key={`pe-${i}`} x1={a.x} y1={a.y} x2={b.x} y2={b.y}
        strokeDasharray={isBad ? "4,3" : "none"} />
  )}
}

{/* временная линия игры */}
{interactive && selected && cursor && (
  <line
    x1={PX(selected.key)} y1={PY(selected.grade)}
    stroke="#818cf8" strokeWidth={2.5} strokeDasharray={[]} />
)}

{points.map((p) => {
  const done = drawnEdges.some((e) => (e.a.key === p.key && e.b.key === p.key))
  const isSel = selected && selected.key === p.key
  return (
    <g
      key={`pp-${p.key}:${p.grade}`}
      className="transition-all duration-500"
      onPointerDown={interactive && onPointDown ? () => onPointDown(p) : null}
      onPointerUp={interactive && onPointUp ? () => onPointUp(p) : null}
      style={interactive ? { cursor: "pointer" } : {}}
    >
      <circle cx={PX(p.key)} cy={PY(p.grade)} r={10}
        stroke={isSel ? "#a5b4fc" : done ? "#38bdf8" : "#ccc"}
        strokeDasharray={isSel ? [4, 3] : []} />
      <text x={PX(p.key)} y={PY(p.grade) + 3.5} font-size={12} font-weight="bold">{p.key}</text>
    </g>
  );
})}

{pending && (
  <g className="transition-all duration-500">

```

```

    seen.add(gpid(cur));
    cur = parentOf(edges, gpid(cur)).a;
  }
  return false;
};

/** Проверка BST леса: у узла ≤1 ребёнка слева и ≤1 справа */
export function forestIsBst(edges: GEdge[], all: Pair[]) {
  const kids = new Map<string, Pair[]>();
  for (const e of edges) kids.set(gpid(e.a), [...(kids.get(gpid(e.b)) || [])]);
  const hasParent = new Set(edges.map((e) => gpid(e.b)));
  const roots = all.filter((p) => !hasParent.has(gpid(p.a)));
  const ok = (n: Pair | undefined, lo: number, hi: number) => {
    if (!n) return true;
    if (n.key <= lo || n.key > hi) return false;
    const ch = (kids.get(gpid(n.a)) ?? []).slice().sort((a, b) => a.key - b.key);
    const left = ch.filter((c) => c.key < n.key);
    const right = ch.filter((c) => c.key > n.key);
    if (left.length > 1 || right.length > 1) return false;
    return ok(left[0], lo, n.key) && ok(right[0], n.key, hi);
  };
  return roots.every((r) => ok(r, -Infinity, Infinity));
}

/** Полный вердикт по текущим линиям. */
export function gameVerdict(edges: GEdge[], all: Pair[]) {
  const heap = edges.every((e) => e.a.grade > e.b.grade);
  const oneParent = new Set(edges.map((e) => gpid(e.b)));
  const acyclic = !edges.some((e) => reaches(edges, e.a, e.b));
  const bst = forestIsBst(edges, all);
  const roots = all.filter((p) => !edges.some((e) => gpid(e.a) === gpid(p.a)));
  const complete = edges.length === all.length - 1 && roots.length === 1;
  return { heap, oneParent, acyclic, bst, complete, win: heap && oneParent && acyclic && bst && complete };
}

/** Попытка хода: почему нельзя – или ОК. */
export function tryConnect(edges: GEdge[], a: Pair, b: Pair) {
  if (sameP(a, b)) return { ok: false, reason: "Это одна и та же линия" };
  if (reaches(edges, a, b)) return { ok: false, reason: "Линия уже есть" };
  if (gpid(a.a) === gpid(b.a) || gpid(a.b) === gpid(b.b)) return { ok: false, reason: "Нельзя соединять линии одного игрока" };
  if (gpid(a.a) === gpid(b.b) || gpid(a.b) === gpid(b.a)) return { ok: false, reason: "Нельзя соединять линии одного игрока" };
  return { ok: true, reason: "Ход корректен" };
}

```

```

.....
    pairs.push({ key: nums[0], grade: nums[1] });
  }
  const xs = pairs.map((p) => p.key);
  const ys = pairs.map((p) => p.grade);
  return { pairs, errors, equalX: new Set(xs).size !== 1 };
};

const DEMO_RAW = "45 8, 3 6, 6 5, 2 4, 9 3, 7 2, 12 0,

/* ----- игра «соедини мышкой»: чистая логика

/** Ребро игры: a = родитель (выше по y), b = ребёнок.
export interface GameEdge {
  a: Pair;
  b: Pair;
}
const pid = (p: Pair) => `${p.key}:${p.grade}`;

export function canConnect(
  edges: GameEdge[],
  x: Pair,
  y: Pair,
): { ok: boolean; edge?: GameEdge; reason?: string } {
  if (x.key === y.key && x.grade === y.grade) return { ok: false, reason: "такая точка уже проведена" };
  for (const e of edges) {
    if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) ||
        (pid(e.a) === pid(y) && pid(e.b) === pid(x)))
      return { ok: false, reason: "такая линия уже проведена" };
  }
  if (x.grade === y.grade) return { ok: false, reason: "ребёнок не может быть старше родителя" };
  const [parent, child] = x.grade > y.grade ? [x, y] : [y, x];
  if (edges.some((e) => pid(e.b) === pid(child)))
    return { ok: false, reason: `у точки (${child.key}, ${child.grade}) уже есть родитель` };
  // цикл: родитель уже сидит в компоненте ребёнка (ребёнок не может быть старше родителя)
  const childComp = new Set<string>([pid(child)]);
  let grew = true;
  while (grew) {
    grew = false;
    for (const e of edges) {
      if (pid(e.a) === pid(parent) && pid(e.b) !== pid(child)) {
        grew = true;
        childComp.add(pid(e.b));
      }
    }
  }
  return { ok: true, edge: edges.find(e => pid(e.a) === pid(parent) && pid(e.b) === pid(child)) };
}

```



```

.....
    if (ra !== rb) comp.set(ra, rb);
  }
  const comps = new Set(points.map((p) => find(pid(p))))
  const connected = comps.size === 1;
  const countOk = edges.length === n - 1;
  // BST: дети по x, inorder не убывает; корень один; з
  const children = new Map<string, Pair[]>();
  for (const e of edges) children.set(pid(e.a), [...(ch
  const roots = points.filter((p) => !parentCount.has(p
  let bstOk = roots.length === 1;
  if (bstOk) {
    const seen = new Set<string>();
    const inorder: number[] = [];
    const walk = (p: Pair): void => {
      if (seen.has(pid(p))) {
        bstOk = false; // цикл
        return;
      }
      seen.add(pid(p));
      const kids = children.get(pid(p)) ?? [];
      if (kids.length > 2) {
        bstOk = false;
        return;
      }
      const sorted = [...kids].sort((u, v) => u.key - v
      let left: Pair | null = null;
      let right: Pair | null = null;
      if (sorted.length === 2) {
        if (sorted[0].key === sorted[1].key || sorted[0
          bstOk = false; // двое детей с равным x или с
          return;
        }
        left = sorted[0];
        right = sorted[1];
      } else if (sorted.length === 1) {
        // одиночный ребёнок: сторона важна – определяет
        if (sorted[0].key <= p.key) left = sorted[0];
        else right = sorted[0];
      }
    }
  }
}

```

```

.....
    if (left[0]) node.left = build(left[0]);
    if (right[0]) node.right = build(right[0]);
    return node;
  };
  return build(rootPair);
}

const BuildMode: React.FC = () => {
  const [raw, setRaw] = useState(DEMO_RAW);
  const [phase, setPhase] = useState<BuildPhase>("sort");
  const [algo, setAlgo] = useState<SortAlgo>("none");
  const [order, setOrder] = useState<ConnectOrder>("y-d");
  const [sortIdx, setSortIdx] = useState(0);
  const [connIdx, setConnIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [sortCost, setSortCost] = useState<string | null>();
  const live = useTreapLive();

  const parsed = useMemo(() => parsePairs(raw), [raw]);
  const userPairs = parsed.errors.length ? [] : parsed.pairs;

  const n = userPairs.length;

  const userTree = useMemo(() => (userPairs.length >= 1
  const canonEdges = useMemo(() => {
    const es: { a: Pair; b: Pair }[] = [];
    const walk = (node: TNode | null) => {
      if (!node) return;
      if (node.left) { es.push({ a: { key: node.key, group: 1,
        left); }
      if (node.right) { es.push({ a: { key: node.key, group: 1,
        de.right); }
    };
    walk(userTree);
    return es;
  }, [userTree]);
  const leftRightEdges = useMemo(() => [...canonEdges]..

```

Универсальное пособие • полный исходный код

```
.....
const [cursor, setCursor] = useState<{ x: number; y: number }>({ x: 0, y: 0 });
const [gameMsg, setGameMsg] = useState<string | null>(null);
const [badEdge, setBadEdge] = useState<GameEdge | null>(null);
const [hintUsed, setHintUsed] = useState(false);

const resetGame = () => {
  setGameEdges([]);
  setSel(null);
  setCursor(null);
  setGameMsg(null);
  setBadEdge(null);
  setHintUsed(false);
};

const startGame = () => {
  resetGame();
  setPhase("game");
  setPlaying(false);
};

const verdict = useMemo(() => checkGameState(userPair, gameEdges));
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree(gameEdges) : null));

const attemptConnect = (x: Pair, y: Pair) => {
  const r = canConnect(gameEdges, x, y);
  if (r.ok && r.edge) {
    setGameEdges((prev) => [...prev, r.edge!]);
    setGameMsg(r.reason ?? null);
    setBadEdge(null);
  } else {
    setGameMsg(r.reason ?? "так нельзя");
    setBadEdge({ a: x, b: y });
    window.setTimeout(() => setBadEdge(null), 900);
  }
};

const pointDown = (p: Pair) => {
  if (phase !== "game") return;
  if (!sel) {
```

```

.....
    <div className="flex flex-wrap items-center justify-center">
      <h4 className="text-sm font-bold text-slate-300">Демо-набор</h4>
      <button onClick={() => { setRaw(DEMO_RAW); setRows(2); }}>
        демо-набор
      </button>
    </div>
    <textarea
      value={raw}
      onChange={(e) => { setRaw(e.target.value); setRows(2); }}
      className="w-full bg-slate-950 border border-emerald-500"
      placeholder="3 5, 45 8, 1 -4 ..."
    />
    {parsed.errors.length > 0 && (
      <div className="mt-2 text-xs text-rose-300 bg-rose-500/20 p-4">
        {parsed.errors.map((e, i) => <div key={i}>{e}</div>)}
      </div>
    )}
    {parsed.errors.length === 0 && (parsed.equalX || parsed.equalY) && (
      <div className="mt-2 text-xs text-amber-300 bg-amber-500/20 p-4">
        {parsed.equalY && <div>⚡ Есть <b>равные y</b> у тех, кто <b>раньше</b> в списке</div> (сортировка стабильная)
        {parsed.equalX && <div>⚡ Есть <b>равные x</b> у тех, у кого <b>раньше</b> (сортировка по де.кей → left).</div>}
      </div>
    )}
  </div>

  { /* Координатная плоскость – НАВЕРХУ */ }
  <div className="bg-emerald-950/20 rounded-xl p-4">
    <div className="flex flex-wrap items-center justify-center">
      <h4 className="text-base font-bold text-emerald-300">Координатная плоскость</h4>
      <div className="flex items-center gap-2">
        {phase === "sort" ? (
          <div>
            <span className="text-xs text-slate-400">Сортировка по де.кей → left</span>
            <span className="text-xs text-slate-400">Сортировка по де.кей → right</span>
          </div>
        ) : (
          <span className="text-xs text-slate-400">Сортировка по де.кей → left</span>
        )}
      </div>
    </div>
  </div>

```

```

        </>
      })
    </div>
  </div>
  <Plane
    points={userPairs}
    drawnEdges={phase === "game" ? gameEdges : ph
    pending={null}
    interactive={phase === "game"}
    selected={sel}
    cursor={cursor}
    badEdge={badEdge}
    onPointDown={pointDown}
    onPointUp={pointUp}
    onMove={planeMove}
  />
  <div className="mt-2 text-xs text-slate-300 bg-
    {phase === "sort" ? (
      algo === "none" ? (
        <>Ввод приходит <b>как попало</b> ({n} па
        ть</b>.</>
      ) : sortIdx < 3 ? (
        algo === "quicksort" ? (
          <> quicksort: счётчик сравнений: <b>sl
          {comparisons}</b> ( $n \cdot \log_2 n$ ).</>
        ) : (
          <> counting: точки в корзины по значени
          , потому что у – маленькие целые.</>
        )
      ) : (
        <>Отсортировано ({sortCost ?? "способ не
        <b>Игру </b></>
      )
    ) : phase === "game" ? (
      gameWon ? (
        <> Готово! {hintUsed ? "Решение подсказан
        ево: линии вниз по y, обход по x отсортирован
      ) : gameMsg ? (

```

```

.....
      <button onClick={resetGame} disabled={!gameI
ate-700 text-white disabled:opacity-30"> C
      <button onClick={showSolution} disabled={gameI
te disabled:opacity-30"> Показать решение<
    </div>
  })
</div>

{/* Победное дерево в игре */}
{phase === "game" && gameTree && (
  <div className="bg-emerald-950/20 rounded-xl p-4">
    <h4 className="text-sm font-bold text-emerald-400">
      <TreeView root={gameTree} subtitle="Слева направо
ществует." />
    </div>
  )}

{/* Дерево */}
{order === "y-desc" && userTree && (
  <div className="bg-slate-900/40 rounded-xl p-4">
    <h4 className="text-sm font-bold text-slate-300">
      <TreeView root={userTree} subtitle="Слева направо
    </div>
  )}
</div>
);
};

/* ===== РЕЖИМ SPLIT ===== */

const SplitMode: React.FC = () => {
  const [x0, setX0] = useState(7);
  const [stepIdx, setStepIdx] = useState(0);
  const fullTree = useMemo(() => buildTree(POINTS), [POINTS]);
  const split = useMemo(() => splitTree(buildTree(POINTS), x0), [fullTree, x0]);
  const steps = split.trace;
  const live = useTreapLive();
  // Компилятор: x – ключ разреза, i – номер шага спуска

```

```

.....
    <div className="text-xs text-slate-300 bg-slate-900"
      {steps[idx]?.note}
    </div>

    {done && (
      <div className="grid grid-cols-1 md:grid-cols-2"
        <div className="bg-sky-950/20 rounded-xl p-3"
          <h5 className="text-sm font-bold text-sky-300"
            <TreeView root={lTree} width={480} height={480}
          </div>
          <div className="bg-rose-950/20 rounded-xl p-3"
            <h5 className="text-sm font-bold text-rose-300"
              <TreeView root={rTree} width={480} height={480}
            </div>
          </div>
        )}

    <div className="text-xs text-slate-400 bg-slate-900"
      <LiveChip live={live} />{' '}
      <b>Как это работает на самом деле:</b> идём с
      деревом), отдаётся <b>вместе с веткой</b> – д
    </div>
  </div>
);
};

```

/ ===== РЕЖИМ MERGE ===== */*

```

const MergeMode: React.FC = () => {
  const [stepIdx, setStepIdx] = useState(0);
  const live = useTreapLive();
  // Компилятор: i – номер шага слияния.
  useEffect(() => { if (live.i !== null) setStepIdx(live.i); }, [live.i]);
  const { l, r } = useMemo(() => splitTree(buildTree(P0, P1, P2)));
  const merged = useMemo(() => mergeTree(cloneTree(l), cloneTree(r)));
  const steps = merged.trace;
  const idx = Math.min(stepIdx, steps.length - 1);
  const done = idx >= steps.length - 1;

```

```

    })

    <div className="text-xs text-slate-400 bg-slate-900" style={bgStyle}>
      <b>Грамотно про «слияние»:</b> у корней сравним
      в (то, что сохраняет порядок x) отправляется
    </div>
  </div>
);
};

/* not used here РЕЖИМ ERASE not used here */

```

```

const EraseMode: React.FC = () => {
  const [key, setKey] = useState(9);
  const [stepIdx, setStepIdx] = useState(0);
  const fullTree = useMemo(() => buildTree(POINTS), [POINTS]);
  const res = useMemo(() => eraseNode(buildTree(POINTS), key), [key]);
  const steps = res.trace;
  const live = useTreapLive();
  // Компилятор: x – ключ удаляемой вершины, i – шаг слияния
  useEffect(() => { if (live.x !== null && POINTS.some((p) => p.id === live.x)) setKey(9); }, [live.x]);
  useEffect(() => { if (live.i !== null) setStepIdx(Math.min(stepIdx, live.i)); }, [live.i]);
  const idx = Math.min(stepIdx, steps.length - 1);
  const done = idx >= steps.length - 1;

  const pathIds = useMemo(() => {
    const ids: NodeId[] = [];
    for (const s of steps.slice(0, idx + 1)) if (s.atId !== null) ids.push(s.atId);
    return ids;
  }, [steps, idx]);
  const pathSet = new Set(pathIds);

  return (
    <div className="space-y-4">
      <div className="flex flex-wrap items-center gap-3">
        <span className="text-sm text-slate-300">Удалить:
        <LiveChip live={live} />
        <select value={key} onChange={(e) => { setKey(Number(e.target.value)); }}>

```



```

</div>
  );
};

/* ----- РЕЖИМ LAYERS (2k/2k+1) -----

const LayersMode: React.FC = () => {
  const [pick, setPick] = useState<"heap" | "treap">("h
  // Полное дерево из 7 узлов (куча): индексы 1..7
  const heapArr = [0, 98, 45, 90, 32, 21, 76, 12];
  const full = useMemo(() => buildTree(POINTS), [1]);
  // Честная слотовая разметка treap: индексы как в куче
  const slots = useMemo(() => {
    const arr: (string | null)[] = Array(16).fill(null);
    const walk = (n: TNode | null, i: number) => {
      if (!n || i >= 16) return;
      arr[i] = n.id;
      walk(n.left, 2 * i);
      walk(n.right, 2 * i + 1);
    };
    walk(full, 1);
    return arr;
  }, [full]);
  const nodeCount = slots.filter(Boolean).length;

  return (
    <div className="space-y-4">
      <div className="flex gap-2">
        <button onClick={() => setPick("heap")} className="
          ext-white" : "bg-slate-700 text-slate-300">`>
        <button onClick={() => setPick("treap")} className="
          text-white" : "bg-slate-700 text-slate-300">
      </div>

      {pick === "heap" ? {
        <div className="bg-slate-900/40 rounded-xl p-4">
          <h4 className="text-sm font-bold text-slate-300">
            4>

```



```

        </div>
    );
  })}
</div>

<div className="text-xs text-slate-300 bg-slate-900" >
  {stage === 0 && <>Массив пар <b>не отсортирован</b>
    а беспорядок.</>}
  {stage === 1 && <> Бинарный поиск посмотрел на
    ю половину... <b>в которой лежала цель</b>. На
    реставил</b>: массив как был в беспорядке, та
  {stage === 2 && bidx < bsSteps.length - 1 && <>
  {stage === 2 && bidx >= bsSteps.length - 1 && <>
    сравнения. Но чтобы поиск вообще заработал,
    сравнения на <b>всех</b>. «Отсортирую бинарный
</div>

<div className="text-xs text-slate-400 bg-slate-900" >
  <b>Есть ли разница, чем сортировать?</b> Срав
  ния граница, каждое сравнение даёт 1 бит. Но
  ля маленьких целых у (вкладка «Собрать», кноп
  задать порядок соединения.
</div>
</div>
);
};

/* not standard but we don't know what we are not not not РЕЖИМ GAME not standard but we don't know what we are not not not */

const GameMode: React.FC = () => {
  const [edges, setEdges] = useState<GEdge[]>([]);
  const [sel, setSel] = useState<Pair | null>(null);
  const [hover, setHover] = useState<Pair | null>(null);
  const [msg, setMsg] = useState<string>("Кликни точку-");
  "));

  const v = gameVerdict(edges, POINTS);

```

```

<svg viewBox={`0 0 ${PL_W} ${PL_H + 20}`} class=
  <defs>
    <pattern id="gm-grid" width="14" height="14">
      <path d="M 14 0 L 0 0 0 14" fill="none" s
    </pattern>
  </defs>
  <rect x="34" y="16" width={PL_W - 46} height=
  <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={P
  <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y

  {edges.map((e, i) => {
    <line
      key={`ge-${i}`}
      x1={PX(e.a.key)} y1={PY(e.a.grade)}
      x2={PX(e.b.key)} y2={PY(e.b.grade)}
      stroke={i === edges.length - 1 ? "#34d399"
      strokeWidth={i === edges.length - 1 ? 3.5
      className="transition-all duration-300"
    />
  }}}

  {POINTS.map((p) => {
    const isSel = sel && sameP(sel, p);
    const isHover = hover && sameP(hover, p) &&
    return (
      <g key={`gp-${p.key}:${p.grade}`} onClick=
    > </g>
  ));
  <circle cx={PX(p.key)} cy={PY(p.grade)}
  <circle cx={PX(p.key)} cy={PY(p.grade)}
  "#a5b4fc" : isHover ? "#94a3b8" : "#38bdf8"
  transition-all duration-200" />
  <text x={PX(p.key)} y={PY(p.grade) + 3.5}
  space">{p.key}</text>
  <text x={PX(p.key)} y={PY(p.grade) + 24.5}
  </text>
  </g>
  );
  });
  });

```

```
.....

/** Значения из Python-компилятора: x – ключ, y – приоритет
function useTreapLive(): { x: number | null; y: number | null; i: number | null; linked: boolean } {
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
  const x = vizNumber(vars?.x);
  const y = vizNumber(vars?.y);
  const i = vizNumber(vars?.i);
  return { x, y, i, linked: x !== null || y !== null || i !== null };
}

const LiveChip: React.FC<{ live: { x: number | null; y: number | null; i: number | null; linked: boolean } }> = ({
  live: {
    x, y, i, linked
  } : { x: number | null; y: number | null; i: number | null; linked: boolean }
}) : null;

export const TreapBuildViz = () => {
  const [tab, setTab] = useState<Tab>("build");

  return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border-1 border-emerald-300">
      <div className="flex flex-wrap items-center justify-between">
        <div className="flex items-center gap-3">
          <div className="p-2.5 bg-emerald-600 text-white rounded-2xl">
            <span className="text-2xl"> </span>
          </div>
          <div>
            <h3 className="text-2xl font-bold text-white">
              <p className="text-sm text-emerald-300">Папки
            </p>
          </div>
        </div>
      </div>
      <div className="flex flex-wrap gap-2 mb-5">
```

Универсальное пособие • полный исходный код

```
.....
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { ColoringViz } from "./ColoringViz";
import { ComponentsViz } from "./ComponentsViz";
import { ComplexityClassesViz } from "./ComplexityClasses";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomaton";
import { SegmentTreeVisualizer } from "./SegmentTreeVisualizer";
import SplayPlayground from "./SplayPlayground";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithms";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimation";
import ChapterImage from "./ChapterImage";
import { PrefixSumViz } from "./visualizers/PrefixSumViz";
import { SparseTableViz } from "./visualizers/SparseTableViz";
import { TreapBuildViz } from "./TreapBuildViz";

export interface VizEntry {
  /** Заголовок панели/модалки. */
  title: string;
  /** Короткое пояснение, что здесь можно потыкать. */
  hint?: string;
  Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающей подсказки
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
```

```

.....
    hint: "Наведите на число — увидите, из чего оно сложилось",
    Component: PrefixSumViz,
},
"dynamic-programming": {
    title: "Динамическое программирование",
    hint: "Таблица подзадач заполняется на глазах.",
    Component: DPVisualizer,
},
"splay-tree": {
    title: "Splay: механика → большое дерево → собери сбалансированное",
    hint: "Три вкладки: покадровая механика поворотов в песочница «подними узел сам» с проверкой порядка.",
    Component: SplayPlayground,
},
"splay-rotations": {
    title: "Splay: механика → большое дерево → собери сбалансированное",
    hint: "Три вкладки: покадровая механика поворотов в песочница «подними узел сам» с проверкой порядка.",
    Component: SplayPlayground,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component: GraphDFS },
"top-sort": { title: "Топологическая сортировка", Component: TopSort },
"scc-kosaraju": { title: "Компоненты сильной связности", Component: SCC },
"graph-components": {
    title: "Компоненты связности: BFS-разметка",
    hint: "Каждый запуск обхода красит один «остров»; чистый граф — белый.",
    Component: ComponentsViz,
},
"graph-articulation": { title: "Точки сочленения", Component: GraphArticulation },
"bridges-code": { title: "Мосты: DFS + tin/low", Component: BridgesCode },
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: EulerPathVsCycle },
"planarity-euler-formula": {
    title: "Планарность: K5/K3,3 + раскраска",
    hint: "Слева — непланарные графы с подсветкой пересечений ребер.",
    Component: () => (
        <div className="space-y-10">
            <PlanarityDemo />
            <ColoringViz />
        </div>
    )
}

```

```
.....
"string-kmp": { title: "Префикс-функция (КМП)", Component: StringKmp },
"string-z-func": { title: "Z-функция", Component: StringZFunc },
"aho-corasick": {
  title: "Ахо-Корасик",
  hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
  Component: () => (
    <div className="space-y-10">
      <WaterfallAnimationWidget />
      <SalmonAutomatonWidget />
    </div>
  ),
},
"complexity-classes": {
  title: "Классы сложности: матрица  $P \subseteq NP \subseteq PSPACE$ ",
  hint: "Кликни класс или задачу; стрелки  $\Leftarrow$  показывают зависимости",
  Component: ComplexityClassesViz,
},
intro: { title: "Мнемокарточки", Component: MnemonicCards },
};

export const getViz = (id?: string): VizEntry | undefined;
```

```
.....
ФАЙЛ 76/99: src/components/WaterfallAnimationWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО
```

```
.....
import React, { useEffect, useState } from 'react';
import { useVizRuntime, vizNumber, vizString } from 'viz-runtime';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

// Структура заранее подготовленного красивого бора для поиска
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
  id: number;
  label: string; // путь от корня
  char: string;
```



```

.....
};

export const WaterfallAnimationWidget: React.FC = () => {
  // Переключатель режимов: 'training' (Полный BFS обход)
  const [activeMode, setActiveMode] = useState<'training' | 'search'>('training');

  // === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
  const [textToScan, setTextToScan] = useState<string>('');
  const [currentIndex, setCurrentIndex] = useState<number>(0);
  const [currentNode, setCurrentNode] = useState<number>(0);
  const [isSearchDone, setIsSearchDone] = useState<boolean>(false);
  const [searchLog, setSearchLog] = useState<string>('');
  'Режим поиска: Введите текст и жмите «Следующий шаг»';
  const [jumpPath, setJumpPath] = useState<{ from: number, to: number }>({ from: 0, to: 0 });
  const [foundMatches, setFoundMatches] = useState<{ index: number, match: string }>({ index: 0, match: '' });
  const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0);

  // === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
  // Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
  const [trainStep, setTrainStep] = useState<number>(0);

  // — Синхронизация с Python-компилятором: s/текст, i/индекс, v/вершина
  // v — текущая вершина автомата. Влияет на режим поиска.
  const runtime = useVizRuntime();
  const liveText = vizString(runtime?.variables?.s) ?? '';
  const liveI = vizNumber(runtime?.variables?.i);
  const liveNode = vizNumber(runtime?.variables?.v) ?? 0;
  useEffect(() => {
    if (liveText) {
      setTextToScan(liveText.toUpperCase().replace(/[^A-Z0-9_ ]/g, ''));
      resetSearchSimulation();
    }
  }, [liveText]);
  useEffect(() => {
    if (liveI !== null) setCurrentIndex(Math.max(0, Math.min(liveI, textToScan.length)));
  }, [liveI, textToScan.length]);
  useEffect(() => {
    if (liveNode !== null) setCurrentNode(Math.max(0, Math.min(liveNode, graph.nodes.length)));
  }, [liveNode, graph.nodes.length]);
};

```

```

.....
    _NODES[nextNode].label})). `;
    setJumpPath({ from: curr, to: nextNode, type: 'no
curr = nextNode;
    setActiveSearchCodeLine(3);
} else {
    setActiveSearchCodeLine(2);
    let jumps = 0;
    while (curr !== 0 && TRIE_TRANSITIONS[curr][char])
        curr = WATERFALL_NODES[curr].fail;
        jumps++;
    }
    const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
    if (originalCurr === 0) {
        logMsg += `В корне нет перехода по '${char}'. Л
        setJumpPath(null);
    } else {
        logMsg += `Поток по '${char}' у вершины #${orig
        рыжок по fail-ссылке в #${curr} (${WATERFALL_
        . `;
        setJumpPath({ from: originalCurr, to: nextNode,
    }
    curr = nextNode;
}

const currentMatches: { index: number; word: string
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
    if (WATERFALL_NODES[temp].is_terminal) {
        WATERFALL_NODES[temp].words.forEach((word) =>
            currentMatches.push({ index: currentInd
            matchedWordsForLog.push(word);
        });
    }
    temp = WATERFALL_NODES[temp].term_link;
}
}

```



```

    },
    {
      targetNodeId: 4,
      title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
      desc: 'Финальная вершина #4 (HERS) на Depth 4. Роль  
#7 (S). fail[HERS] = #7 (S). Автомат полностью обучен',
      codeLine: 4,
      scoutPos: 0,
      inspectedParentFail: 0,
      checkTargetChar: 'S',
      checkingBranchesFrom: 0,
    },
  ];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainInfo.targetNodeId];

```

```

return (
  <div className="bg-slate-800/95 rounded-2xl p-6 border-1px border-slate-700">
    {/* Шапка и переключатель режимов */}
    <div className="flex flex-wrap items-center justify-between">
      <div className="flex items-center space-x-3">
        <span className="text-3xl"> </span>
        <div>
          <h4 className="text-2xl font-bold text-white">
            Анимация водопада: Полное обучение автомата
          </h4>
          <p className="text-sm text-slate-400">
            Визуализация пошагового построения всех функций
          </p>
        </div>
      </div>
    </div>

    {/* Переключатель режимов */}
    <div className="bg-slate-900 p-1.5 rounded-xl border-1px border-slate-800">
      <button
        onClick={() => setActiveMode('training')}

```

```

.....
      {TRAINING_STEPS_INFO.map((step, idx) =>
        <button
          key={idx}
          onClick={() => setTrainStep(idx)}
          className={`w-full text-left px-3 p-2
            ${
              idx === trainStep
                ? 'bg-indigo-600 text-white font-medium'
                : 'bg-slate-800 text-slate-400'
            }`}
        >
          <span className="line-clamp-1">{step}
            {idx === trainStep && <span className="font-medium">
              </span>
            }
          </span>
        </button>
      )}
    </div>
  </div>

  <div className="flex gap-2">
    <button
      onClick={() => setTrainStep((prev) => Math.max(0, prev - 1))}
      disabled={trainStep === 0}
      className="flex-1 bg-slate-700 hover:bg-slate-600 transition-all"
    >
      Назад
    </button>
    <button
      onClick={() => setTrainStep((prev) => Math.min(TRAINING_STEPS_INFO.length - 1, prev + 1))}
      disabled={trainStep === TRAINING_STEPS_INFO.length - 1}
      className="flex-1 bg-indigo-600 hover:bg-indigo-500 transition-all shadow-lg shadow-indigo-900/50"
    >
      {trainStep === TRAINING_STEPS_INFO.length - 1 ? 'Вперед' : 'Далее'}
    </button>
  </div>
</div>

```

```

        </div>
        <div className={`p-1.5 rounded flex items-center justify-between border-blue-500 text-blue-300 font-bold`} >
          <span>5. else fail[u] = root; // Bero
          {currentTrainInfo.codeLine === 5 && <
        root</span>}
        </div>
      </div>
    </div>

    <div>
      <h5 className="text-slate-200 font-bold mb-2">
        <Lightbulb className="w-4 h-4 text-amber-400"/>
      </h5>
      <div className="bg-slate-800 p-4 rounded-3xl w-inner min-h-[72px] flex items-center">
        {currentTrainInfo.desc}
      </div>
    </div>
  </div>
</div>
) : (
  /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
  <div className="grid grid-cols-1 lg:grid-cols-3">
    <div className="bg-slate-900/60 p-5 rounded-xl">
      <div>
        <h5 className="text-white font-bold mb-2">
          <span>✎</span> Текст для сканирования
        </h5>
        <p className="text-xs text-slate-400 mb-4">
          Введи текст, чтобы лосось просканировал
        </p>
        <input
          type="text"
          value={textToScan}
          onChange={handleTextChange}
          maxLength={15}
          placeholder="ВВЕДИТЕ ТЕКСТ..."

```

```

        >
        <span>{currentIndex === 0 ? 'Начать пла
        <ArrowRight className="w-4 h-4" />
        </button>
    </div>
</div>

<div className="lg:col-span-2 bg-slate-900/60" >
    <div>
        <h5 className="text-white font-bold mb-3" >
            <span></span> Текущее действие в коде
        </h5>
        <div className="bg-slate-950 p-4 rounded-1" >
            <div className={`p-1.5 rounded flex item
border-blue-500 text-blue-300 font-bold` :
                <span>1. curr = state; char = text[i]
                {activeSearchCodeLine === 1 && <span>
инициализация</span>}
            </div>
            <div className={`p-1.5 rounded flex item
border-rose-500 text-rose-300 font-bold an
                <span>2. while (curr !== root && !trie
                {activeSearchCodeLine === 2 && <span>
рыжок по fail!</span>}
            </div>
            <div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold` :
                <span>3. curr = trie[curr].get(char)
                {activeSearchCodeLine === 3 && <span>
ем по течению</span>}
            </div>
            <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 fon
                <span>4. if (is_terminal[curr]) report
                {activeSearchCodeLine === 4 && <span>
опадение!</span>}
            </div>
        </div>
    </div>

```

```

.....
    </div>
  </div>

  { /* Само SVG дерево водопада */ }
  <div className="w-full overflow-x-auto custom-s<
    <svg viewBox="0 0 800 520" className="w-full

    { /* Горизонтальные линии и подписи уровней
    {
      { depth: 0, y: 60, label: 'Depth 0 (Корень
      { depth: 1, y: 160, label: 'Depth 1 (H, S
      { depth: 2, y: 260, label: 'Depth 2 (HE, I
      { depth: 3, y: 360, label: 'Depth 3 (HER,
      { depth: 4, y: 460, label: 'Depth 4 (HERS
    ].map((level) => {
      <g key={`depth-line-${level.depth}`}>
        <line
          x1="20"
          y1={level.y}
          x2="780"
          y2={level.y}
          stroke="#334155"
          strokeWidth="1"
          strokeDasharray="4,4"
          opacity="0.4"
        />
        <rect x="25" y={level.y - 12} width="14
      />
      <text x="32" y={level.y + 4} fill="#94a
        {level.label}
      </text>
    </g>
  }}}

  { /* Рендер прямых переходов (потоков воды)
  {Object.entries(TRIE_TRANSITIONS).map(({[from
    const fromNode = WATERFALL_NODES[Number(f
    return Object.entries(children).map(({[cha

```



```

.....
      <g transform={`translate(${(fromNode.x - toNode.x) / 2})`}>
        <rect x="-15" y="-12" width="30" height="24"
strokeWidth="2" />
        <text x="0" y="5" fontSize="14"
          {isMatchBranch ? ' ' : ' '}>
        </text>
      </g>
    })
  </g>
);
});
}}

```

```

{/* Рендер fail-ссылок (пунктирные водопады) */}
Object.values(WATERFALL_NODES).map((node) => {
  if (node.id === 0) return null;
  const targetNode = WATERFALL_NODES[node.fail];

  // В режиме обучения показываем fail-ссылку
  // Найдем индекс шага, на котором обработали fail-ссылку
  const stepIdxForNode = TRAINING_STEPS_INFO[node.fail].stepIdx;
  if (activeMode === 'training' && trainStep === stepIdxForNode) {
    let isHighlighted = activeMode === 'search' || activeMode === 'fail';
    let isJustEstablished = activeMode === 'training' && trainStep === stepIdxForNode;

    // Вычисляем изогнутую кривую для красоты
    const dx = targetNode.x - node.x;
    const dy = targetNode.y - node.y;
    const cx = node.x + dx / 2 + (node.id % 2 === 0 ? -10 : 10);
    const cy = node.y + dy / 2 - 30;

    return (
      <g key={`fail-${node.id}`}>
        <path

```

```

.....
    r={isCurrent || isTargetChild ? '24' : '12'}
    fill={isCurrent ? '#2563eb' : isTargetChild ? '#f8d7da' : '#d6d8db'}
    stroke={isCurrent ? '#ffffff' : isTargetChild ? '#000000' : '#000000'}
    strokeWidth={isCurrent || isTargetChild ? 1 : 0}
    className="transition-all duration-300"
  />

```

```

  { /* Текст внутри вершины */ }

```

```

<text
  x={node.x}
  y={node.y + 5}
  fill={isCurrent || isTargetChild ? 'white' : 'black'}
  fontSize={isCurrent || isTargetChild ? 12 : 10}
  fontWeight="bold"
  textAnchor="middle"
>
  {node.label}
</text>

```

```

  { /* Метка искомой вершины в режиме обхода */ }
  {isTargetChild && (
    <g transform={`translate(${node.x}, ${node.y})`} >
      <rect x="-35" y="-12" width="70" height="24" fill={fill} stroke={stroke} strokeWidth={strokeWidth} />
      <text x="0" y="2" fontSize="10" fill="white" >
        ИЩЕМ FAIL
      </text>
    </g>
  )}

```

```

  { /* Индикатор словарных слов */ }

```

```

  {hasWords && (
    <g transform={`translate(${node.x + 10}, ${node.y})`} >
      <circle cx="0" cy="0" r="9" fill={fill} stroke={stroke} strokeWidth={strokeWidth} />
      <text x="0" y="3" fontSize="10" fill="white" >
        *
      </text>
    </g>
  )}

```

```

    <span> </span> Улов лосося {Найденные слова}
  </h5>
  {foundMatches.length === 0 ? (
    <p className="text-sm text-slate-400 italic">
      Пока ничего не поймали. Запустите шаги сирены
    </p>
  ) : (
    <div className="flex flex-wrap gap-2">
      {foundMatches.map((match, idx) => (
        <div
          key={idx}
          className="bg-emerald-950 border border-emerald-500
            text-white text-sm font-bold shadow animate-[scaleUp_0.3s_ease-out]"
        >
          <CheckCircle2 className="w-4 h-4 text-emerald-500"/>
          <span>{match.word}</span>
          <span className="text-[10px] bg-emerald-950 text-white">
            Индекс {match.index}
          </span>
        </div>
      ))}
    </div>
  )}
</div>
</div>
);
};

```

РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 77/99: src/components/visualizers/PrefixSumViz.tsx
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 415 | РАЗМ: 10.5 КБ

```

.....
useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, mode === "2d"
}, [chapterId, mode]);

return (
  <div className="w-full bg-slate-950 rounded-2xl border
    <div className="flex gap-2 mb-4 overflow-x-auto no
      {{
        [
          ["1d", "1. Одномерные суммы"],
          ["2d", "2. Двумерные суммы"],
        ] as const
      }).map(([key, label]) => (
        <button
          key={key}
          onClick={() => setMode(key)}
          className={`px-3 py-2 rounded-lg text-xs sm
            mode === key ? "bg-indigo-600 text-white"
          }`}
        >
          {label}
        </button>
      )))
    </div>
    {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
  </div>
);
};

/* ..... ID .....
const Prefix1D: React.FC = () => {
  const pref = useMemo(
    () => A1.reduce<number[]>((acc, v) => [...acc, acc[v]
    [],
  );

  /** Что сейчас под курсором: префикс P[i] или элемент
  const [hover, setHover] = useState<{ kind: "pref" | "a

```

```

.....
<div className="flex gap-1.5 items-center mb-1"
  <div className="w-[46px] shrink-0 text-right"
    {A1.map((v, i) => {
      const inCover = covered && i >= covered.l
      const inRange = i >= range.l && i <= range.r
      const isDrivenAdd = shownDriven !== null
      return (
        <div
          key={i}
          onMouseEnter={() => setHover({ kind: "add" })}
          onMouseLeave={() => setHover(null)}
          onClick={() => setRange((r) => (i < r ? r : i))}
          className={` ${cellBase} cursor-pointer
            isDrivenAdd
              ? "bg-amber-500 text-black ring-2"
              : inCover
                ? "bg-indigo-500 text-white ring-2"
                : inRange
                  ? "bg-indigo-900/70 text-indigo-300"
                  : "bg-slate-800 text-slate-300"
            }`}
          title={`a[${i}] = ${v}`}
        >
          {v}
        </div>
      )
    })}
  </div>
);
}}
</div>

```

```

{/* массив P */}
<div className="flex gap-1.5 items-center">
  <div className="w-[46px] shrink-0 text-right"
    {pref.map((v, i) => {
      const active = hover?.kind === "pref" && i === prefIndex;
      const isL = i === range.l;
      const isR = i === range.r + 1;
      const liveValue = liveP?.[i];
      const blank = hasLiveP && (liveValue === null || liveValue === 0)
    })}
  </div>

```

```

.....
    <span className="font-mono text-white">i =
    <span className="font-mono text-emerald-300
    {hasLiveP && <= <span className="font-mono
    <span className="ml-1 text-slate-500">Пусто
</p>
) : hover?.kind === "pref" ? {
  hover.i === 0 ? {
    <p className="text-slate-300">
      <b className="text-emerald-400">P[0] = 0<
      отрезка, начинающегося с нуля.
    </p>
  ) : {
    <p className="text-slate-300">
      <b className="text-emerald-400">
        P[{hover.i}] = {pref[hover.i]}
      </b>{" "}
      – это сумма всего, что набежало от начала
    <span className="font-mono text-indigo-300">
      a[0..{hover.i - 1}] = {A1.slice(0, hove
    </span>
    </p>
  )
) : hover?.kind === "a" ? {
  <p className="text-slate-300">
    <b className="text-indigo-400">
      a[{hover.i}] = {A1[hover.i]}
    </b>{" "}
    входит во все префиксы, начиная с{" "}
    <span className="font-mono text-emerald-300">
    подавинуть границу запроса.
  </p>
) : {
  <p className="text-slate-500">
    Наведите курсор на любое число: у <b classN
    у <b className="text-indigo-400">a</b> – ку,
  </p>
)}
</div>

```

```

    return p;
  }, [n, m]));

const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

useEffect(() => {
  const r1 = vizNumber(vars?.r1);
  const c1 = vizNumber(vars?.c1);
  const r2 = vizNumber(vars?.r2);
  const c2 = vizNumber(vars?.c2);
  if (r1 === null || c1 === null || r2 === null || c2 === null) return;
  // Python-шаблон (и глава) используют 1-индексные координаты.
  // клетка (1, 1) – левый верхний элемент матрицы. В JavaScript же
  // 0-индексные, поэтому переводим: (1,1) → (0,0).
  const to0 = (v: number, hi: number) => Math.max(0, v - 1);
  setRect({
    r1: to0(r1, n - 1),
    c1: to0(c1, m - 1),
    r2: to0(r2, n - 1),
    c2: to0(c2, m - 1),
  });
}, [vars, n, m]);

const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;

const cornerOf = (i: number, j: number) => {
  if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
  if (i === rect.r1 && j === rect.c2 + 1) return "B";
  if (i === rect.r2 + 1 && j === rect.c1) return "C";
  if (i === rect.r1 && j === rect.c1) return "D";
  return null;
};

```


.....
.....
ФАЙЛ 78/99: src/components/visualizers/SparseTableViz.t
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТР0К: 830 | РАЗ

```
import React, { useContext, useEffect, useState } from
import {
  emitVizDemo,
  useVizRuntime,
  vizArray,
  vizNumber,
  vizRecord,
  VizChapterContext,
} from "../../data/vizStepBus";
import { motion, AnimatePresence } from "motion/react";
import {
  Play,
  RotateCcw,
  ChevronRight,
  ChevronLeft,
  Lock,
  ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () =>
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
  for (let i = 0; i + (1 << j) <= N; i++) {
    st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 <
  }
}

// Data for 2D Matrix (Sparse Table 2D)
```

```

.....
        ST2D[r][c + (1 << (ky-1))][kx][ky-1]
    );
    }
  }
}
}
}
}
}
}

```

```

export const SparseTableViz: React.FC = () => {
  // Эта страница посвящена именно двумерной таблице: 1D
  // вкладкой, но больше не подменяет 2D-визуализацию и
  const [tab, setTab] = useState<"1d" | "2d_build" | "2d_query">("1d");
  const chapterId = useContext(VizChapterContext);

  // Каждая вкладка = своё демо с собственным скелетом:
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, tab === "2d_build" ? "2d_build" : "2d_query", [chapterId, tab]);
  }, [chapterId, tab]);

  return (
    <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-5">
      <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800">
        <button
          onClick={() => setTab("1d")}
          className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white` : `bg-slate-800 text-slate-400`}
        >
          1. Сворачивание 1D
        </button>
        <button
          onClick={() => setTab("2d_build")}
          className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white` : `bg-slate-800 text-slate-400`}
        >
          2. Сворачивание 2D (Построение)
        </button>
        <button
          onClick={() => setTab("2d_query")}
          className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white` : `bg-slate-800 text-slate-400`}
        >
          3. Сворачивание 2D (Запрос)
        </button>
      </div>
    </div>
  );
}

```

```

.....
    liveI !== null && liveJ !== null && Number.isInteger
      ? { i: liveI, j: liveJ }
      : null;

// Total steps: we animate computing each element for
const computeSteps: { i: number; j: number }[] = [];
for (let j = 1; j < LOG; j++) {
  for (let i = 0; i + (1 << j) <= N; i++) {
    computeSteps.push({ i, j });
  }
}

const maxStep = computeSteps.length;
// Эта таблица читает i/j/st напрямую. Не двигаем её
// строки произвольного Python-кода.
const pointedCell = hover ?? liveTarget;
const covered = pointedCell ? { from: pointedCell.i,

return {
  <div className="flex flex-col gap-5">
    <div className="flex items-center justify-between">
      <div>
        <h3 className="text-lg font-bold text-indigo-500">
          Построение Sparse Table (Min)
        </h3>
        <p className="text-sm text-slate-400">
          Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + 2^{j-1}][j-1])$ 
        </p>
      </div>
      <div className="flex gap-2">
        <button
          onClick={() => setStep(Math.max(0, step - 1))}
          className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
          disabled={step === 0}
        >
          <ChevronLeft size={20} />
        </button>
        <button

```

```

    }}}
  </div>
  <div className="mt-2 text-xs min-h-[36px]">
    {hover && covered ? (
      <p className="text-slate-300">
        <b className="text-sky-400">
          ST[{hover.i}][[{hover.j}]] = {stID[hover.i]}
        </b>{" "}
        – это минимум на отрезке{" "}
        <span className="font-mono text-sky-300">
          [{covered.from}, {covered.to}]
        </span>{" "}
        длины  $2^{\{hover.j\}}$  = {1 << hover.j}:{ " "}
        <span className="font-mono text-slate-400">
          min({arrID.slice(covered.from, covered.to)}
        </span>
      </p>
    ) : (
      <p className="text-slate-500">
        Наведите курсор (или тапните) на любое чи
      </p>
    )}
  </div>
</div>

<div className="overflow-x-auto pb-2">
  <div className="inline-block min-w-full bg-slate-500">
    <div className="grid grid-cols-{auto_repeat(4)}">
      {/* Headers */}
      <div className="text-slate-500 font-mono text-sm">
        i \ j
      </div>
      <div className="text-center font-bold text-slate-500">
        rap">
           $2^0$  (L=1)
        </div>
      <div className="text-center font-bold text-slate-500">
        rap">

```

```

.....
    } else if (!compilerLinked && step > 0) {
      const currentStep = computeSteps[step];
      isActive = currentStep.i === i && currentStep.j === j;
      if (currentStep.j - 1 === j) {
        if (currentStep.i === i) isSrc1 = true;
        if (currentStep.i + (1 << (currentStep.j - 1)) === i) isSrc2 = true;
      }
    }
  }

  // Готовые вычисленные уровни демо не обновляются
  // уровень j=0 всегда берём из уже сущих
  const shownValue = compilerLinked ? compilerLinked : 0;
  const displayValue =
    shownValue === undefined || shownValue === null
      ? "."
      : typeof shownValue === "object"
        ? JSON.stringify(shownValue)
        : String(shownValue);

  return (
    <div key={j} className="relative flex flex-direction: column align-items: center">
      {!isValid ? (
        <div className="w-[clamp(30px,9vw,17px)] h-[clamp(11px,3vw,17px)] font-bold transition-colors">
          <motion.div
            onMouseEnter={() => (compilerLinked ? setCompilerLinked(0) : null)}
            onMouseLeave={() => setHover(false)}
            onClick={() => (compilerLinked ? setCompilerLinked(0) : null)}
            style={{
              opacity: compilerLinked ? 1 : 0,
              scale: compilerLinked || isSrc1 || isSrc2 ? 1 : 0.5,
            }}
            className={`w-[clamp(30px,9vw,17px)] h-[clamp(11px,3vw,17px)] font-bold transition-colors`}
          />
        ) : (
          <div
            style={{
              opacity: compilerLinked ? 1 : 0,
              scale: compilerLinked || isSrc1 || isSrc2 ? 1 : 0.5,
            }}
            className={`w-[clamp(30px,9vw,17px)] h-[clamp(11px,3vw,17px)] font-bold transition-colors`}
          />
        )
      )}
    </div>
  );
}

```

```

        />
      </motion.svg>
    })
  </div>
  );
  }}}
</React.Fragment>
  )))
</div>
</div>
</div>

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-
  {compilerLinked && liveTarget ? (
    <p className="text-base text-slate-300">
      <span className="text-emerald-400 font-bold">
        <span className="font-mono text-white">i =
        – активна ячейка <span className="font-mono
        {(hasLiveTable || hasLiveSource) && <math>= <span
        ?.[liveTarget.j]} ?? {liveTarget.j === 0 ? l
      </p>
    ) : step > 0 && step <= maxStep ? (
      (() => {
        const c = computeSteps[step - 1];
        const half = 1 << (c.j - 1);
        return (
          <p className="text-lg text-slate-300">
            <span className="text-white font-bold">
              ST[{c.i}][[{c.j}]]
            </span>{" "}
            = min(
              <span className="text-emerald-400 font-i
                ST[{c.i}][[{c.j} - 1]]
              </span>
            ,
            <span className="text-amber-400 font-bo
              ST[{c.i + half}][[{c.j} - 1]]

```



```

const isActive = !!activeCell &
ll.c + L) && ((r - activeCell.r) % (1 << st

    if (isActive) {
        if (isCurr) {
            highlightClass = `bg-indigo
{locked && locked.r === r && locked.c === c
            zIndex = 'z-10';
        } else {
            const prevL = 1 << (shownK
            const isTop = r < activeCell
            const isLeft = c < activeCe

                if (isTop && isLeft) {
                    highlightClass = 'bg-ros
md:scale-[1.08]';
                } else if (isTop && !isLeft
                    highlightClass = 'bg-blur
] md:scale-[1.08]';
                } else if (!isTop && isLeft
                    highlightClass = 'bg-eme
9,0.3)] md:scale-[1.08]';
                } else {
                    highlightClass = 'bg-amb
3)] md:scale-[1.08]';
                }
                zIndex = 'z-10';
            }
        }

    return (
        <div
            key={idx}
            onMouseEnter={() => { if(is
            onMouseLeave={() => { if(is
            onClick={() => {
                if(isCurr) {

```



```

    ) : (
      <div className="flex flex-col gap-4 items-center justify-between p-4">
        <p className="mb-4">При <span className="font-size-16 font-weight-medium">двух</span>  

          драты имеют размер <span className="font-size-16 font-weight-medium">10 см</span>  

          <div className="bg-[#0a0f1e] rounded-2xl p-4 shadow-inner overflow-x-auto">
            <span className="text-slate-500">// Текущая температура:</span>  

            <span className="text-indigo-400">{temp}</span><br/>
          </div>
          <p className="mt-6 italic text-indigo-400">Пример использования:  

            <Play size={14} className="fill-indigo-400 stroke-white"></Play>  

          </p>
        </div>
      ) : (
        <div className="flex flex-col gap-4 animate-fade-in">
          <div className="bg-[#0a0f1e] rounded-xl p-4 shadow-inner overflow-x-auto">
            <span className="text-slate-500">// Температура:</span>  

            <span className="text-purple-400">intensity</span> <span className="text-amber-300">1</span></div><br/><br/>
            <span className="text-slate-500">// Количество точек:</span>  

            <span className="text-indigo-400">ST</span> <span className="text-rose-400">1</span>  

            &nbsp;&nbsp;&nbsp;<span className="text-amber-300">1</span></div>, <span className="text-blue-400">1</span>  

            &nbsp;&nbsp;&nbsp;<span className="text-blue-400">ext-amber-300">1</span></div>[[k-<span className="text-emerald-400">1</span>  

            &nbsp;&nbsp;&nbsp;<span className="text-emerald-400">ext-amber-300">1</span></div>[[k-<span className="text-emerald-400">1</span>  

            &nbsp;&nbsp;&nbsp;<span className="text-amber-300">c + pl</span>]]<span className="text-emerald-400">1</span>  

            &nbsp;&nbsp;&nbsp;<span className="text-slate-500">// Правый Низ</span><br/>
          {''}});
        </div>
      ) : (

```

```

.....
        <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
        <span className="text
[activeCell.r + prevL][activeCell.c + prevL
        </div>
        </>
        )
        }}()
    </div>

    <div className="pt-3 mt-3 border-t
        ) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][shownK
    </div>
    <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
    </div>
    ) : {
        <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
        Наведите курсор на матрицу {L}x{L
        </div>
        )}
    </div>
    )}
</div>
</div>
</div>
};
};

```

```

const Viz2DQuery: React.FC = () => {
  const [r1, setR1] = useState(1);
  const [c1, setC1] = useState(1);
  const [r2, setR2] = useState(5);
  const [c2, setC2] = useState(6);
  const runtime = useVizRuntime();

```

```

.....
        return (
            <div key={idx} className={`
y-center rounded text-[clamp(9px,2.4vw,12px)
0 border-rose-500/50 border scale-105 z-10'
            {val2D[r][c]}
            </div>
        )
    }}}

    <div
        className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
        style={{
            top: `${(r1 / 8) * 100}%`,
            left: `${(c1 / 8) * 100}%`,
            marginTop: '8px',
            marginLeft: '8px',
            height: 'calc(' + (H/8*100) +
            width: 'calc(' + (W/8*100) +
        }}
    />

    {/* Показываем 4 угла в самой матрице */}
    <div className="absolute pointer-e
shadow-[0_0_10px_#f43f5e]" style={{ top: `
lc(`${(Ly / 8) * 100}% - 16px)`, height: `ca
        <div className="absolute pointer-e
shadow-[0_0_10px_#3b82f6]" style={{ top: `
width: `calc(`${(Ly / 8) * 100}% - 16px)`, i
        <div className="absolute pointer-e
ld-400 shadow-[0_0_10px_#10b981]" style={{
8px)`, width: `calc(`${(Ly / 8) * 100}% - 16
        <div className="absolute pointer-e
00 shadow-[0_0_10px_#f59e0b]" style={{ top:
00}% + 8px)`, width: `calc(`${(Ly / 8) * 100
    </div>

    <div className="bg-slate-950 p-4 roun

```

```

.....
    <div className="absolute right-3 top-3
    <span className="text-slate-500">// 1.
    <span className="text-purple-400">int<
ext-amber-300">1</span>; <span className="
    <span className="text-purple-400">int<
ext-amber-300">1</span>; <span className="
    <span className="text-purple-400">int<
"text-slate-500">// Вьс: {Lx}</span><br/>
    <span className="text-purple-400">int<
"text-slate-500">// Шир: {Ly}</span><br/><b

    <span className="text-slate-500">// 2.
    <span className="text-indigo-400">int<
    &nbsp;&nbsp;&nbsp;<span className="text-slate-500">В запроса)</span><br/>
    &nbsp;&nbsp;&nbsp;<span className="text-rose-900">r1</span>|{<span className="text-white bg-r

    &nbsp;&nbsp;&nbsp;<span className="text-slate-500">во, чтобы правый край уперся в c2)</span><b
    &nbsp;&nbsp;&nbsp;<span className="text-blue-500">r1</span>|{<span className="text-white bg-b

    &nbsp;&nbsp;&nbsp;<span className="text-slate-500">рх, чтобы нижний край уперся в r2)</span><b
    &nbsp;&nbsp;&nbsp;<span className="text-emerald-500">nded">r2 - Lx + 1</span>|{<span className="

    &nbsp;&nbsp;&nbsp;<span className="text-slate-500">и влево от c2)</span><br/>
    &nbsp;&nbsp;&nbsp;<span className="text-amber-500">">r2 - Lx + 1</span>|{<span className="text
    {'}}'}));
    </div>

    <div className="w-full flex justify-cent
    <div className="flex flex-col items-ces
    <h4 className="text-slate-400 font-bo

```

```

.....

        <div className="mt-auto bg-slate-950 font
    )]">
        <div className="text-center font-bold
wrap bg-[#0a0f1e] py-3 px-2 rounded-lg bord
        <span className="text-slate-400 font
            <span className="text-rose-400">
            <span className="text-blue-400 m
n>
            <span className="text-emerald-40
span>
            <span className="text-amber-400
            &nbsp;<span className="text-slate-4
            <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
            </div>
        </div>

        </div>
    </div>
)
}

```

only not editing has enough old was was always also was before has enough old want old enough not editing also was before was was the enough not editing has enough old want editing has enough old want old

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

number the enough was the the want not enough not editing not enough the want not enough enough has the the the want not enough was the the was not editing want not editing the has the enough was the

.....

ФАЙЛ 79/99: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 137 | РАЗМЕР

.....

```

        </div>
    </main>
</div>

```

.....

```
// Just some visual feedback on buttons
document.querySelectorAll('[id^="theme-"]')
    .forEach(btn => btn.classList.remove('outline', 'outline-1'));
```

```
if (theme === 'dark') document.getElementById('btn-theme-dark').classList.add('text-white transition-colors');
if (theme === 'light') document.getElementById('btn-theme-light').classList.add('text-white transition-colors');
if (theme === 'terminal') document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors');
```

```
document.getElementById('btn-theme-dark').classList.add('text-white transition-colors');
document.getElementById('btn-theme-light').classList.add('text-white transition-colors');
document.getElementById('btn-theme-notebook').classList.add('text-white transition-colors');
document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors');
```

```
}
```

```
// Initialize theme and mode
setTheme('dark');
```

```
// Setup intersection observer for TOC highlighting
document.addEventListener('DOMContentLoaded', () => {
    const mobileMenuBtn = document.getElementById('mobile-menu');
    const mobileDrawer = document.getElementById('mobile-drawer');
    const closeDrawerBtn = document.getElementById('close-drawer');
    const drawerOverlay = document.getElementById('drawer-overlay');
```

```
function toggleDrawer() {
    const isClosed = mobileDrawer.classList.contains('closed');
    if (isClosed) {
        mobileDrawer.classList.remove('closed');
        if (drawerOverlay) {
            drawerOverlay.classList.remove('hidden');
```

```
.....

    const observer = new IntersectionObserver((
      entries.forEach(entry => {
        if (entry.isIntersecting && !document
          // Highlight active TOC link lo
          const id = entry.target.id;
          document.querySelectorAll('aside
            if (link.getAttribute('href
              link.classList.add('bg-
            } else {
              link.classList.remove('
            }
          }));
        }
      }));
  }, observerOptions);

  document.querySelectorAll('section[id^="que
    observer.observe(section);
  });
});
</script>
</body>
</html>
```

.....
ФАЙЛ 80/99: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР: 1000

```
.....

<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  <title>Пособие по алгебре: От пиццы к предикатам</t
  <script src="https://cdn.tailwindcss.com"></script>
```

```

.....
    }
    .formula-scroll::-webkit-scrollbar {
        display: none;
    }

    mjx-container {
        max-width: 100% !important;
    }
    mjx-container[display="true"] {
        overflow-x: auto;
        overflow-y: hidden;
        max-width: 100%;
        scrollbar-width: none;
        -ms-overflow-style: none;
        -webkit-overflow-scrolling: touch;
    }
    mjx-container[display="true"]::-webkit-scrollbar
        display: none;
    }

    @media (max-width: 640px) {
        html { font-size: 13px; }

        .uno-card { width: 40px; height: 60px; font-
        .uno-card::before, .uno-card::after { font-
        .uno-card::before { top: 1px; left: 2px; }
        .uno-card::after { bottom: 1px; right: 2px; }

        .uno-card.mini { width: 28px !important; he
        us: 4px; }
        .uno-card.mini::before, .uno-card.mini::aft

        .uno-equation .text-2xl { font-size: 1.2rem
        .uno-equation.gap-4 { gap: 0.5rem !important

        /* Allow horizontal scroll for equations in
        .formula-scroll { flex-wrap: nowrap !importan
        px; }
    
```



```
.....  
body.theme-light #top-controls { background-color: #f8d7da; }
```

```
body.theme-notebook {  
    background-color: #fcf8eb !important; /* light beige */  
    color: #3b2818 !important; /* brown ink */  
    background-image: repeating-linear-gradient(45deg, transparent, transparent 2px, #3b2818 2px, #3b2818 4px);  
    background-attachment: local !important;  
    font-family: "Comic Sans MS", "Caveat", "Serif";  
}
```

```
body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-700 {  
    background-color: rgba(255, 255, 255, 0.7);  
    border-color: #cbd5e1 !important;  
    box-shadow: 2px 2px 5px rgba(0,0,0,0.05);  
}
```

```
body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-400 { color: #cbd5e1; }
```

```
body.theme-notebook .text-slate-400 { color: #546e7a; }
```

```
body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-800 { border-color: #94a3b8 !important; }
```

```
body.theme-notebook .font-mono { font-family: "Monospace"; }
```

```
body.theme-notebook #top-controls { background-color: #f8d7da; }
```

```
/* CREEPY BLINKING */
```

```
@keyframes creepy-blink {  
    0%, 95%, 98%, 100% { transform: scaleY(1); }  
    96.5% { transform: scaleY(0.1); }  
}
```

```
.creepy-eye {  
    transform-origin: center;  
    transform-box: fill-box;  
    animation: creepy-blink 4s infinite;  
}
```

```
.creepy-eye-alt {  
    transform-origin: center;  
    transform-box: fill-box;  
    animation: creepy-blink 5.2s infinite;  
}
```

```

.....
    <marker id="arrow-yellow" markerWidth="6" m
      <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d"
    </marker>
    <marker id="arrow-orange" markerWidth="6" m
      <path d="M0,0 L6,3 L0,6 Z" fill="#f9731d"
    </marker>
    <marker id="arrow-pink" markerWidth="6" mar
      <path d="M0,0 L6,3 L0,6 Z" fill="#f472b1"
    </marker>
    <marker id="arrow-cyan" markerWidth="6" mar
      <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4bf"
    </marker>
    <marker id="arrow-indigo" markerWidth="6" m
      <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8"
    </marker>
    <marker id="arrow-lime" markerWidth="6" mar
      <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d"
    </marker>
    <marker id="arrow-green" markerWidth="6" ma
      <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80"
    </marker>

    <!-- ЛЕГО БЛОКИ -->
    <g id="lego-yellow">
      <rect x="0" y="0" width="40" height="20"
      <rect x="6" y="-5" width="8" height="6"
      <rect x="26" y="-5" width="8" height="6"
      <line x1="2" y1="2" x2="38" y2="2" stro
    </g>
    <g id="lego-red">
      <rect x="0" y="0" width="40" height="20"
      <rect x="6" y="-5" width="8" height="6"
      <rect x="26" y="-5" width="8" height="6"
      <line x1="2" y1="2" x2="38" y2="2" stro
    </g>
    <g id="lego-rem">
      <rect x="0" y="0" width="40" height="10"
      <rect x="6" y="-5" width="8" height="6"

```

```
.....  
<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-
```

```
  <!-- Левое меню (Оглавление) -->
```

```
  <aside id="mobile-drawer" class="fixed inset-y-0  
    translate-x-full transition-transform duration-300
```

```
    w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
```

```
    <div class="h-full flex flex-col p-6 overflow-y-auto
```

```
    lg:rounded-2xl lg:border lg:border-slate-700
```

```
      <div class="flex justify-between items-center
```

```
        <h3 class="font-black text-transparent bg-clip-text  
        ider text-sm">
```

```
          Содержание
```

```
        </h3>
```

```
        <button id="close-drawer-btn" class="text-slate-500
```

```
          <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6
```

```
            <path stroke-linecap="round" stroke-width="2" d="M6 9l9 9
```

```
          </svg>
```

```
        </button>
```

```
      </div>
```

```
    <nav class="space-y-3 text-sm font-medium">
```

```
      <details class="group" open>
```

```
        <summary class="list-none cursor-pointer">
```

```
          <a href="#question-1" class="text-slate-500 pl-3 font-bold text-blue-300">
```

```
            1. Алгебраические структуры
```

```
          </a>
```

```
        </summary>
```

```
        <div class="pl-6 space-y-2 text-sm">
```

```
          <a href="#section-1-1" class="text-slate-500">1.1. Группы
```

```
          <a href="#section-1-2" class="text-slate-500">1.2. Кольца
```

```
          <a href="#section-1-3" class="text-slate-500">1.3. Поля
```

```
          <a href="#section-1-4" class="text-slate-500">1.4. Модули
```

```
        </div>
```

```
      </details>
```

```
    <details class="group">
```

```
.....
    <a href="#q4-root" class="block
    <a href="#q4-count" class="bloc
</div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-5" class:
er-fuchsia-500 pl-3 font-bold text-fuchsia-1
        5. Группа корней из единицы
    </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q5-def" class="block
        <a href="#q5-group" class="bloc
        <a href="#q5-prim" class="block
        <a href="#q5-crit" class="block
        <a href="#q5-cyclic" class="blo
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-6" class:
er-lime-500 pl-3 font-bold text-lime-300">
        6. Делимость, НОД и Евклид
    </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q6-1" class="block ho
        <a href="#q6-2" class="block ho
        <a href="#q6-3" class="block ho
        <a href="#q6-4" class="block ho
        <a href="#q6-5" class="block ho
        <a href="#q6-6" class="block ho
    </div>
</details>
```

er-green-500 pl-3 font-bold text-green-400">

9. Кольцо многочленов

</summary>

<div class="pl-6 space-y-2 text

<a href="#q9-1" class="bloc

<a href="#q9-2" class="bloc

<a href="#q9-3" class="bloc

<a href="#q9-4" class="bloc

</div>

</details>

<details class="group">

<summary class="list-none curso

<a href="#question-10" clas

der-rose-500 pl-3 font-bold text-rose-400">

10. Делимость и Схема Г

</summary>

<div class="pl-6 space-y-2 text

<a href="#q10-1" class="blo

<a href="#q10-2" class="blo

<a href="#q10-3" class="blo

a>

<a href="#q10-4" class="blo

<a href="#q10-5" class="blo

</div>

</details>

<details class="group">

<summary class="list-none curso

<a href="#question-11" clas

der-cyan-500 pl-3 font-bold text-cyan-400">

11. НОД и НОК многочлен

</summary>

<div class="pl-6 space-y-2 text

<a href="#q11-1" class="blo

```
.....
    <details class="group">
      <summary class="list-none cursor-pointer text-fuchsia-500 pl-3 font-bold text-fuchsia-400">
        14. Кратность корня. Корень
      </a>
    </summary>
    <div class="pl-6 mt-2 space-y-2">
      <a href="#q14-1" class="block text-fuchsia-400">
      <a href="#q14-2" class="block text-fuchsia-400">
      <a href="#q14-3" class="block text-fuchsia-400">
        deg)</a>
      <a href="#q14-4" class="block text-fuchsia-400 border border-rose-500/30"> Кратность корня
    </div>
  </details>

  <details class="group">
    <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
      15. Производная. Критерий
    </a>
  </summary>
  <div class="pl-6 mt-2 space-y-2">
    <a href="#q15-1" class="block text-cyan-400">
    <a href="#q15-2" class="block text-cyan-400">
    <a href="#q15-3" class="block text-cyan-400">
    <a href="#q15-4" class="block text-cyan-400">
  </div>
</details>

  <details class="group">
    <summary class="list-none cursor-pointer text-red-500 pl-3 font-bold text-red-400">
      16. Неприводимые многочлены
    </a>
```

.....

```

</nav>
<script>
    document.querySelectorAll('details.'
        detail.addEventListener('toggle
            if (detail.open) {
                document.querySelectorA
                    if (other !== detail
                        other.removeAtt
                    }
            });
        }
    });
});

// Scroll spy logic
document.addEventListener('DOMConten
    const sections = document.query
    ="q5-"], div[id^="q6-"], div[id^="q7-"], di
    v[id^="q13-"], div[id^="q14-"], div[id^="q1
    let isScrolling = false;

    document.querySelectorAll('.gro
        link.addEventListener('click
            isScrolling = true;
            setTimeout(() => isScro
        });
    });

    const observer = new Intersection
        if (isScrolling) return;

        let visibleId = null;
        entries.forEach(entry => {
            if (entry.isIntersectin
                visibleId = entry.t

// Highlight active

```

```
.....
modal.style.width = '100vw';
modal.style.height = '100vh';
modal.style.backgroundColor = 'white';
modal.style.zIndex = '99999';
modal.style.display = 'flex';
modal.style.flexDirection = 'column';
modal.style.alignItems = 'center';
modal.style.justifyContent = 'center';

const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a1a1a';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('div');
header.innerText = 'Экспорт в PDF';
header.style.marginBottom = '10px';

const subHeader = document.createElement('div');
subHeader.innerText = 'Из-за не  
ние веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';
```



```

        box.appendChild(textarea);
        btnContainer.appendChild(closeB);
    }
    btnContainer.appendChild(closeB);
    box.appendChild(btnContainer);
    modal.appendChild(box);
    document.body.appendChild(modal);
}

```

```

function downloadMD() {
    let clone = document.querySelector('#clone');

    // Smart markdown parsing based on DOM
    // Smart markdown parsing based on DOM
    clone.querySelectorAll('h2').forEach(el => {
        clone.querySelector('h3').forEach(el => {
            clone.querySelector('.analog').forEach(el => {
                clone.querySelector('li').forEach(el => {
                    clone.querySelector('img').forEach(el => {
                        let caption = '';
                        if (el.nextElementSibling && el.nextElementSibling.tagName === 'CAPTION') {
                            caption = el.nextElementSibling.textContent;
                        }
                        el.textContent = `\\n\\n
                    });
                });
            });
        });
    });

    clone.querySelectorAll('.img-caption').forEach(el => {
        let text = clone.innerHTML;
        text = text.replace(/\\n{3,}/g, '\\n\\n\\n');

        if (window !== window.top) {
            fallbackModal(text, 'Markdown');
            return;
        }

        const blob = new Blob([text], { type: 'text/markdown' });
        const url = URL.createObjectURL(blob);
        const link = document.createElement('a');
        link.href = url;
        link.textContent = 'Download Markdown';
        link.click();
        URL.revokeObjectURL(url);
    });
}

```

.....

```

/* Specific backgrounds
body.bg-white aside > d
body.bg-white [class*="f
nt; }
body.bg-white [class*="f
nt; }
body.bg-white [class*="f
body.bg-white .analogy-
body.bg-white .img-plac
body.bg-white .info-blo

body.bg-white .analogy-
or: #94a3b8 !important; }
body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya

```

```

.....
        <div class="flex flex-col gap-2">
            <div class="text-xs text-slate-500">
                <button onclick="downloadWord()"
ont-bold flex items-center justify-center gap-2
                    <span> </span> Word
                </button>
                <a href="algebra.pdf" target="_blank"
-sm font-bold flex items-center justify-center gap-2
                    <span> </span> PDF
                </a>
                <button onclick="downloadMD()"
-bold flex items-center justify-center gap-2
                    <span> </span> Markdown
                </button>
                <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-center gap-2
                    <span> </span> HTML
                </button>
            </div>

        <!-- Смена темы -->
        <div>
            <div class="text-xs text-slate-500">
                <div class="flex items-center gap-2">
                    <button id="theme-dark" onclick="toggleTheme('dark')"
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
                    <button id="theme-light" onclick="toggleTheme('light')"
r justify-center text-amber-50 hover:text-white
">* </button>
                    <button id="theme-terminal"
-center justify-center text-green-400 hover:text-white
терминала"> </button>
                </div>
            </div>
        </div>
    </div>
</div>

```

```

        </ul>
    </div>
</header>

<!-- the rest of the code for the container WTF CONTAINER the rest of the code -->
<div id="wtf-container" class="hidden">
    <h2 class="text-3xl font-black text-cen

        <div class="overflow-x-auto w-full pb-4"
            <div class="grid grid-cols-3 gap-4"
                <!-- Column headers -->
                <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
                <div class="bg-orange-900/40 bo
ольцо полиномов</div>
                <div class="bg-cyan-900/40 bord
сные числа</div>

                <!-- Row 1: База -->
                <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
                    <div class="text-xs text-yellow
                    <div class="font-bold text-slat
                </a>
                <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
                    <div class="text-xs text-orange
                    <div class="font-bold text-slat
                </a>
                <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
                    <div class="text-xs text-cyan-5
                    <div class="font-bold text-slat
                </a>

                <!-- Row 2: Базовые операции / Евкли
                <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra

```

```

.....
        </a>
        <a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-500"
        <div class="font-bold text-slate-700"
        </a>

        <!-- Row 5: Корни / Уравнения -->
        <div class="hidden md:block"></div>
        <a href="#question-13" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange-500"
        <div class="font-bold text-slate-700"
        </a>
        <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-500"
        <div class="font-bold text-slate-700"
        </a>

        <!-- Row 6: Кратность -->
        <div class="hidden md:block"></div>
        <a href="#question-14" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange-500"
        <div class="font-bold text-slate-700"
        </a>
        <div class="hidden md:block"></div>

        <!-- Row 7: Производная -->
        <div class="hidden md:block"></div>
        <a href="#question-15" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange-500"
        <div class="font-bold text-slate-700"
        </a>
        <div class="hidden md:block"></div>

```

```
.....
border-b border-red-500/30 pb-2">Кластер 4:
```

```
    <a href="#proof-algorithms" onclick=
:col-span-3 bg-red-900/20 p-4 border-l-4 bo
    <div class="text-xs text-red-500">
    <div class="font-bold text-slate-500">
    </a>
  </div>
</div>
```

```
<!-- ЛИКБЕЗ SECTION INSERTED HERE -->
<section id="proof-algorithms" class="m
  <div class="flex items-center mb-6">
    <div class="bg-red-500/20 p-3 r
      <svg class="w-8 h-8 text-red-500"
rg/2000/svg">
        <path stroke-linecap="r
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
      </svg>
    </div>
    <h2 class="text-3xl pr-4 border
  </div>
```

```
  <div class="prose prose-invert max-w
    <p class="text-lg">
      Хватит тупить. Доказательств
    есь для того, чтобы жать на кнопки. Вот жес
    </p>
```

```
    <h2 class="text-2xl font-bold b
(Меметика)</h2>
```

```
<!-- Прямое -->
<div class="bg-slate-800/80 p-6">
  <h3 class="text-xl text-blue-400">
  <p class="text-sm text-slate-400">
  <ul class="list-disc pl-5 s
    <li><span class="text-w
```

```
.....
        <ul class="list-disc pl-5 sp-1">
            <li><span class="text-white">
                валось.</li>
            <li><span class="text-white">
                Просто перепиши формулу с буквой  $k$ .</li>
            <li><span class="text-white">
                усок  $k$ , замени его на жареного карася из
            </li>
        </ul>
    </div>
```

```

    <!-- Двойная делимость -->
    <div class="bg-slate-800/80 p-6">
        <h3 class="text-xl text-orange-400">
        <p class="text-sm text-slate-200">
        <ul class="list-disc pl-5 sp-1">
            <li><span class="text-white">
                вверх по уравнениям).</li>
            <li><span class="text-white">
                рху вниз, показывая, что  $\Delta \vdots r_i$ 
            <li><span class="text-white">
                х. Твой  $d$  больше.</li>
        </ul>
    </div>
```

```

    <h2 class="text-2xl font-bold border-bottom">
        ритмы экзекуции)</h2>
```

```

    <div class="grid grid-cols-1 md:grid-cols-2">
        <div class="bg-slate-800 p-6">
            <h4 class="text-red-400">
            <p class="text-sm text-slate-200">
            <p class="text-xs font-sans">
                со следующим. Если в конце 0 – пациент мертв
            </div>
```

```

    <div class="bg-slate-800 p-6">
        <h4 class="text-red-400">
```

```
.....

    <li class="bg-slate-800/50"
      <span class="text-2xl">
        <div>
          <h4 class="text-cyan">
            <p class="text-sm">
              x двух, потом последних. <br>
              <code class="text-x">
            </div>
          </li>

        <li class="bg-slate-800/50"
          <span class="text-2xl">
            <div>
              <h4 class="text-cyan">
                <p class="text-sm">
                  есть сложение — есть ноль. Если есть умноже
                  <code class="text-x">
                </div>
              </li>

            <li class="bg-slate-800/50"
              <span class="text-2xl">
                <div>
                  <h4 class="text-cyan">
                    <p class="text-sm">
                      авь их драться. Умножение всегда врывается
                      <code class="text-x">
                    </div>
                  </li>
                </ul>
              </div>
            </section>
          </div>

        <div id="questions-container">
          <!-- what the sharded sharded what and not knowles what and knowles what not ВОПРОС 1 whatles whatles what what the sharded what les
```



```
.....

/** Собирает CSS страницы: сначала пробуем правила, при
async function collectCss(): Promise<string> {
  const parts: string[] = [];
  for (const sheet of Array.from(document.styleSheets))
    try {
      const rules = (sheet as CSSStyleSheet).cssRules;
      parts.push(Array.from(rules).map((r) => r.cssText));
    } catch {
      const href = (sheet as CSSStyleSheet).href;
      if (!href) continue;
      try {
        const res = await fetch(href);
        if (res.ok) parts.push(await res.text());
      } catch {
        /* чужой домен — пропускаем */
      }
    }
  }
  return parts.join("\n");
}
```

```
const EXTRA_CSS = `
/* — правки для автономного файла — */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family
a,term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding
.export-head { border-bottom: 1px solid #1e293b; padding
.export-note { background: #0f172a; border: 1px solid #
  border-radius: .5rem; padding: .85rem 1rem; font-size
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; fon
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; fon
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem
```

```
.....
    el.replaceWith(link);
  });

  // <details> в файле лучше раскрыть: иначе код и разб
  host.querySelectorAll("details").forEach((d) => d.set

  return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[]): string {
  if (!termIds.length) return "";
  const items = termIds
    .map((id) => {
      const e = GLOSSARY_BY_ID.get(id);
      if (!e) return "";
      return `<div class="export-term" id="term-${escape
<h3>${escapeHtml(e.title)}</h3>
<p>${escapeHtml(e.short)}</p>
${e.formal ? `<p class="formal">${escapeHtml(e.formal
${e.complexity ? `<p class="cx">Сложность: ${escapeHt
</div>`;
    })
    .join("\n");

  return `<section class="export-gloss">
<h2>Словарик терминов из этой темы</h2>
<p style="font-size:.8rem;color:#94a3b8;margin:0 0 1r
  ны сюда.</p>
  ${items}
</section>`;
}

function wrap(opts: { title: string; subtitle: string;
  const stamp = new Date().toLocaleString("ru-RU");
  return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
```

```

.....
    a.click();
    a.remove();
    setTimeout(() => URL.revokeObjectURL(url), 2000);
}

/** Имя файла: «04-splay-derevo.html» – без кириллицы и
function safeName(title: string, id: string): string {
    const num = title.match(/^\\s*(\\d+)/)?.[1];
    const base = id.replace(/[^a-z0-9-]+/gi, "-").toLowerCase();
    return `${num ? String(num).padStart(2, "0") + "-" : ""}${base}.html`;
}

/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter) {
    const css = await collectCss();
    const { body, termIds } = prepareBody(chapter.content);
    const html = wrap({
        title: chapter.title,
        subtitle: chapter.category || "Универсальное пособие",
        css,
        body,
        gloss: glossarySection(termIds),
        origin: window.location.origin,
    });
    triggerDownload(html, safeName(chapter.title, chapter.id));
}

/** Выгрузить всё пособие одним файлом со сквозным оглавлением. */
export async function downloadBookHtml(chapters: Chapter[]) {
    const css = await collectCss();
    const allTerms: string[] = [];
    const bodies: string[] = [];

    const toc = chapters
        .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}">${c.title}</a>`
        .join("\\n");

    chapters.forEach((c) => {

```

Универсальное пособие • полный исходный код

```
.....
* (блок «Аналогия») и/или 2D-визуализации (интерактивны
*
*   node scripts/audit-coverage.mjs           # отчёт
*   node scripts/audit-coverage.mjs --md      # markd
*/
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
  entryPoints: [path.join(ROOT, "src/data/content.ts")],
  bundle: true,
  format: "esm",
  platform: "node",
  outfile: bundle,
  logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/registry.json"), "utf-8");
const registryBody = registry.slice(registry.indexOf("VIZUALIZATORY"), registry.length);
const vizIds = new Set([...registryBody.matchAll(/^\s{2}[a-zA-Z0-9_-]+/g)].map(m => m[0]));

const rows = chapters.map((c) => {
  const html = c.content ?? "";
  const analogies = (html.match(/Аналогия/gi) ?? []).length;
  return {
    id: c.id,
    title: c.title,
```

Универсальное пособие • полный исходный код

```
.....
  dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
  dump("НЕТ 2D (аналогия есть)", gapViz);
  console.log(`Пропущенные номера билетов 1-24: ${missi
  console.log(`Визуализаторы без главы: ${orphanViz.join
}
```

.....
ФАЙЛ 84/99: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```
.....
/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 *   node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
  entryPoints: ["src/data/content.ts", "src/data/glossa
  bundle: true, format: "esm", platform: "node",
  outdir: "tmp/audit-terms", outExtension: { ".js": ".m
  external: ["react", "react-dom", "motion", "lucide-re
});

const { chapters } = await import(path.join(ROOT, "tmp/
const { GLOSSARY_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[\.\*\+?\^\$\{\}\(\)|[\]\[\]\]/g, "\\
const re = new RegExp(
  `(?!([\\p{L}\\p{N}]-|))(${GLOSSARY_LABELS.map((l) => e
  "giu"
);
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label
```

Универсальное пособие • полный исходный код

```
.....
console.log("\nГлав без единой подсказки:", bad.length,
console.log("Терминов в глоссарии:", GLOSSARY.length, "
console.log("Не встретились:", GLOSSARY.filter((g) => !
```

.....
ФАЙЛ 85/99: scripts/export/build-guide-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 28 | РАЗМ

```
.....
/**
 * Оркестратор: бандлит TSX-генератор и запускает его.
 * Использование: node scripts/export/build-guide-pdf.m
 */
```

```
import { build } from "esbuild";
import { pathToFileURL } from "node:url";
import path from "node:path";
import fs from "node:fs";
```

```
const root = path.resolve(import.meta.dirname, "../..");
const outfile = path.join(root, "tmp/guide-pdf-bundle.c
```

```
await build({
  entryPoints: [path.join(root, "scripts/export/guide-p
  outfile,
  bundle: true,
  platform: "node",
  format: "cjs",
  target: "node20",
  jsx: "automatic",
  loader: { ".jpg": "dataurl", ".png": "dataurl", ".css
  external: ["pdfkit", "svg-to-pdfkit", "pdfkit/js/pdfk
  logLevel: "warning",
});
```

```
const outArg = process.argv[2];
await import(pathToFileURL(outfile).href);
```

```
.....
    info: {
      Title: "Универсальное пособие — полный исходный код",
      Author: "CI: rebuild-pdf",
      Subject: "Экспорт всего кода проекта (код -> txt)",
      CreationDate: new Date(),
    },
  ));

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
  doc.addPage({ size: [A4.width, A4.height], margin: 0 });
  doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
}

doc.end();
await new Promise((resolve, reject) => {
  stream.on("finish", resolve);
  stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`    страниц: ${pages.length}`);
console.log(`    выход:   ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
  console.error("Ошибка сборки PDF:", err.message);
  process.exit(1);
});
```

Универсальное пособие • полный исходный код

```
.....  
const categoryOf = (rel) => CATEGORIES.find(({rel}) => r
```

```
/** Порядок разделов в документе. */
```

```
const ORDER = [  
  "Конфигурация проекта",  
  "Ядро приложения",  
  "Контент и данные пособия",  
  "Билеты (учебный контент)",  
  "Интерактивные компоненты и симуляторы",  
  "Визуализаторы (вложенные)",  
  "HTML-разметка (layout)",  
  "Утилиты",  
  "Скрипты сборки и экспорта",  
  "CI / автоматизация",  
  "Прочее",  
];
```

```
function listFiles() {  
  let files;  
  try {  
    files = execFileSync("git", ["ls-files", "--cached"  
      cwd: ROOT,  
      encoding: "utf8",  
    })  
      .split("\n")  
      .filter(Boolean);  
  } catch {  
    files = [];  
    const walk = (dir) => {  
      for (const e of fs.readdirSync(dir, { withFileType  
        const abs = path.join(dir, e.name);  
        const rel = path.relative(ROOT, abs).split(path  
        if (EXCLUDE_DIRS.some((d) => rel === d || rel.s  
        if (e.isDirectory()) walk(abs);  
        else files.push(rel);  
      }  
    };  
    walk(ROOT);  
  }
```



```

const chapters = extractChapters(files);
ensureDir(OUT_DIR);

const out = [];
const push = (s = "") => out.push(s);

const stamp = new Date().toLocaleString("ru-RU", { time: true });
const totalBytes = files.reduce((s, f) => s + fs.statSync(f).size, 0);

push(HR);
push("      УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ ДАННЫХ");
push("      ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ И КОД)");
push(HR);
push();
push(`Дата генерации:                ${stamp}`);
push(`Файлов исходного кода:         ${files.length}`);
push(`Суммарный объём кода:           ${humanSize(totalBytes)}`);
push(`Глава/билетов в пособии:        ${chapters.length}`);
push();

push(HR);
push("      ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
push(HR);
chapters.forEach((c, i) => {
  push(`  ${String(i + 1).padStart(3, " ")}. ${c.title}`);
});
push();

push(HR);
push("      ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
push(HR);
let current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push();
    push(`  # ${cat}`);
  }
});

```

Универсальное пособие • полный исходный код

```
.....
    console.log(`      строк:    ${txt.split("\n").length}`
    console.log(`      выход:    ${path.relative(ROOT, TXT)}
  }
```

```
main();
```

```
.....
ФАЙЛ 88/99: scripts/export/common.mjs
```

```
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМ:
```

```
.....
/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код   -> full_code_all_pages.txt      (collect-code.mjs)
 * txt   -> page-XXXX.png                (render-pages.mjs)
 * png   -> full_code_all_pages.pdf      (build-pdf.mjs)
 */
```

```
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";
```

```
export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)),
```

```
/** Куда кладём финальные артефакты (эта папка отдаётся клиенту)
export const OUT_DIR = path.join(ROOT, "public", "export");
/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают)
export const PAGES_DIR = path.join(ROOT, "tmp", "export");
```

```
export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");
```

```
/** Параметры вёрстки страницы (подобраны под A4 @150dpi)
export const PAGE = {
  /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100
  density: Number(process.env.EXPORT_DENSITY ?? 150),
```

```
}  
  
.....  
}
```

```
.....  
ФАЙЛ 89/99: scripts/export/guide-pdf-entry.tsx
```

```
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 673 | РАЗМЕР: 10.5 КБ  
.....
```

```
/**  
 * Автосборка PDF-конспекта `guide_all_topics.pdf`.  
 *  
 * Что внутри:  
 * * весь ТЕКСТ страниц пособия (больше текста, чем HTML)  
 * * картинки визуализаций – React-компоненты рендерятся  
 *   (renderToStaticMarkup), из разметки берутся SVG-ссылки  
 * * НИКАКОГО кода компилятора и интерфейсных кнопок.  
 *  
 * Результат: public/export/guide_all_topics.pdf, ~40–70 КБ  
 */
```

```
import * as fs from "node:fs";  
import * as path from "node:path";  
import { createElement } from "react";  
import { renderToStaticMarkup } from "react-dom/server";  
import PDFDocument from "pdfkit";  
// @ts-ignore - у пакета нет типов  
import SVGtoPDF from "svg-to-pdfkit/source.js";  
import { chapters } from "../../src/data/content";  
import { VizChapterContext } from "../../src/data/vizState";  
import { VIZ_REGISTRY, getViz } from "../../src/components";
```

```
// ----- шрифты -----
```

```
const FONT_DIRS = ["/usr/share/fonts/truetype/dejavu",  
function findFont(file: string): string | null {  
  for (const dir of FONT_DIRS) {  
    const p = path.join(dir, file);  
    if (fs.existsSync(p)) return p;  
  }  
}
```

```

.....
    .replace(/<br\s*\s*>/gi, "\n")
    .replace(/<\/(p|div|li|h[1-6]|tr|summary)>/gi, "\n")
    .replace(/<[>]+>/g, " ")
  )
  .replace(/[ \t]+/g, " ")
  .replace(/ *\n */g, "\n")
  .replace(/\n{2,}/g, "\n")
  .trim();
}

/** Найти закрывающий тег с учётом вложенности однотипных
function findClose(html: string, openEnd: number, tag: string): number {
  const re = new RegExp(`<\/?${tag}(\s[>]*)?>`, "gi")
  re.lastIndex = openEnd;
  let depth = 1;
  let m: RegExpExecArray | null;
  while ((m = re.exec(html))) {
    depth += m[0][1] === "/" ? -1 : 1;
    if (depth === 0) return m.index;
  }
  return html.length;
}

function findCloseAny(html: string, openEnd: number, tag: string): { end: number; inner: string } {
  const close = html.toLowerCase().indexOf(`<\/${tag}`);
  if (close < 0) return { end: html.length, inner: html };
  const inner = html.slice(openEnd, close);
  const end = html.indexOf(">", close) + 1;
  return { end, inner };
}

/** Разобрать HTML главы на учебные блоки (текст → 90% )
export function htmlToBlocks(html: string): Block[] {
  const blocks: Block[] = [];
  let i = 0;
  const skipTo = ["<pre", "<h2", "<h3", "<h4", "<summary"];
  while (i < html.length) {
    const lower = html.toLowerCase();

```

```

.....
    // содержимое details выводим тоже: оно идёт сразу
    const detEnd = html.toLowerCase().indexOf("</details>");
    if (detEnd >= 0) {
        blocks.push(...htmlToBlocks(html.slice(end, detEnd)));
        i = detEnd + "</details>".length;
    } else i = end;
    continue;
}
if (lower.startsWith("<li", at) || (liM && at === 1)) {
    const openEnd = html.indexOf(">", at) + 1;
    const close = html.toLowerCase().indexOf("</li>", at);
    const text = stripTags(html.slice(openEnd, close < 0 ? html.length : close));
    if (text) blocks.push({ kind: "li", text });
    i = close < 0 ? html.length : close + 5;
    continue;
}
if (lower.startsWith("<tr", at)) {
    const openEnd = html.indexOf(">", at) + 1;
    const close = html.toLowerCase().indexOf("</tr>", at);
    const rowHtml = html.slice(openEnd, close < 0 ? html.length : close);
    const cells: string[] = [];
    const tdRe = /<(t[dh])(\s{^>]*)?>([\s\S]*?)<\/\1>/g;
    let tm: RegExpExecArray | null;
    while ((tm = tdRe.exec(rowHtml))) cells.push(stripTags(tm[2]));
    if (cells.some(Boolean)) blocks.push({ kind: "table", cells });
    i = close < 0 ? html.length : close + 5;
    continue;
}
// <p>
const openEnd = html.indexOf(">", at) + 1;
const close = html.toLowerCase().indexOf("</p>", at);
const text = stripTags(html.slice(openEnd, close < 0 ? html.length : close));
if (text) blocks.push({ kind: "p", text });
i = close < 0 ? html.length : close + 4;
}
return blocks;
}

```

```

        idx = pos;
    }
    const imgRe = /<img[^>]*src="(data:image\/(png|jpeg
    let im: RegExpExecArray | null;
    while ((im = imgRe.exec(html))) images.push(im[1]);
    const grids = extractDivGrids(html);
    return { svgs, images, grids };
} catch (e) {
    return { svgs: [], images: [], grids: [], error: St
}
}

/**
 * Визуализации бывают и без SVG: матрицы из div-ячеек
 * префиксные суммы). Собираем такие «таблицы» в текст
 */
function extractDivGrids(html: string): string[][] {
    const grids: string[][] = [];
    const rows: string[] = [];
    const rowMark = /<div class="[^"]*(?:flex gap-1\.5|gr
    let m: RegExpExecArray | null;
    while ((m = rowMark.exec(html))) {
        // сбалансированно находим конец строки-контейнера
        let depth = 1;
        let pos = m.index + m[0].length;
        while (depth > 0 && pos < html.length) {
            const no = html.indexOf("<div", pos);
            const nc = html.indexOf("</div>", pos);
            if (nc < 0) break;
            if (no >= 0 && no < nc) { depth++; pos = no + 4;
            else { depth--; pos = nc + 6; }
        }
        const inner = html.slice(m.index + m[0].length, pos
        // теги → разделители: порядок значений сохраняется
        const cells = decodeEntities(inner.replace(/<[^>]+>
            .split(/\s+/)
            .filter((v) => v.length > 0 && v.length <= 6);
        if (cells.length >= 2 && cells.length <= 24) rows.p
    
```

```

.....
  setupFonts() {
    if (F_REG) this.doc.registerFont("Body", F_REG);
    if (F_BOLD) this.doc.registerFont("BodyBold", F_BOLD);
    if (F_MONO) this.doc.registerFont("Mono", F_MONO);
    if (F_MONO) this.doc.registerFont("MonoBold", F_MONO);
  }
  bodyFont() { this.doc.font(F_REG ? "Body" : "Helvetica"); }
  boldFont() { this.doc.font(F_BOLD ? "BodyBold" : "HelveticaBold"); }
  monoFont() { this.doc.font(F_MONO ? "Mono" : "Courier"); }

  get y() { return this.doc.y; }
  set y(v: number) { this.doc.y = v; }
  get pageBottom() { return A4[1] - M.bottom; }
  ensureSpace(need: number) {
    if (this.y + need > this.pageBottom) this.doc.addPage();
  }
  footerAll(skipCover = true) {
    const range = this.doc.bufferedPageRange();
    const oldBottom = this.doc.page.margins.bottom;
    for (let p = range.start; p < range.start + range.count; p++) {
      if (skipCover && p === range.start) continue;
      this.doc.switchToPage(p);
      this.doc.page.margins.bottom = 0;
      this.doc.font(F_REG ? "Body" : "Helvetica").fontSize(12)
        .text(String(p + 1), 0, A4[1] - 30, { width: A4[1] });
      this.doc.page.margins.bottom = oldBottom;
    }
  }
}

```

```

const CH_LINE = 13.4;      // межстрочный в теле
const BODY_SIZE = 9.6;

```

```

function drawTextBlock(d: Doc, text: string, opts: { size: number,
  boolean } = {}) {
  const size = opts.size ?? BODY_SIZE;
  const lead = size * 1.42;
  d.ensureSpace(lead * 2);

```

```

.....
    const to = Math.min(lines.length, from + fit);
    drawPortion(from, to, yTop);
    from = to;
    if (from < lines.length) { d.doc.addPage(); yTop = 1
  }
  d.y = yTop + Math.min(lines.length, Math.floor((d.page
  d.doc.moveDown(3);
}

```

```

function drawTableBlock(d: Doc, rows: string[]) {
  const size = 8.4;
  d.ensureSpace(size * 3);
  const text = rows.map((c) => c.replace(/\s*\n\s*/g, "
  d.doc.fontSize(size).fillColor("#374151").text(text, 1
  d.doc.moveDown(2.2);
}

```

```

function drawGrid(d: Doc, rows: string[]) {
  const size = 8;
  const lead = size * 1.6;
  d.ensureSpace(rows.length * lead + 16);
  const top = d.y;
  d.monoFont().fontSize(size).fillColor("#111827");
  const cellW = Math.min(
    34,
    (CONTENT_W - 16) / Math.max(...rows.map((r) => r.sp
  );
  rows.forEach((row, ri) => {
    const cells = row.split("\u2009");
    cells.forEach((cval, ci) => {
      const x = M.left + 6 + ci * cellW;
      const y = top + 4 + ri * lead;
      d.doc.save().rect(x - 2, y - 2, cellW - 4, lead -
      d.doc.text(cval, x + 2, y + lead * 0.12, { width:
    });
  });
  d.y = top + 8 + rows.length * lead + 8;
  d.doc.moveDown(2);
}

```



```
.....
    const avail = d.pageBottom - d.y;
    if (avail < 200) d.doc.addPage();
    try {
        d.doc.image(buf, M.left + (CONTENT_W - maxW) / 2, d
    } catch { return; }
    d.y = d.y + 10 + maxW * 0.6; // примерная высота фото
    d.doc.moveDown(2);
}
```

```
// ----- сборка -----
```

```
const FRONT_TOC_LINES = 44; // строк содержания на стра
```

```
function tocEntries() {
    return chapters.map((c) => ({ id: c.id, title: c.titl
})
```

```
function tocPagesNeeded(): number {
    return Math.max(1, Math.ceil(tocEntries().length / FR
})
```

```
interface BuildOpts {
    outPath: string;
    record: boolean;
    /** id → стартовая страница (для второго прохода) */
    pageMap?: Map<string, number>;
    frontPages: number;
}
```

```
function buildOne(opts: BuildOpts): Stats {
    return new Promise<Stats>((resolve, reject) => {
        const stream = opts.record ? fs.createWriteStream("
        const d = new Doc(stream as unknown as fs.WriteStre
        const doc = d.doc;

        const finish = (stats: Stats) => {
            d.footerAll(!opts.record);
            doc.end();
```

```

}
const pageNo2 = (opts.pageMap?.get(ch.id) ?? 0)
const isTopic = /^\\d/.test(ch.title);
doc.fontSize(isTopic ? 10.5 : 10).font(isTopic ? 'serif' : 'sans-serif')
  .fillColor(isTopic ? "#1f2937" : "#6b7280");
// заголовок + точечная отбивка + номер страницы
const title = ch.title.length > 86 ? ch.title.slice(0, 86) + '...' : ch.title;
const w = CONTENT_W - 40;
const tw = doc.widthOfString(title);
const pw = doc.widthOfString(String(pageNo2));
doc.text(title, M.left, d.y, { width: w, lineHeight: 1.2 });
const dots = ".".repeat(Math.max(3, Math.floor((w - tw - pw) / 2)));
doc.fillColor("#d1d5db").text(dots, M.left + tw + pw, d.y);
doc.fillColor("#111827").text(String(pageNo2), M.left + tw + pw, d.y);
d.y += 16.2;
line++;
});
// добиваем до расчётного числа страниц содержания
const used = Math.ceil(entries.length / FRONT_TOC_ENTRIES);
for (let p = used; p < opts.frontPages - 1; p++) {
  doc.addPage();
  if (!opts.record) console.log(`TOC: ${entries.length} pages`);
}

// — главы —
const stats: Stats = { chapterStart: new Map(), total: 0 };

chapters.forEach((ch, ci) => {
  const startPageOf = () => {
    // текущая страница = pages (0-based) + 1
    return doc.bufferedPageRange().start + doc.bufferedPageRange().end;
  };
  if (ci > 0) doc.addPage();
  const pageNow = opts.record ? doc.bufferedPageRange().end : startPageOf();
  d.chapterStart.set(ch.id, pageNow);
  if (!opts.record && ci < 3) console.log(`глава ${ci + 1}: ${ch.title}`);
  d.bodyFont();
  doc.y = M.top;
});

```

```

.....
    const n = (art?.svgs.length ?? 0) + (art?.ima
    console.log(` viz ${ch.id}: svg=${art?.svgs.
    for ? " ОШИБКА: " + art.error : ""}`);
    if (n !== 0) console.log(` Δ ${ch.id}: визуа
}
if (art && (art.svgs.length || art.images.length
d.ensureSpace(60);
doc.moveDown(0.6);
doc.fontSize(10).font(F_BOLD ? "BodyBold" : "
    .text("■ Визуализация: " + vizEntry.title,
d.y = doc.y + 6;
let drawn = 0;
for (const img of art.images) {
    if (drawn >= 4) break;
    drawImage(d, img);
    drawn++;
}
for (const svg of art.svgs) {
    if (drawn >= 5) break;
    drawSvg(d, svg);
    drawn++;
}
for (const grid of art.grids) {
    if (drawn >= 7) break;
    drawGrid(d, grid);
    drawn++;
}
} else if (art?.error) {
    // SSR упал – рисуем текстовую заглушку, PDF
}
}
});

stats.chapterStart = d.chapterStart;
stats.total = doc.bufferedPageRange().count;
finish(stats);
});
}

```

ФАЙЛ 90/99: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМЕР: 3.5 КБ

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
  ROOT,
  PAGES_DIR,
  TXT_PATH,
  PAGE,
  ensureDir,
  humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
  const expanded = line.replace(/\t/g, " ").replace(/<br>/g, "\n");
  if (expanded.length <= PAGE.cols) return [expanded];
  const parts = [];
  const indent = " ".repeat(Math.min(expanded.match(/^<br>/g, "\n");
  const rest = expanded;
  let first = true;
  while (rest.length > 0) {
    const width = first ? PAGE.cols : PAGE.cols - indent.length;
    parts.push((first ? "" : indent) + rest.slice(0, width));
    rest = rest.slice(width);
    first = false;
  }
}
```

```

    "-page", PAGE.size,
    "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "Helvetica",
    "-pointsizesize", String(PAGE.pointsizesize),
    `text:${job.txtFile}`,
    "-background", "white",
    "-flatten",
    "-colorspace", "Gray",
    ...(PAGE.monochrome ? ["-monochrome"] : []),
    "-strip",
    "-define", "png:compression-level=9",
    job.pngFile,
};

```

```

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

```

```

async function worker() {
  while (cursor < jobs.length) {
    const job = jobs[cursor++];
    await execFileAsync(im, args(job));
    fs.rmSync(job.txtFile, { force: true });
    done += 1;
    if (done % 25 === 0 || done === total) {
      process.stdout.write(`отрендерено ${done} страниц\n`);
    }
  }
}

```

```

await Promise.all(Array.from({ length: concurrency }, () => worker()));
process.stdout.write("\n");

```

```

const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
if (missing.length) throw new Error(`Не отрендерены ${missing.length} страниц`);

```

```

const bytes = jobs.reduce((s, j) => s + fs.statSync(j.pngFile).size, 0);
console.log("[2/3] txt -> png");
console.log(`объем: ${bytes} байт, страниц: ${total}`);

```

Универсальное пособие • полный исходный код

```
.....  
const used = new Set<string>();  
for (const m of html.matchAll(/data-term-id="([^\"]+)"/g)  
console.log("== data-term-id в контенте → нет в глоссарии ==");  
for (const t of used) if (!GLOSSARY_BY_ID.has(t)) console.log("термин в глоссарии: ", GLOSSARY_BY_ID.get(t));  
console.log("терминов в глоссарии:", GLOSSARY_BY_ID.size);  
.....
```

ФАЙЛ 93/99: scripts/selftest/_harness-entry.tmp.ts
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 3 | РАЗМЕР: 1000 байт

```
.....  
export { PAGE_SYNC } from "../../src/data/vizSync";  
export { buildDebugRunner, DEBUG_STEP_CAP } from "../../src/data/vizSync";  
.....
```

ФАЙЛ 94/99: scripts/selftest/_regcheck.tmp.ts
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 9 | РАЗМЕР: 1000 байт

```
.....  
import { chapters } from "../../src/data/content";  
import { VIZ_REGISTRY as vizRegistry } from "../../src/data/vizRegistry";  
const regIds = new Set(Object.keys(vizRegistry as any));  
console.log("== главы без визы ==");  
for (const c of chapters) if (!regIds.has(c.id)) console.log(c.id);  
console.log("== визы без главы ==");  
for (const id of regIds) if (!chapters.some((c: any) => c.id === id)) console.log(id);  
console.log("глав:", chapters.length, "| виз в реестре:", regIds.size);  
.....
```

ФАЙЛ 95/99: scripts/selftest/_runner.tmp.ts
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 14 | РАЗМЕР: 1000 байт

```
.....  
// Прогон КАЖДОГО шаблона через настоящий харнесс дебага  
// полный код и init-версия; проверяем error/truncated/timeout  
.....
```

```

// 2. splay: числовой key + кадр i
const sp = init("splay-tree");
ok(sp.includes("key = 32") && sp.includes("keys = {50"});

// 3. КМП/Z: pattern/text/s попадают в init
const kmp = init("string-kmp");
ok(kmp.includes('pattern = "aba"') && kmp.includes('s =

// 4. Форду-Беллману — рёбра реального графа (S→A = 4,
const bf = PAGE_SYNC["bellman-ford"].code ?? "";
ok(bf.includes('("S", "A", 4)') && bf.includes('("D", "
ok(!bf.includes('("S", "A", 5)'), "bellman-ford: загрузи
ok(!/break/.test(bf), "bellman-ford: без break");

// 5. Джонсон: h(B) = -2, ребро -2, w' = 0
const jh = PAGE_SYNC["johnson-algo"].code ?? "";
ok(jh.includes('"B": -2') && jh.includes("w_new = w + h

// 6. Краскал/Прим/Борувка: 9 вершин и вес 51
for (const id of ["mst-kruskal", "mst-boruvka"]) {
  const c = PAGE_SYNC[id].code ?? "";
  ok(/ABCDEFGHI/.test(c), `${id}: 9 вершин A..I`);
}
ok((PAGE_SYNC["mst-prim"].code ?? "").match(/"I":/g)?,
ok((PAGE_SYNC["mst-kruskal"].code ?? "").includes('{13,

// 7. prefix-sums-2d#2d: пометка 1-индексных клеток
const p2 = PAGE_SYNC["prefix-sums-2d#2d"].code ?? "";
ok(p2.includes("клетки 1-индексные"), "prefix#2d: комме

// 8. графы top-sort/scc/graph-dfs-bfs/graph-components
ok((PAGE_SYNC["top-sort"].code ?? "").includes("0: [1,
ok((PAGE_SYNC["scc-kosaraju"].code ?? "").includes("2:
  емо");
ok((PAGE_SYNC["graph-dfs-bfs"].code ?? "").includes('ad
ok((PAGE_SYNC["graph-components"].code ?? "").includes(

```

```
.....

let fails = 0;
for (const [uL, uR, add] of cases) {
  const steps = genLazySteps(base, uL, uR, add);
  const last = steps[steps.length - 1];
  const wantRoot = sum(0, 5) + add * (uR - uL + 1);
  const wantQ = sum(uL, uR) + add * (uR - uL + 1);
  const gotRoot = last.treeState[1];
  const gotQ = q(last, 1, 0, 5, uL, uR, 0);
  const okR = gotRoot === wantRoot, okQ = gotQ === wantQ;
  console.log('(uL,uR,add)=(${uL},${uR},${add}) root=${gotRoot} q=${gotQ} wantRoot=${wantRoot} wantQ=${wantQ} "FAIL" | шагов=${steps.length}');
  if (!okR || !okQ) fails++;
}
// query без обновлений = обычная сумма
const q0 = q({ treeState: buildTreeFull(base), lazyState: {} });
console.log("query(1,3) без обновлений =", q0, q0 === 23);
if (q0 !== 23) fails++;
console.log(fails === 0 ? "LAZY SELFTEST: ALL OK" : `LAZY SELFTEST: FAILS=${fails}`);
process.exit(fails === 0 ? 0 : 1);
```

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

.....
ФАЙЛ 98/99: [.github/workflows/deploy-pages.yml](#)

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9 КБ

name: Deploy app to GitHub Pages

```
# ДВЕ настройки репозитория нужно сделать один раз ВРУЧНУЮ
# (у бота-интеграции нет admin-прав на них – API отвечает за них)
#
# 1. Settings → Pages → Build and deployment → Source
#    Если там «Deploy from a branch» (legacy) – шаг D
```


- ```
.....
- name: Build for the repository subpath
 env:
 VITE_BASE: /algv0/
 run: npm run build

- name: Make GitHub Pages fallback
 run: cp dist/index.html dist/404.html

- name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist
```

#### deploy:

```
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 # Ранняя и понятная ошибка вместо невнятного «Fail»
 # когда в репозитории Pages не переключён на сборку
 - name: Preflight – Pages собирается через GitHub Pages
 env:
 GH_TOKEN: ${ secrets.GITHUB_TOKEN }
 run: |
 bt=$(gh api "repos/${ github.repository }/pages" |
 echo "Pages build_type: ${bt}")
 if ["${bt}" != "workflow"]; then
 echo "::error title=Pages не настроен::Сетте этот запуск. См. комментарий в начале деплоя"
 exit 1
 fi

 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
```

## Универсальное пособие • полный исходный код

```
.....
concurrency:
 group: rebuild-pdf-${{ github.ref }}
 cancel-in-progress: true

permissions:
 contents: write

jobs:
 rebuild-pdf:
 name: код → txt → png → pdf
 runs-on: ubuntu-latest
 # не реагируем на собственный коммит бота
 if: "!contains(github.event.head_commit.message, '[bot])"
 steps:
 - name: Checkout
 uses: actions/checkout@v6
 with:
 fetch-depth: 0
 persist-credentials: true

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: "24"
 cache: npm

 - name: Install ImageMagick + DejaVu fonts
 run: |
 sudo apt-get update -qq
 sudo apt-get install -y --no-install-recommends imagemagick
 # На Ubuntu политика ImageMagick иногда запрещает конвертировать текст в png
 # разрешаем то, что нужно пайплайну (text -> png)
 for policy in /etc/ImageMagick-6/policy.xml /usr/lib*/ImageMagick-6/policy.xml; do
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\(*\|text\)" /rights="write" pattern="\1" /g' "$policy"
 fi
 done
```

```
.....
 uses: actions/upload-artifact@v7
 with:
 name: full-code-export
 path: |
 public/export/full_code_all_pages.pdf
 public/export/full_code_all_pages.txt
 public/export/guide_all_topics.pdf
 if-no-files-found: error
 retention-days: 30

- name: Upload artifacts (PNG-страницы)
 uses: actions/upload-artifact@v7
 with:
 name: full-code-pages-png
 path: tmp/export-pages/*.png
 if-no-files-found: error
 retention-days: 7

- name: Commit rebuilt export back to the branch
 run: |
 git config user.name "github-actions[bot]"
 git config user.email "41898282+github-action@users.noreply.github.com"
 git add -- public/export/full_code_all_pages.*
 if git diff --cached --quiet; then
 echo "Экспорт не изменился — коммит не нужен"
 exit 0
 fi
 git commit -m "chore(export): пересборка full-code-export"
 git push origin "HEAD:${GITHUB_REF_NAME}"
```